

PrimeWave™ Design Environment Calculator Function Reference Manual

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	11
Related Products, Publications, and Trademarks	11
Conventions	12
Customer Support	13
Statement on Inclusivity and Diversity	13

1. PrimeWave Design Environment Calculator Overview	15
Using the PrimeWave Design Environment Calculator	15
Creating Expressions with the Calculator	16
Adding and Viewing Expressions in History	18
Adding Expressions to Output	18
Adding Favorite Expressions	19
Changing the Order of Favorite Expressions	19
Removing Favorite Expressions	20

2. Supported Calculator Functions	21
Data Access Functions	21
aceSignal()	23
data()	24
dBCPh()	26
elem()	28
extData()	29
fftOutput()	30
file()	31
flatten()	32
getPhasenoise()	33
harmonic()	34
i()	35
imProd()	36
inoise()	37
ipnh()	38

Contents

l()	40
lstb()	41
lv()	42
lx()	43
mcMean()	44
mcStddev()	45
meas()	46
o()	47
onoise()	48
op()	48
opt()	50
poles()	51
power()	52
signal()	53
stateye()	54
stateyeM()	55
temp()	56
v()	57
var()	58
vlogA()	58
zeros()	60
Operators, Constants, and Logic Operations	61
Supported Operators	61
Supported Constants	61
Supported Logic Operations	62
Mathematic Functions	63
abs	64
acos	65
acosh	66
asin	66
asinh	67
atan	67
atanh	68
complex	68
concat	69
conjugate	69
cos	70
cosh	70

Contents

db10	71
db20	72
dbm	72
exp	73
floor	74
imag	74
int	75
ln	75
log10	76
mag	77
max	77
min	78
phase	79
pwr	80
real	80
round	81
sgn	82
sin	82
sinh	83
sqrt	83
tan	84
tanh	85
Measurement Functions	85
avgdev	89
avgfreq	90
bw	92
bw3db	93
correlation	95
cross	96
dBCP	100
dcgain	103
dcgamos	105
dcswing	108
delay	110
direct_ipn	115
duty cycle	115
enb	118
eye_mask	120

Contents

falltime	122
falltrig	125
gain1_f	127
gain_m	129
gbp	131
get_harmonic_freq	133
globalmax	134
globalmean	135
globalmin	136
globalstddev	137
imProd	138
integ	139
ipn	141
ipnhVRI	143
mean	144
measure_aperture	146
measure_eyeopen	148
measure_eyesize	149
median_val	152
overshoot	154
p2p	156
pamn_level_at_eye_center	158
pamn_level_at_user_ref	160
pamn_open_at_eye_center	161
pamn_open_at_user_ref	162
paramvalue	164
phase_m	164
risetime	166
risetrig	169
rmsval	171
root	172
settling_t	174
sfdr	177
signalPower	180
slewrates	182
slope	185
snb	186
sndr	188
snr	191

Contents

snr_f	194
snrh	195
stddev	197
sweepValueDiff	198
thd	199
variance	201
x_at_ymax	203
x_at_ymin	205
xmax	207
xmin	209
yield	210
ymax	211
ymin	213
yvalue	216
Waveform Functions	217
a2d	221
average	224
clip	227
cPwrContour	230
create_bus	232
create_eye	232
crossings	237
d2a	240
dBc2V2	243
delayvst	243
derivative	247
dft	249
dftbb	254
dnl	258
duty_vs_t	261
fallvst	263
fft	265
flip	269
fourEval	270
func	273
fvst	274
gdelay	276
histo	278

Contents

hold_times	280
hold_times2	283
ift	287
inl	288
integral	290
ipnhVRICurves()	291
join	292
jvst_freq	294
jvst_ideal	298
jvst_llse	301
jvst_refsig	305
loadpull	308
max_window_average	309
median	311
parametric	312
pareto_r	313
pareto_r2	315
pareto_s	317
pavg	320
peak	322
phasenoise	325
prms	327
psd	329
psdbb	332
pstddev	334
pwc	336
pzFreqRealFilter()	338
reflectionContour	339
repeat	340
risevst	341
sample	343
setup_times	346
setup_times2	349
signalAtFreq	352
signalcompare	353
slewratesvst	353
smooth	356
spectralPower	358
sPowerDiffVI	359

Contents

sPowerDiffVR	360
swapsweep	360
sweepavg	362
sweepmax	363
sweepmin	364
tangent	366
text2wave	367
trace	368
v2todBc	370
window_average	370
x_at_y	371
xscale	372
xshift	373
xval	374
yvsi	375
RF Functions	376
apgc	378
b1f	379
dBCPhVRI	380
dBCPVRICurves	381
ga	382
gac_freq	383
gac_gain	383
gmax	383
gmin	384
gmsg	384
gp	385
gpc_freq	385
gpc_gain	386
gt	386
gumx	386
hp	387
kf	388
lsb	388
lq	389
mu1	390
mu2	390
nc_freq	390

Contents

nc_gain	391
nf	392
nfmin	392
noisec	392
opgc	393
rn	395
Rn	395
sp	395
spm	396
ssb	397
stabc_in	397
stabc_out	398
vswr	399
vswrc_in	400
vswrc_out	401
ym	402
yp	403
zm	404
zp	404

About the PrimeWave Design Environment Calculator Reference Manual

The *PrimeWave Design Environment Calculator Reference Manual* provides reference information for the PrimeWave Design Environment's Calculator functions, which are used to analyze simulation data. The material includes syntax, arguments, and examples.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeWave Design Environment Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeWave™ Design Environment tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® PrimeWave™ Design Environment

Synopsys® Custom Compiler

PrimeSim™ Continuum

PrimeSim™

PrimeSim™ Pro

PrimeSim™ SPICE

- PrimeSim™ HSPICE®
- PrimeSim™ XA
- PrimeSim™ CCK
- PrimeSim™ MOSRA
- VCS® PrimeSim™ AMS
- Synopsys® FineSim®
- Synopsys® HSPICE®
- Synopsys® VCS®
- Synopsys Verdi®
- Cadence® Spectre® Circuit Simulator
- MathWorks® MATLAB

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>

Convention	Description
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also

includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

PrimeWave Design Environment Calculator Overview

This chapter contains information on how to use the PrimeWave Design Environment's Calculator.

The PrimeWave Design Environment Calculator provides a collection of functions used to analyze simulation data.

This chapter contains the following topics:

- [Using the PrimeWave Design Environment Calculator](#)
- [Creating Expressions with the Calculator](#)
- [Adding and Viewing Expressions in History](#)
- [Adding Expressions to Output](#)
- [Adding Favorite Expressions](#)
- [Changing the Order of Favorite Expressions](#)
- [Removing Favorite Expressions](#)

Using the PrimeWave Design Environment Calculator

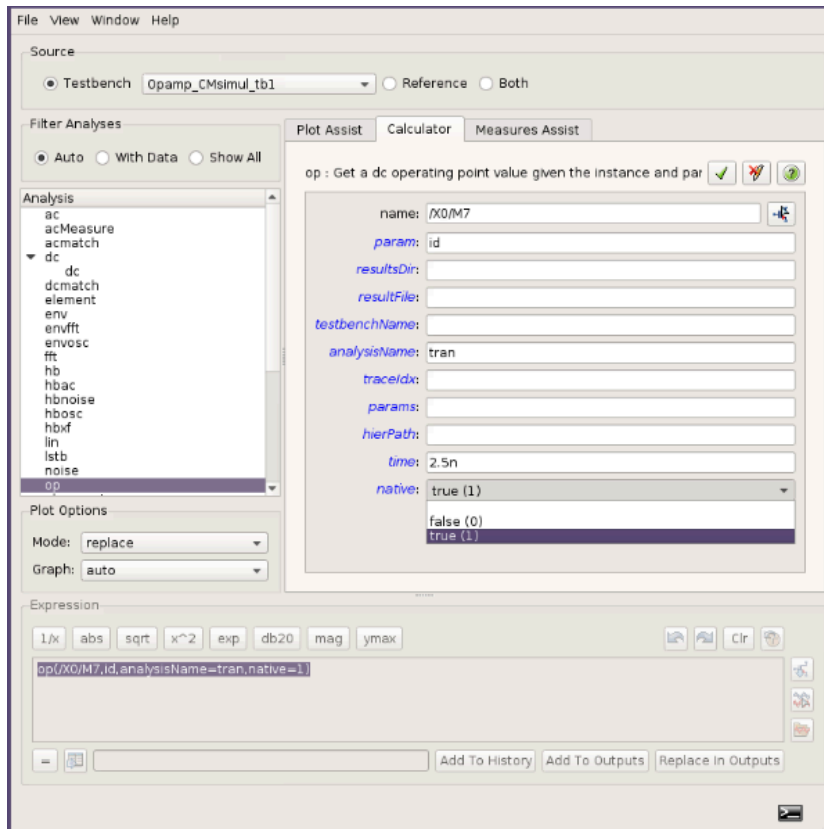
To access the PrimeWave Design Environment Calculator, click the **Calculator** tab in the Results Analyzer.

As you type a function or expression in the Calculator, the syntax of those functions or expressions is highlighted, which can help ensure you use correct syntax.

Click the **Help** button (the question mark) in the upper-right corner of the calculator to open the help topic for the selected function.

For example, the following figure shows an `op()` function as it is set up in the Calculator.

```
op(/X0/M7, id, time=2.5n)
```



[Show me](#) how to use the calculator to create output expressions.

See also Creating Custom Functions.

Creating Expressions with the Calculator

To create an expression in the Calculator:

1. Choose **Results > Analyze** from the PrimeWave Design Environment main menu bar.

The Results Analyzer window opens.

2. Click the **Calculator** tab in the Results Analyzer window.
3. Enter a signal (or expression) directly into the **Expression** field, or use one of the following methods:

- Use the Plotting Assistant.

See Plotting Signals With the Plotting Assistant for more information on how to use this method.

- Select identifiers in the schematic.

Click **Pick from Schematic**  to the right of the **Expression** field. Next, select one or more wire segments or instance terminals in the design. Selecting wires creates an expression for the voltage of the corresponding net, while selecting instance terminals creates an expression for the current through that terminal.

- Select signals from the waveform viewer.

Click **Pick from waveform viewer**  to the right of the **Expression** field. waveform viewer opens in "export-signal" mode. After you pick a signal in a waveform viewer window, the signal(sigName, filename) expression is added into the **Expression** field of the Results Analyzer.

- Select signals from the signal browser.

Click **Pick from signal browser**  to the right of the **Expression** field. waveform viewer opens in "export-signal" mode. After you pick a signal from the signal browser, the signal(<sigName>, /full/path/filename.ext) expression is added into the **Expression** field of the Results Analyzer.

- Click **Results Analyzer** in the **Output** pane of the PrimeWave Design Environment main window.

The expression in the corresponding row of the **Output** pane appears in the **Expression** field.

4. Locate and double-click a function from the list of available functions.

You can view the functions by category by choosing from the **Category** menu. You can also type part of the name of a function in the **Filter** text box.

If the function requires input in addition to the expression, a panel appears and replaces the function list.

For example, double-clicking the `cross` function displays a panel with the following arguments:

- s: The input signal.
- y: The threshold level.
- n: The required stop value.
- x: The delay value.

Moving your pointer over any of these text boxes gives you descriptions of the function arguments. If an expression is in the **Expression** field when the function is activated, it is automatically inserted as the value of the first argument.

If the function does not require additional input (if it is a single-argument function), then the function is immediately inserted into the expression buffer. The position of insertion depends on the position of the cursor. If the cursor is placed immediately before or after an expression, it treats the expression as the first (or only argument) of the function. If anything is selected in the **Expression** field, the selected items are treated as the function argument.

- When you have selected a function in the Calculator, the green check mark button indicates either **Add** or **Update**.
 - **Add** appears when there are no items selected in the **Expression** field. Click **Add** to insert the function in the **Expression** field at the cursor position.
 - **Update** appears when there is an item selected in the **Expression** field. Click **Update** to replace the selection in the **Expression** field with the function.
- You can edit functions by clicking the function name. The existing function expression is loaded in the Calculator panel, and can be edited and updated.

If you single-click an expression, the syntax and type of expression is displayed just below the list of expressions.

For an example of how to create expressions using the Calculator, see SolvNetPlus article, [“How to save and plot the parameters of a VerilogA file in the PrimeWave Design Environment”](#).

Adding and Viewing Expressions in History

The expression history is populated with the last 16 expressions are successfully evaluated. After evaluation, an expression is copied to the history and can be recalled at any time and used to build future expressions. To add the current contents of the **Expression** field to the history without evaluating the expression, click the **Add to History** button.

To view the expression history, click **History** , which is located to the right of the **Clear** button, and select an expression from the list.

Adding Expressions to Output

To add an expression to the Output Set in the PrimeWave Design Environment main window, click the **Add to Outputs** button.

The contents of the expression field are added to the next row of the **Outputs** table.

Adding Favorite Expressions

To add one or more favorite expressions in the Calculator:

1. Ensure the Results Analyzer is open (**Results > Analyzer**), and click the Calculator tab.
2. Choose **<Favorites>** from the **Category** menu.

The **<Edit Favorites>** text appears along with any previously specified favorite expressions.

3. Double-click the **<Edit Favorites>** text.

The **Edit Favorites** dialog box opens.

4. (Optional) Choose the expression category that contains the expression you want to add as a favorite.

All available expressions are listed by default.

5. Select an expression from the list that is displayed.

You can also type the first few letters of an expression in the **Filter** text box to find a specific expression, and then select that expression.

6. Click **Add** to add just the selected expression, or click **Add All** to add all the currently displayed expressions.

Expressions are added to the list of favorites.

7. Click **OK** to save your changes.

See Also

- [Removing Favorite Expressions](#)
- [Changing the Order of Favorite Expressions](#)

Changing the Order of Favorite Expressions

To change the order of the expressions listed in the expression favorites:

1. Ensure the Results Analyzer is open (**Results > Analyzer**), and click the Calculator tab.
2. Choose **<Favorites>** from the **Category** menu.

The **<Edit Favorites>** text appears along with any previously specified favorite expressions.

3. Double-click the **<Edit Favorites>** text.

The **Edit Favorites** dialog box opens.

4. In the list of **Favorites**, select an expression and click **Up** or **Down** to move the expression up or down in the list.

Any changes you make are also reflected in the list of favorite expressions available from the **<Favorites>** category in the Calculator.

5. Click **OK** to save your changes.

Removing Favorite Expressions

To remove one or more favorite expressions:

1. Ensure the Results Analyzer is open (**Results > Analyzer**), and click the **Calculator** tab.
2. Choose **<Favorites>** from the **Category** menu.

The **<Edit Favorites>** text appears along with any previously specified favorite expressions.

3. Double-click the **<Edit Favorites>** text.

The **Edit Favorites** dialog box opens.

4. Select an expression from the list of favorites that is displayed.
5. Click **Remove** to remove just the selected expression, or click **Remove All** to remove all the currently displayed expressions in the favorites list.

Expressions are removed from the list of favorites.

6. Click **OK** to save your changes.

2

Supported Calculator Functions

The following topics describe the available Calculator functions:

- [Data Access Functions](#)
 - [Operators, Constants, and Logic Operations](#)
 - [Mathematic Functions](#)
 - [Measurement Functions](#)
 - [Waveform Functions](#)
 - [RF Functions](#)
-

Data Access Functions

This section contains information on how to use the supported data access functions.

The following topics summarize the data access functions available in the PrimeWave Design Environment Calculator.

Function	Description
aceSignal()	Reads the saved signal data from an ACE script.
data()	Returns a specified signal from a specified result in a specified directory.
dBCPh()	Returns the nth order intercept point.
elem()	Returns an element parameter value given the instance and parameter name.
extData()	Generic socket for accessing an external data access Tcl procedure that is not formally registered with the PrimeWave Design Environment expression language (no <code>wf::createFunction</code> was used).

Function	Description
fftOutput()	Returns a single signal from a specified result in a specified directory.
file()	Returns a file handle for built-in functions.
flatten()	Flattens the given signal with the given parameter as the new x-axis.
getPhasenoise()	Returns the phasenoise from a phasenoise analysis.
harmonic()	Returns the signal value for a specific intermodulation product.
i()	Returns the specified current signal.
imProd()	Returns the out versus in curve for intermodulation product.
inoise()	Returns the equivalent input noise from a noise analysis.
ipnh()	Returns the nth order intercept point.
l()	Returns the specified logic signal.
lstb()	Returns the loop stability results from an LSTB analysis.
lv()	PrimeSim HSPICE LV#() function.
lx()	PrimeSim HSPICE LX#() function.
mcMean()	Returns the mean.
mcStddev()	Returns the stddev.
meas()	Returns a specified simulator measurement from a specified result in a specified directory.
o()	Returns the result of another expression.
onoise()	Returns the equivalent output noise from a noise analysis.
op()	Returns a DC operating point value given the instance and parameter name.
opt()	Returns a transient operating point value given the instance and parameter name.
poles()	Returns the poles from a pz analysis.
power()	Returns an instance power or, if the name is not specified, total power.

Function	Description
signal()	Returns the specified signal from the specified file.
stateye()	Returns a specified stateye signal from a specified result in a specified directory.
stateyeM()	Returns a specified stateye signal from a specified result in a specified directory.
temp()	Returns the value of the temperature design variable.
v()	Returns the specified voltage signal.
var()	Returns the value of a design variable.
vlogA()	Returns a Verilog-A internal parameter value given the instance and parameter name.
zeros()	Returns the zeros from a pz analysis.

aceSignal()

Description

Reads the saved signal data from an ACE script. Typically, this function is automatically added when results from ACE scripts are found.

Syntax

```
aceSignal (name <string>
    [, resultType <string> | resultFile <string>]
    [, analysisName <string>]
    [, time <string>]
    [, dataType <string>]
    [, resultsDir <string> | testbenchName <string>]
    [, mapType <string>]
    [, traceIdx <string> | params <string>]
    [, hierPath <string>])
```

Arguments

name

The name of the signal.

resultType

(Optional) The result type.

`analysisName`

(Optional) The user-specified name for the analysis.

`time`

(Optional) The time value.

`dataType`

(Optional) The data type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`mapType`

(Optional) The signal type (instance, net, or terminal). (Default: net)

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

`hierPath`

(Optional) A hierarchical path to appended to relative name value (no beginning "/").

data()

Description

Returns a specified signal from a specified result in a specified directory. The main purpose of this function is to handle data internally. For specialized direct use, such as while using PrimeSim HSPICE MOSFET Output Templates, you can use this by specifying only (one) first argument. The other arguments are optional and are resolved automatically.

Syntax

```
data ([name <signalName>]  
    [, param <string>])
```

```
[, resultType <string> | resultFile <string>]  
[, resultsDir <string> | testbenchName <string>]  
[, analysisName <string>]  
[, mapType <string>]  
[, dataType <string>]  
[, time <string>]  
[, traceIdx <string>]  
[, params <string>]  
[, hierPath <string>]  
[, native <boolean>])
```

Arguments

name

(Optional) The identifier for the result.

param

(Optional) The parameter name for the result for specifying a primary parameter needed to access the desired data.

resultType

(Optional) The type of analysis whose data is being accessed.

resultFile

(Optional) The file where the results reside.

resultsDir

(Optional) The directory where the results reside.

testbenchName

(Optional) The name of the testbench.

analysisName

(Optional) The user-specified name for the analysis.

mapType

(Optional) The type of object signal (instance, net, or terminal) whose data is being accessed. (Default: none)

dataType

(Optional) An integrator-defined data type of the signal.

time

(Optional) The time point for the transient operating point/node voltage.

`traceIdx`

(Optional) The trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions used to filter a family of signals created by iterative simulation.

`hierPath`

(Optional) A hierarchical path to append to relative name value (not beginning with "/").

`native`

(Optional) The param is directly found in the simulation results, otherwise an expression is sought first. (Default: 0)

Examples

The following example plots a transfer function of the voltage source v1 from XF analysis:

```
data(v1)
```

The following example plots loop gain from STB/HBSTB/PSTB analysis:

```
data(loopGain)
```

dBCPh()

Description

Returns the nth order intercept point.

Syntax

```
dBCPh (signal <string>,  
      harmonicIndices <string>  
      [, gainDecrease <string>]  
      , extrapolationPoint <string>  
      [, referred <enum>]  
      [, sweepUnits <enum>]  
      [, resultType <string>]  
      [, resultsDir <string>]  
      [, resultFile <string>]  
      [, testbenchName <string>]  
      [, analysisName <string>]  
      [, params <string>])
```

Arguments

`signal`

The signal from which to extract the harmonic.

`harmonicIndices`

The harmonic indices, space delimited.

`gainDecrease`

(Optional) The gain decrease. (Default: 1)

`extrapolationPoint`

Input signal extrapolation point.

`referred`

(Optional) The Input/Output referred measurement. Valid values: input, output.
(Default: input)

`sweepUnits`

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none. (Default: dBm)

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`params`

(Optional) The parameter conditions.

elem()

Description

Returns an element parameter value given the instance and parameter name.

Syntax

```
elem (name <instance>  
      [, param <string>]  
      [, resultsDir <string> | testbenchName <string>]  
      [, resultFile <string>]  
      [, analysisName <string>]  
      [, traceIdx <string> | params <string>]  
      [, hierPath <string>])
```

Arguments

name

The hierarchical instance name from the design. Can substitute for the param argument when hierPath.

param

(Optional) The name of the parameter.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The name of the results file.

testbenchName

(Optional) The name of the testbench to use.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to be appended to the relative name value (no beginning "/").

extData()

Description

Generic socket for accessing external data access Tcl procedure that is not formally registered with the PrimeWave Design Environment expression language (no `wf::createFunction` was used).

Syntax

```
extData (tclProcName <string>
        [, arg1 <string>]
        [, arg2 <string>]
        [, arg3 <string>]
        [, arg4 <string>]
        [, arg5 <string>]
        [, arg6 <string>]
        [, arg7 <string>]
        [, arg8 <string>]
        [, arg9 <string>]
        [, arg10 <string>]
        [, name <signalName>]
        [, resultType <string>]
        [, resultsDir <string>]
        [, resultFile <string>]
        [, testbenchName <string>]
        [, analysisName <string>]
        [, mapType <string>]
        [, dataType <string>]
        [, traceIdx <string>]
        [, params <string>]
        [, hierPath <string>]
        [, time <string>])
```

Arguments

tclProcName

The name of the external Tcl data access type procedure to call from the PrimeWave Design Environment Expression Language.

arg1-arg10

(Optional) A generic string argument number 1-10.

name

(Optional) The hierarchical design name of the object.

resultType

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The name of the results file.

`testbenchName`

(Optional) The name of the testbench to use.

`analysisName`

(Optional) The user-specified name for the analysis.

`mapType`

(Optional) The signal type (instance, net, or terminal). (Default: none)

`dataType`

(Optional) The integrator-defined data type of the signal.

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions used to filter a family of signals created by iterative simulation.

`hierPath`

(Optional) A top-level path to append to relative instance names (not beginning with "/").

`time`

(Optional) An optional time point for transient operating point or node voltage.

fftOutput()

Description

Returns a single signal from the specified result in a specified directory.

Syntax

```
fftOutput ([resultType <string> | resultFile <string>]  
          [, resultsDir <string> | testbenchName <string>]  
          [, analysisName <string>]  
          [, signalName <string>]  
          [, traceIdx <string> | params <string>])
```

Arguments

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`signalName`

(Optional) An optional signal name (when empty, the single signal name is extracted from the data file).

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

file()

Description

Returns a file handle for built-in functions.

Syntax

```
file ([filename <filePath>]  
    [, resultType <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, params <string>])
```

Arguments

`filename`

(Optional) The name of the file to open.

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`params`

(Optional) The parameter conditions.

flatten()

Description

Flattens the specified signal with a specified parameter as the new x-axis. The `flatten()` function can be used if you have a set of sweep scalar values that you want to process further using calculator functions that take a signal as an argument. The `flatten()` function is particularly useful when you have single point analyses (DC, AC) over a parametric sweep or corners and you want to convert the result into a suitable form for calculator function measurements.

Syntax

```
flatten (signal <string>,  
        param <string>  
        [, resultType <string> | resultFile <string>]  
        [, resultsDir <string> | testbenchName <string>]  
        [, analysisName <string>]  
        [, params <string>]  
        [, hierPath <string>]  
        [, time <string>])
```

Arguments

`signal`

The signal to flatten.

`param`

The parameter to become the x-axis.

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The name of the result file.

`testbenchName`

(Optional) The name of the testbench to use.

`analysisName`

(Optional) The user-specified name for the analysis.

`params`

(Optional) The parameter conditions.

`hierPath`

(Optional) A hierarchical path to appended to relative name value (no beginning "/").

`time`

(Optional) An optional time point for transient operating point or node voltage.

getPhasenoise()

Description

Gets the phase noise from a phase noise analysis.

Syntax

```
getPhasenoise ([resultType <string> | resultFile <string>]  
               [, resultsDir <string> | testbenchName <string>]  
               [, analysisName <string>]  
               [, traceIdx <string> | params <string>])
```

Arguments

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

harmonic()

Description

Returns the signal value for a specific intermodulation product.

Syntax

```
harmonic ([signal <string>]
          [, coeff1 <string>]
          [, coeff2 <string>]
          [, coeff3 <string>]
          [, coeff4 <string>]
          [, coeff5 <string>]
          [, coeff6 <string>]
          [, coeff7 <string>]
          [, coeff8 <string>]
          [, coeff9 <string>]
          [, coeff10 <string>]
          [, xUnits <enum>]
          [, resultType <string> | resultFile <string>]
          [, resultsDir <string> | testbenchName <string>]
          [, analysisName <string>]
          [, params <string>])
```

Arguments

signal

(Optional) The signal expression that will be evaluated and reduced to a specific intermodulation product. For exampleFor snac and hbac data, if no coefficients are specified, the entire signal is reformatted such that a plot will display wavelets for each `sideband.v(/out)`. If a signal expression is not specified, the function returns the intermodulation frequency value.

`coeff1-coeff10`

(Optional) The frequency coefficients used to calculate the intermodulation frequency of the target harmonic. The number of coefficients specified must match the number of primary tones in the circuit.

`xUnits`

(Optional) Performs conversion of the xAxis units when not a frequency sweep. “asls” means no conversion. “toW” means the xAxis sweep units in the data is in Dbm and will be converted to Watts. “toDbm” means the xAxis sweep units in the data is in Watts and will be converted to Dbm.

Valid values: asls, isW, isDbm, toW, and toDbm. (Default: asls)

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`params`

(Optional) The parameter conditions.

i()

Description

Returns the specified current signal.

Syntax

```
i (name <pick>
    [, resultType <string> | resultFile <string>]
    [, resultsDir <string> | testbenchName <string>]
    [, analysisName <string>]
    [, mapType <string>]
    [, traceIdx <string> | params <string>]
    [, hierPath <string>])
```

Arguments

`name`

The hierarchical design name of the object.

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`mapType`

(Optional) The signal type.

Valid values: instance, net, or terminal. (Default: terminal)

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

`hierPath`

(Optional) A hierarchical path to appended to relative name value (no beginning "/").

imProd()

Description

Returns the out-versus-in curve for an intermodulation product.

Syntax

```
imProd (signal <string>,  
        freq <string>  
        [, resultType <string> | resultFile <string>])
```



```
[, resultsDir <string> | testbenchName <string>]  
[, analysisName <string>]  
[, params <string>]  
[, hierPath <string>]  
[, time <string>])
```

Arguments

signal

The signal on which to do the out-versus-in conversion.

freq

The intermodulation product frequency.

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

time

(Optional) An optional time point for transient operating point or node voltage.

inoise()

Description

Gets the equivalent input noise from a noise analysis.

Syntax

```
inoise ([resultType <string> | resultFile <string>]  
        [, resultsDir <string> | testbenchName <string>]  
        [, analysisName <string>]  
        [, traceIdx <string> | params <string>])
```

Arguments

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

ipnh()

Description

Returns the nth-order intercept point.

Syntax

```
ipnh (signal <string>,  
      firstOrderIndices <string>,  
      nthOrderIndices <string>  
      [, extrapolationPoint <string>]  
      [, order <integer>]  
      [, referred <enum>]  
      [, sweepUnits <enum>]  
      [, resultType <string>]  
      [, resultsDir <string>]  
      [, resultFile <string>])
```

```
[, testbenchName <string>]  
[, analysisName <string>]  
[, params <string>])
```

Arguments

signal

The signal from which to extract the harmonics.

firstOrderIndices

The first order indices, space delimited.

nthOrderIndices

The nth order indices, space delimited.

extrapolationPoint

(Optional) The input signal extrapolation point. If unspecified, defaults to lowest value.

order

(Optional) The intermodulation order. (Default: 3)

referred

(Optional) The Input/Output referred measurement.

Valid values: input, output (Default: input)

sweepUnits

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none (Default: dBm)

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) Testbench name.

analysisName

(Optional) The user-specified name for the analysis.

params

(Optional) The parameter conditions.

I()

Description

Returns the specified logic signal.

Syntax

```
l (name <signalName>  
    [, a2dLoThreshold <real>]  
    [, a2dHiThreshold <real>]  
    [, resultType <string> | resultFile <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, mapType <string>])
```

Arguments

name

The hierarchical design name of the object.

a2dLoThreshold

(Optional) The low (0) threshold to use if a signal is being converted from a voltage signal. If not specified, the `saA2DLoThresholdDefault` value is used.

a2dHiThreshold

(Optional) The high (a) threshold to use if a signal is being converted from a voltage signal. If not specified, the `saA2DHiThresholdDefault` value is used.

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

mapType

(Optional) The signal type (instance, net, or terminal). (Default: net)

lstb()

Description

Returns the loop stability results from an LSTB analysis.

Syntax

```
lstb (name <string>  
    [, resultType <string> | resultFile <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, traceIdx <string> | params <string>])
```

Arguments

name

The loop gain format to output to results.

The `lstb()` function can take the following format identifiers to output the loop gain:

- db
- mag
- imag
- real
- phase

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

lv()

Description

Returns PrimeSim HSPICE Output Template LV#() values.

Syntax

```
lv (instance <instance>,  
    index <integer>  
    [, resultType <string> | resultFile <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, traceIdx <string> | params <string>])
```

Arguments

instance

The name of the instance.

index

The parameter number in LV#() as defined by PrimeSim HSPICE (an integer equal or greater than 0).

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

lx()

Description

Returns PrimeSim HSPICE Output Template LX#() values.

Syntax

```
lx (instance <instance>,  
    index <integer>  
    [, resultType <string> | resultFile <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, traceIdx <string> | params <string>])
```

Arguments

`instance`

The name of the instance.

`index`

The parameter number in LX#() as defined by PrimeSim HSPICE (an integer equal or greater than 0).

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

mcMean()

Description

Returns the mean.

Syntax

```
mcMean (signal <string>
        [, param <string>]
        [, resultType <string> | resultFile <string>]
        [, resultsDir <string> | testbenchName <string>]
        [, analysisName <string>]
        [, params <string>]
        [, hierPath <string>]
        [, time <string>])
```

Arguments

signal

The signal to for which to calculate mcMean.

param

(Optional) The parameter name.

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

time

(Optional) A time value.

mcStddev()

Description

Returns the stddev.

Syntax

```
mcStddev (signal <string>  
    [, param <string>]  
    [, resultType <string> | resultFile <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, params <string>]  
    [, hierPath <string>]  
    [, time <string>])
```

Arguments

signal

The signal to for which to calculate mcStddev.

param

(Optional) The parameter name.

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

time

(Optional) A time value.

meas()

Description

Returns a specified simulator measurement from a specified result in a specified directory.

Syntax

```
meas (name <string>  
    [, resultsDir <string> | testbenchName <string>]  
    [, resultType <string> | resultFile <string>]  
    [, analysisName <string>]  
    [, traceIdx <string> | params <string>])
```

Arguments

name

The measurement name.

resultsDir

(Optional) The directory where the results reside.

testbenchName

(Optional) The testbench name.

resultType

(Optional) The result type.

resultFile

(Optional) The result file.

`analysisName`

(Optional) The user-specified name for the analysis.

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

o()

Description

Returns the result of another expression.

Syntax

```
o (name <string>
    [, resultsDir <string> | testbenchName <string>]
    [, resultType <string>]
    [, analysisName <string>]
    [, traceIdx <string> | params <string>])
```

Arguments

`name`

The measurement name.

`resultsDir`

(Optional) The results directory for the expression.

`testbenchName`

(Optional) The testbench name.

`resultType`

(Optional) The result type.

`analysisName`

(Optional) The user-specified name for the analysis.

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

onoise()

Description

Returns the equivalent output noise from a noise analysis.

Syntax

```
onoise ([resultType <string> | resultFile <string>]  
        [, resultsDir <string> | testbenchName <string>]  
        [, analysisName <string>]  
        [, traceIdx <string>] | [params <string>])
```

Arguments

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

op()

Description

Returns the DC operating point value given the instance and parameter name.

Syntax

```
op (name <instance>  
    [, param <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, resultFile <string>])
```

```
[, analysisName <string>]  
[, traceIdx <string> | params <string>]  
[, hierPath <string>]  
[, time = <string>]  
[, native <boolean>])
```

Arguments

name

The hierarchical instance name from the design. It can be picked from the design in the PrimeWave Design Environment Results Analyzer window. If name is blank, it is assumed that the `-hierPath` argument of `wf: :expr` will be used to specify the instance. When blank, the parameter can be specified via the name argument. For example, an operating point expression in the PrimeWave Design Environment Outputs pane can be `op(/I5/M7, id)`, whereas the same expression for back-annotation would be `op(id)` within the `opParamExprs` property.

param

(Optional) The parameter name.

resultsDir

(Optional) The directory where the results reside. You can specify this directory if there are previous simulation and analysis results.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

time

(Optional) The time point for transient operating point/node voltage.

native

(Optional) If true, the name argument refers directly to a device; otherwise, the name argument is treated as a sub-expression. (Default: 0)

Examples

```
op(/X0/M7, id, time=2.5n)
```

opt()

Description

Returns a transient operating point value given the instance and parameter name.

Syntax

```
opt (name <instance>  
    [, param <string>]  
    [, time <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, resultFile <string>]  
    [, analysisName <string>]  
    [, traceIdx <string> | params <string>]  
    [, hierPath <string>])
```

Arguments

name

The hierarchical instance name from the design. It can be picked from the design in the PrimeWave Design Environment Results Analyzer window. If name is blank, it is assumed that the `-hierPath` argument of `wf::expr` will be used to specify the instance. When blank, the parameter can be specified via the name argument.

param

(Optional) The parameter name.

time

(Optional) The time point for transient operating point or node voltage.

resultsDir

(Optional) The directory where the results reside. You can specify this directory if there are previous simulation and analysis results.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

poles()

Description

Returns the poles from a pz analysis.

Syntax

```
poles ([resultType <string> | resultFile <string>]  
      [, resultsDir <string> | testbenchName <string>]  
      [, analysisName <string>]  
      [, traceIdx <string> | params <string>])
```

Arguments

resultType

(Optional) The type of analysis whose data is being accessed.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The file where the results reside.

testbenchName

(Optional) The name of the testbench.

analysisName

(Optional) The user-specified name for the analysis.

`traceIdx`

(Optional) The trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions used to filter a family of signals created by iterative simulation.

Errors

Possible error messages include:

- Signal not found or variable/type undefined.

power()

Description

Returns an instance power or, if the name is not specified, total power.

Syntax

```
power ([name <instance>]  
      [, resultType <string> | resultFile <string>]  
      [, resultsDir <string> | testbenchName <string>]  
      [, analysisName <string>]  
      [, traceIdx <string> | params <string>]  
      [, hierPath <string>])
```

Arguments

`name`

(Optional) An instance name (defaults to total power).

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

`hierPath`

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

signal()

Description

Returns the specified signal from the specified file.

Syntax

```
signal (name <string>,  
        filename <filePath>  
        [, resultType <string>]  
        [, resultsDir <string> | testbenchName <string>]  
        [, analysisName <string>]  
        [, traceIdx <string> | params <string>])
```

Arguments

`name`

The name of the signal.

`filename`

The name of the waveform data file.

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

stateye()

Description

Returns a specified stateye signal from a specified result in a specified directory.

Syntax

```
stateye (port <string>,  
         type <enum>  
         [, param <string>]  
         [, resultType <string> | resultFile <string>]  
         [, resultsDir <string> | testbenchName <string>]  
         [, analysisName <string>])
```

Arguments

port

A port number.

type

(Optional) A measurement type. The parameter depends on the type argument.

Valid values: eye, eyeV, eyeT, BER, bathtubV, and bathtubT

type Argument	param
eye	No parameter needed.
eyeV	voltage
eyeT	time
BER	No parameter needed.
bathtubV	voltage
bathtubT	time

`param`

(Optional) A parameter name, which depends on the type argument (see descriptions for type above).

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`resultType`

(Optional) The result type.

`analysisName`

(Optional) The user-specified name for the analysis.

stateyeM()

Description

Returns a specified stateye signal from a specified result in a specified directory.

Syntax

```
stateyeM (port <string>,  
         type <enum>,  
         name <string>  
         [, param1 <string>]  
         [, param2 <string>]  
         [, resultType <string> | resultFile <string>]  
         [, resultsDir <string> | testbenchName <string>]  
         [, analysisName <string>])
```

Arguments

`port`

A port number.

`type`

(Optional) A measurement type. The parameters depend on the `type` argument.

Valid values: Veye, Heye, WorstBits, Eye, or BER

type Argument	param1	param2
Veye	time	tolerance
Heye	voltage	tolerance
WorstBits	time	state
Eye	voltage	time
BER	voltage	time

`param1`

(Optional) The first parameter, which depends on the type argument (see previous descriptions for `type`).

`param2`

(Optional) The second parameter, which depends on the type argument (see previous descriptions for `type`).

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`resultType`

(Optional) The result type.

`analysisName`

(Optional) The user-specified name for the analysis.

temp()

Description

Returns the value of the temperature design variable. `testbenchName` `resultsDir`

Syntax

```
temp ([resultsDir <string> | testbenchName <string>]
[, params <string>])
```

Arguments

resultsDir

(Optional) The results directory for the expression.

testbenchName

(Optional) The testbench name.

params

(Optional) The parameter conditions.

v()

Description

Returns the specified voltage signal.

Syntax

```
v (name <signalName>  
    [, resultType <string> | resultFile <string>]  
    [, resultsDir <string> | testbenchName <string>]  
    [, analysisName <string>]  
    [, mapType <string>]  
    [, traceIdx <string> | params <string>]  
    [, hierPath <string>])
```

Arguments

name

The hierarchical design name of the object.

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

`mapType`

(Optional) Signal type (instance, net, or terminal). (Default: net)

`traceIdx`

(Optional) A trace identifier for obtaining a single signal from a family.

`params`

(Optional) The parameter conditions.

`hierPath`

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

var()

Description

Returns the value of a design variable. You can specify the temperature (`.temp`) as a parameter for this function.

Syntax

```
var (name <string>
    [, resultsDir <string> | testbenchName <string>]
    [, params <string>])
```

Arguments

`name`

The name of the output expression.

`resultsDir`

(Optional) The results directory for the expression.

`testbenchName`

(Optional) The testbench name.

`params`

(Optional) The parameter conditions.

vlogA()

Description

Returns a Verilog-A internal parameter value given the instance and parameter name.

Syntax

```
vlogA (name <instance>
      [, param <string>]
      [, resultType <string>]
      [, resultsDir <string> | testbenchName <string>]
      [, resultFile <string>]
      [, analysisName <string>]
      [, traceIdx <string> | params <string>]
      [, hierPath <string>])
```

Arguments

name

The hierarchical instance name from the design.

param

(Optional) The parameter name.

resultType

(Optional) The result type.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The result file.

testbenchName

(Optional) The testbench name.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) A trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions.

hierPath

(Optional) A hierarchical path to append to the relative name value (no beginning "/").

zeros()

Description

Returns the zeros from a pz analysis.

Syntax

```
zeros ([resultType <string> | resultFile <string>]  
      [, resultsDir <string> | testbenchName <string>]  
      [, analysisName <string>]  
      [, traceIdx <string> | params <string>])
```

Arguments

resultType

(Optional) The type of analysis whose data is being accessed.

resultsDir

(Optional) The directory where the results reside.

resultFile

(Optional) The file where the results reside.

testbenchName

(Optional) The name of the testbench.

analysisName

(Optional) The user-specified name for the analysis.

traceIdx

(Optional) The trace identifier for obtaining a single signal from a family.

params

(Optional) The parameter conditions used to filter a family of signals created by iterative simulation.

Errors

Possible error messages include:

- Signal not found or variable/type undefined.

Operators, Constants, and Logic Operations

This section contains information on how to use the supported operators, constants, and logic operations.

This chapter contains the following topics:

- [Supported Operators](#)
- [Supported Constants](#)
- [Supported Logic Operations](#)

Supported Operators

The following table summarizes the supported operators.

Operator/Function	Alias	Description
+		Addition operator
-		Subtraction operator
*		Multiplication operator
/		Division operator
^	**	Power operator
?		Ternary operator
%		Integer operator

Mathematic functions can operate on scalars or waveforms. The returned value can be a scalar (or a waveform) if its input argument is a scalar (or signal).

Supported Constants

The following table summarizes the supported constants.

Constant Name	Variable Name	Value
Pi (π)	c_pi	3.14159265358979

Constant Name	Variable Name	Value
2*Pi	c_2pi	6.28318530717959
Elementary Charge (qe)	c_qe	1.60217662e-19
Euler's number (<i>e</i>)	c_e	2.71828182845905
K	c_k	1.380649e-23
sqrt(2)	c_sqrt2	1.41421356237310

Supported Logic Operations

The following table summarizes the supported logic operations.

Operator	Logic Function	Notes
&	AND	Bitwise or unary
	OR	Bitwise or unary
^	XOR	Bitwise or unary
~	NOT	Bitwise only
>>	SHIFT right	
<<	SHIFT left	
~&	NAND	Unary or binary
~	NOR	Unary or binary
~^	XNOR	Unary or binary

Note:

You can add the 'b' 'o' 'd' or 'h' prefix to a logic operator to specify binary, octal, decimal, or hexadecimal, respectively. For example:

```
6'b010=6'b000010
9'o077=9'b000111111
4'd010=4'b1010
12'h0F0=12'b000011110000
```

Mathematic Functions

This topic contains information on how to use the supported mathematic functions.

The following topics summarize the mathematic functions available in the Calculator.

Function	Description
abs	Measures the absolute value of a given signal or value.
acos	Measures the arc cosine value of a given signal or value.
acosh	Calculates the hyperbolic arch-cosine of the signal.
asin	Measures the arc sine value of a given signal or value.
asinh	Calculates the hyperbolic arch-sine of the signal.
atan	Measures the arc tangent value of a given signal or value.
atanh	Calculates the hyperbolic arch-tangent of the signal.
complex	Returns a complex scalar or waveform with x as the real part and y as the imaginary part.
concat	Concatenates up to 32 signals or scalar values into one signal. The result signal's X axis values are data point indices.
conjugate	Returns the conjugate of a complex number.
cos	Measures the cosine value of a given signal or value (in radian units).
cosh	Measures the hyperbolic cosine value of a given signal.
db10	Measures the db10 value of a given signal.
db20	Measures the db20 value of a given signal.
dbm	Measures the dbm value of a given signal or value.
exp	Measures the exponential value of a given signal or value.
floor	Calculates the largest integer not greater than the value of given signal or value.
imag	Measures the imaginary part of a value of the given signal or value.
int	Calculates the rounded integer of the value of a given signal or value.

Function	Description
ln	Measures the natural logarithm value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.
log10	Measures the base-10 logarithmic value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.
mag	Measures the magnitude value of given signal or value.
max	Compares two signals or values and calculates the larger of the two signals or values.
min	Compares two signals or values and calculates the smaller of the signals or values.
phase	Measures the phase value of given signal or value.
pwr	Calculates the power value of a given signal or value.
real	Measures the real part of a value of the given signal or value.
round	Rounds the given signal or value to an integer value and returns the nearest integer value of the float, double, or long double argument.
sgn	Calculates the sgn value of a given signal or value.
sin	Measures the sine value of the given signal or value (in radian units).
sinh	Measures the hyperbolic sine value of the given signal or value.
sqrt	Measures the square root value of the given signal or value (in radian units).
tan	Measures the tangent value of the given signal or value (in radian units).
tanh	Measures the hyperbolic tangent value of the given signal or value.

abs

Measures the absolute value of a given signal or value.

Syntax

`abs(<signal>, [<type>])`

Returns

A signal or scalar value.

Arguments

`<signal>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<type>`

(Optional) Specifies to perform the function in a discrete or continuous way. Default value is discrete.

Errors

Possible error messages include:

- Signal not found

Examples

```
abs('0|v(out)')  
abs(-5)=5
```

```
abs($sig, 'discrete')
```

will perform `abs()` in a discrete way.

```
abs($sig, 'continuous')
```

will perform `abs()` in a continuous way.

acos

Measures the arc cosine value of a given signal or value.

Syntax

```
acos(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error (>1 , <-1)

Examples

```
acos('0|v(out)')  
acos(0.6)=927m
```

acosh

Description

Calculates the hyperbolic arch-cosine of the signal. If the value is out of domain $[1, \infty)$, a warning message will appear.

Syntax

```
acosh(<signal>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured. The value should be in the range of $[1, \infty)$.

asin

Measures the arc sine value of a given signal or value.

Syntax

```
asin(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error (>1 , <-1)

Examples

```
asin('0|v(out) ')  
asin(0.5)=524m
```

asinh

Description

Calculates the hyperbolic arch-sine of the signal.

Syntax

```
asinh(<signal>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

atan

Measures the arc tangent value of a given signal or value.

Syntax

```
atan(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
atan('0|v(out)')  
atan(10)=1.47
```

atanh

Description

Calculates the hyperbolic arch- tangent of the signal. If the value is out of domain (-1,1), then a warning will be given.

Syntax

```
atanh(<signal>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured. The value should be in the range of (-1,1).

complex

Returns a complex scalar or waveform with x as the real part and y as the imaginary part. Both x and y need to be real values or signals.

Syntax

```
complex(<real signal>|<value>,<imag signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<real_signal>|<value>

The real signal or value.

<imag_signal>|<value>

The imaginary signal or value.

Errors

Possible error messages include:

- Signal not found(Real/Imag)

Examples

```
complex('0|v(out)', '0|v(out)')  
complex(2,3)=0
```

concat

Description

Concatenates up to 32 signals or scalar values into one signal. The result signal's X axis values are data point indices.

Syntax

```
concat <signal1> <signal2> [<signal3> ... <signal32>]
```

Returns

A signal.

Arguments

<signal1>

The first signal to concatenate. It can be a scalar value.

<signal2>

The second signal to concatenate. It can be a scalar value.

[<signal3> ... <signal32>]

(Optional) More signals to concatenate. They can be scalar values.

conjugate

Description

Returns the conjugate of a complex number. The conjugate of a complex number is a number with opposite sign at the imaginary part.

Syntax

```
conjugate(<signal>)
```

Returns

A complex signal.

Arguments

<signal>

The signal to be calculated. Complex signals are supported.

COS

Measures the cosine value of a given signal or value (in radian units).

Syntax

```
cos(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
cos('0|v(out) ')  
cos(60)=-952m
```

cosh

Measures the hyperbolic cosine value of a given signal. The hyperbolic cosine function uses the following equation for calculations: $\cosh(x) = \frac{e^x + e^{-x}}{2}$

Syntax

```
cosh(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
cosh('0|v(out)')  
cosh(60)=-952m
```

db10

Measures the db10 value of a given signal. The db10 function uses the following equation for calculations: $db10(x) = 10 \cdot \log(x)_{10}$

Syntax

db10(<signal>|<value>)

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error($x \leq 0$)

Examples

```
db10('0|v(out)')  
db10(10)=10
```

db20

Measures the db20 value of a given signal. The db20 function uses the following equation for calculations: $db20(x) = 20 \cdot \log(x)_{10}$

Syntax

```
db20(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error($x \leq 0$)

Examples

```
db20('0|v(out)')  
db20(10)=20
```

dbm

Measures the dbm value of a given signal or value. The dbm function uses the following equation for calculations: $dbm(x) = 30 + \log(x)_{10}$

Syntax

```
dbm(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error($x \leq 0$)

Examples

```
dbm('0|v(out)')  
dbm(10) = 40
```

exp

Measures the exponential value of a given signal or value.

Syntax

`exp(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
exp('0|v(out)')  
exp(2) = 7.39
```

floor

Calculates the largest integer not greater than the value of given signal or value.

Syntax

```
floor(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
floor('0|v(out) ')  
floor(2.5)=2
```

imag

Measures the imaginary part of a value of the given signal or value.

Syntax

```
imag(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The imaginary part of a signal or value, or 0 if the signal or value is real.

Errors

Possible error messages include:

- Signal not found

Examples

```
imag('0|v(out)')  
imag(23)=0
```

int

Calculates the rounded integer of the value of a given signal or value.

Syntax

```
int(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
int(-1.2)=-1
```

ln

Measures the natural logarithm value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.

Syntax

```
ln(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error($x < 0$)

Examples

```
ln('0|v(out) ')  
ln(2.5)=916m
```

log10

Measures the base-10 logarithmic value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.

Syntax

`log10(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error($x < 0$)

Examples

```
log10('0|v(out) ')  
log10(10)=1
```

mag

Measures the magnitude value of given signal or value.

Syntax

```
mag(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
mag('0|v(out)')  
mag(-2.5)=2.5
```

max

Compares at least two (and up to 32) signals or values and calculates the larger of the signals or values.

Syntax

```
max(<signal1>|<value1>,<signal2>|<value2>,[<signal3>|<value3> ...  
  <signal32>|<value32>])
```

Returns

A signal or scalar value.

Arguments

<signal1>|<value1>

The first signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

<signal2>|<value2>

The second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

<signal3>|<value3> ... <signal32>|<value32>

(Optional) The third through thirty-second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
max('0|v(out)', '0|v(out)')  
max(2, 3)=3
```

min

Compares at least two (and up to 32) signals or values and calculates the smaller of the signals or values.

Syntax

```
min(<signal1>|<value1>, <signal2>|<value2>, [<signal3>|<value3> ...  
  <signal32>|<value32>])
```

Returns

A signal or scalar value.

Arguments

<signal1>|<value1>

The first signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

<signal2>|<value2>

The second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

<signal3>|<value3> ... <signal32>|<value32>

(Optional) The third through thirty-second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
min('0|v(out)', '0|v(out)')  
min(1, 2) = 1
```

phase

Measures the phase value of given signal or value.

Syntax

```
phase(<signal>|<value>, [<type>], [<wrapped>])
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported.

[<type>]

(Optional) The return signal shows in degrees or radians.

[<wrapped>]

(Optional) The return signal range from $-180(-\pi\pi)$ to $180(\pi)$ or not.

Errors

Possible error messages include:

- Signal not found

Examples

```
phase('0|v(out)')  
phase(10) = 0
```

pwr

Calculates the power value of a given signal or value using the following

equation:
$$pwr(x) = \begin{cases} x^y & x \geq 0 \\ (-x)^y & x < 0 \end{cases}$$

Syntax

`pwr(<signal1>|<value1>,<signal2>|<value2>)`

Returns

A signal or scalar value.

Arguments

`<signal1>|<value1>`

The first signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<signal2>|<value2>`

The second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- (Sig1/Sig2)

Examples

```
pwr('0|v(out) ','0|v(out) ')\npwr(2,4)=16
```

real

Measures the real part of a value of the given signal or value.

Syntax

`real(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported.

Errors

Possible error messages include:

- Signal not found

Examples

```
real('0|v(out)')  
real(2)=2
```

round

Rounds the given signal or value to an integer value and returns the nearest integer value of the float, double, or long double argument passed to this function. If a decimal value is between .1 and .5, an integer value less than the argument is returned. If a decimal value is between .6 and .9, the integer value is greater than the argument.

Syntax

round(<signal>|<value>)

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
round('0|v(out)')  
round(2.6)=3
```

sgn

Calculates the sgn value of a given signal or value using the following

$$\text{equation: } \text{sgn}(x) = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Syntax

`sgn(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
sgn('0|v(out)')  
sgn(2)=1
```

sin

Measures the sine value of the given signal or value (in radian units).

Syntax

`sin(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
sin('0|v(out)')  
sin(20)=913m
```

sinh

Measures the hyperbolic sine value of the given signal or value using the following equation: $\sinh(x) = \frac{e^x - e^{-x}}{2}$

Syntax

```
sinh(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
sinh('0|v(out)')  
sinh(2)=3.63
```

sqrt

Measures the square root value of the given signal or value (in radian units).

Syntax

```
sqrt(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error($x < 0$)

Examples

```
sqrt('0|v(out)')  
sqrt(5)=2.24
```

tan

Measures the tangent value of the given signal or value (in radian units).

Syntax

tan(<signal>|<value>)

Returns

A signal or scalar value.

Arguments

<signal>|<value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
tan('0|v(out)')  
tan(30)=-6.41
```


tanh

Measures the hyperbolic tangent value of the given signal or value using the following equation: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Syntax

`tanh(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
tanh('0|v(out)')
tanh(2)=964m
```

Measurement Functions

This topic contains information on how to use the supported measurement functions.

The following sections summarize the measurement functions available in the Calculator.

Function	Description
avgdev	Calculates the average deviation of a given signal.
avgfreq	Calculates the average frequency of a given signal.
bw	Returns the bandwidth of a given signal when the signal strength drops below a bound dB from its DC value.
bw3db	Equivalent to specifying <code>bw(<signal>, 3)</code> , where the dB threshold is fixed to 3dB.

Function	Description
correlation	Calculates the correlation coefficient of the Y values of given signal versus the X values of a given signal.
cross	Calculates the correlation coefficient of the Y values of given signal versus the X values of a given signal.
dBCP	Calculates a 1 dB Compression Point (dBCP).
dcgain	Calculates the X value of a given signal at the min gain.
dcgamos	Calculates the minimum value of a signal equivalent to the min(gain1,gain2) function.
dcswing	Calculates the Y value of a given signal at the min gain.
delay	Calculates the time difference between two signals by the edge index and at the specified switching edges.
direct_ipn	Calculates hbac/pac ipn.
dutycycle	Calculates the duty cycle of a given period signal at the specified edges of edge indexes.
enb	Calculates the effective number of bits of a given signal. The FFT is calculated first, then the SNDR.
eye_mask	Adds a mask on to the existing eye object.
falltime	Calculates the time difference of a given signal between the high y_threshold crossing and the low y_threshold crossing.
falltrig	Calculates the X value of the nth falling edge over threshold Y.
gain1_f	Calculates the frequency of a given signal at the signal magnitude, which is 0db(1).
gain_m	Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency.
gbp	Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency.
get_harmonic_freq	Calculates the frequency of the nth harmonic.
globalmax	Returns the global maximum Y value of all data points for signal.
globalmean	Returns the global discrete mean value of all data points for a signal.
globalmin	Returns the global minimum Y value of all data points for signal.

Function	Description
globalstddev	Returns the global standard deviation value of all data points for signal.
imProd()	Calculates the intermodulation produce of a given signal and frequency.
integ	Calculates the integral value of a given signal from x1 to x2.
ipn	Calculates the n-order intercept point between the target signal and reference signal.
ipnhVRI	N dB intercept point VRI.
mean	Calculates the mean value of a given signal or value from x1 to x2.
measure_aperture	Applies the Aperture measurements to the eye object returned from the create_eye function.
measure_eyepopen	Measures the height and width of the eye opening.
measure_eyesize	Applies the Eye Size measurements to the eye object returned from the create_eye function.
median_val	Calculates the median value of a given signal or value.
overshoot	Calculates the overshoot value of a given signal.
p2p	Calculates peak-to-peak, the difference between the maximum and minimum Y values of the given signal.
pamn_level_at_eye_center	Measures the PAMN eye diagram level at eye center.
pamn_level_at_user_ref	Measures the PAMN eye diagram level at user reference.
pamn_open_at_eye_center	Measures the PAMN eye diagram open at eye center.
pamn_open_at_user_ref	Measures the PAMN eye diagram open at user reference.
paramvalue	Calculates the value of a P parameter.
phase_m	Calculates the phase margin.
risetime	Calculates the time difference of given signal between low threshold crossing and high threshold crossing beginning at the X value.
risetrig	Measures the X value of the nth rising edge over the y_threshold starting from the x_start value.

Function	Description
rmsval	Measures the RMS value of given signal.
root	Computes the value of x at which f(x) equals the specified threshold.
settling_t	Measures the settling time of given signal.
sfdr	Measures the spurious free dynamic range.
signalPower	Measures the Signal Power in support of spectrum measurements.
slewrates	Measures the slew rate of given signal.
slope	Measures the slope value of the given signal at x value.
snb	Measures the Noise floor/Bin in support of spectrum measurements.
snr	Measures the signal to noise-and-distortion ratio of a given signal.
snr	Measures the signal-to-noise ratio of given signal.
snr_f	Measures the signal-to-noise ratio of given signal, which is the resulting waveform of a Fast Fourier Transform (FFT) signal
snrh	Measures the Noise floor/rthZ in support of spectrum measurements.
stddev	Measures the standard deviation of the Y values for given signal and the X values are discarded.
thd	Measures the total harmonic distortion of a given signal.
variance	Measures the variance of the Y values for given signal.
x_at_ymax	Measures the x value at the minimum y value of given signal from x_start to x_stop.
x_at_ymin	Measures the x value at the minimum y value of given signal from x_start to x_stop.
xmax	Measures the maximum x value of a given signal.
xmin	Measures the minimum x value of a given signal.
yield	Measures the count of the value specified by spec_min and spec_max and provides the value for (valid_count/total_count)*100.
ymax	Measures the maximum y value of a given signal.
ymin	Measures the minimum y value of a given signal.
yvalue	Measures the y value of given signal at x.

avgdev

Calculates the average deviation of a given signal.

Syntax

```
avgdev(<signal>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

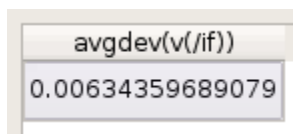
Possible error messages include:

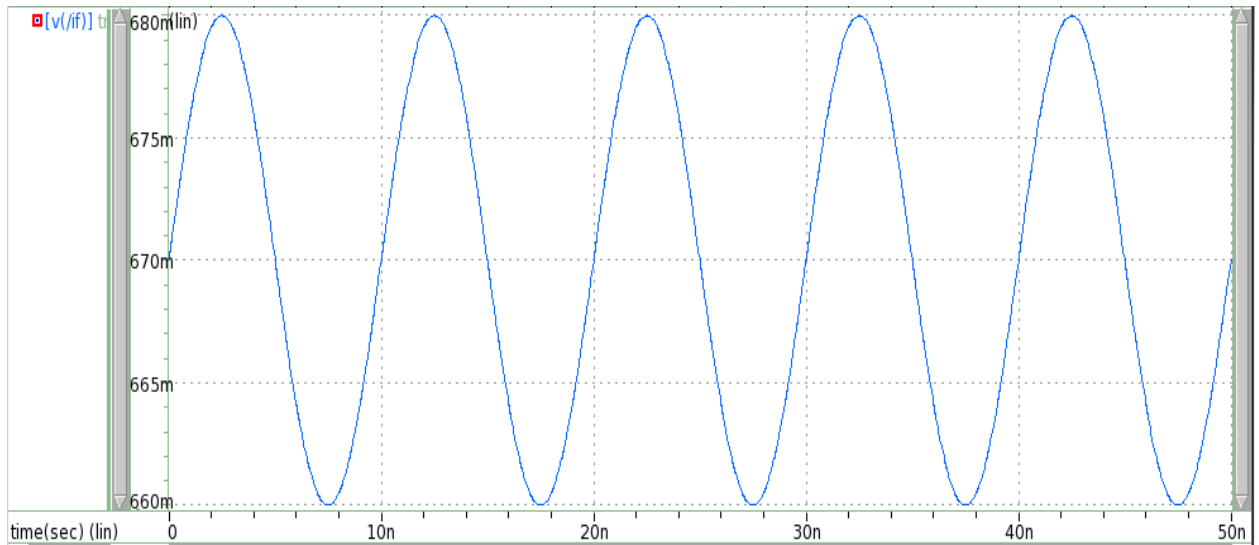
- Signal not found
- Incorrect signal type

Examples

```
avgdev(v(/if))
```

```
avgdev(1)=0
```





avgfreq

Calculates the average frequency of a given signal.

Syntax

```
avgfreq(<signal>, [<x_start>], [<x_stop>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

[<x_start>]

(Optional) The starting x value. The default is the signal starting time.

[<x_stop>]

(Optional) The stopping x value. The default is the signal ending time.

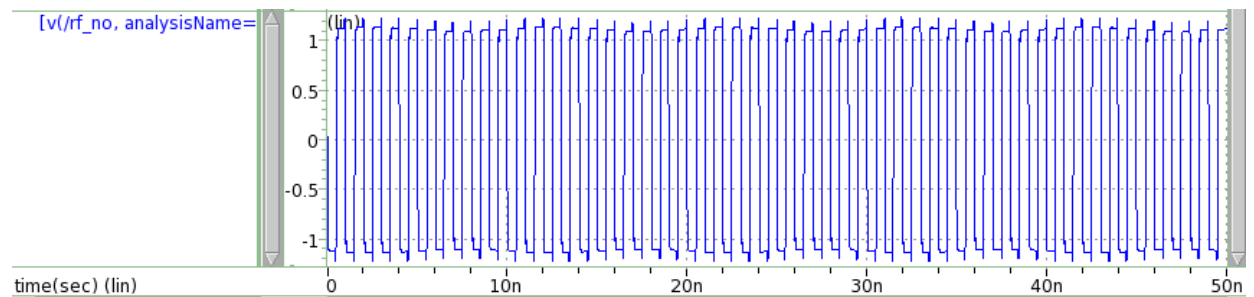
Errors

Possible error messages include:

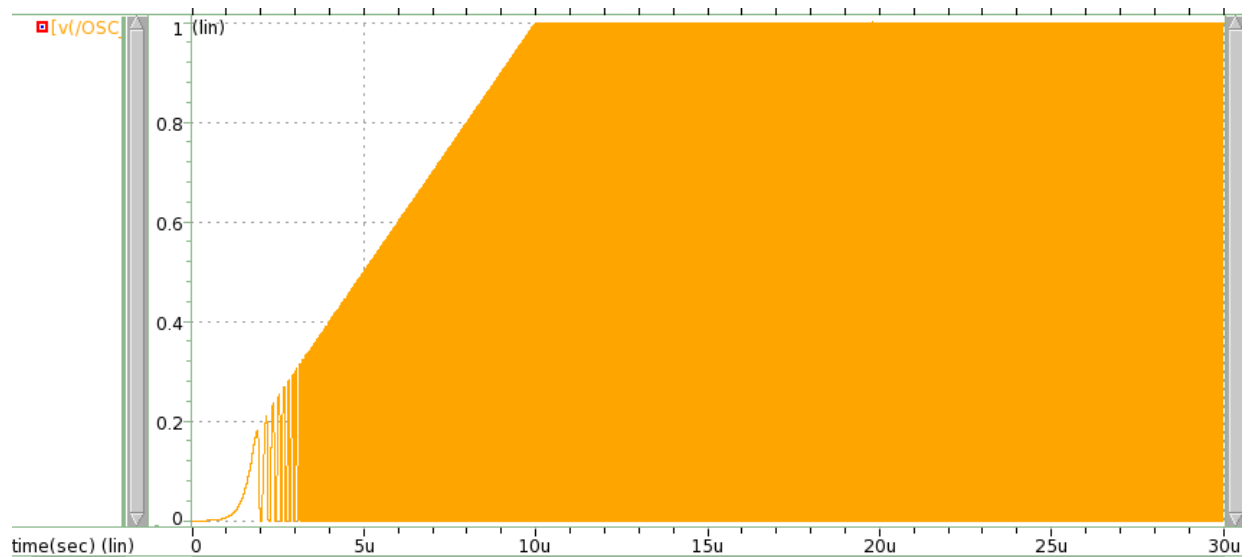
- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
avgfreq(v(/rf_no))  
avgfreq(v(/OSC_OUT), 20u, 25u)
```



```
avgfreq(v(/rf_no))  
999568219.832
```



avgfreq(v(/OSC_OUT),0,5u)	15.8986M
avgfreq(v(/OSC_OUT),5u,15u)	56.7492M
avgfreq(v(/OSC_OUT),15u,20u)	63.3351M
avgfreq(v(/OSC_OUT),20u,25u)	63.3371M
avgfreq(v(/OSC_OUT),25u,30u)	63.3368M

bw

Returns the bandwidth of a given signal when the signal strength drops below a bound dB from its DC value.

Syntax

bw(<signal>,<db_band>,[<band_type>],[<topline>],[<percent>])

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<db_band>

The dB threshold.

[<band_type>]

(Optional) The type of measurement (lowpass, highpass, bandpass, or stopband). The default is lowpass.

[<topline>]

(Optional) The top line reference (firstpoint, detect, a dB value, or a percentage value). The default is firstpoint.

[<percent>]

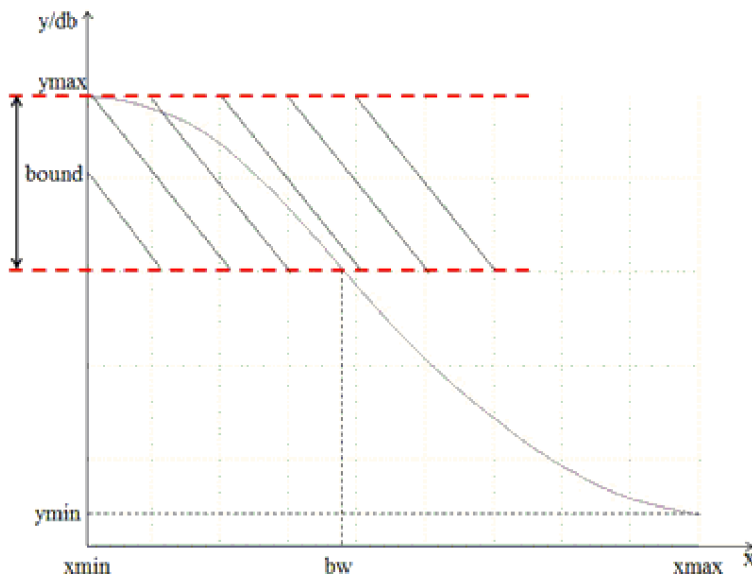
(Optional) When specified as 1, the <topline> is a percentage value.

Errors

Possible error messages include:

- Signal not found Incorrect signal type
- Db value out of range

Examples



`bw(v(/out),4,LowPass,firstpoint,1)`

will return the following:



The example inputs of the arguments shown as follows:

- Signal: `v(/out)`
- db bound: 4
- band type: LowPass
- topline: firstpoint
- Percent: 1(True)

bw3db

Equivalent to specifying `bw(<signal>, 3)`, where the dB threshold is fixed to 3dB.

Syntax

```
bw3db(<signal>, [<band_type>], [<topline>], [<percent>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

[<band_type>]

(Optional) The type of measurement (lowpass, highpass, bandpass, or stopband). The default is lowpass.

[<topline>]

(Optional) The top line reference (firstpoint, detect, a dB value, or a percentage value). The default is firstpoint.

[<percent>]

(Optional) When specified as 1, the <topline> is a percentage value.

Errors

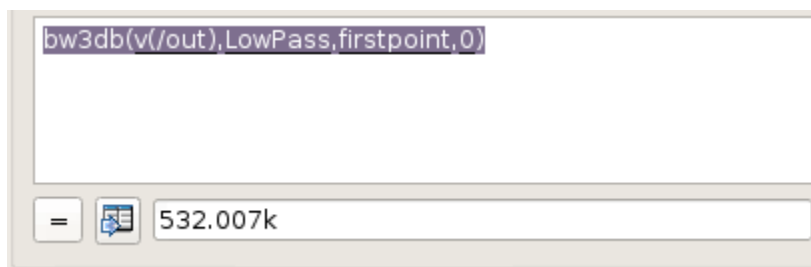
Possible error messages include:

- Signal not found.
- Incorrect signal type.
- DB value out of range.

Examples

```
bw3db(v(/out), LowPass, firstpoint, 0)
```

will return the following:



The example inputs of the arguments shown as follows:

- Signal: v(/out)
- band type: LowPass
- topline: firstpoint
- Percent: 0(False)

correlation

Calculates the correlation coefficient of the Y values of given signal versus the X values of a given signal. The following functions represent the correlation coefficient:

$$\text{corr}(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} \quad \text{cov}(X, Y) = \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y}) \quad \sigma_X, \sigma_Y \text{ are the standard deviations of X and Y, where: } \sigma_X = \sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \quad \sigma_Y = \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

Syntax

`correlation(<signal>)`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found.

Examples

`correlation('v(/out)')`

will return the following:



where v(/out) is the input net.

cross

Calculate the x-value of a given signal at y value crossing. Supports both positive and negative count indexes. If positive, it will find the cross from the beginning to the end. If negative, it will find the cross from the end to beginning.

Syntax

```
cross(<signal>,<y_level>,<edge_index>,[<x_start>],  
[<percent>],[<trigger_type>],[<occurrences>],  
[<output_xAxisType>],[<tolerance>])
```

Returns

An analog scalar value. The `cross( )` function returns the x value of the nth crossing edge over threshold `y_level` beginning at `x_start`.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

<y_level>

The threshold of the Y value.

<edge_index>

An integer value, which can be 0, 1, ..., n.

[<x_start>]

(Optional) The starting point of an X value of a signal.

[<percent>]

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is not be percentage value.

[<trigger_type>]

(Optional) The trigger type (*fall*, *rise*, or *either*). The *fall* argument specifies the wave crossing from above the y_level to below the y_threshold.

[<occurrences>]

(Optional) Specifies to retrieve only one occurrence of an overshoot event for the given waveform (*single*), or all occurrences of overshoot for the given waveform which you can later plot or print (*multiple*).

[<output_xAxisType>]

(Optional) Specifies the X axis type of the result waveform. The value can be *time* or *idx*.

[<tolerance>]

(Optional) Determines the threshold crossings and filters crossings with amplitudes smaller than tolerance.

Errors

Possible error messages include:

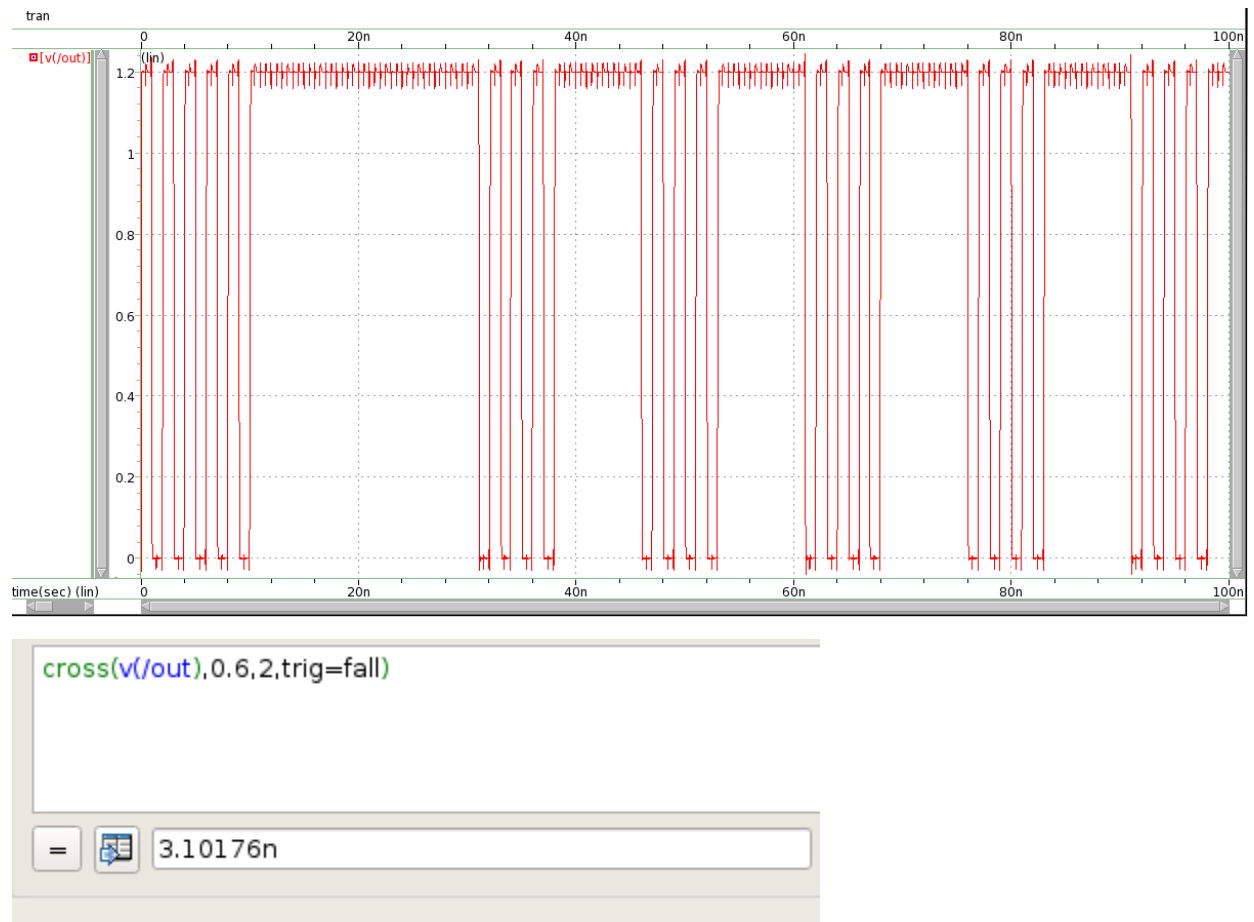
- Signal not found. Y value out of range.

Examples

Example 1:

```
cross('v(/out)',0.6,2,trig='fall')
```

will return the following:



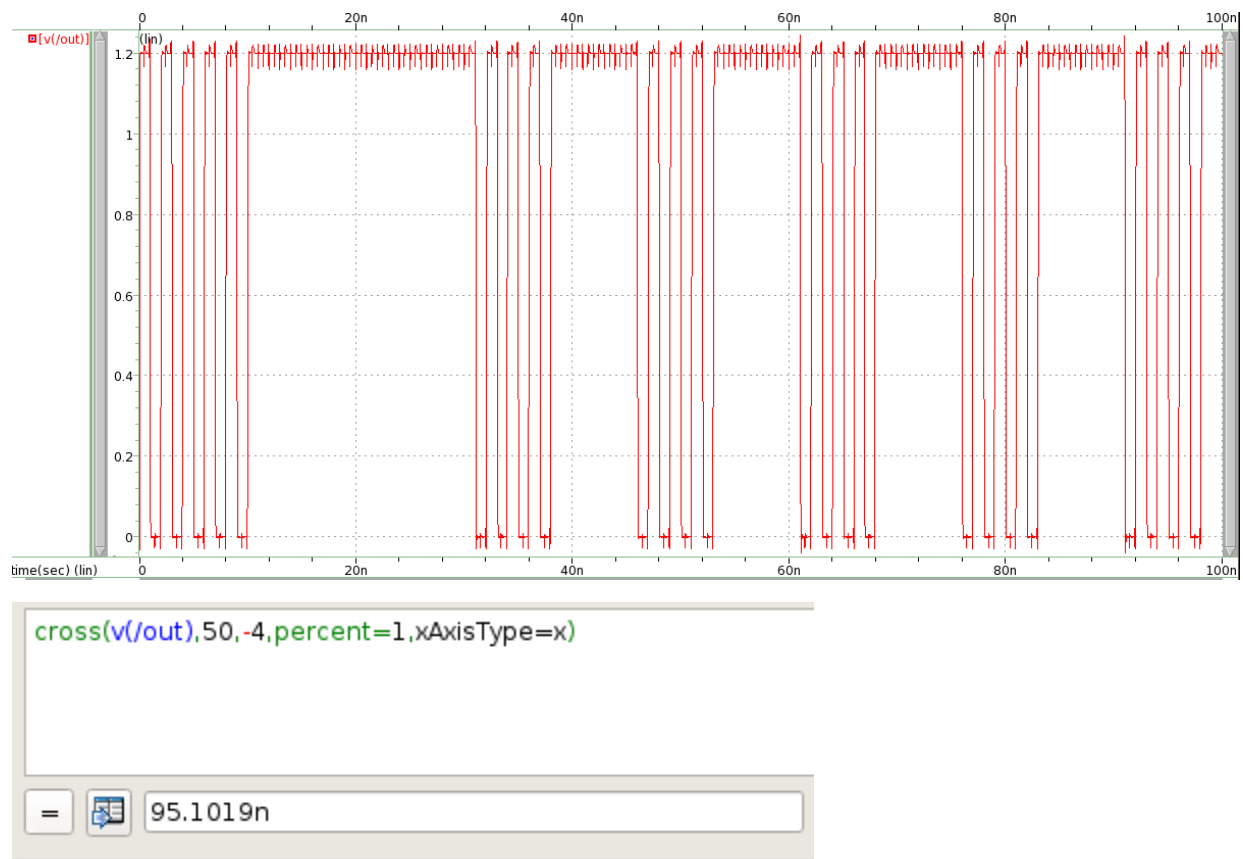
The example inputs of the arguments are as follows:

- Signal: v(/out)
- Y level: 0.6
- Edge index: 2
- Trig=fall

Example 2:

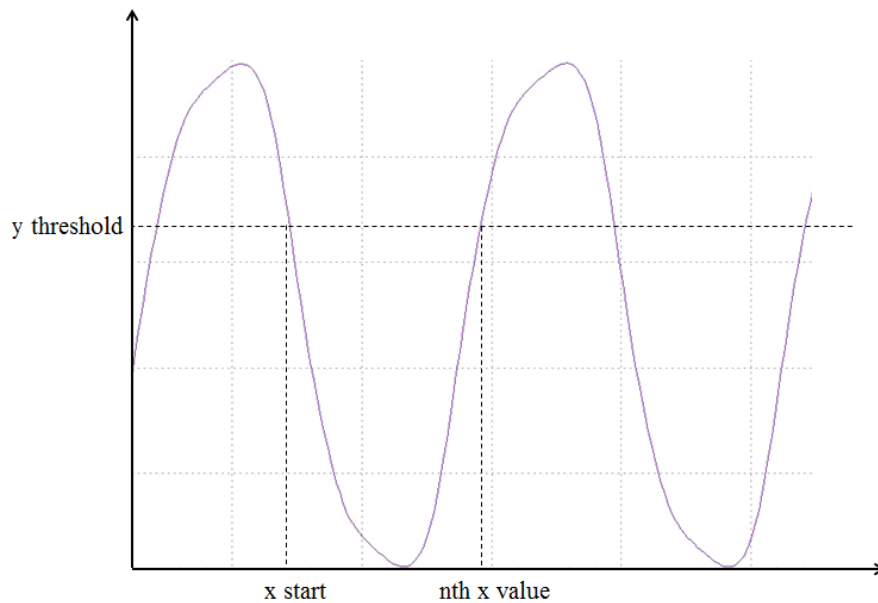
```
cross(v(/out),50,-4,percent=1,xAxisType=x)
```

will return the following:



The example inputs of the arguments are as follows:

- Signal: v(/out)
- Y level as percentage: 50
- Edge index: -4, cross will be calculated at 4th edge from the end
- xAxisType: x, this will return the x-axis value at the crossing



Both positive and negative count indexes are supported. If the count index is positive, the cross is found from the beginning to the end. If the count index is negative, the cross is found from the end to the beginning.

In the following example, the x-value of the second edge from the beginning of the waveform when it crosses the 0.5 level is reported.

```
cross(v(/vin),0.5,2,trig=rise)
```

In the following example, the x-value of the second edge from the end of the waveform (or the second-last edge) when it crosses the 0.5 level is reported.

```
cross(v(/vin),0.5,-2,trig=rise)
```

dB CP

Description

Calculates a 1 dB Compression Point (dB CP). When the amplitude of the input signal is small enough, which is almost the same as (linear) AC analysis. When the input signal grows larger, the circuit response is saturated and the output amplitude does not linearly increase.

You can calculate the 1dB compression point from HB SWEEP results. Sweep the input amplitude and measure the output amplitude. The algorithm includes:

- Calculating the value of gain by using extrapolation point and the waveform $y=f(x)$.
- Generating a linear function with $\text{slope}=1$: $y=x+\text{gain}$ (where gain is known).
- Finding the point on the waveform that satisfies the following parameters: the difference between the Y value of the point (on the waveform) and the Y value of the point (on the linear function) equals N. N is gain decrease (for example, for P1dB, N=1; and for P2dB, N=2).

Syntax

```
dBCP(<signal>,<gain_decrease>,<extrapolation_point>,  
[<input|output>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<gain_decrease>`

The difference of the Y value between a point on waveform and a point on the $y=x+\text{gain}$ linear function.

`<extrapolation_point>`

The X value that needs to be input. According to the X value and the $y=f(x)$ input signal, the system gets the Y value and then generates a $y=x+\text{gain} \rightarrow \text{gain}=y-x$ linear function.

`[<input|output>]`

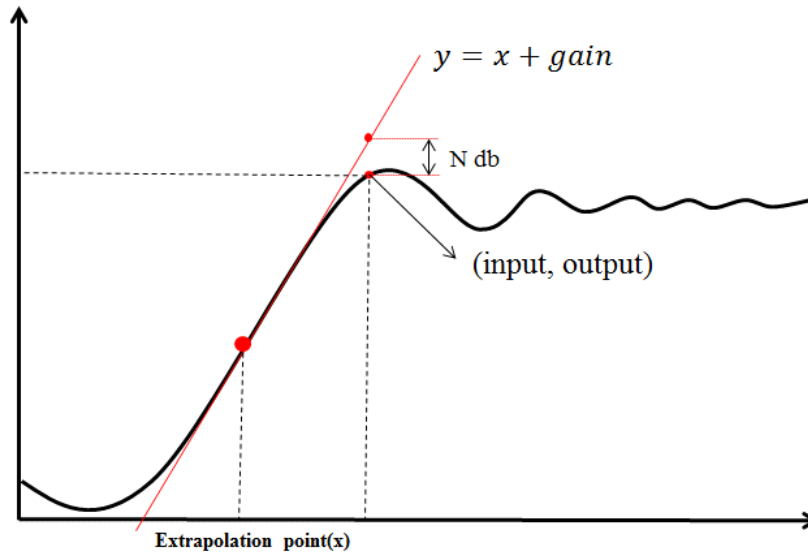
(Optional) Option to choose whether the results are input-based or output-based.

Errors

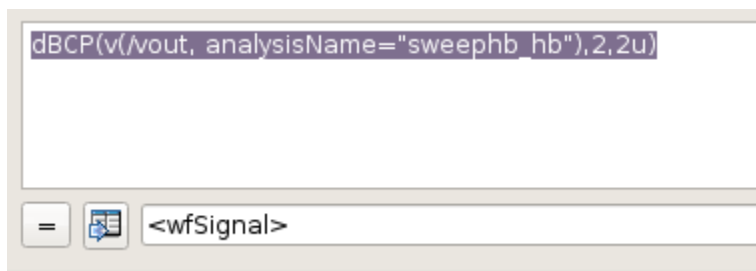
Possible error messages include:

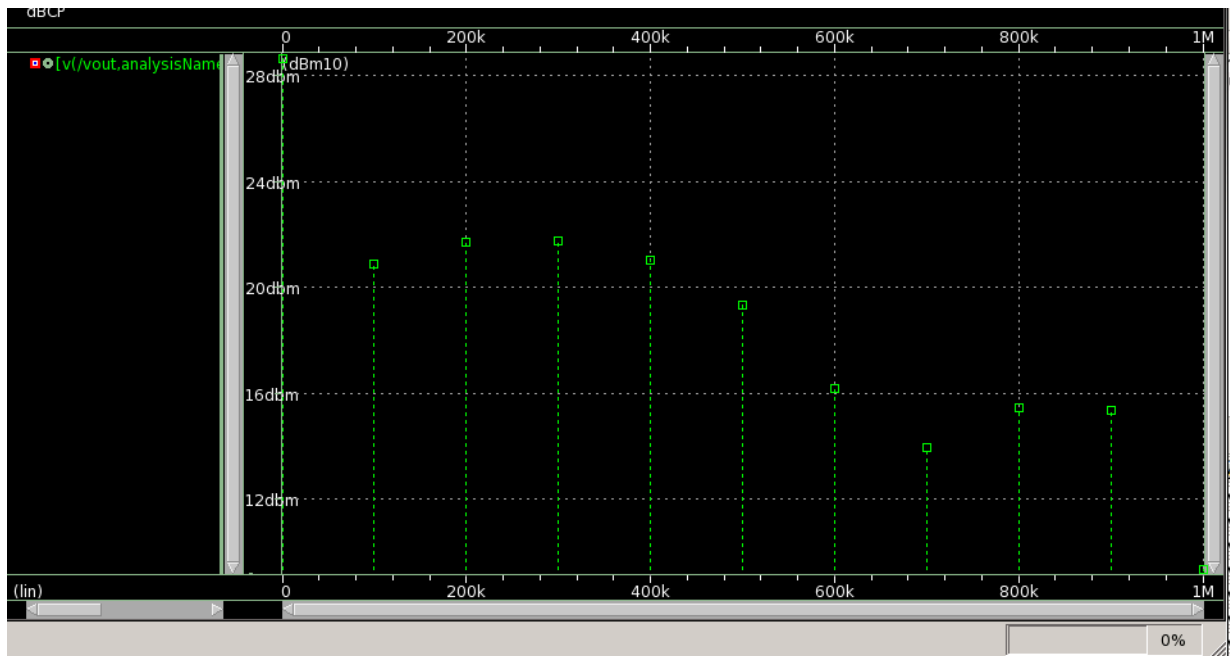
- Signal not found. Extrapolation point X value out of range.

Examples



`dBBCP(v(/vout, analysisName="sweep_hb"), 2, 2u)` will return the following:





The example inputs of the arguments are as follows:

- Signal: v(/out)
- Gain Decrease: 2
- Extrapolation point: 2u

dcgain

Calculates the dcgain value of a given signal at the min gain. The min gain is calculated using the $d(out)/d(in)$ equation, where $d(out)$ is the difference in Y value ($y_2 - y_1$) and $d(in)$ is the difference in X value ($x_2 - x_1$). When this function is executed, a plot of $d(out)/d(in)$ vs $d(in)$ is displayed, which gets the values of $d(in)$ that correspond to each min gain.

Syntax

`dcgain(<signal>, <min_gain>)`

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

<min_gain>

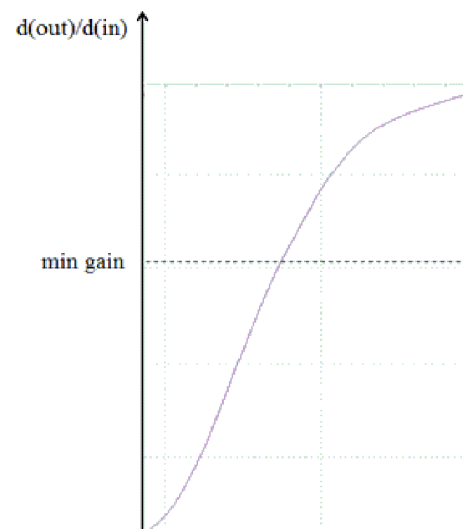
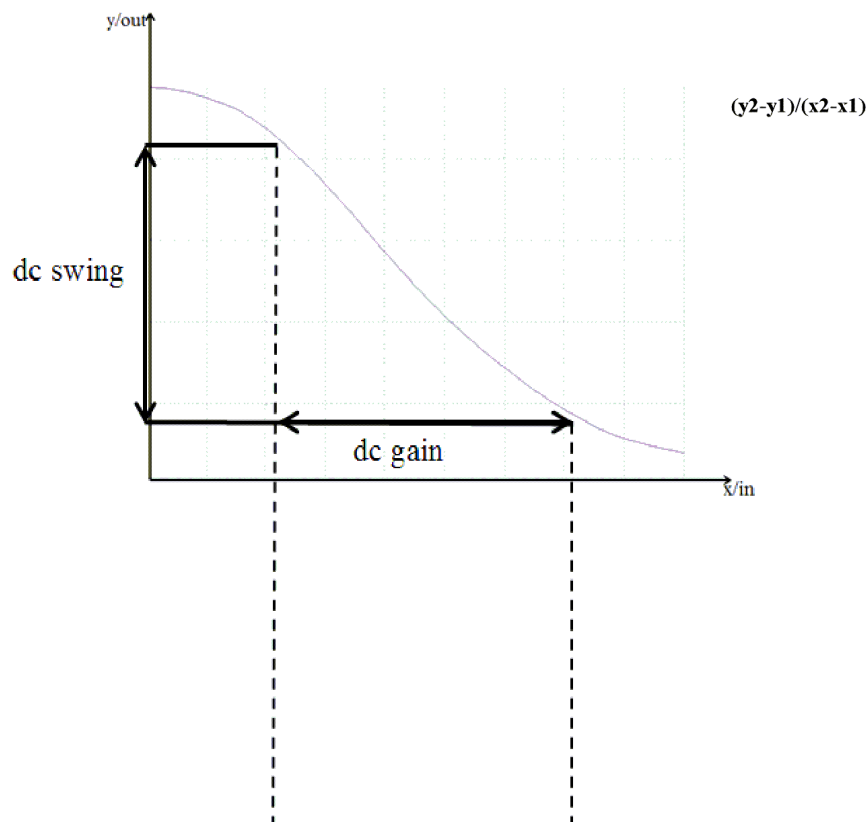
The value of $d(out)/d(in)$.

Errors

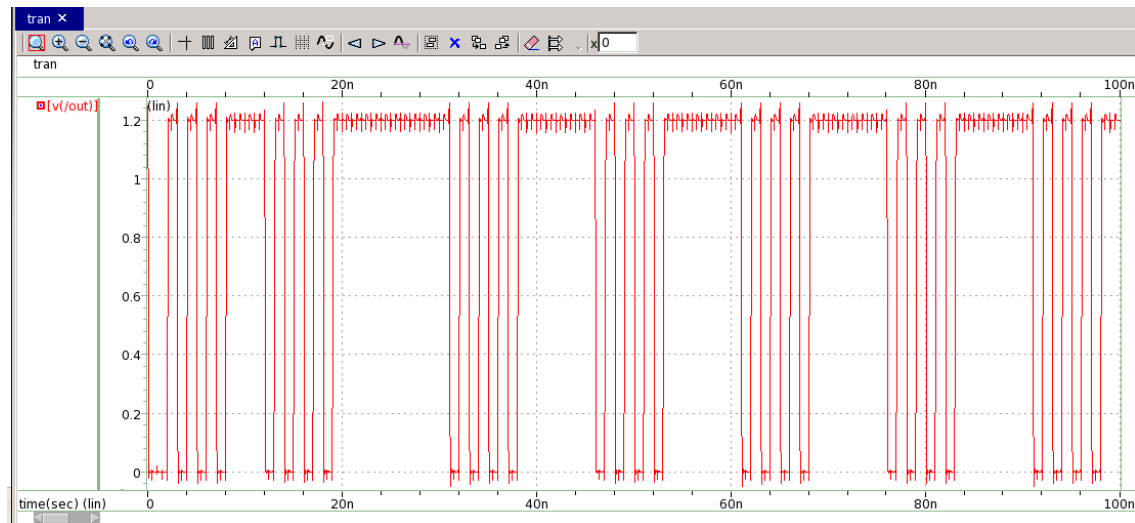
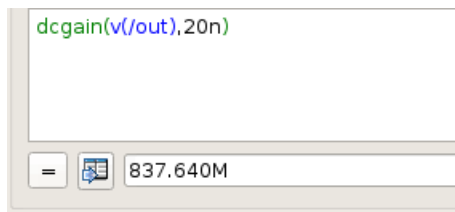
Possible error messages include:

- Signal not found
- DC gain curve does not cross minimum gain level
- Ill-conditioned DC gain curve

Examples



`dcgain(v(/out),20n)` will return the following:



dcgamos

Calculates the minimum value of a signal equivalent to the `min(gain1,gain2)` function. `gain1` is the value of `d(out)/d(in)` for the lower X value, `gain2` is the value of `d(out)/d(in)` for the higher X value—the DC gain of given signal at maximum output swing is returned.

Syntax

`dcgamos(<signal>,<output_swing>,[<percent>])`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<output_swing>`

The Y value. Two corresponding X values exist for each output swing.

`[<percent>]`

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is not be percentage value.

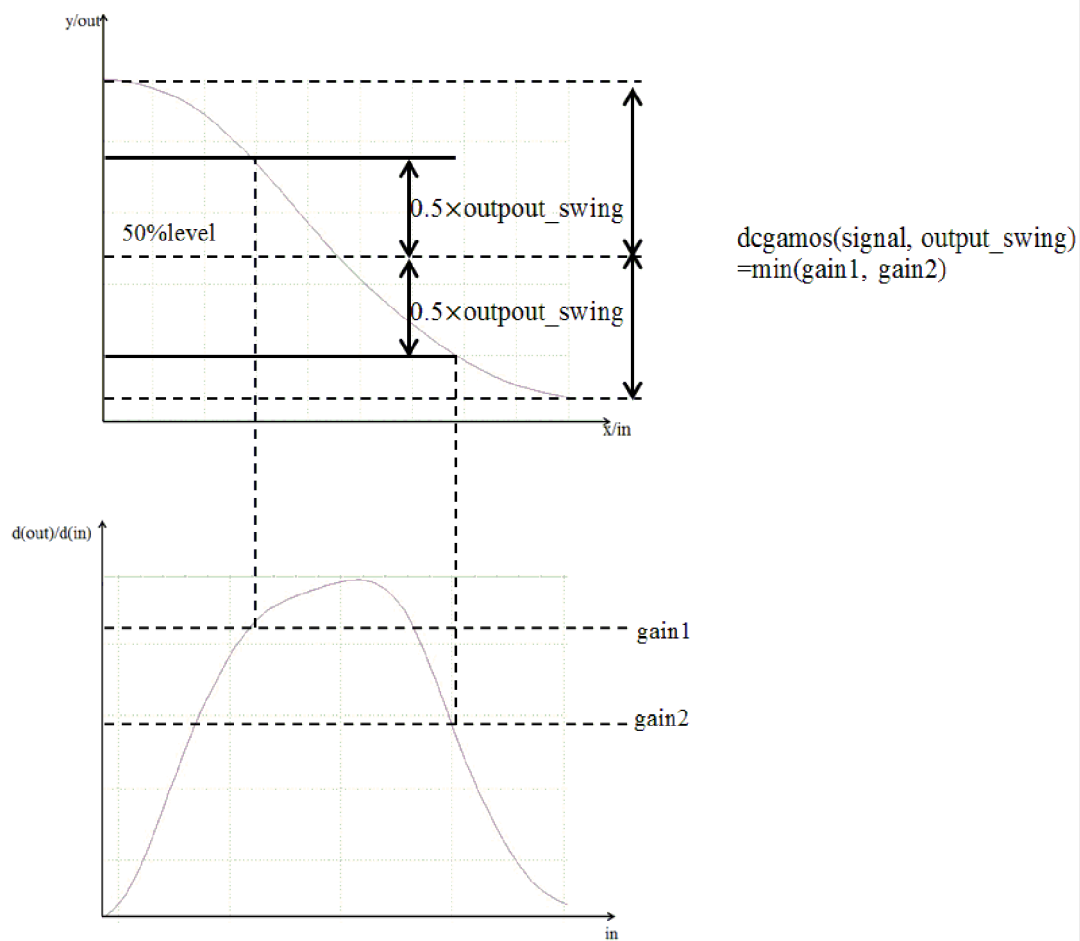
Errors

Possible error messages include:

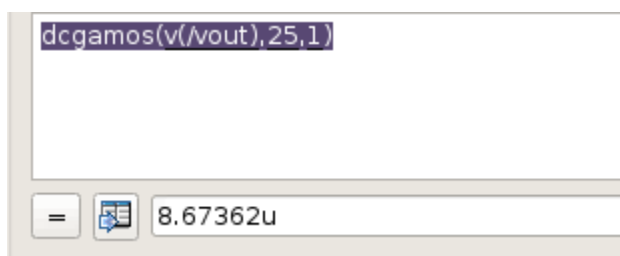
- Signal not found
- Ill-conditioned DC gain curve

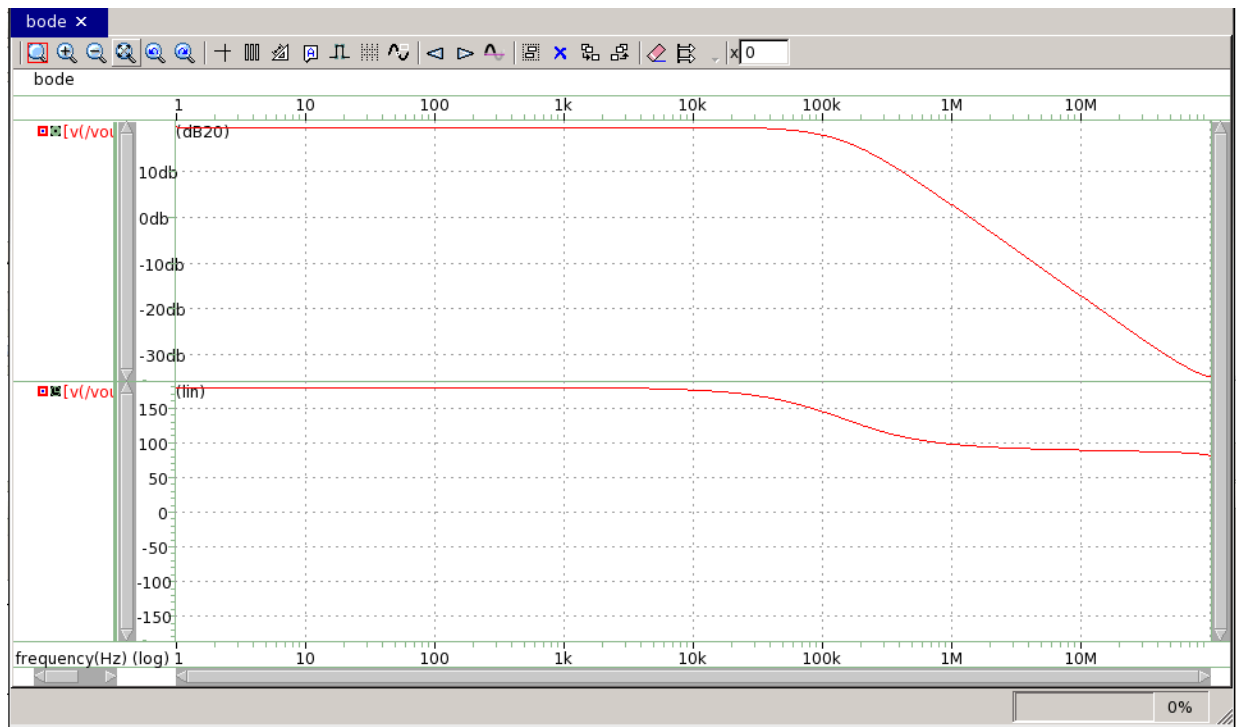
Examples

```
dcgamos(v(/vout),25,1)
```



$dcgamos(v(/vout), 25, 1)$ will return the following:





dcswing

Calculates the Y value of a given signal at the min gain. The min gain is calculated using the $d(out)/d(in)$ equation, where $d(out)$ is the Y value and $d(in)$ is the X value. When this function is executed, a plot of $d(out)/d(in)$ versus $d(in)$ is displayed, which gets the values of $d(in)$ that correspond to each min gain. The Y value is returned according to the value of $d(in)$.

Syntax

`dcswing(<signal>,<min_gain>)`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<min_gain>`

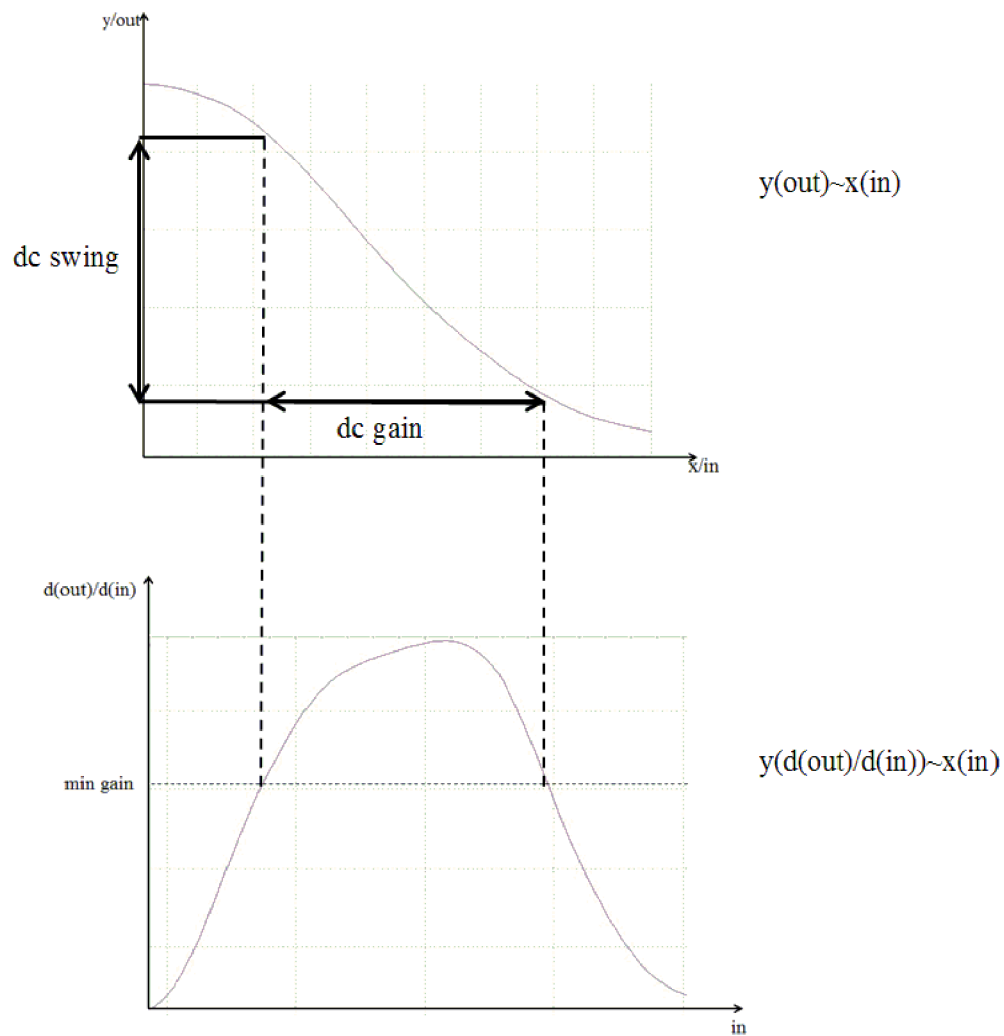
The value of $d(out)/d(in)$.

Errors

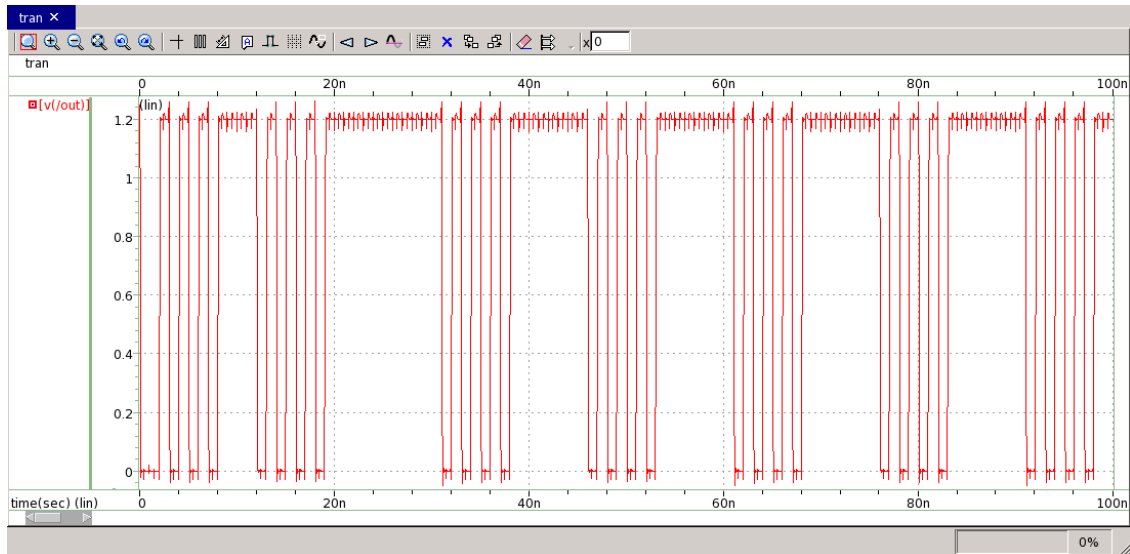
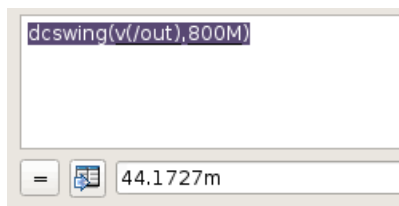
Possible error messages include:

- Signal not found
- DC gain curve does not cross minimum gain level
- Ill-conditioned DC gain curve

Examples



`dcswing(v(/out),800M)` will return the following:



delay

Calculates the time difference between two signals by the edge index and at the specified switching edges.

Syntax

```
delay(<trigger_signal>,<target_signal>,<trigger_y_level>,<target_y_level>,<trigger_x_index>,<target_x_index>,<trigger_x_start>,<percent>,<target_x_start>,<target_start_relative_to>,<x_stop>,<trigger_edge_type>,<target_edge_type>,<trigger_periodicity>,<target_periodicity>,<trigger_tolerance>,<target_tolerance>,<occurrences>,<output_xaxistype>)
```

Returns

An analog scalar value.

Arguments

`<trigger_signal>`

The reference signal.

`<target_signal>`

The signal to be calculated. Analog signals are supported.

`<trigger_y_level>`

The y_threshold for the trigger signal.

`<target_y_level>`

The y_threshold for the target signal.

`<trigger_x_index>`

The index value for the trigger signal.

`<target_x_index>`

The index value for the target signal.

[`<trigger_x_start>`]

The time value on the X-axis (the start time for the trigger signal).

[`<percent>`]

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

[`<target_x_start>`]

The start time for the target signal.

[`<target_start_relative_to>`]

Specifies whether the start time for target signal is relative to trigger signal start time or time.

[`<x_stop>`]

The stop time for target signal.

[`<trigger_edge_type>`]

Specifies the edge type of the crossing for trigger signal. There are three options: rising, falling, or either.

[`<target_edge_type>`]

Specifies the edge type of the crossing for target signal. There are three options: rising, falling, or either.

[<trigger_periodicity>]

Periodic interval for the trigger signal. The delay algorithm for multiple occurrences returns the difference between the X points for the specified edges on the target and trigger waveforms (with the specified periodicity).

For example, if the edge number specified for both the waveforms is 1 and periodicity is 2, then it will compute the difference between the 1,3,5,7 ... edges for both the waveforms.

[<target_periodicity>]

Periodic interval for the target signal.

[<trigger_tolerance>]

Tolerance value to detect the threshold crossings for trigger signal.

[<target_tolerance>]

Tolerance value to detect the threshold crossings for target signal.

[<occurrences>]

Specifies whether you want to retrieve only one occurrence of a delay time event (*single*), or all occurrences of delay time events (*multiple*). The default value is *single*.

[<output_xaxistype>]

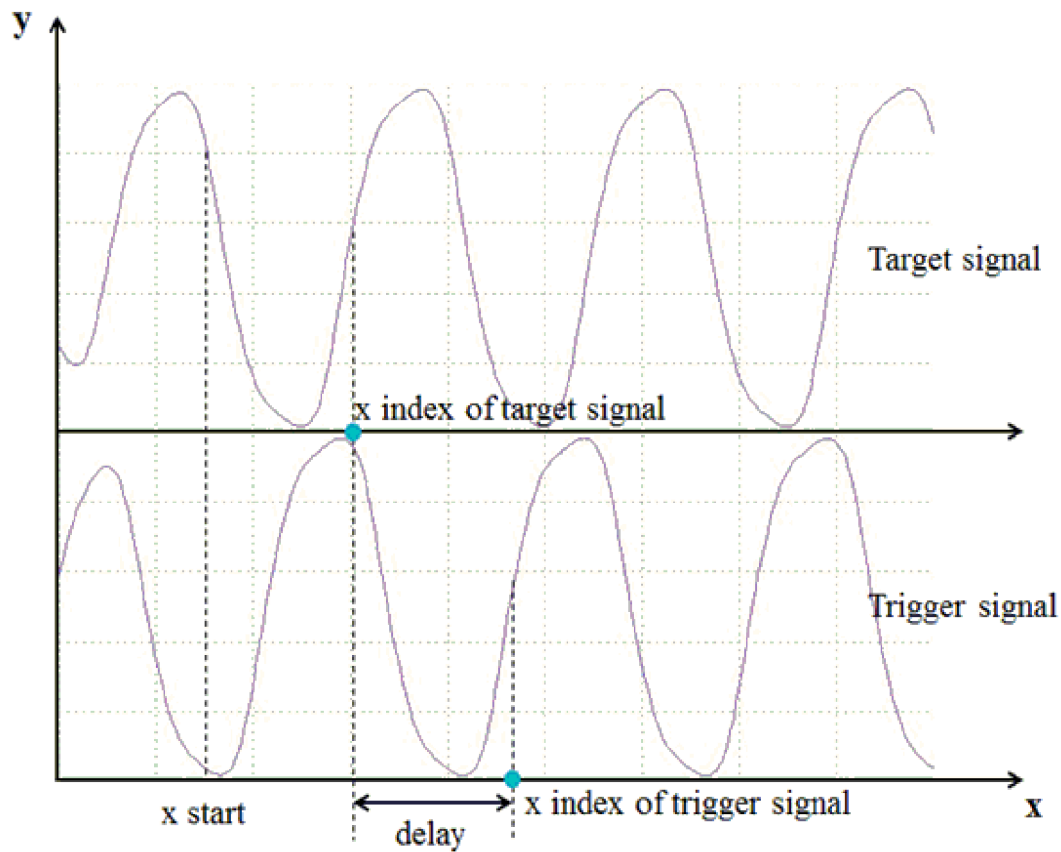
Specifies whether you want to output delay time data against trigger time, target time or cycle. Cycle numbers refer to the nth occurrence of the delay time. This option is valid only when [<occurrences>] is set to *multiple*.

Errors

Possible error messages include:

- Signal not found
- (Target/Trig)Y value out of range

Examples

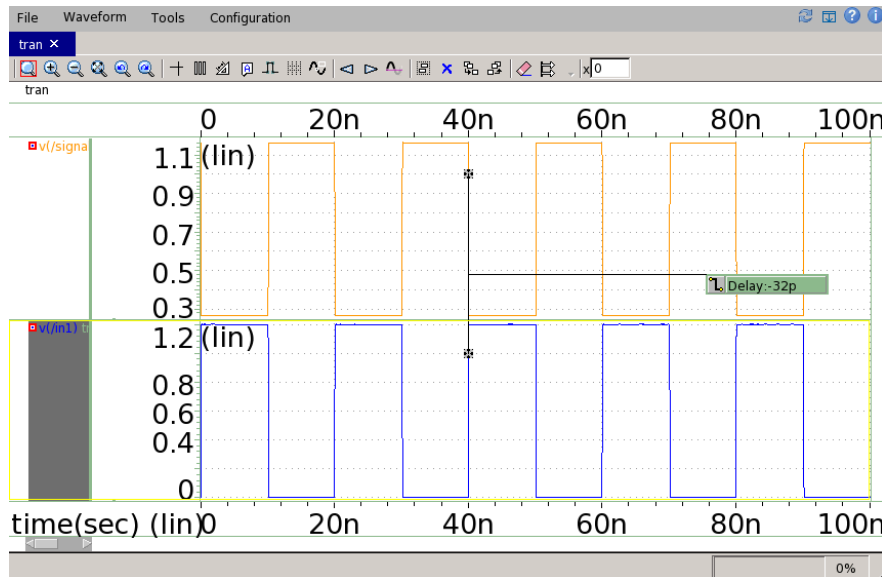


The following example shows how the function works:

```
delay(v(/in1),v(/signal1),
1,1,2,2,10n,x2=10n,xstop=80n,trig=rise,target=opposite)
```

```
delay(v(/in1),v(/signal1),1,1,2,2,10n,x2=10n,xstop=80n,trig=rise,target=opposite)]
```

= -31.9883p Add To History Add To On



The arguments are as follows:

- Trigger Signal - `v(/in1)`
- Target Signal - `v(/signal1)`
- Trigger Y level- Y level threshold of trigger signal, equal to 1 in this case.
- Target Y level- Y level threshold of target signal, equal to 1 in this case.
- Trigger X index- The edge index of trigger signal. In this case, delay is calculated on the second index of trigger signal.
- Target X index- The edge index of target signal. In this case, delay is calculated between the second index of target signal and second index of trigger signal.
- Trigger X start- The start x-value of trigger signal.
- Target X start- The start x-value of target signal.
- X stop - X-value stop times for both trigger and target signal.
- Trigger edge type- The edge type of trigger signal. It is `rise` in this case.
- Target edge type- The edge type of target signal. In this case, it is the opposite of trigger signal edge type, and hence it is `false`.

direct_ipn

Description

Calculates hbac/pac ipn.

Syntax

```
direct_ipn(file, [resistance], [input/output])
```

Arguments

file

The waveform file.

resistance

(Optional) The resistance.

input/output

(Optional) Input/output. (Default is input.)

dutycycle

Calculates the duty cycle of a given period signal at the specified edges of edge indexes. This function uses the following algorithm, where T is the time the signal is active, and P is the total period of the signal: $D = \frac{T}{P} \cdot 100\text{percent}$

Syntax

```
dutycycle(<signal>, <y_value>, <rise_edge>, [<nth_index>],  
[<percent>], <vs_time>], [<output_xAxisType>], [<time_value>], [<average>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

<y_value>

The reference signal.

<rise_edge>

The y_threshold for the trigger signal.

[<nth_index>]

(Optional) The index value, which can be 0, 1, 2, . . . , or n for a period signal.

[<percent>]

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

[<vs_time>]

(Optional) Specifies the output to be all of duty cycles.

[<output_xAxisType>]

(Optional) Specifies to retrieve value data against time or cycle. This feature depends on vs_time. Default is time.

[<time_value>]

(Optional) Specifies the time value, which is the cycle begin value or cycle end value. This feature depends on vs_time. Default is cycle_end.

[<average>]

(Optional) Specifies the output value to be the average of duty cycle.

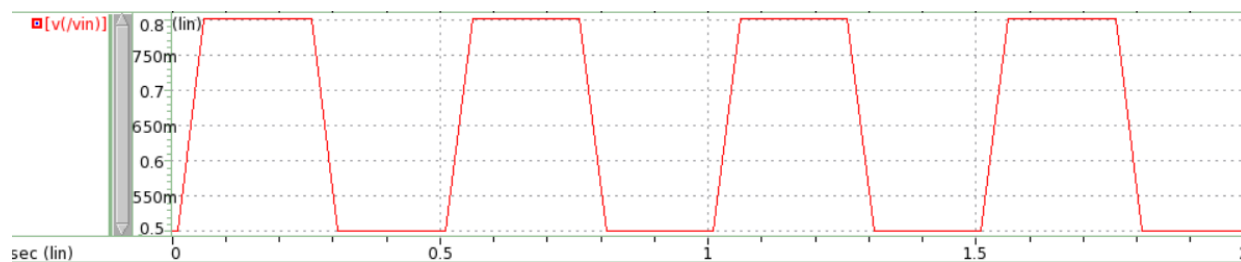
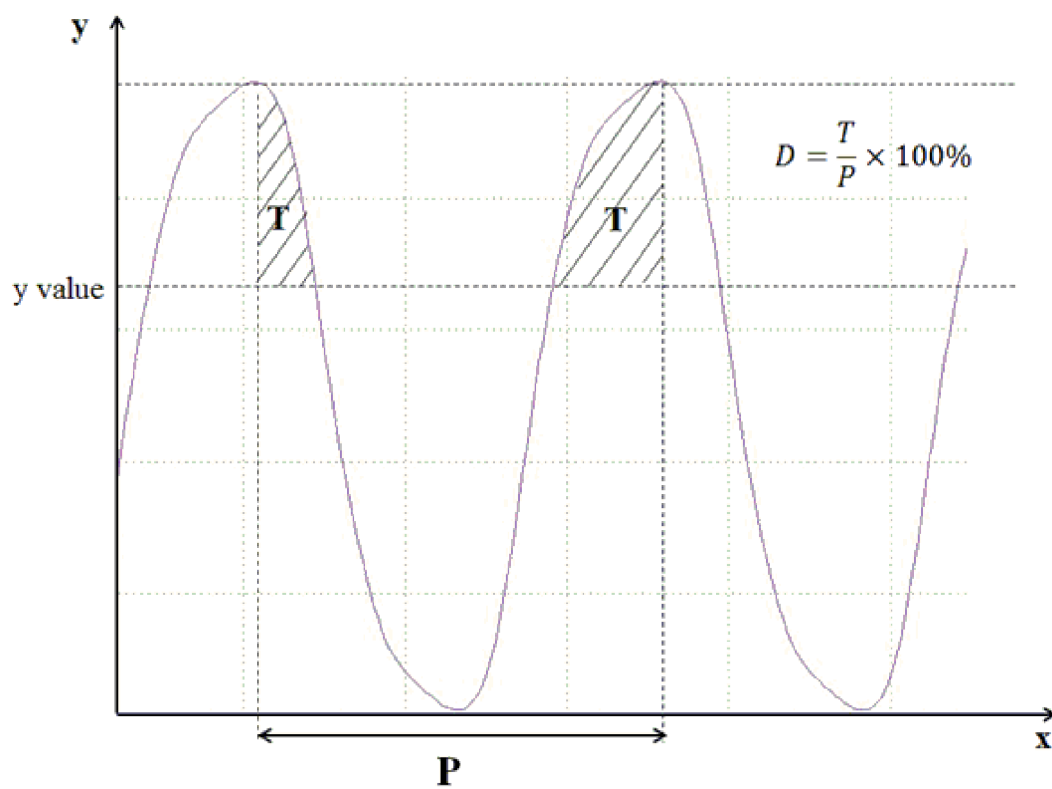
Errors

Possible error messages include:

- Signal not found
- Cannot find a complete pulse cycle
- Pulse period too small

Examples

```
dutycycle(v(/vin))  
dutycycle(v(/vin),0.7)  
dutycycle(v(/vin),75,percent=1)  
dutycycle(v(/vin),vst=false,csample=cycle_end)
```

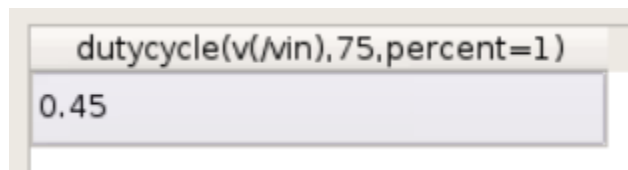



`dutycycle(v(/vin))`

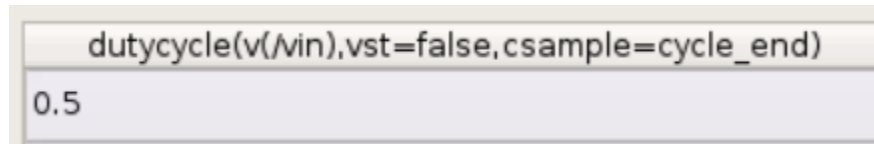
`dutycycle(v(/vin),0.7)`

`dutycycle(v(/vin),0.7)`
0.466666671965

```
dutycycle(v(/vin),75,percent=1)
```



```
dutycycle(v(/vin),vst=false,csample=cycle_end)
```



enb

Calculates the effective number of bits of a given signal. The FFT is calculated first, then the SNDR. The formula is $ENB = (SNDR - 1.76) / 6.02$.

Syntax

```
enb(<signal>,<x_start>,<x_stop>,<num_of_points>,  
    <signal_freq>,[<band_limit>],[<windowing_type>],[<decimation_rate>]),  
    [<bin_size>],[<window_alpha>],[<normalization>],[<user_defined_value>],[  
    <suppress_dc>],[<dual_sideband>],[<max_harmonic_index>],[include_floor_no  
ise],[<full_scale>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated.

<x_start>

The starting point of X in the given signal.

<x_stop>

The stopping point of X in the given signal.

<num_of_points>

The number of points needing to be measured.

<signal_freq>

The frequency of an input signal.

[<band_limit>]

(Optional) Sets the band limit frequency of the signal.

[<windowing_type>]

Specifies the windowing function. The default is `rectangle`.

[<decimation_rate>]

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Normally, this argument is an integer or a rational fraction greater than one dividing the sampling rate.

[<bin_size>]

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

[<window_alpha>]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in calculation.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.
And the formula is changed to $ENB = (SNR - 1.76) / 6.02$.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
enb('0|v(out)',0,8n,1024,100M,deci=20)
```

eye_mask

Adds a mask on to the existing eye object and returns 0 (fail) if the mask touches the eye; otherwise, this function returns 1 (pass).

Syntax

```
eye_mask(<eye_obj>,<width>,<height>,<x1>,  
[<absolute>],[<lower_bound>],[<upper_bound>],[<center>])
```

Returns

An analog scalar value.

Arguments

<eye_object>

The target eye object.

<width>

The width of the eye mask.

<height>

The height of the eye mask.

<x1>

The X value.

[<absolute>]

(Optional) Sets absolute size values.

[<lower_bound>]

(Optional) Specifies the vertical lower boundary of the eye mask.

[<upper_bound>]

(Optional) Specifies the vertical upper boundary of the eye mask.

[<center>]

(Optional) Centers the mask to the eye opening.

Errors

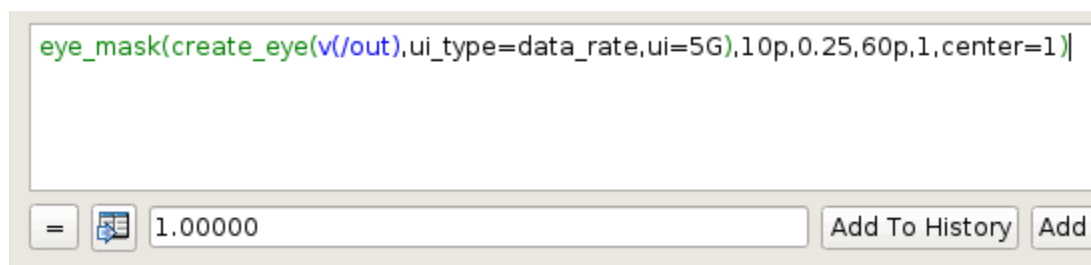
Possible error messages include:

- Width is not specified
- Height is not specified
- x1 is not specified
- Signal must be eye object
- No valid eye object

Examples

The following example shows how the function works:

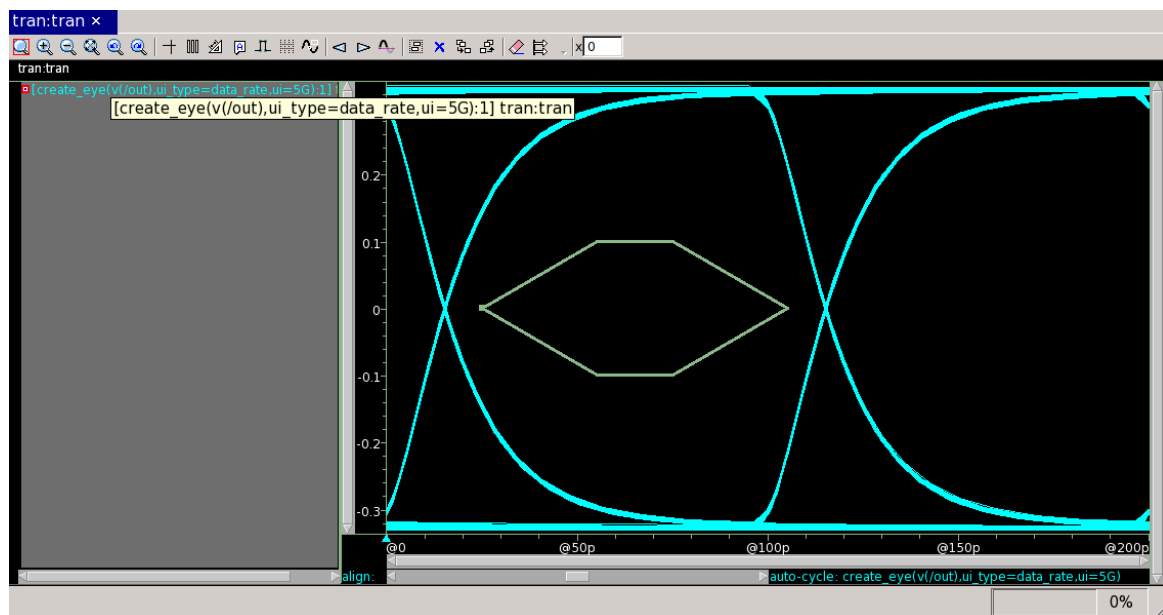
```
eye_mask(create_eye(v(/out),ui_type=data_rate,ui=5G),  
10p,0.25,60p,1,center=1)
```



Where,

- `create_eye(v(/out),ui_type=data_rate,ui=5G)` : Eye object in which eye mask is added.
- Width: 10p
- Height: 0.25
- x1: 60p
- Absolute Value: 1(True)
- Center the mask to eye opening: 1

This returns 1 if the mask touches eye. If you try to add the same mask through WaveView, you get the following:



falltime

Calculates the fall time measured between theta1 (percent_high) to theta2 (percent_low) of the difference between the initial value and the final value.

Syntax

```
falltime(<signal>,<high_y_level>,<low_y_level>,  
[<x_start>],[<percent>],[<edge_index>],[<min>],[<max>],[<min_value_type  
>],[<max_value_type>],[<occurrences>],[<output_xAxisType>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated.

`<high_y_level>`

Initial value at which the computation is to be started.

`<low_y_level>`

Final value at which the computation is to be stopped.

`[<x_start>]`

(Optional) The starting point of the X time value.

`[<percent>]`

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

`[<edge_index>]`

(Optional) The index of the edge. The default value is 1.

`[<min>]`

(Optional) Min value for percentage y init/final. Default is xmin or ymin depending on type.

`[<max>]`

(Optional) Max value for percentage y init/final, default is xmax or ymax depending on type.

`[<min_value_type>]`

(Optional) Specifies whether the [`<min>`] value is an X-axis (x) or Y-axis value (y). The default value is y.

`[<max_value_type>]`

(Optional) Specifies whether the [`<max>`] value is an X-axis (x) or Y-axis value (y). The default value is y.

`[<occurrences>]`

(Optional) Specifies whether you want to retrieve only one occurrence of a slewrate time event (single), or all occurrences of slewrate time events (multiple). The default value is single.

[<output_xaxistype>]

(Optional) Specifies whether you want to output slewrate data against time, or cycle. Cycle numbers refer to the nth occurrence of the slewrate event. This option is valid only when [<occurrences>] is set to “multiple”.

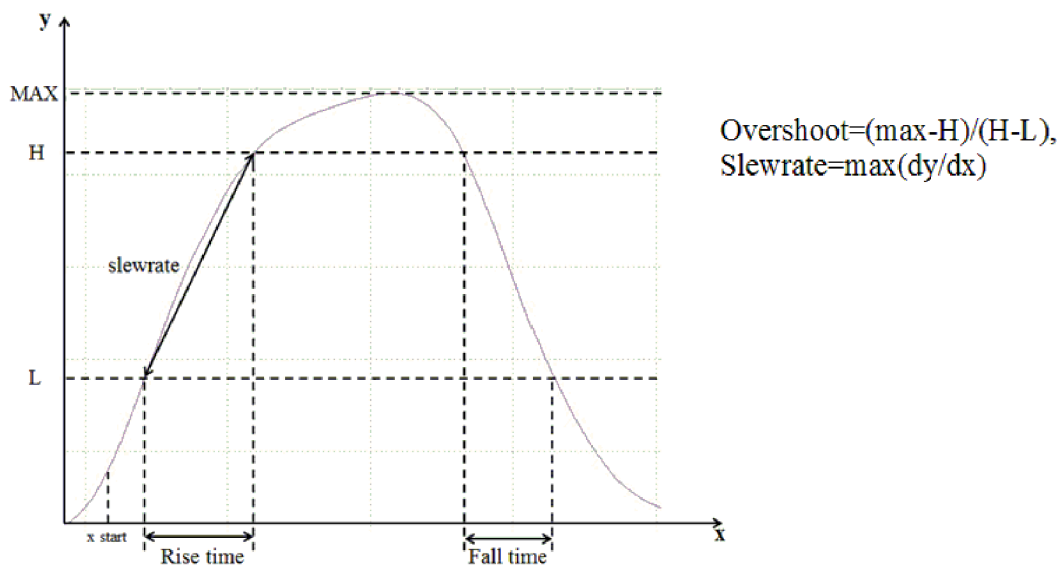
Errors

Possible error messages include:

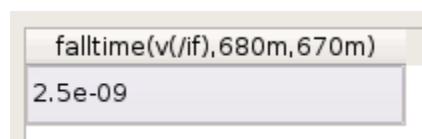
- Signal not found
- Y value out of range
- X value out of range

Examples

```
falltime(v(/if),680m,670m)
falltime(v(/if),100,75,percent=1,occu=multiple)
```

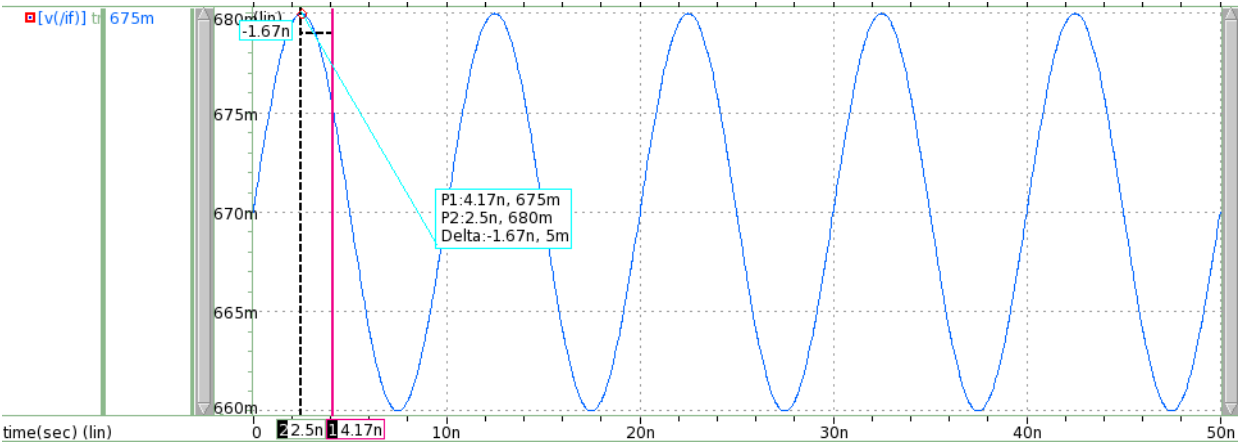


```
falltime(v(/if),680m,670m)
```



x-axis	falltime(v(/if),100,75,percent=1,occu=multiple)
2.5000n	1.66665n
12.5000n	1.66666n
22.5000n	1.66666n
32.5000n	1.66666n
42.5000n	1.66666n

falltime(v(/if),100,75,percent=1,occu=multiple)



falltrig

Calculates the X value of the nth falling edge over threshold Y.

Syntax

falltrig(<signal>,<y_level>,<edge_index>,[<x_start>],[<percent>])

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated.

<y_level>

The y_threshold value.

<edge_index>

The number of the nth falling edge over the threshold.

[<x_start>]

(Optional) The starting point of the X time value.

[<percent>]

(Optional) The percentage threshold value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

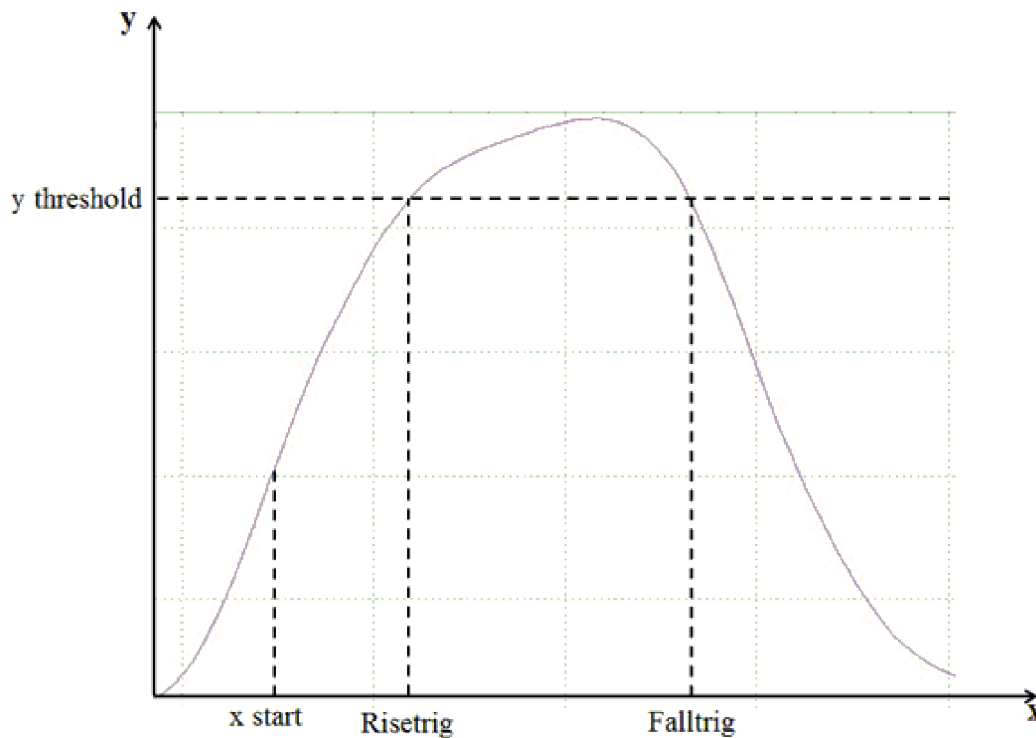
Errors

Possible error messages include:

- Signal not found
- Y value out of range

Examples

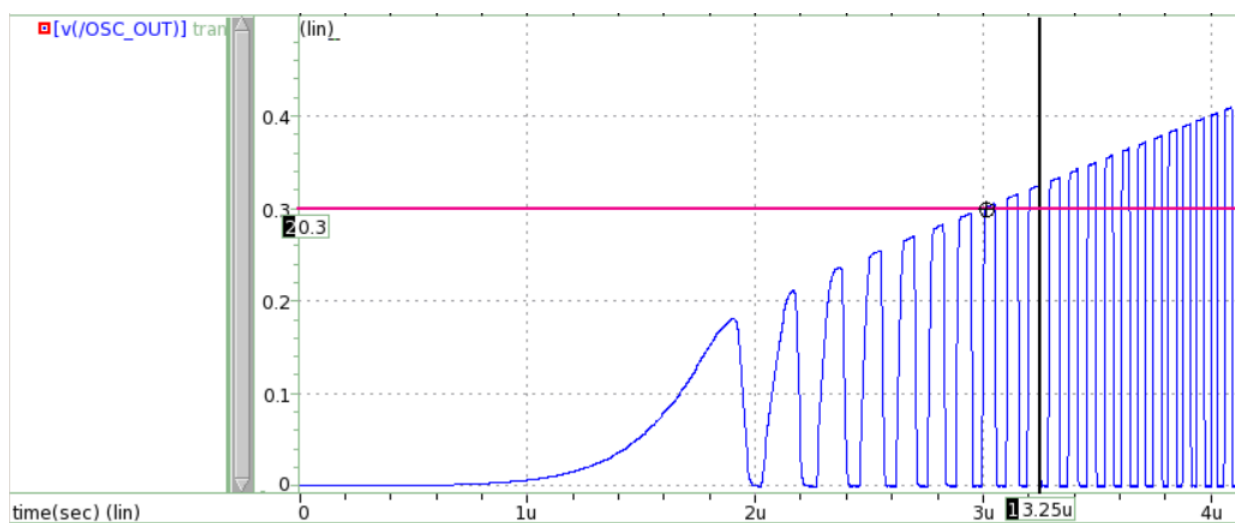
```
falltrig(v(/OSC_OUT),0.3,3)  
falltrig(v(/OSC_OUT),60,3,percent=1)
```



`falltrig(v(/OSC_OUT),0.3,3)` and `falltrig(v(/OSC_OUT),60,3,percent=1)`

`falltrig(v(/OSC_OUT),0.3,3)`
3.2492654209e-06

`falltrig(v(/OSC_OUT),60,3,percent=1)`
3.24928423563e-06



gain1_f

Calculates the frequency of a given signal at the signal magnitude, which is 0db (1) .

Syntax

`gain1_f(<signal>)`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

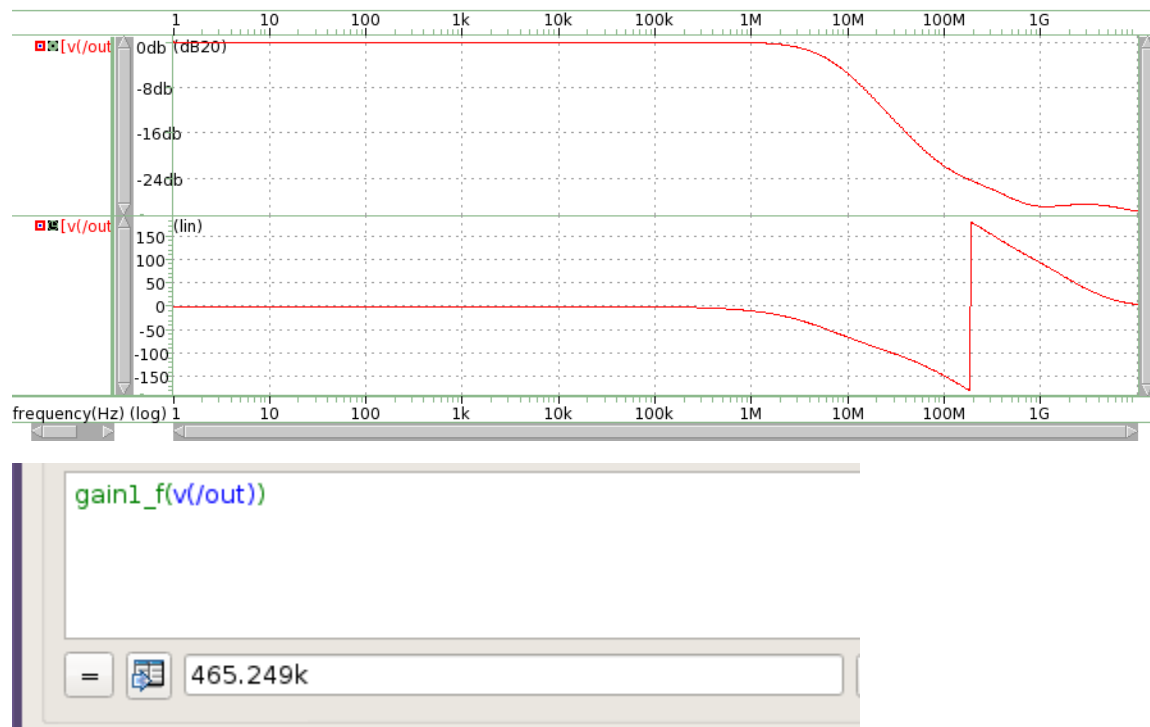
Possible error messages include:

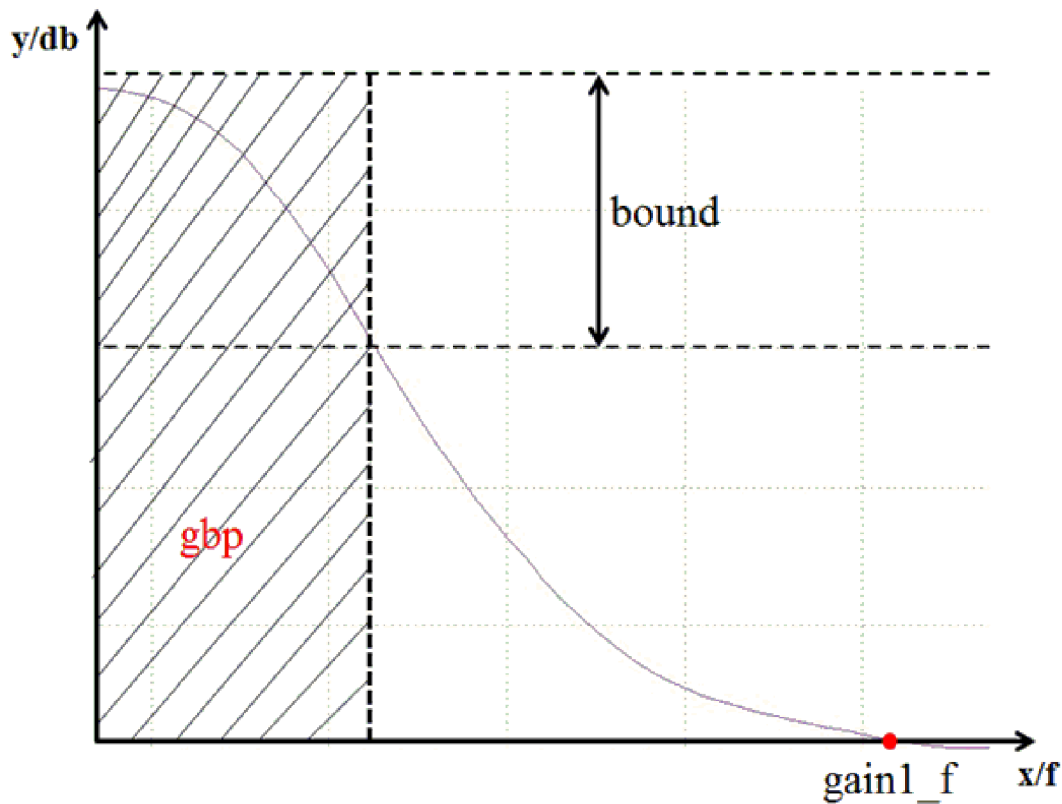
- Signal not found
- Incorrect signal type

Examples

```
gain1_f('0|v(out)')
```

will return the following:





gain_m

Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency. The phase angle is -180 degrees (modulate 360 degrees), and the gain margin is $1/g$ if g is the gain at -180 degrees.

Syntax

```
gain1_m(<signal>, [<phase_degree>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

[<phase_degree>]

(Optional) The phase degree, which can be -180 or 0. The default value is -180.

Errors

Possible error messages include:

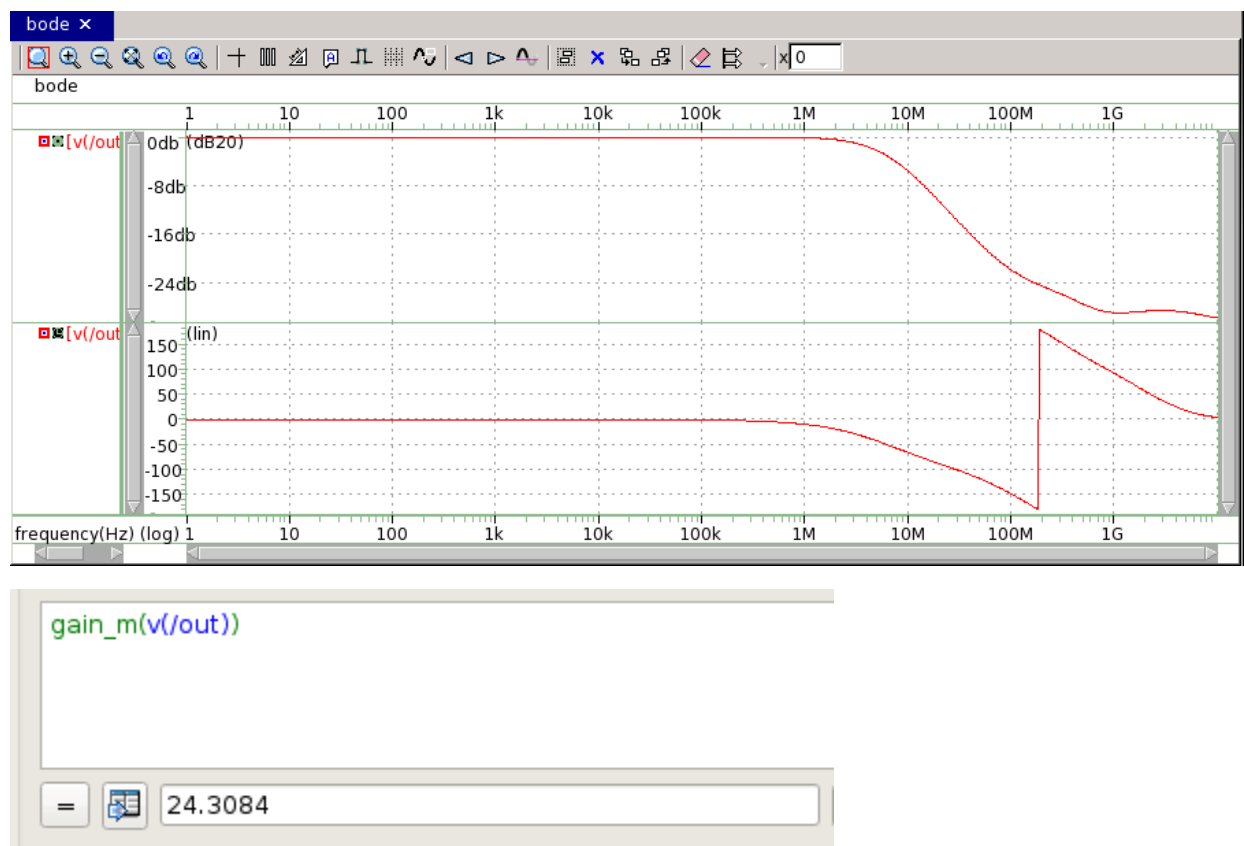
- Signal not found
- Incorrect signal type
- Waveform phase does not go through -180 degree

Examples

Example 1:

```
gain_m('v(/out)')
```

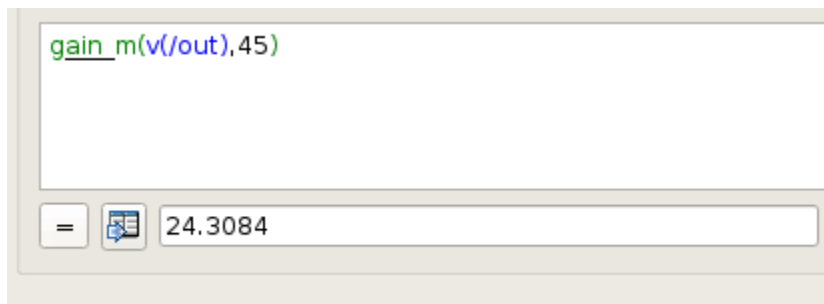
will return the following:



Example 2:

```
gain_m('v(/out)', 45)
```

will return the following:



gbp

Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency. The phase angle is -180 degrees (modulate 360 degrees), and the gain margin is $1/g$ if g is the gain at -180 degrees.

Syntax

```
gbp(<signal>, [<db_bound>], [<gain>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

<db_bound>

(Optional) The upper dB threshold.

<gain>

(Optional) User-defined gain value.

Errors

Possible error messages include:

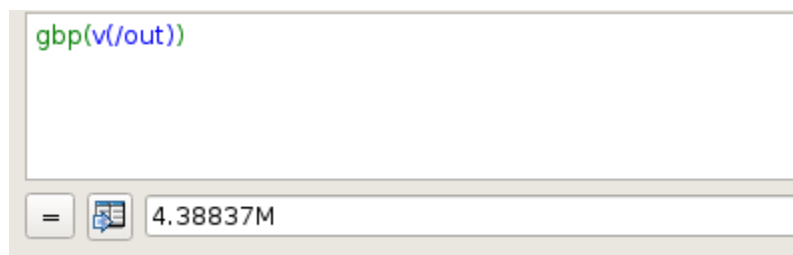
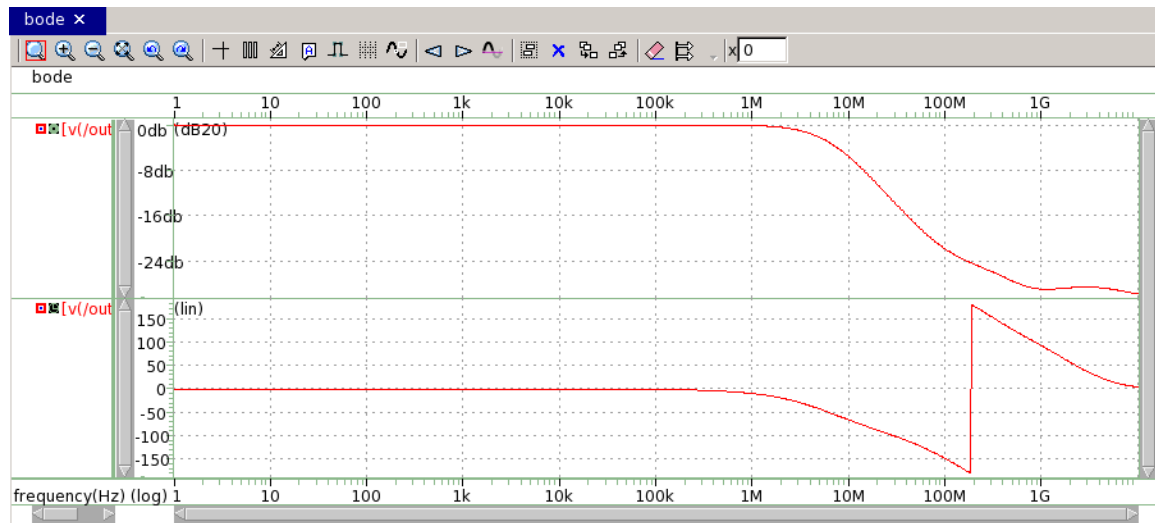
- Signal not found
- Incorrect signal type
- Cannot find the bandwidth

Examples

Example 1:

```
gbp('v(/out)')
```

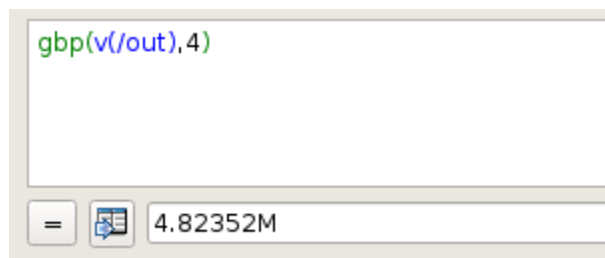
will return the following:

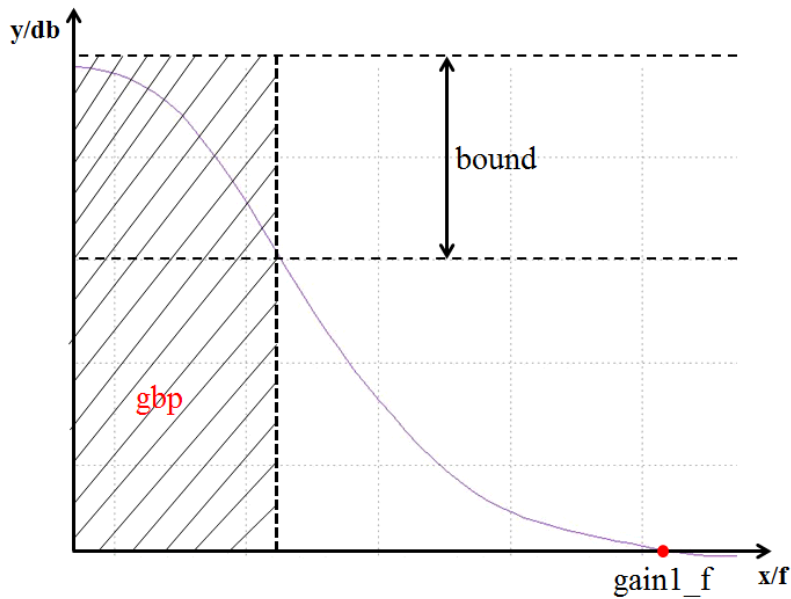


Example 2:

```
gbp('v(/out)',4)
```

will return the following:





get_harmonic_freq

Calculates the frequency of the nth harmonic.

Syntax

```
get_harmonic_freq(<signal>, <0-based_index>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<0-based_index>`

The index number.

Errors

Possible error messages include:

- Signal not found
- Invalid input index

globalmax

Returns the global maximum Y value of all data points for signal. (Also accepts scalar inputs.)

Syntax

```
globalmax(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

Errors

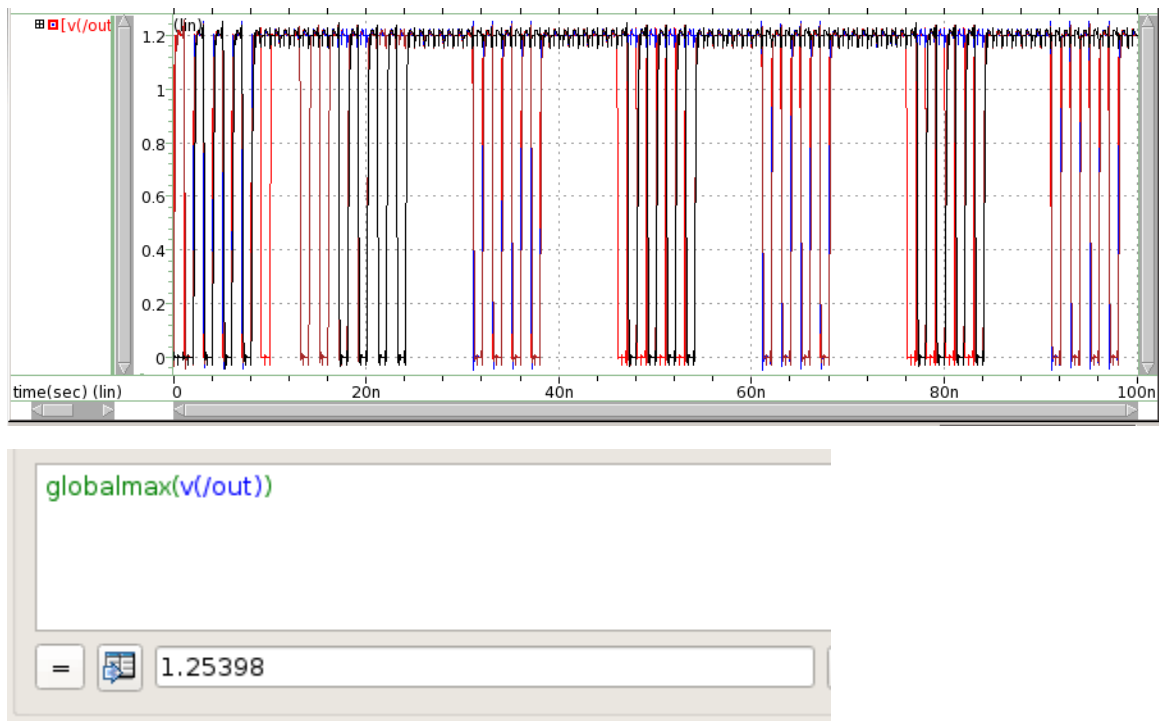
Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
globalmax('v(/out)')
```

will return the following:



globalmean

Returns the global discrete mean value of all data points for a signal. (Also accepts scalar inputs.)

Syntax

```
globalmean(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

Errors

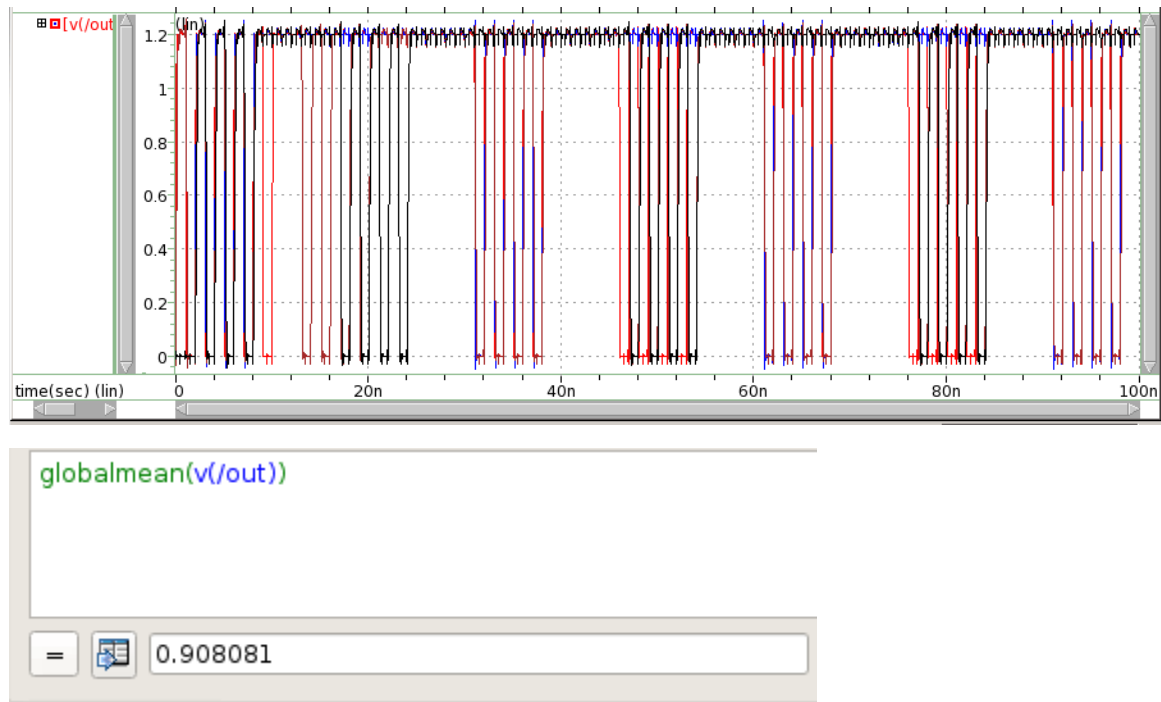
Possible error messages include:

- Signal not foundIncorrect signal type

Examples

```
globalmean('v(/out)')
```

will return the following:



globalmin

Returns the global minimum Y value of all data points for signal. (Also accepts scalar inputs.)

Syntax

```
globalmin(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

Errors

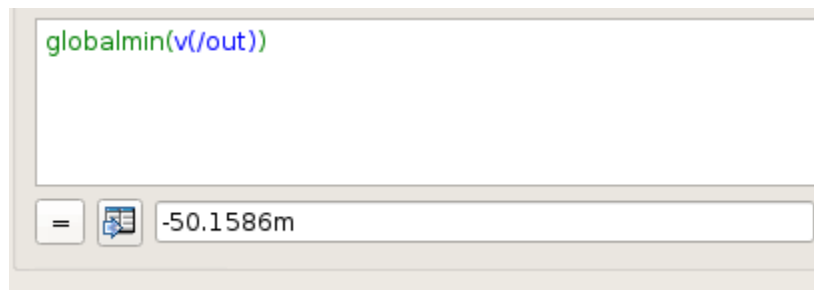
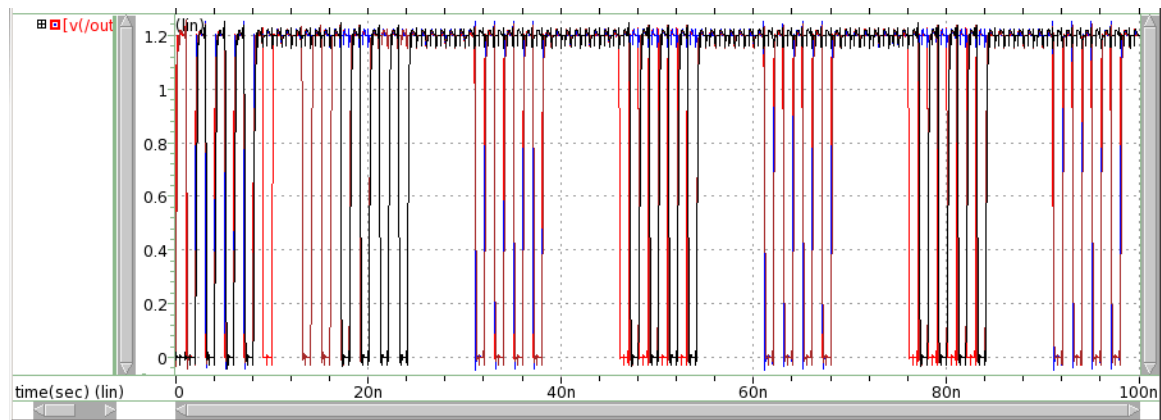
Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
globalmin('v(/out)')
```

will return the following:



globalstddev

Returns the global standard deviation value of all data points for signal. (Also accepts scalar inputs.)

Syntax

```
globalstddev(<signal>|<value> [, mc])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`mc`

(Optional) When `mc` is set to `true`, `globalstddev()` flattens Monte Carlo only and calculates the stddev. (By default, `mc` is `false`.)

For example, given a signal `sig:temp=1~4, monte_carlo=1~4`

`globalstddev(sig) = 1 value` (flatten temp and monte_carlo)

`globalstddev(sig, 0) = 1 value`

`globalstddev(sig, 1) = 4 value` (flatten monte_carlo only)

imProd

Description

Calculates the intermodulation product of a given signal and frequency.

Syntax

`imProd(<signal>, <freq>, [<param_name>])`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Supports analog signals. For complex signals, the magnitude is automatically applied.

`<freq>`

The frequency value.

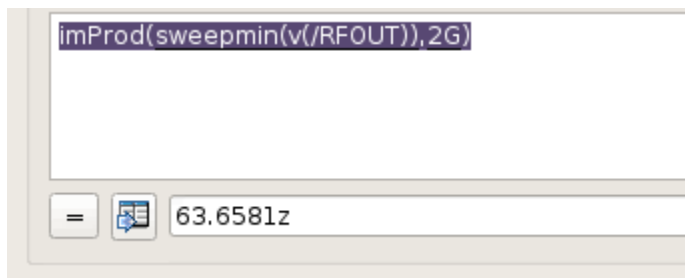
`<param_name>`

(Optional) The parameter condition of the input signal.

Examples

`imProd(sweepmin(v(/RFOUT)), 2G)`

will return the following:



integ

Calculates the integral value of a given signal from x1 to x2.

Syntax

```
integ(<signal>,[<x_start>],[<x_stop>],[<extrapolate>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`[<x_start>]`

(Optional) The starting point of the time X value.

`[<x_stop>]`

(Optional) The ending point of the time X value.

`[<extrapolate>]`

(Optional) When enabled, approximates a value if the x value is out of range. When disabled, the `x_start` and `x_stop` out of range values will be reset to `xmin` and `xmax`.

Errors

Possible error messages include:

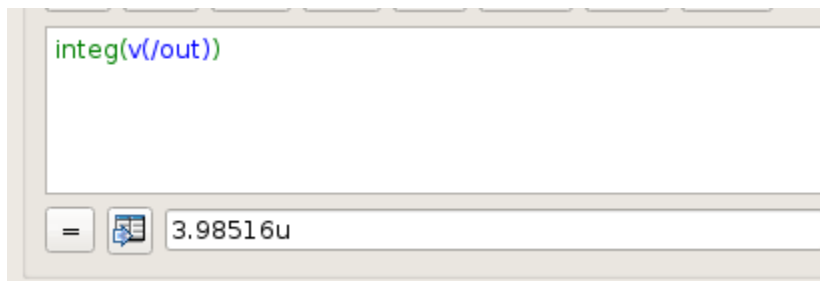
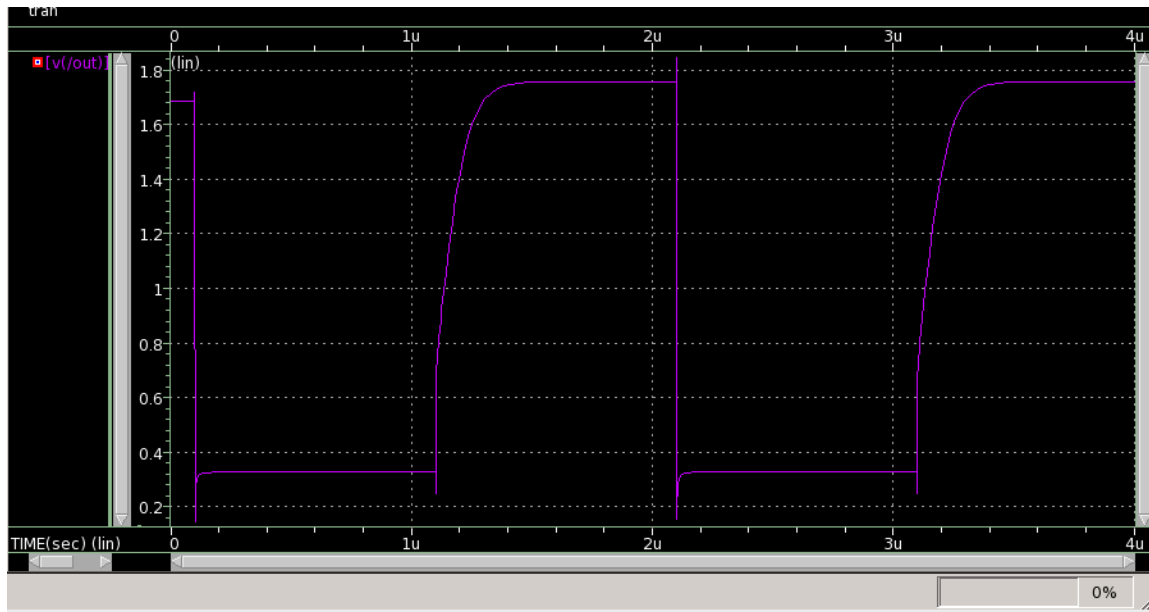
- Signal not found
- X value out of range

Examples

Example 1:

```
integ('v(/out)')
```

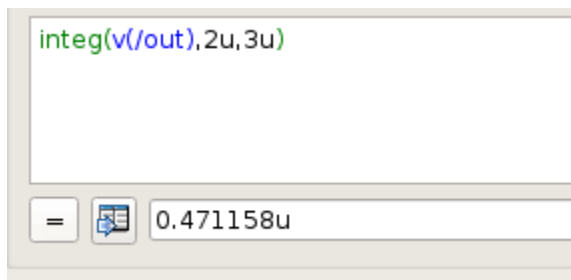
will return the following:



Example 2:

```
integ('v(/out)', 2u, 3u)
```


will return the following:



ipn

Calculates the n -order intercept point between the target signal and reference signal. For a non-linear system with an output and two fundamental excitation f_1 and f_2 frequencies, the n th order Intercept Point is the point on a set of PowerOut ($PowerOut_1$ and $PowerOut_n$) versus PowerIn curves at which a line extrapolated from $PowerOut_1$ with a slope of 1 and a line with a slope of n extrapolated from the $PowerOut_n$ intersect. $PowerOut_1$ represents the first order term, and $PowerOut_n$ is the n th order term. Both lines must be extrapolated from a region with sufficiently low input power. OIP_n is the value of $PowerOut$ at which IP_n occurs. IIP_n is the value of the $PowerIn$ value at which IP_n occurs.

Syntax

```
ipn(<signal>,<reference_signal>,<term_order>,<extrapolation_point>,[<input|output>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

<reference_signal>

The reference signal.

<term_order>

An integer value, which can be 0, 1, . . . , or n . n is the slope of the linear function.

<extrapolation_point>

The extrapolated X value at which two signals start to intercept each other.

[<input|output>]

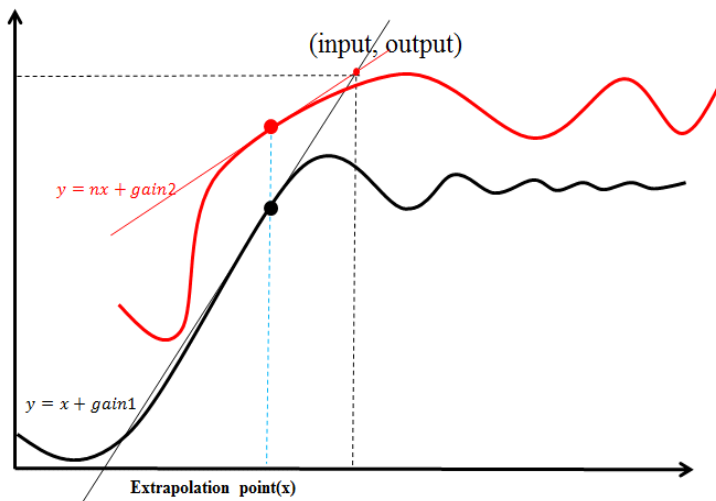
(Optional) The X and Y components on the waveform, where X is the `input` and Y is the `output`. The default is `output`.

Errors

Possible error messages include:

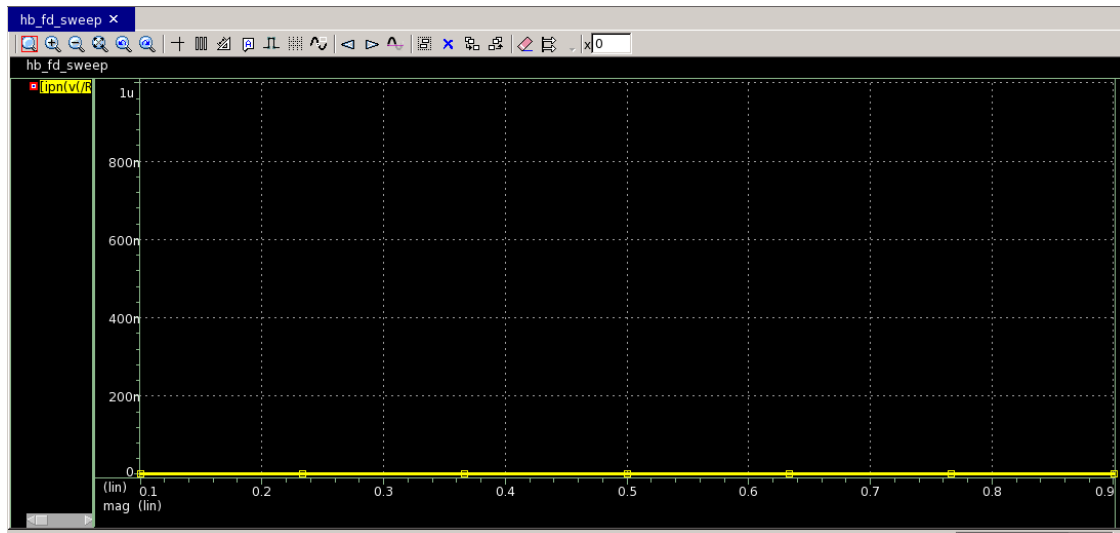
- Signal not found
- X value out of range

Examples



`ipn(v/RFOUT),v(/net3),1,50)` will return the following:





ipnhVRI

Description

N dB intercept point VRI.

Syntax

```
ipnhVRI (voltage signal <string>,
        harmonic indices 1 <string>,
        harmonic indices 2 <string>
        [, current signal]
        [, resistance]
        [, order 1 <integer>]
        [, order 2 <integer>]
        [, extrapolationPoint 1 <string>]
        [, extrapolationPoint 2 <string>]
        [, sweepUnits <enum>]
        [, referred <enum>])
```

Arguments

voltage signal

Input voltage signal.

harmonic indices 1

The first indices, space delimited.

harmonic indices 2

The second indices, space delimited.

`current signal`

(Optional) The input current signal.

`resistance`

(Optional) The resistance.

`order 1`

(Optional) Harmonic indices 1 signal's order. (Default: 1)

`order 2`

(Optional) Harmonic indices 2 signal's order. (Default: 3)

`extrapolationPoint 1`

(Optional) Harmonic indices 1 signal extrapolation point. If unspecified, defaults to lowest value.

`extrapolationPoint 2`

(Optional) Harmonic indices 2 | extrapolation point. If unspecified, defaults to lowest value.

`sweepUnits`

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none (Default: dBm)

`referred`

(Optional) The Input/Output referred measurement.

Valid values: input, output (Default: input)

mean

Calculates the mean value of a given signal or value from `x1` to `x2`.

Syntax

```
mean(<signal>|<value>,[<x_start>],[x_stop],[<type>])
```

Returns

An analog scalar value.

Arguments

`<signal|value>`

The signal or value to be calculated. Analog signals are supported.

[<x_start>]

(Optional) The starting point of the X value.

[x_stop]

(Optional) The ending point of the X value.

[<type>]

(Optional) The type of given signal, which can be `continuous` or `discrete`. The default is `continuous`.

Errors

Possible error messages include:

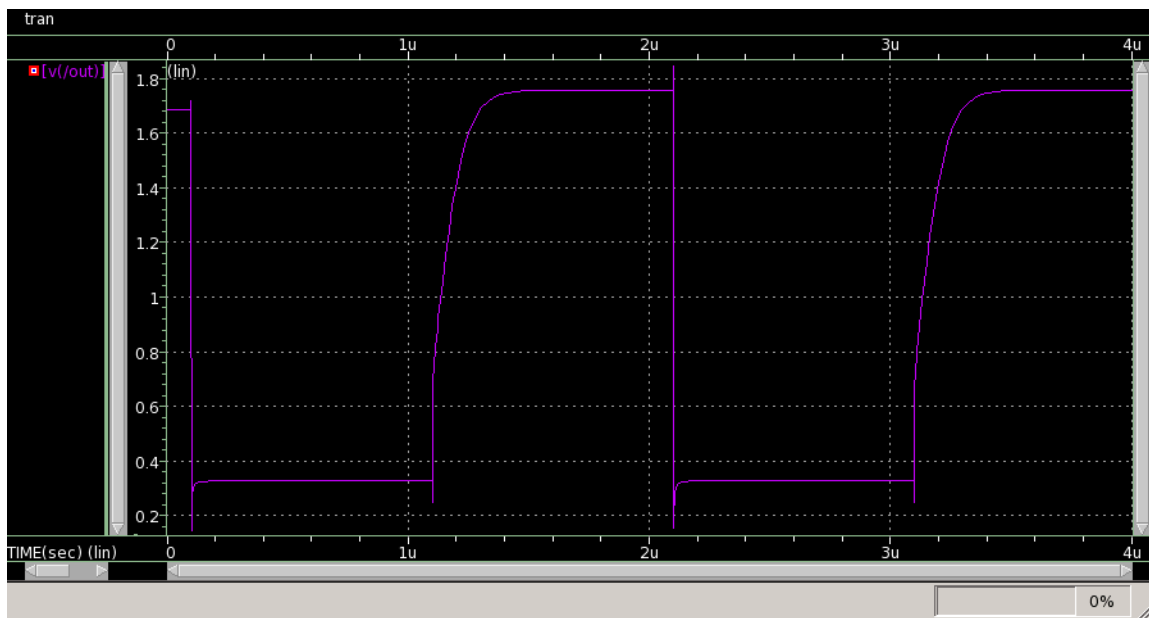
- Signal not found
- X value out of range

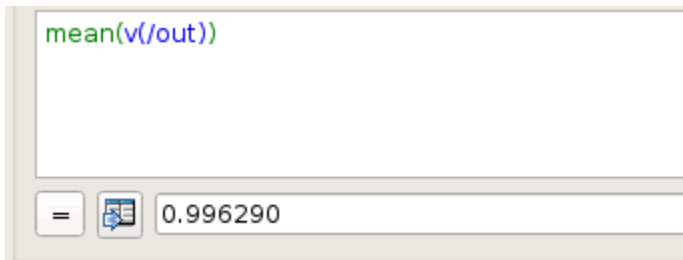
Examples

Example 1:

```
mean('v(/out)')
```

will return the following:

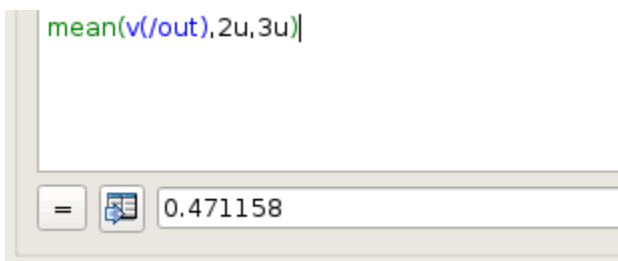




Example 2:

```
mean('0|v(out)',2u,3u
```

will return the following:



measure_aperture

Applies the aperture measurements to the eye object returned from the `create_eye` function.

Syntax

```
measure_aperture(<eye_object>,<vref>,[<dc>],[<ac>],<return_type>)
```

Returns

N/A

Arguments

`<eye_object>`

The output of the `create_eye` function.

`<vref>`

The central reference voltage level.

`[<dc>]`

(Optional) The DC voltage offset from the reference level.

[<ac>]

(Optional) The AC voltage offset from the reference level.

[<return_type>]

(Optional) The returned measurement type, which includes enumerated values in a list of all the measurements that the aperture returns.

The following are values are included:

- Aperture_Time (the default)
- Min_Delay
- Max_Delay
- Skew
- Jitter_P2P
- Jitter_RMS
- Setup
- Hold
- rho/rhi: The outer/inner most crossing time of the eye open at VREF+VAC on rising edge
- fto/fti: The inner/outer most crossing time of the eye open at VREF+VDC on falling edge
- fho/fhi: The outer/inner most crossing time of the eye open at VREF-VAC on falling edge
- rto/rti: The inner/outer most crossing time of the eye open at VREF-VDC on rising edge

Errors

Possible error messages include:

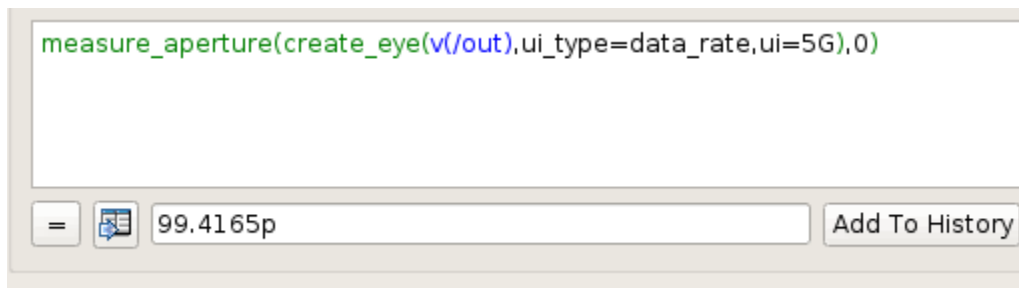
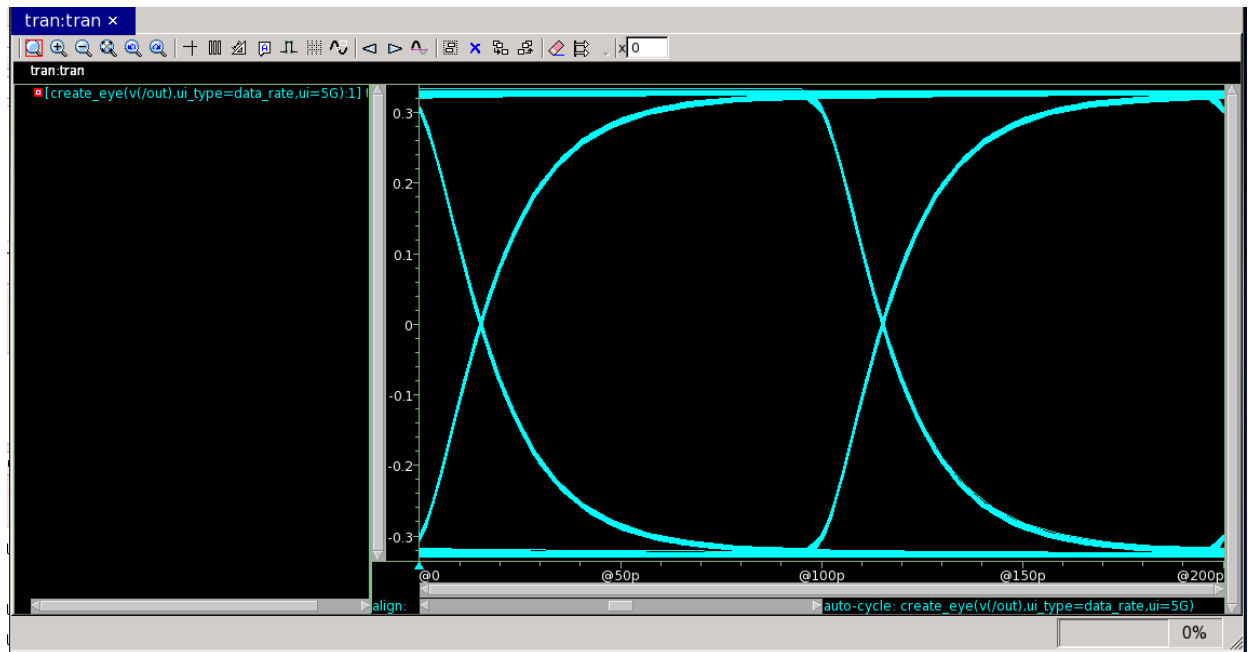
- Reference voltage is not specified
- Signal must be eye object
- No valid eye object

Examples

```
measure_aperture(create_eye(v(/out),ui_type=data_rate,ui=5G),0)
```

The following example shows how the function works on the eye object.

`create_eye(v(/out),ui_type=data_rate,ui=5G)` You first need to create the eye object using the `create_eye` function from the calculator:



measure_eyepopen

Measures the height and width of the eye opening.

Syntax

```
measure_eyepopen(<eye_object>,[<opening_type>],[<accuracy(>)>],[<vref>],  
[<return_type>])
```

Returns

The height or width of the eye object.

Arguments

`<eye_object>`

The output of the `create_eye` function.

`[<opening_type>]`

(Optional) The type of eye opening to be measured. Valid values are Diamond, Rectangle, Hexagon. Default: Diamond.

`[<accuracy(>)]`

(Optional) The accuracy percentage of the diamond opening measurement. Default: 1.

`[<vref>]`

(Optional) The VREF value used for the rectangle and hexagon opening measurements.

`[<return_type>]`

(Optional) The returned measurement type. Valid values are Height and Width. Default: Width.

Errors

Possible error messages include:

- Reference voltage is not specified
- Signal must be eye object
- No valid eye object
- Evaluation failed

Examples

```
measure_eyepopen('x|create_eye(signal)', accu=2, type='Height')
measure_eyepopen('x|create_eye(signal)', 'Rectangle', vref=0.6, type='Height')
measure_eyepopen('x|create_eye(signal)', 'Hexagon', vref=0.6)
```

measure_eyesize

Applies the eye size measurements to the eye object returned from the `create_eye` function.

Syntax

```
measure_eyesize(<eye_object>, <vref>, [<dc>], [<ac>], <return_type>)
```

Returns

N/A

Arguments

`<eye_object>`

The output of the `create_eye` function.

`<vref>`

The central reference voltage level.

`[<dc>]`

(Optional) The DC voltage offset from the reference level.

`[<ac>]`

(Optional) The AC voltage offset from the reference level.

`[<return_type>]`

(Optional) The returned measurement type, which includes enumerated values in a list of all the measurements that the Eye Size returns.

The following are values are included:

- WH (the default)
- WL
- MH
- ML
- OH
- OL
- FUSS
- FUSS2
- DVDT
- DVDT_AVG
- DVDT_MIN
- DVDT_MAX
- Skew1
- Skew2

- CenterX
- IHEIGHT_AtCenterX
- OHEIGHT_AtCenterX

Errors

Possible error messages include:

- Reference voltage is not specified
- Signal must be eye object
- No valid eye object

Note that the `measure_eyesize()` function requires an eye object. The picker for the eye object field in the Calculator tab opens the design editor, asking for a net, terminal or instance term.

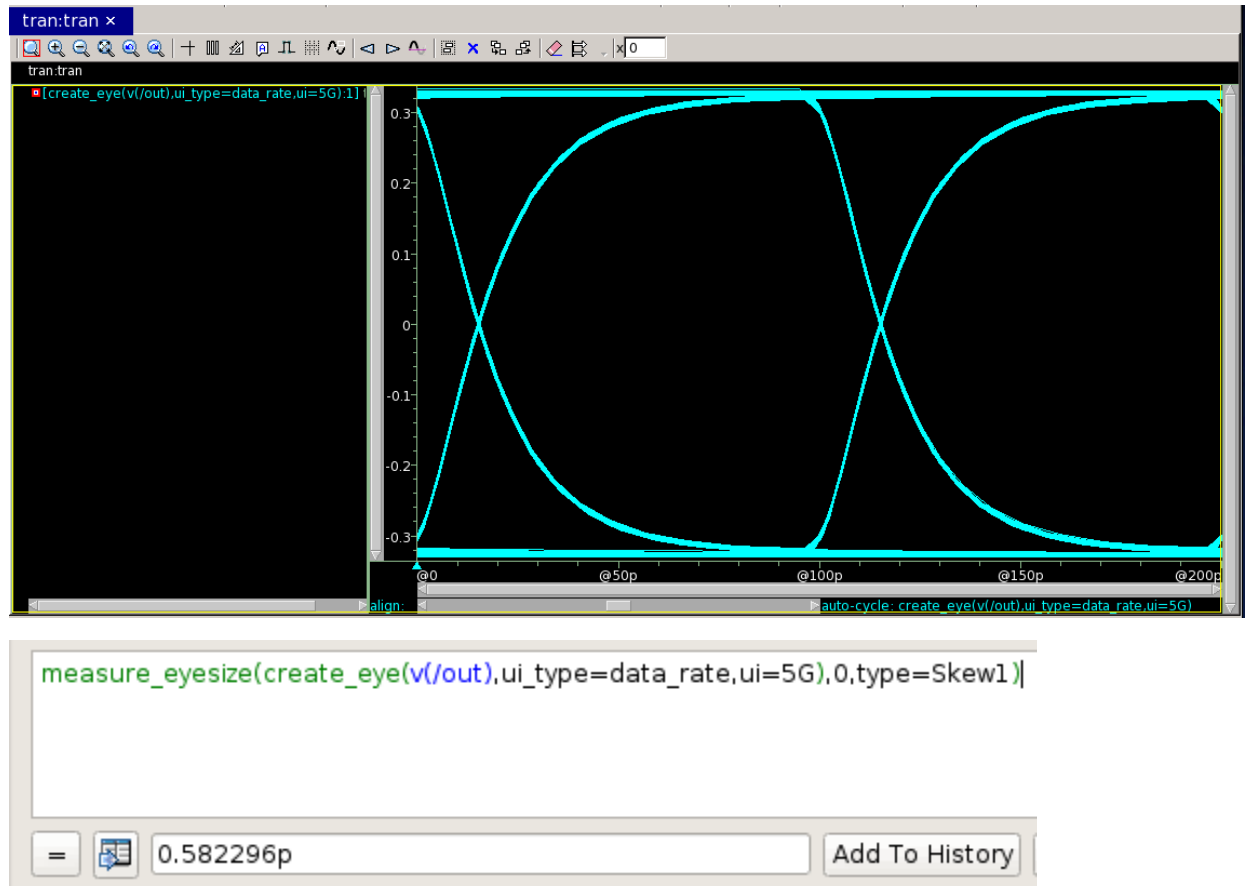
You must provide a valid eye object, but do not use the picker. Manually enter the name of a `create_eye()` expression in the eye object field in the Calculator. If your `create_eye()` expression is unnamed, you must first enter a name for the expression in the Outputs pane.

Examples

```
measure_eyesize(create_eye(v(/out),ui_type=data_rate,ui=5G),0,type=Skew1)
```

The following example shows how the function works on the eye object.

`create_eye(v(/out),ui_type=data_rate,ui=5G)` You first need to create the eye object using the `create_eye` function from the calculator. This specific example returns the skew value:



median_val

Calculates the median value of a given signal or value. The signals are sorted, where n is odd in the top equation and n is even in the bottom equation:

$$\begin{cases} Val\left(\frac{n+1}{2}\right) \\ Val\left(\frac{n}{2}\right) + Val\left(\frac{n}{2} + 1\right) \end{cases}$$

Syntax

`median_val(<signal>|<value>)`

Returns

An analog scalar value.

Arguments

<signal|value>

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

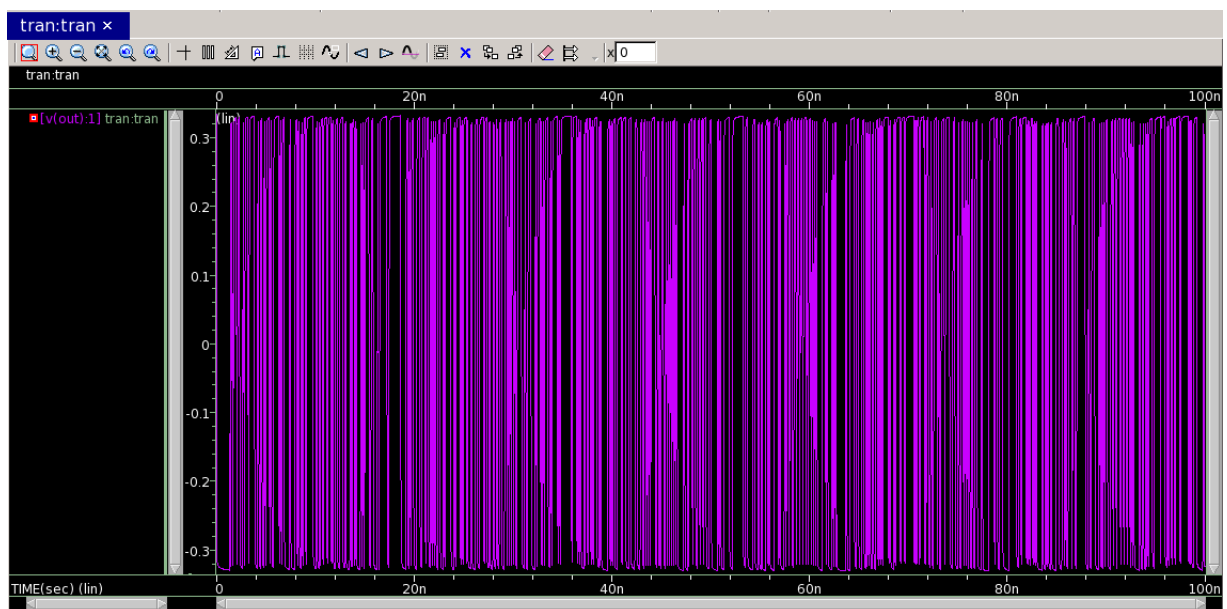
Errors

Possible error messages include:

- Signal not found
- Input signal has no data point
- Input signal has too many data point
- Out of memory

Examples

```
median_val('v(/out)')
```





overshoot

Calculates the overshoot value of a given signal, which uses the following algorithm:

$$(\text{Max}(\text{signal}) - \text{high_y_level}) / (\text{high_y_level} - \text{low_y_level})$$

Syntax

```
overshoot(<signal>,<low_y_level>,<high_y_level>,<percent>,<low_value_type>,<high_value_type>,<occurrences>,<output_xAxisType>,<output_type>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated. Analog signals are supported.

<low_y_level>

The low threshold of the Y signal value.

<high_y_level>

The high threshold of the Y signal value.

[<percent>]

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

[<low_value_type>]

(Optional) Determines how to retrieve the low y value.

[<high_value_type>]

(Optional) Determines how to retrieve the high y value.

[<occurrences>]

(Optional) Specifies result to be only one occurrence of an overshoot event for the given waveform (*single*) or all occurrences of overshoot for the given waveform which you can later plot or print (*multiple*).

[<output_xAxisType>]

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx.

[<output_type>]

(Optional) Default is *absolute*. If *percentage* is selected, the threshold values are in percentages, otherwise, absolute values.

Errors

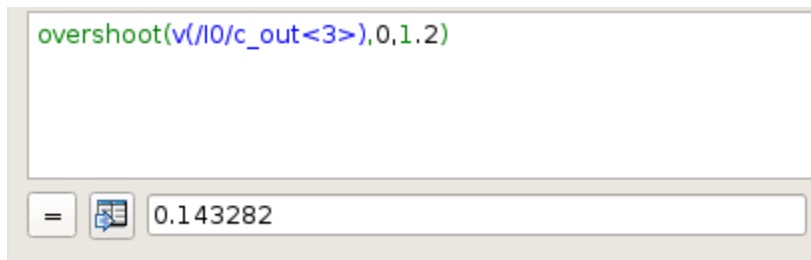
Possible error messages include:

- Signal not found
- Y value out of range

Examples

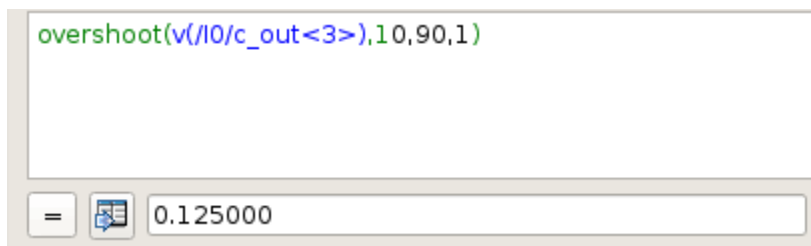
Example 1:

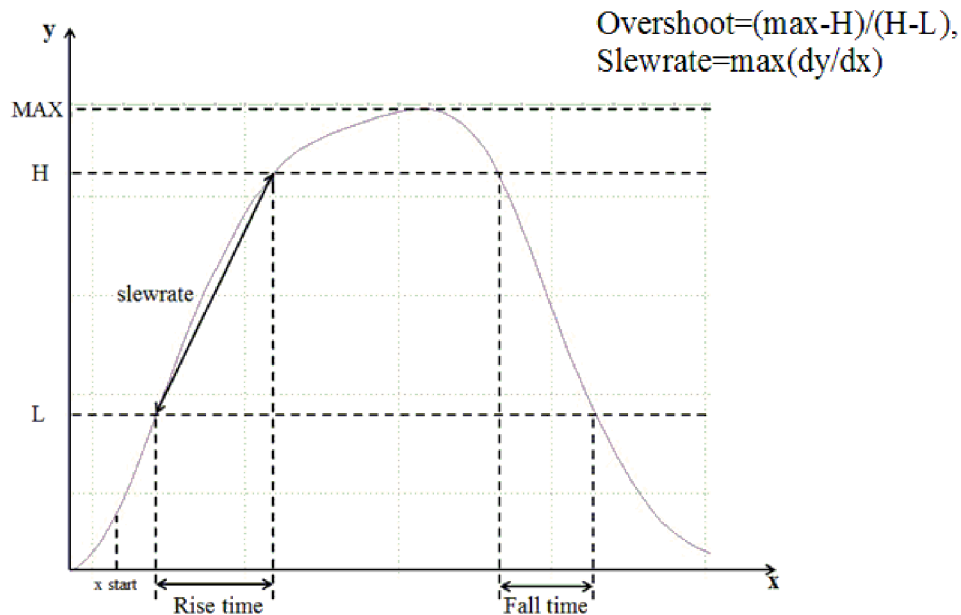
```
overshoot(v(/I0/c_out<3>),1.2,0)
```



Example 2:

```
overshoot(v(/I0/c_out<3>),10,90,1)
```





p2p

Measures peak-to-peak, the difference between the maximum and minimum Y values of the given signal. When the argument `measEdge` is 1 (true), `p2p` measures peak-to-peak value of edges.

Syntax

```
p2p(<signal>, [x1=<x_start>], [x2=<x_stop>], [measEdge=0|1],  
[n=<nth_index>], [minpp=<min_p2p>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[x1=<x_start>]`

(Optional) The starting point of the X range.

`[x2=<x_stop>]`

(Optional) The stop point of the X range.

[<measEdge>=0|1]

(Optional) Specifies whether to measure peak-to-peak value of edges. Default is 0 (false).

When `measEdge=0`, arguments `n` and `minpp` will be ignored even if they are specified.

[n=<nth_index>]

(Optional) The integer number. Specifies the index of edge to be measured. It takes effect only when `measEdge=1`. If unspecified and `measEdge=1`, the function finds the edge with biggest peak-to-peak and returns the value.

[minpp=<min_p2p>]

(Optional) The minimum peak-to-peak value that an edge must have. It takes effects only when `measEdge=1` and `nth_index` is specified. Its default value is 5% of the peak-to-peak value of the biggest edge.

If all edges' peak-to-peak value are smaller than `min_p2p`, the function will return `failed`.

Errors

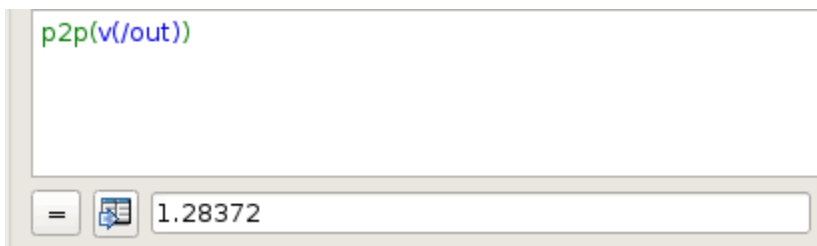
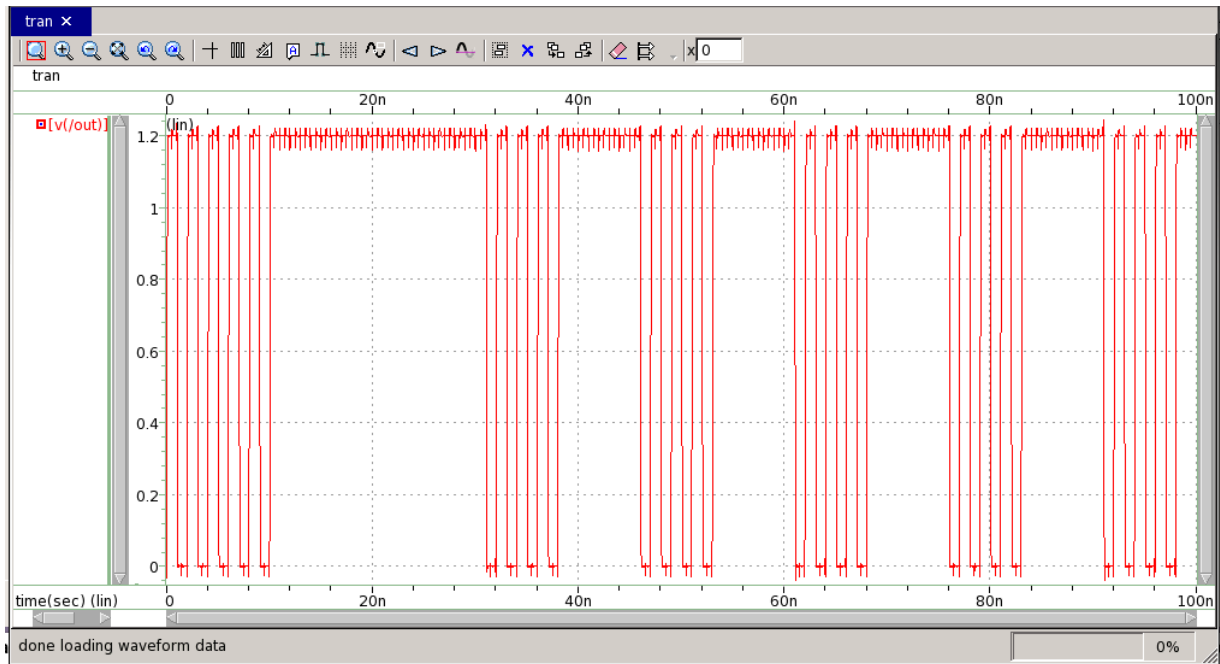
Possible error messages include:

- Signal not found
- X value out of range

Examples

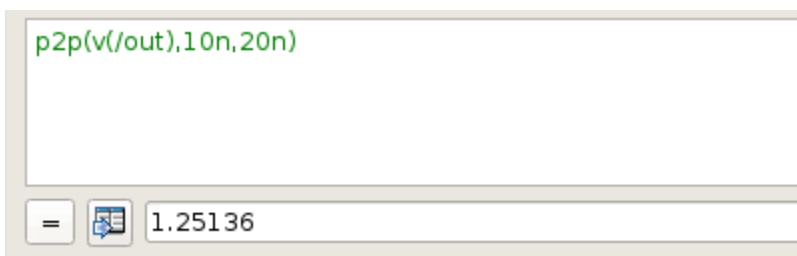
The following calculates: $\text{ymax}('v(\text{out})') - \text{ymin}('v(\text{out})')$

```
p2p('v(out)')
```



The following calculates: $\text{ymax}('v(out)', 10n, 20n) - \text{ymin}('v(out)', 10n, 20n)$

`p2p('v(out)', 10n, 20n)`



pamn_level_at_eye_center

Measures eye level at eye center.

Syntax

```
pamn_level_at_eye_center(<eye object>,[number of levels],[levels],[time  
reference],[eye center],[time location type],[time window %],[eye open],  
[return type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Level Measurement

Arguments

<eye object>

(Required) The measurement PAMn eye object that is returned from the `create_eye` function.

<number of levels>

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported. The default is 4.

<levels>

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

<time reference>

(Optional) The default is the center eye. It is based on the center eye and each eye is independent.

<eye center>

(Optional) The location of the eye center for time reference. The default is maximum eye width. For **Maximum Eye Width**, the center of the maximum eye width of the Middle/Center eye is used as the time reference. For **Maximum Eye Height**, the maximum of the eye height of the Middle/Center eye is used as the time reference.

<time location type>

(Optional) The default is eye center. The available values are **Eye center** and **Minimum rms**.

<time window %>

(Optional) The time window from the eye center as a percent of UI. The default is 10.

<eye open>

(Optional) The eye level that is measured. Level 1 is the bottom eye opening.
The default is level1.

The available values are level0, level1, level2, level3, level4, level5.

<return type>

(Optional) The returned measurement type. The default is mean.

The available values are mean, point to point, rms, skew, rlm, rlm, oif.

pamn_level_at_user_ref

Measures eye level at user reference.

Syntax

```
pamn_level_at_user_ref(<eye object>, [number of levels], [levels],  
[vref], [x value], [time location type], [time window %], [eye open],  
[return type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Level Measurement

Arguments

<eye object>

(Required) The measurement PAMn eye object that is returned from the
create_eye function.

<number of levels>

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported.
The default is 4.

<levels>

(Optional) The amplitude levels. The default is empty, which means eye
openings are even spaced. PW calculates default levels, otherwise, use input
levels as a string of numbers separated by “,”.

<vref>

(Optional) The user specified VREF. Use middle point of VREF as the time
reference if the X value is not given.

<x value>

(Optional) The user specified X value as the time reference.

<time location type>

(Optional) The default is eye center. The available values are **Eye center** and **Minimum rms**.

<time window %>

(Optional) The time window from the eye center as a percent of UI. The default is 10.

<eye open>

(Optional) The eye level that is measured. Level 1 is the bottom eye opening. The default is level1.

The available values are level0, level1, level2, level3, level4, level5.

<return type>

(Optional) The returned measurement type. The default is mean.

The available values are mean, point to point, rms, skew, rlm, rlm, oif.

pamn_open_at_eye_center

Measures eye open at eye center.

Syntax

```
pamn_open_at_eye_center(<eye object>, [number of levels], [levels], [time  
reference], [eye center], [offset from midpoint], [eye open], [return  
type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Open Measurement

Arguments

<eye object>

(Required) The measurement PAMn eye object that is returned from the create_eye function.

<number of levels>

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported.
The default is 4.

<levels>

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

<time reference>

(Optional) The default is the center eye. It is based on the center eye and each eye is independent.

<eye center>

(Optional) The location of the eye center for time reference. The default is maximum eye width. For **Maximum Eye Width**, the center of the maximum eye width of the Middle/Center eye is used as the time reference. For **Maximum Eye Height**, the maximum of the eye height of the Middle/Center eye is used as the time reference.

<offset from midpoint>

(Optional) The offset value from the midpoint for calculating DVDT min/max/avg.
The default is 0.1.

<eye open>

(Optional) The eye opening that is measured. The default is eye1.

The available values are eye0, eye1, eye2, eye3, eye4.

<return type>

(Optional) The returned measurement type. The default is midpoint.

The available values are midpoint, skew, height, width, dvdt_min_rise, dvdt_max_rise, dvdt_avg_rise, dvdt_min_fall, dvdt_max_fall, dvdt_avg_fall, dvdt_min_both, dvdt_max_both, dvdt_avg_both.

pamn_open_at_user_ref

Measures eye open at user reference.

Syntax

```
pamn_open_at_user_ref(eye object, [number of levels], [levels], [vref],  
[x value], [offset from mid-point], [eye open], [return type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Open Measurement

Arguments

`<eye object>`

(Required) The measurement PAMn eye object that is returned from the `create_eye` function.

`<number of levels>`

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported. The default is 4.

`<levels>`

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

`<vref>`

(Optional) The user specified VREF. Use middle point of VREF as the time reference if the X value is not given.

`<x value>`

(Optional) The user specified X value as the time reference.

`<offset from mid-point>`

(Optional) The offset value from the midpoint for calculating DVDT min/max/avg. The default is 0.1.

`<eye open>`

(Optional) The eye opening that is measured. The default is eye1.

The available values are eye0, eye1, eye2, eye3, eye4.

`<return type>`

(Optional) The returned measurement type. The default is midpoint.

The available values are midpoint, skew, height, width, dvdt_min_rise, dvdt_max_rise, dvdt_avg_rise, dvdt_min_fall, dvdt_max_fall, dvdt_avg_fall, dvdt_min_both, dvdt_max_both, dvdt_avg_both.

paramvalue

Calculates the value of a sweeping parameter, which is mainly used with multi-trace waveforms to extract the values of the specified sweeping parameter.

Syntax

```
paramvalue(<sweep_variable>)
```

Returns

Several analog scalar values.

Arguments

<sweep_variable>

The multi-trace sweeping signal to be calculated.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
paramvalue('1|pin_dbm')
```

phase_m

Calculates the phase margin, which is the difference between the phase of the response and -180 degrees when the loop gain is 1.0.

Syntax

```
phase_m(<signal>, [<base_degree>])
```

Returns

Several analog scalar values.

Arguments

<signal>

The signal to be measured.

[<base_degree>]

(Optional) The base degree.

Errors

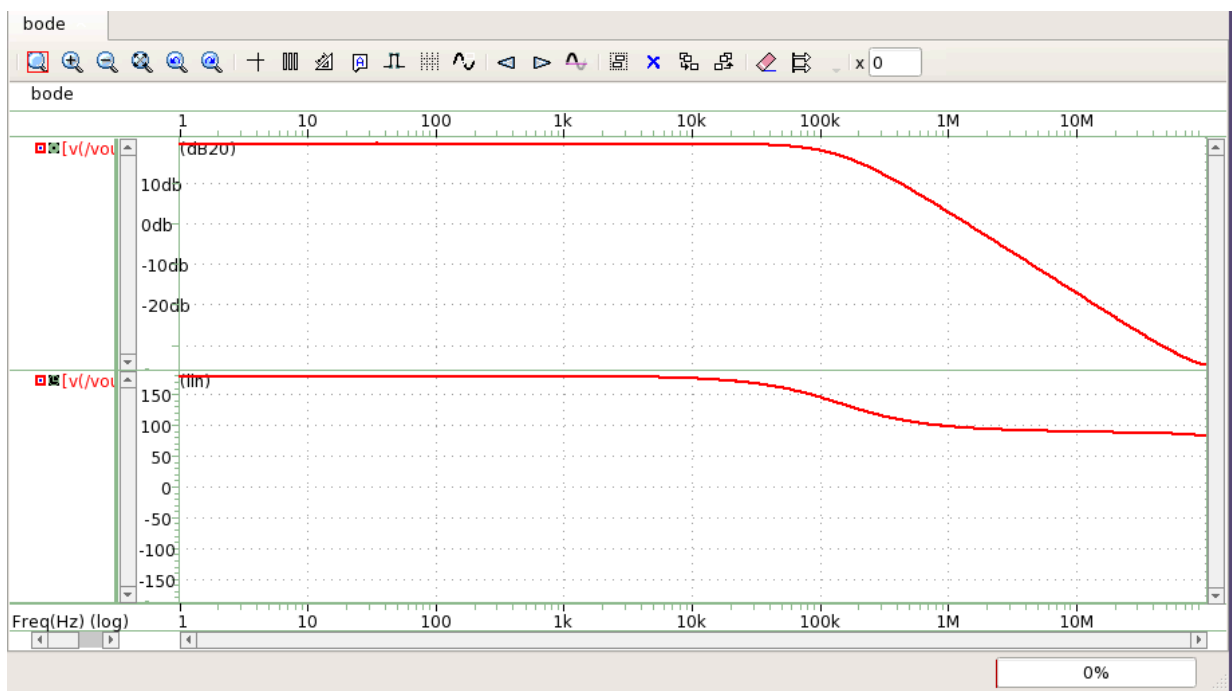
Possible error messages include:

- Signal not found
- Incorrect signal type
- Waveform phase does not go through -180 degree

Examples

Example 1:

```
phase_m('v(/out)')
```

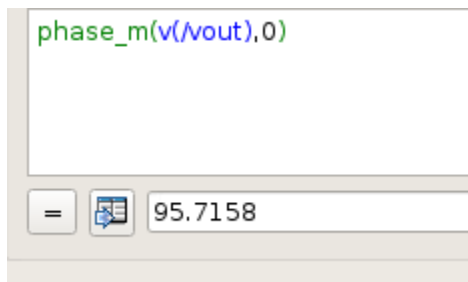


```
phase_m(v(/out))
```

= 275.716

Example 2:

```
phase_m('v(/out)',0)
```



risetime

Calculates the rise time for a signal, which is the time taken by a signal to change from a specified low value to a specified high value. Typically, these values are 10% and 90% of the step height.

Syntax

```
risetime(<signal>,<low_y_level>,<high_y_level>,  
[<x_start>],[<percent>],[edge_index],[<min>],[<max>],  
[<min_value_type>],[max_value_type],[<occurrences>],  
[<output_xaxistype>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be calculated.

<low_y_level>

Initial value at which the computation is to be started.

<high_y_level>

Final value at which the computation is to be stopped.

[<x_start>]

(Optional) The starting point of the X time value.

[<percent>]

(Optional) The percentage threshold value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

[<edge_index>]

(Optional) The index of the edge. The default value is 1.

[<min>]

(Optional) Min value for percentage y init/final. Default is xmin or ymin depending on type.

[<max>]

(Optional) max value for percentage y init/final, default is xmax or ymax depending on type.

[<min_value_type>]

(Optional) Specifies whether the [<min>] value is an X-axis (x) or Y-axis value (y). The default value is y.

[<max_value_type>]

(Optional) Specifies whether the [<max>] value is an X-axis (x) or Y-axis value (y). The default value is y.

[<occurrences>]

(Optional) Specifies whether you want to retrieve only one occurrence of a slewrate time event (single), or all occurrences of slewrate time events (multiple). The default value is single.

[<output_xaxistype>]

(Optional) Specifies whether you want to output slewrate data against time, or cycle. Cycle numbers refer to the nth occurrence of the slewrate event. This option is valid only when [<occurrences>] is set to "multiple".

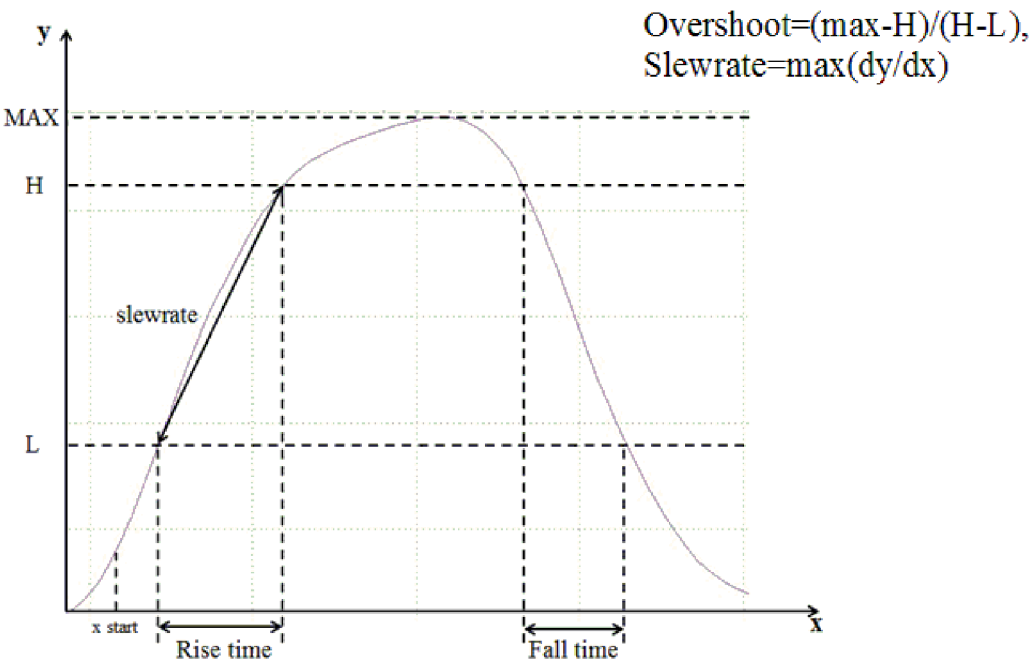
Errors

Possible error messages include:

- Signal not foundY value out of rangeX value out of range

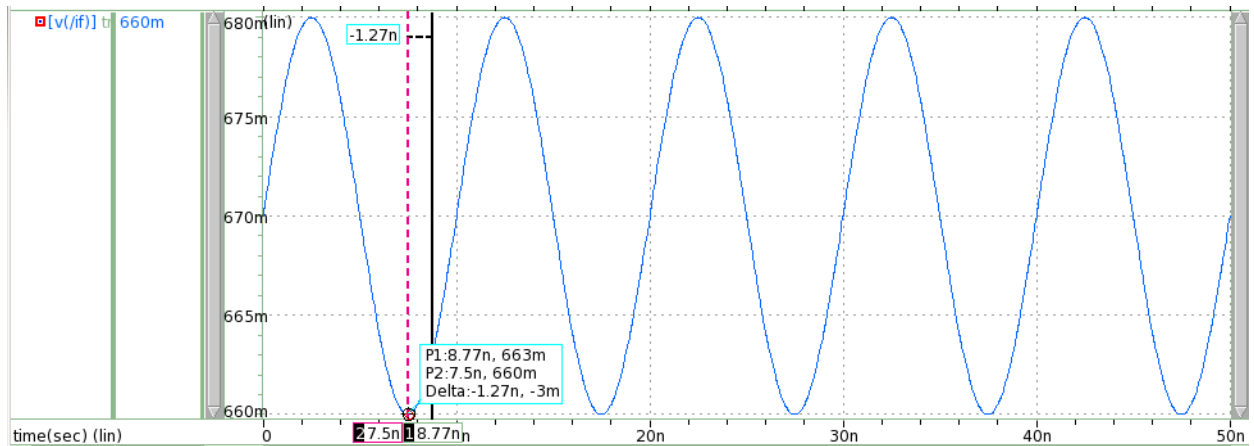
Examples

```
risetime(v(/if),0,15,percent=1,occu=multiple)
risetime(v(/if),670m,680m)
```

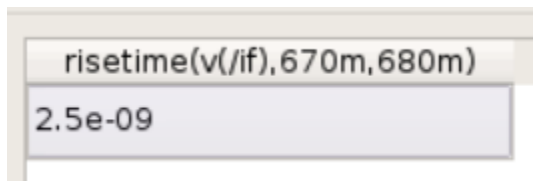


`risetime(v(/if),0,15,percent=1,occu=multiple)`

x-axis	risetime(v(/if),0,15,percent=1,occu=multiple)
7.50000n	1.26571n
17.5000n	1.26553n
27.5000n	1.26553n
37.5000n	1.26584n
47.5000n	1.26557n



```
risetime(v(/if), 670m, 680m)
```



risetrig

Measures the X value of the nth rising edge over the y_threshold starting from the x_start value.

Syntax

```
risetrig(<signal>, <y_level>, <edge_index>, [<x_start>], [<percent>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<y_level>

The threshold of the Y value.

<edge_index>

The nth rising edge.

[<x_start>]

(Optional) The starting point of the X value.

[<percent>]

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the y_level is considered a percentage value; if not, Y is 0.

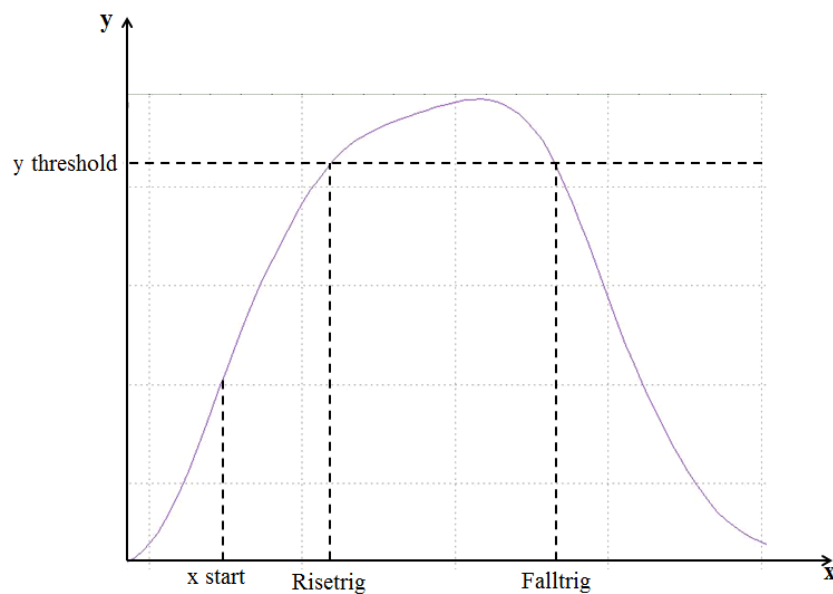
Errors

Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
risetrig(v(/OSC_OUT),0.3,3)  
risetrig(v(/OSC_OUT),60,3,percent=1)
```

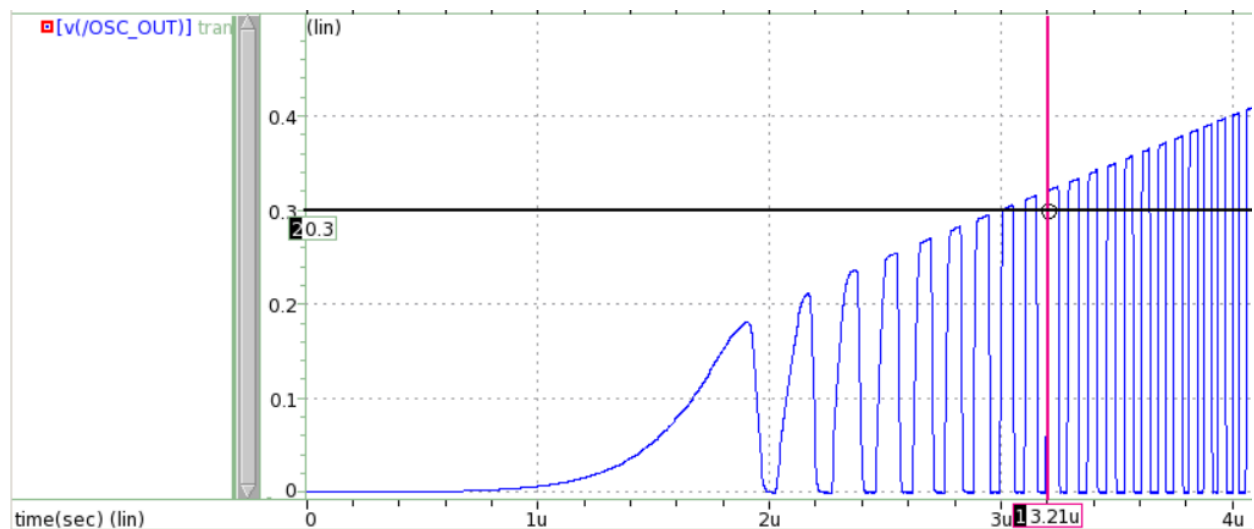


```
risetrig(v(/OSC_OUT),0.3,3)
```

```
risetrig(v(/OSC_OUT),0.3,3)
3.20607871854e-06
```

```
risetrig(v(/OSC_OUT),60,3,percent=1)
```

```
risetrig(v(/OSC_OUT),60,3,percent=1)
3.20603021534e-06
```



rmsval

Measures the RMS value of given signal, where RMS is Root Mean Square. The following

equation is for continuous waveform:
$$RMS = \sqrt{\frac{1}{x_{stop}-x_{start}} \int_{x_{start}}^{x_{stop}} [f(x)]^2 dx}$$

Syntax

```
rmsval(<signal>,[<x_start>],[<x_stop>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

[<x_start>]

(Optional) The starting value of X.

[<x_stop>]

(Optional) The ending value of X.

Errors

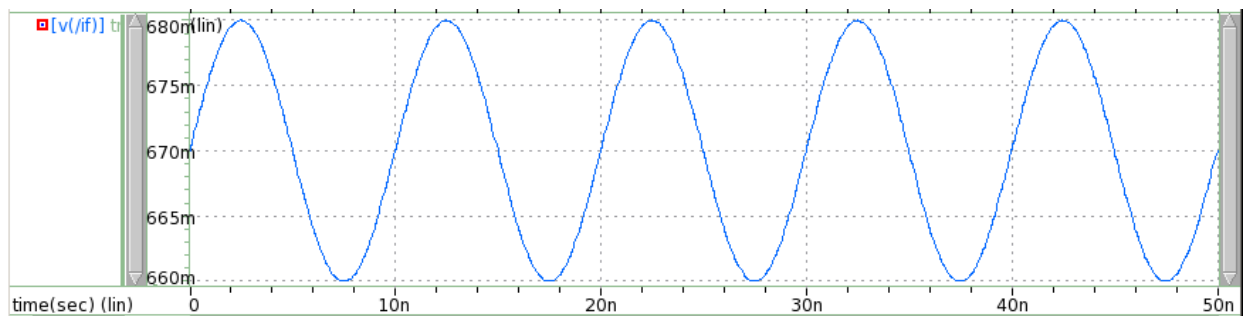
Possible error messages include:

- Signal not found
- X value out of range

Examples

```
rmsval(v(/if))  
rmsval(v(/rf_no),20n,30n)
```

rmsval(v(/if))
0.670037322106



root

Description

Computes the value of x at which f(x) equals the specified threshold.

Syntax

```
root(<signal>,<threshold>,<nth_root>)
```


Returns

An analog scalar value or an analog signal.

Additionally:

- In case of `nth_root=1`, return value is the 1st value of `x` at which `f(x)` equals the specified threshold.
- In case of `nth_root=2`, return value is the second value of `x` at which `f(x)` equals the specified threshold.
- In case of `nth_root<0`, return value is the maximum value of `x` at which `f(x)` equals the specified threshold.
- In case of `nth_root=0`, return value is a waveform of `x` at which `f(x)` equals the specified threshold.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<threshold>`

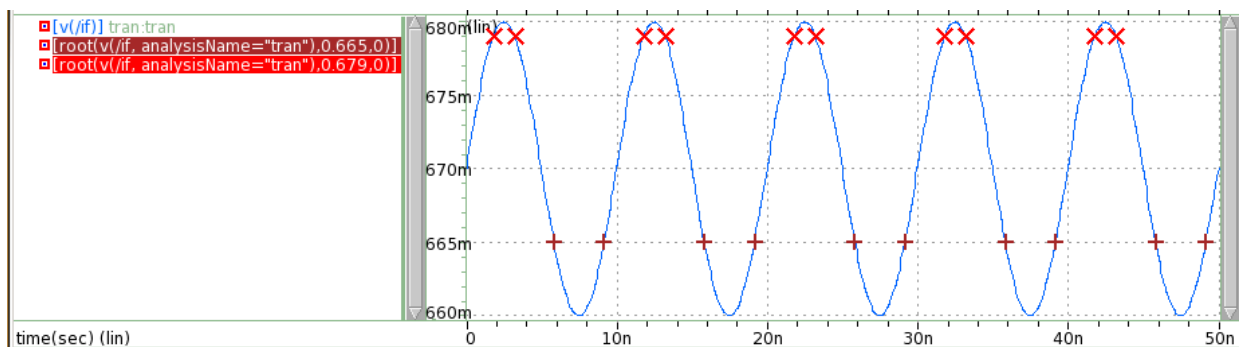
Waveform value at which the root value is to be computed.

`<nth_root>`

Root value you want to calculate.

Examples

```
root(v(/if),0.679,0)  
root(v(/if, analysisName="tran"),0.665,0)
```

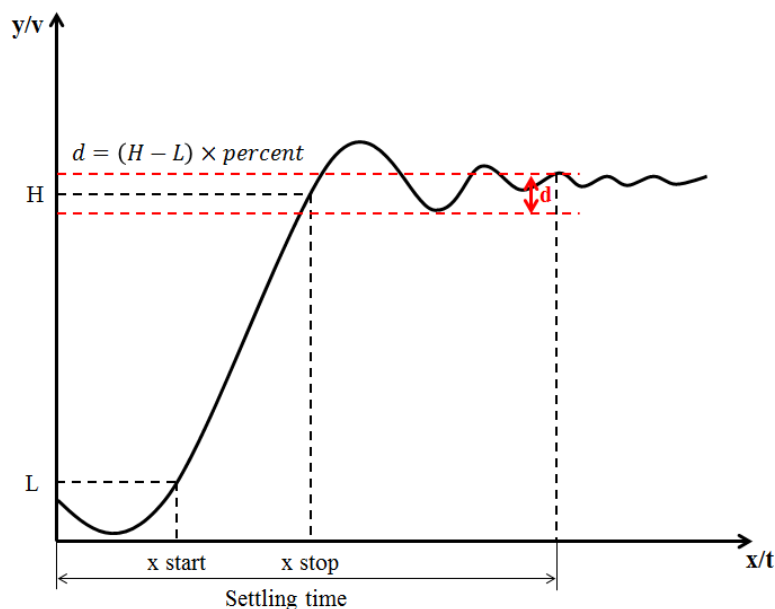


settling_t

Description

Measures the settling time of given signal. Settling time is the time required for the response curve to reach and stay within a range of certain percentage (usually 5% or 2% of the final value).

Figure 1 settling_t Example



To calculate the d value:

$$d = (\text{yvalue at } x_{\text{stop}} - \text{yvalue at } x_{\text{start}}) \times \text{settle_band\%}$$

If `half_band_error=false`, the signal is considered settled when the y values of the signals are within $(\text{yvalue at } x_{\text{stop}} + (d/2))$ and $(\text{yvalue at } x_{\text{stop}} - (d/2))$, and the tool reports the `xvalue(time)` at which this signal entered into this range of settling time.

If `half_band_error=true`, the signal is considered settled when the y values of the signals are within $(\text{yvalue at } x_{\text{stop}} + (d))$ and $(\text{yvalue at } x_{\text{stop}} - (d))$, and tool reports the `xvalue(time)` at which this signal entered into this range of settling time.

Syntax

```
settling_t(<signal>,<start>,<stop>,<settle_band>,[<x_end>],[start_by
=x/y],[stop_by=x/y],[<half_error_band>],
[<occurrences>],[<output_xaxistype>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<start>`

The starting value.

`<stop>`

The stopping value.

`<settle_band>`

A percentage value. For example, to set the settle band to 3%, set this value to 3. This is equivalent to the d value in [Figure 1](#).

`[<x_end>]`

(Optional) The ending value of X. If `<x_end>` is specified, the function calculates the settle time in a sub-window waveform.

`[<start_by>]`

(Optional) The starting value, which can be X or Y. If you specify `start_by='x'`, the starting value is for the x-axis (the x -start value in [Figure 1](#)). If you specify `start_by='y'`, the starting value is for the y-axis (the L value in [Figure 1](#)). The default value is `x`.

`[<stop_by>]`

(Optional) The stopping value, which can be X or Y. If you specify `stop_by='x'`, the stopping value is for the x-axis (the x -stop value in [Figure 1](#)). If you specify `stop_by='y'`, the stopping value is for the y-axis (the H value in [Figure 1](#)). The default value is `'x'`.

`[<half_error_band>]`

(Optional) If `half_error_band` is `false` (the default), the signal should settle between $H+(d/2)$ and $H-(d/2)$. If `half_error_band` is `true`, the user-defined `settle_band` value is equal to half of the error band. In that case, the signal should settle between $H+d$ & $H-d$.

`[<occurrences>]`

Specifies whether you want to retrieve only one occurrence of a settling time event (single), or all occurrences of settling time (multiple). The default value is single.

[<output_xaxistype>]

Specifies whether you want to output settling time data against time or cycle. Cycle numbers is the nth occurrence of the setting time event in the input waveform. This option is valid only when [<occurrences>] is set to “multiple”.

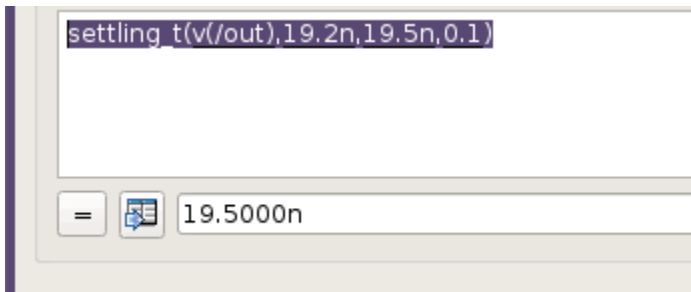
Errors

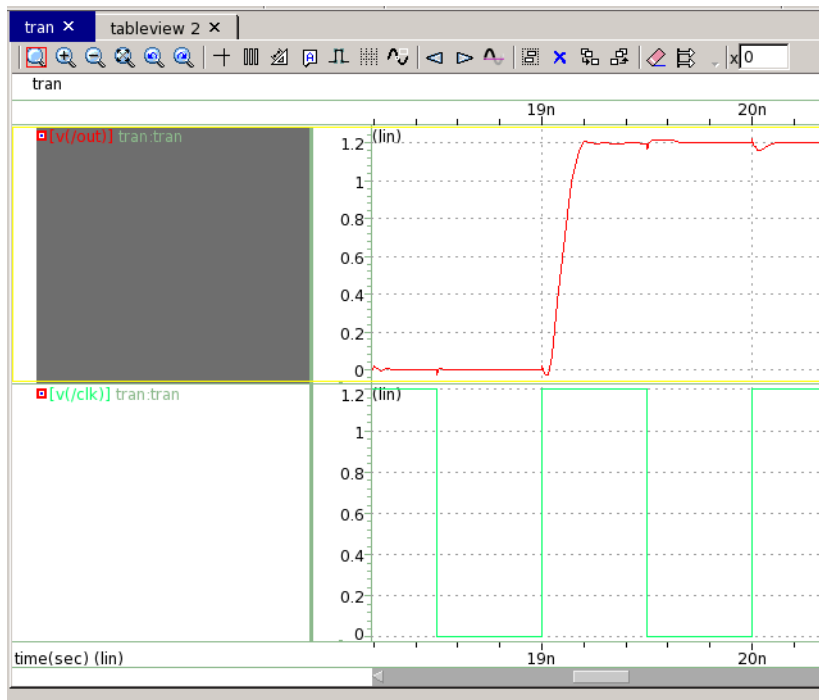
Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
settling_t(v(/out),19.2n,19.5n,0.1)
```





sfd

Description

Measures the spurious free dynamic range. SFDR is the strength ratio of the fundamental signal to the strongest spurious signal in the specified frequency band set by the Start and End frequency fields.

$$SFDR(dBc) = 20 \log \left(\frac{V_1}{\sqrt{\max(V_2^2, V_2^2, \dots, V_n^2)}} \right)$$

or:

$$SFDR(dBFS) = 20 \log \left(\frac{1}{\sqrt{\max(V_2^2, V_2^2, \dots, V_n^2)}} \right)$$

when the `full_scale` option is true.

Syntax

```
sfd(<signal>,<x_start>,<x_stop>,<num_of_points>,<signal_freq>,<band_limit>,<windowing_type>,<decimation_rate>,<bin_size>,<window_alpha>,<normalization>,<user_defined_value>,<suppress_dc>)
```

```
[<dual_sideband>],[<max_harmonic_index>],[<include_floor_noise>],[<full_scale>])
```

Arguments

<signal>

The signal to be measured.

<x_start>

The starting value of x.

<x_stop>

The stopping value of x.

<num_of_points>

The number of points needing to be measured.

<signal_freq>

The input signal frequency.

[<band_limit>]

(Optional) The limit band frequency to be set.

[<windowing_type>]

(Optional) Specifies the windowing function. The default is rectangle.

[<decimation_rate>]

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Normally, this argument is an integer or a rational fraction greater than one dividing the sampling rate.

[<bin_size>]

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

[<window_alpha>]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types.

[<normalization>]

(Optional) There are three options for this argument:

- none — FFT results are not normalized.
- normalized — FFT results are normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results are normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) When enabled, the `sfdr` function takes spurs from harmonics and noise.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
sfdr('0|v(out)', 2n, 4n, 1024, 500M)  
sfdr('0|v(out)', 2n, 4n, 1024, 500M, 550M, 'rectangle', deci=2)
```

signalPower

Measures the Signal Power in support of spectrum measurements.

Syntax

```
signalPower(<signal>, <x_start>, <x_stop>, <num_of_points>,  
<signal_freq>, [<band_limit>], [<windowing_type>],  
[<decimation_rate>]), [<bin_size>], [<window_alpha>],  
[<normalization>], [<user_defined_value>], [<suppress_dc>],  
[<dual_sideband>], [<max_harmonic_index>], [include_floor_noise],  
[<full_scale>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<x_start>

The starting value of x.

<x_stop>

The stopping value of x.

<num_of_points>

The number of points needing to be measured.

<signal_freq>

The input signal frequency.

[<band_limit>]

(Optional) The limit band frequency to be set.

[<windowing_type>]

Specifies the windowing function. The default is rectangle.

[<decimation_rate>]

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

[<bin_size>]

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

[<window_alpha>]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if `normalization` is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in SNR calculation.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

slewrates

Measures the slew rate of given signal, where slew rate is defined as the maximum rate of change of the output value. The following equation represents the slew rate calculation:

$$SR = \max\left(\left|\frac{dy}{dx}\right|\right)$$

Syntax

slewrates(<signal>,<y_init>,<y_final>,<percent>,<min>,<max>,<min_value_type>,<max_value_type>,<occurrences>,<output_xaxistype>)

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<y_init>

The initial crossing of the y_threshold.

<y_final>

The final crossing of the y_threshold.

[<percent>]

(Optional) The percentage value, which can only be 0 or 1. If specified as 1, the threshold value is considered a percentage value; if not, the threshold value is 0.

[<min>]

(Optional) Min value for percentage y init/final. Default is xmin or ymin depending on type.

[<max>]

(Optional) max value for percentage y init/final, default is xmax or ymax depending on type.

[<min_value_type>]

(Optional) Specifies whether the [<min>] value is an X-axis (x) or Y-axis value. (y). The default value is y.

[<max_value_type>]

(Optional) Specifies whether the [<max>] value is an X-axis (x) or Y-axis value (y). The default value is y.

[<occurrences>]

(Optional) Specifies whether you want to retrieve only one occurrence of a slewrate time event (single), or all occurrences of slewrate time events (multiple). The default value is single.

[<output_xaxistype>]

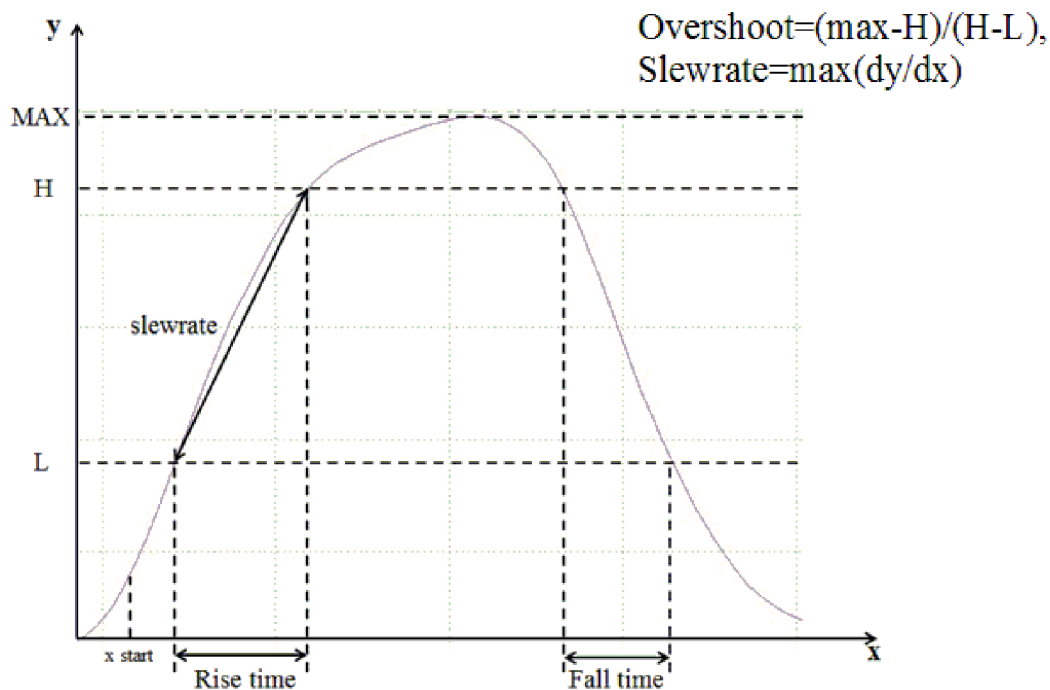
(Optional) Specifies whether you want to output slewrate data against time, or cycle. Cycle numbers refer to the nth occurrence of the slewrate event. This option is valid only when [<occurrences>] is set to multiple.

Errors

Possible error messages include:

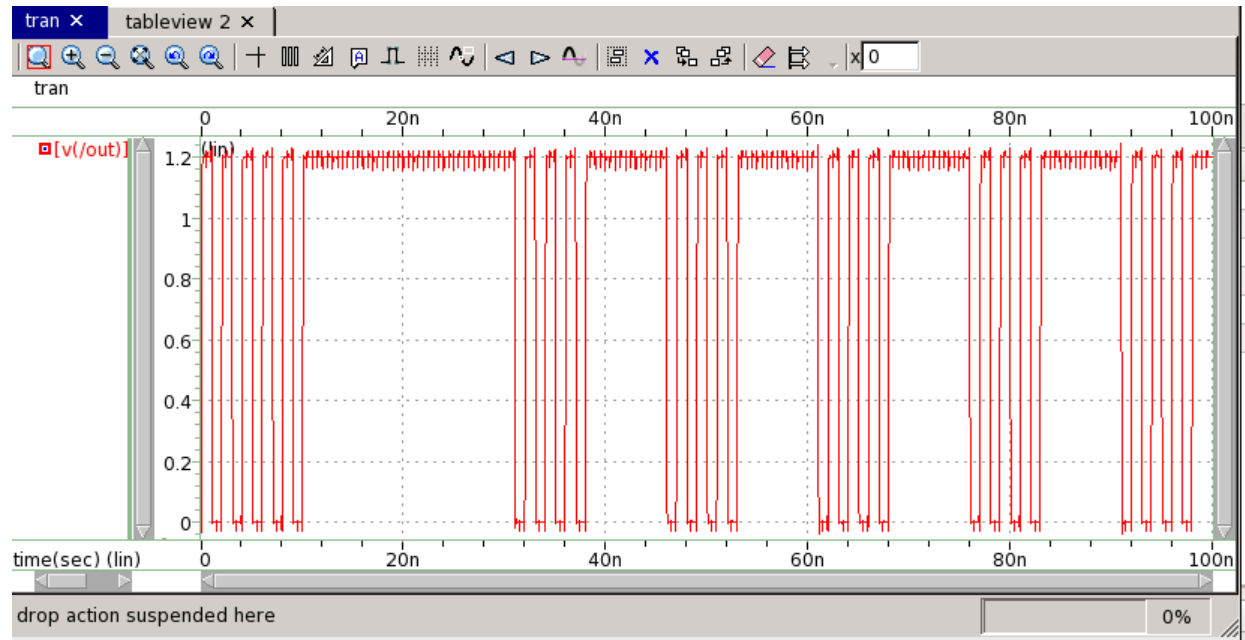
- Signal not found
- Y value out of range

Examples



Example 1:

```
slewrage('v(/out)',0.1,1.1)
```



```
slewrage(v(/out),0.1,1.1)
```

= 11.0321G

Example 2:

```
slewrage('v(/out)',10,90,1)
```

```
slewrage(v(/out),10,90,1)
```

= 10.9515G

slope

Measure the slope value of the given signal at x value.

Syntax

```
slope(<signal>,<x_value>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<x_value>

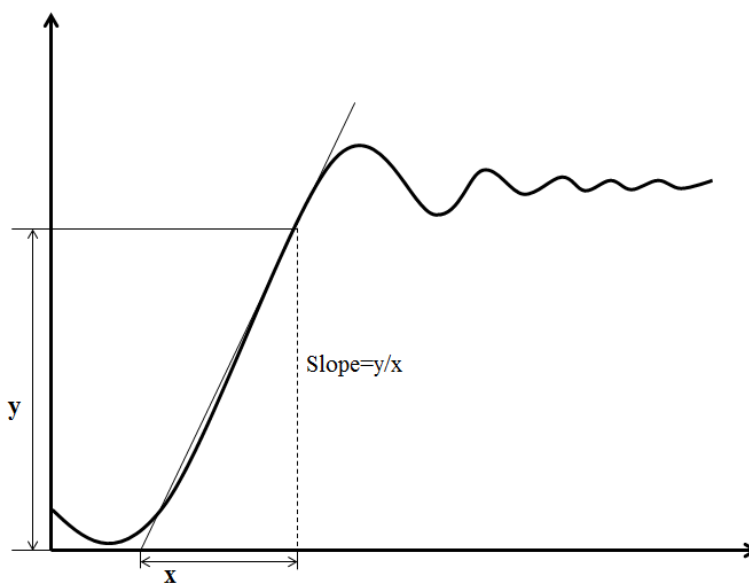
The x value to be set.

Errors

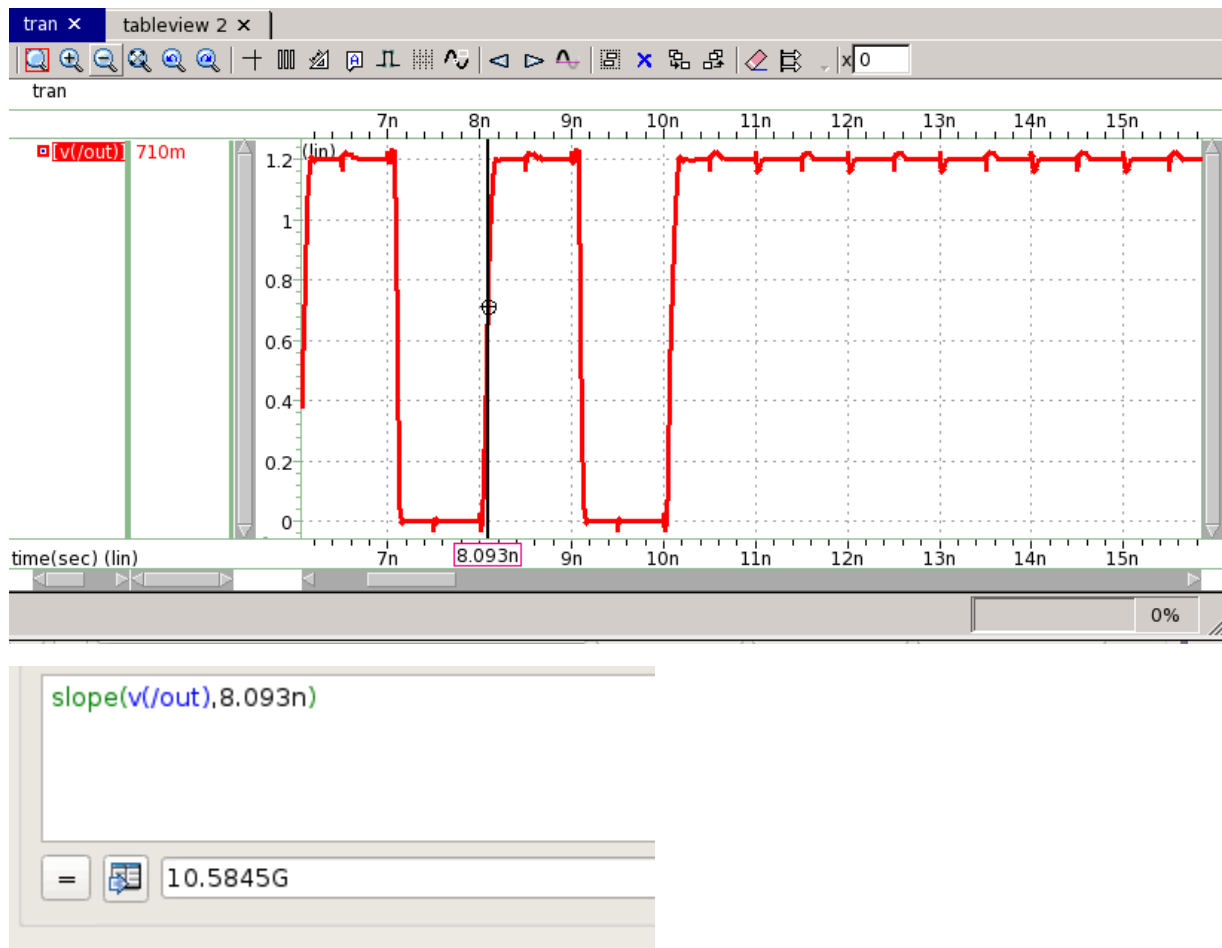
Possible error messages include:

- Signal not found
- Y value out of range

Examples



```
slope('v(/out)',8.093n)
```



snb

Measures the Noise floor/Bin in support of spectrum measurements.

Syntax

```
snb(<signal>,<x_start>,<x_stop>,<num_of_points>,<signal_freq>,<band_limit>,<windowing_type>,<decimation_rate>,<bin_size>,<window_alpha>,<normalization>,<user_defined_value>,<suppress_dc>,<dual_sideband>,<max_harmonic_index>,<include_floor_noise>,<full_scale>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

[`<band_limit>`]

(Optional) The limit band frequency to be set.

[`<windowing_type>`]

Specifies the windowing function. The default is rectangle.

[`<decimation_rate>`]

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

[`<bin_size>`]

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

[`<window_alpha>`]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if `normalization` is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in SNR calculation.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

sndr

Description

Measure the signal to noise-and-distortion ratio of a given signal (in dB units). The Fast Fourier Transform (FFT) evenly Piece-Wise-Constant (PWC) sample signals from

x_start to x_stop for n points. The algorithm of SNDR (Signal to Noise-and-Distortion Ratio) can be: $20 \log \sqrt{\frac{V_1^2}{V_2^2 + V_3^2 + V_4^2 + \dots V_n^2}}$

V_1 is a fundamental component, here is signal frequency, within the range of x_start and x_stop , among which there are n points, so $V_2 \dots V_n$ include harmonic frequency and distortion components.

Syntax

```
sndr(<signal>,<x_start>,<x_stop>,<num_of_points>,<signal_freq>,<band_limit>,<windowing_type>,<decimation_rate>,<bin_size>,<window_alpha>,<normalization>,<user_defined_value>,<suppress_dc>,<dual_sideband>,<include_floor_noise>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x .

`<x_stop>`

The stopping value of x .

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation_rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Usually this argument is an integer or a rational fraction greater than one dividing the sampling rate.

[<bin_size>]

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

[<window_alpha>]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in SNDR calculation.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
snr('0|v(out)',2n,40n,1024,500M)
snr('0|v(out)',2n,40n,1024,500M,550M,'rectangle',deci=2)
```

snr

Description

Measures the signal-to-noise ratio of given signal in dB units. The Fast Fourier Transform (FFT) evenly Piece-Wise-Constant (PWC) samples signals from `x_start` to `x_stop` for `n` points. The SNR is the ratio of the remaining harmonics to the fundamental. The following function is the algorithm for the SNR (Signal to Noise Ratio):

$$20 \log \left(\frac{V_1}{\sqrt{V_{i+1}^2 + V_{i+2}^2 + \dots + V_n^2}} \right)$$

V_1 is the fundamental component, here is signal frequency, within the range of `x_start` and `x_stop`, among which there are `n` points. Where i is the value of `max_harmonic_index`, so $V_{i+1} \dots V_n$ are remaining harmonic frequency.

Syntax

```
snr(<signal>,<x_start>,<x_stop>,<num_of_points>,<signal_freq>,[<band_limit>],[<windowing_type>],[<decimation_rate>],[<bin_size>],[<window_alpha>],[<normalization>],[<user_defined_value>],[<suppress_dc>],[<dual_sideband>],[<max_harmonic_index>],[<include_floor_noise>],[<full_scale>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

[`<band_limit>`]

(Optional) The limit band frequency to be set.

[`<windowing_type>`]

Specifies the windowing function. The default is rectangle.

[`<decimation_rate>`]

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

[`<bin_size>`]

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

[`<window_alpha>`]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if `normalization` is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in SNR calculation.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Invalid input value

- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
snr('0|v(out)',4n,8n,1024,750M)  
snr('0|v(out)',4n,8n,1024,750M,10G,deci=20)
```

snr_f

Measures the signal-to-noise ratio of a given signal (in dB units), which is the resulting waveform of a Fast Fourier Transform (FFT) signal. A similar algorithm with SNR. V_1 is the signal component in the range between `signal_from` and `signal_to`; $V_2... V_n$ are the noise components in the range of `noise_from` and `noise_to`.

$$20 \log \sqrt{\frac{V_1^2}{V_2^2 + V_3^2 + V_4^2 + \dots + V_n^2}}$$

Syntax

```
snr_f(<signal>,<signal_from>,<signal_to>,<noise_from>,<noise_to>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<signal_from>

The starting point of signal around harmonic.

<signal_to>

The stopping point of signal around harmonic.

<noise_from>

The point where noise begins.

<noise_to>

The point where noise stops.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Invalid input value

Examples

```
snr_f('0|v(out)',75M,80M,100m,10G)
```

snrh

Measures the Noise floor/rHz in support of spectrum measurements.

Syntax

```
snrh(<signal>,<x_start>,<x_stop>,<num_of_points>,  
<signal_freq>,[<band_limit>],[<windowing_type>],  
[<decimation_rate>],[<bin_size>],[<window_alpha>],  
[<normalization>],[<user_defined_value>],[<suppress_dc>],  
[<dual_sideband>],[<max_harmonic_index>],[<include_floor_noise>],  
[<full_scale>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<x_start>

The starting value of x.

<x_stop>

The stopping value of x.

<num_of_points>

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation_rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

`[<bin_size>]`

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in SNR calculation.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

stddev

Measures the standard deviation of the Y values for given signal and the X values are discarded. For discrete random variable, the standard deviation is $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \bar{Y})^2}$

Syntax

stddev(<signal/scalar>)

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

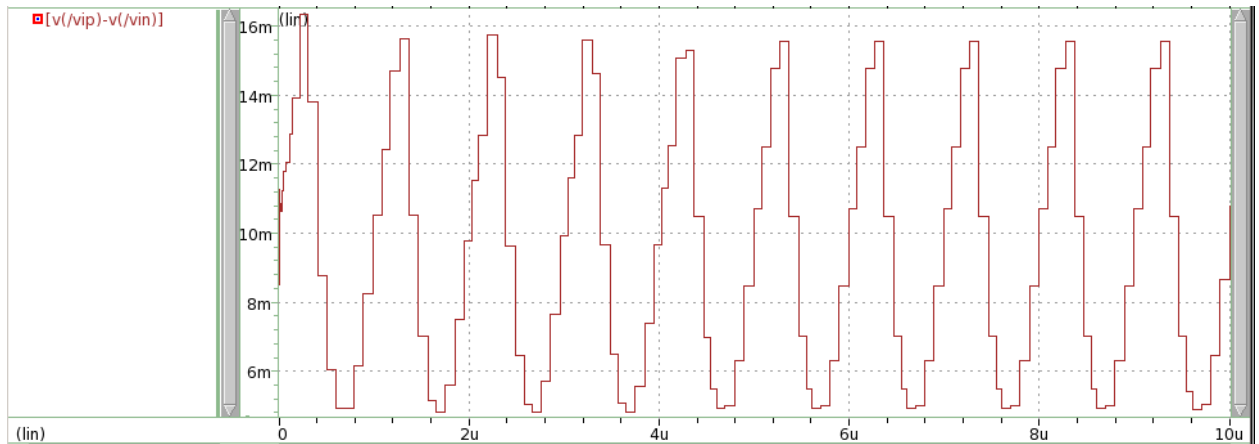
Errors

Possible error messages include:

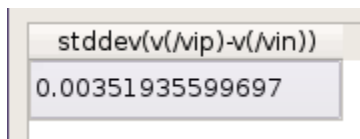
- Signal not found

Examples

stddev(v(/vip)-v(/vin))



will give the following results:



The example inputs of the arguments shown are as follows:

- Signal: $v(/vip) - v(/vin)$

sweepValueDiff

Calculates the value difference between values of sweep scalars.

Syntax

`sweepValueDiff(<signal>)`

Returns

A waveform.

Arguments

`<signal>`

The input sweep scalar signal.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

For a usage example, see SolvNetPlus article #000042278, "[How can I use the sweepValueDiff function to calculate differences in simulation results in the PrimeWave Design Environment?](#)"

thd

Measures the total harmonic distortion of a given signal (in dB units). The Fast Fourier Transform (FFT) evenly Piece-Wise-Constant (PWC) sample signals from `x_start` to `x_stop` for `n` points. The following function is the algorithm for the THD (Total Harmonic Distortion). V_1 is the fundamental component, so $V_2, \dots, V_i$ include harmonic frequency

where i is the value of `max_harmonic_index`. $20 \log \left(\frac{\sqrt{V_2^2 + V_3^2 + \dots + V_i^2}}{V_1} \right)$

Syntax

```
thd(<signal>,<x_start>,<x_stop>,<num_of_points>,<signal_freq>,<band_limit>,<windowing_type>,<harmonic_in_index>,<decimation_rate>,<bin_size>,<window_alpha>,<normalization>,<user_defined_value>,<suppress_dc>,<dual_sideband>,<max_harmonic_index>,<full_scale>,<output_type>,<all_frequencies>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of `x`.

`<x_stop>`

The stopping value of `x`.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is `rectangle`.

`[<harmonic_in_index>]`

(Optional) The maximum harmonic index flag, which can only be 0 or 1. When specified as 1, the `band_limit` value is considered as maximum harmonic index. This option is disabled if the `max_harmonic_index` value is set.

`[<decimation_rate>]`

(Optional) Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Normally, this argument is an integer or a rational fraction greater than one dividing the sampling rate.

`[<bin_size>]`

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. The default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None — FFT results are not normalized.
- Normalized — FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined — FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual-sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<full_scale>]

(Optional) If set to true, the THD is calculated with respect to full-scale. Default is false.

[<output_type>]

(Optional) Specifies the output as dB or percentage.

[all_frequencies]

When enabled, considers all frequencies.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
thd('0|v(out)', 2n, 40n, 1024, 650M)  
thd('0|v(out)', 2n, 40n, 1024, 650M, 8, 'rectangle', ish=1, deci=30)
```

variance

Measures the variance of the Y values for given signal (the X values are discarded). For

discrete random variable, the variance is $Var(Y) = \frac{1}{n} \sum_{i=1}^n (Y_i - \bar{Y})^2$

Syntax

`variance(<signal>)`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

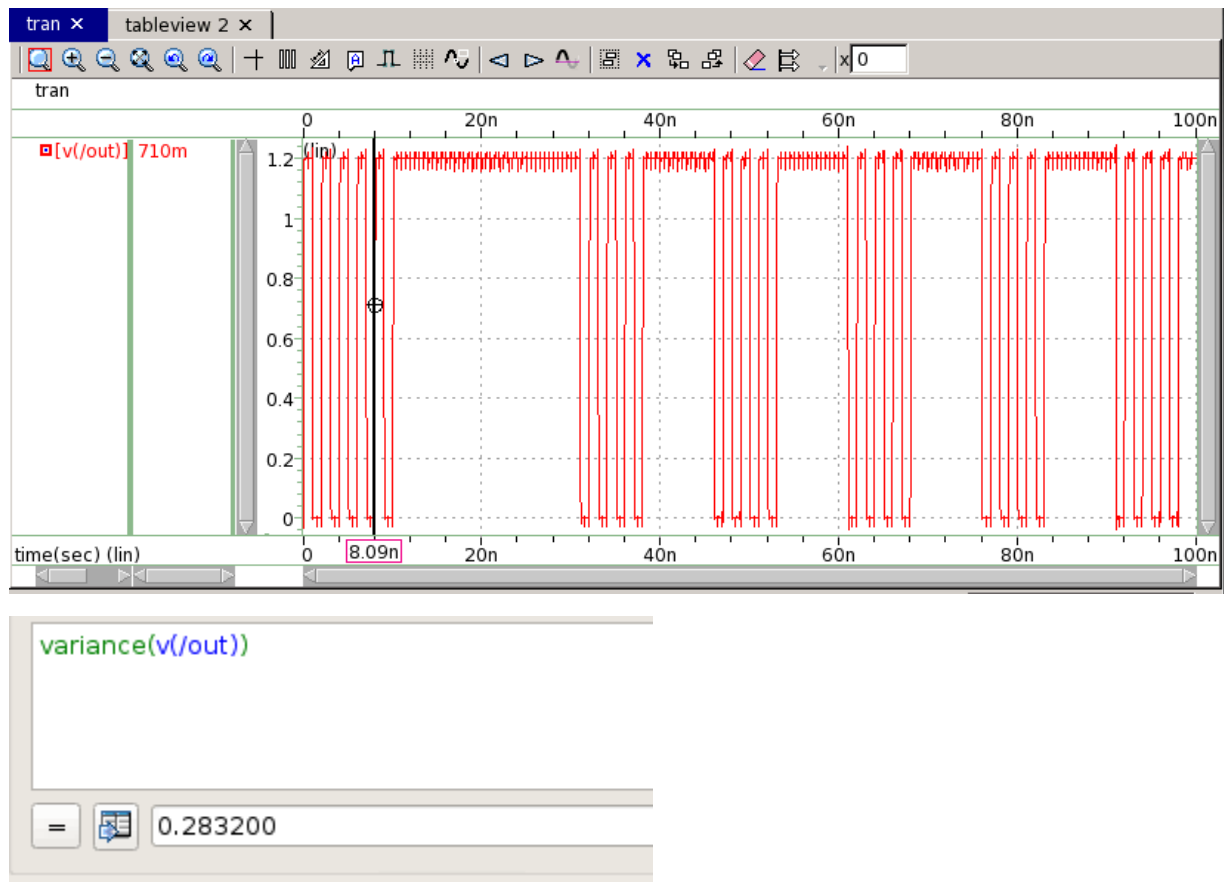
Errors

Possible error messages include:

- Signal not found

Examples

`variance('v(/out)')`



x_at_ymax

Measures the x value at the maximum y value of given signal from x_start to x_stop.

Syntax

```
x_at_ymax(<signal>, [<x_start>], [<x_stop>], [<nth_max>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

[<x_start>]

(Optional) The starting point of x.

[<x_stop>]

(Optional) The stopping point of x.

[<nth_max>]

(Optional) The nth maximum of the given signal counting from x_start to x_stop. If you enter a negative integer, it will count from x_stop to x_start.

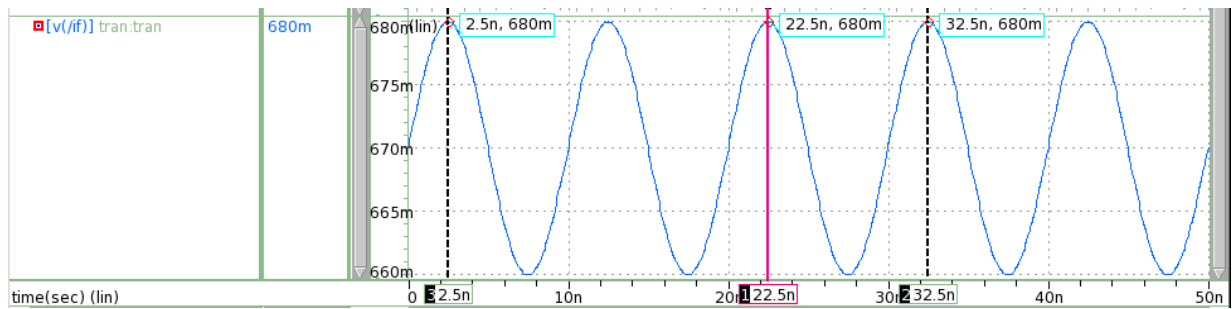
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
x_at_ymax(v(/if))  
x_at_ymax(v(/if), 5n, 35n, 2)  
x_at_ymax(v(/if), 15n, 35n, -1)
```



`x_at_ymax(v(/if))` will return the following:

```
x_at_ymax(v(/if))
2.5e-09
```

The example inputs of the arguments shown are as follows:

- Signal: `v(/if)`

`x_at_ymax(v(/if), 5n, 35n, 2)` will return the following:

```
x_at_ymax(v(/if), 5n, 35n, 2)
2.25e-08
```

The example inputs of the arguments are shown are as follows:

- Signal: `v(/if)`
- X start: `5n`
- X stop: `35n`
- Nth maximum = `2`

`x_at_ymax(v(/if), 15n, 35n, -1)` will return the following:

```
x_at_ymax(v(/if), 15n, 35n, -1)
3.25e-08
```

The example inputs of the arguments shown are as follows:

- Signal: `v(/if)`
- `x_start`: `5n`

- `x_stop`: 35n
 - Nth maximum = -1 (This counts the maximum from `x_stop` to `x_start` in reverse direction)
-

x_at_ymin

Measures the x value at the minimum y value of given signal from `x_start` to `x_stop`.

Syntax

```
x_at_ymin(<signal>, [<x_start>], [<x_stop>], [<nth_min>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The stopping point of x.

`[<nth_min>]`

(Optional) The nth minimum of the given signal counting from `x_start` to `x_stop`. If you enter a negative integer, it will count from `x_stop` to `x_start`.

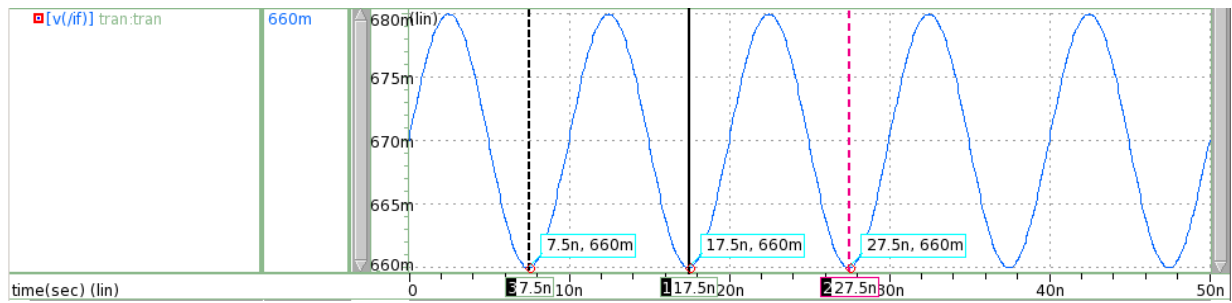
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
x_at_ymin(v(/if))  
x_at_ymin(v(/if), 5n, 35n, 2)  
x_at_ymin(v(/if), 15n, 35n, n=-1)
```



`x_at_ymin(v(/if))` will return the following:

```
x_at_ymin(v(/if))
7.5e-09
```

The example inputs of the arguments shown are as follows:

- Signal: `v(/if)`

`x_at_ymin(v(/if), 5n, 35n, 2)` will return the following:

```
x_at_ymin(v(/if), 5n, 35n, 2)
1.75e-08
```

The example inputs of the arguments shown are as follows:

- Signal: `v(/if)`
- X start: `5n`
- X stop: `35n`
- Nth minimum = `2`

`x_at_ymin(v(/if), 15n, 35n, n=-1)` will return the following:

```
x_at_ymin(v(/if), 15n, 35n, n=-1)
2.75e-08
```

The example inputs of the arguments shown are as follows:

- Signal: `v(/if)`
- X start: `5n`

- X stop: 35n
 - Nth minimum = -1 (This counts the minimum from `x_stop` to `x_start` in reverse direction)
-

xmax

Measures the maximum x value of a given signal.

Syntax

```
xmax(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

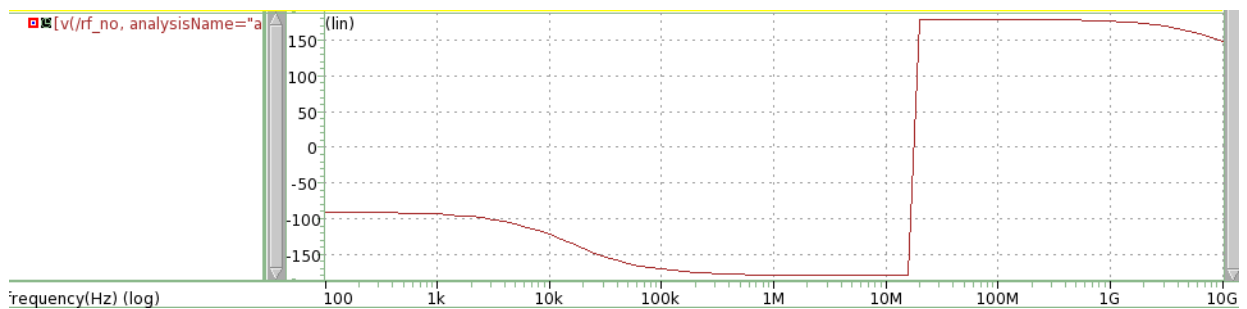
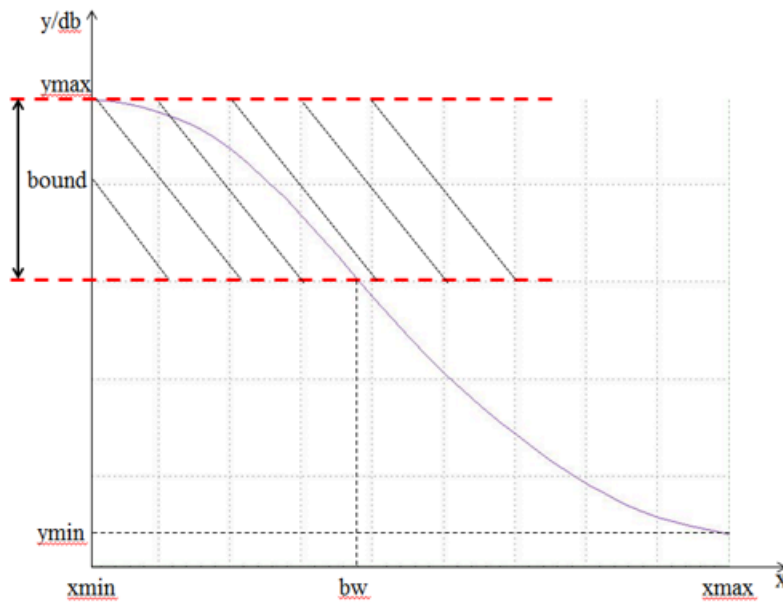
Errors

Possible error messages include:

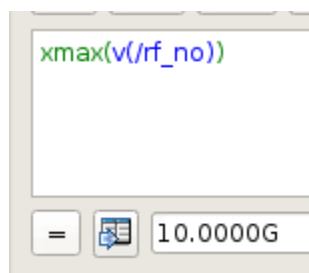
- Signal not found

Examples

```
xmax(v(/rf_no))
```



`xmax(v(/rf_no))` will return the following:



The example inputs of the arguments are as follows:

- Signal: `v(/rf_no)`

xmin

Measures the minimum x value of a given signal.

Syntax

```
xmin(<signal>)
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

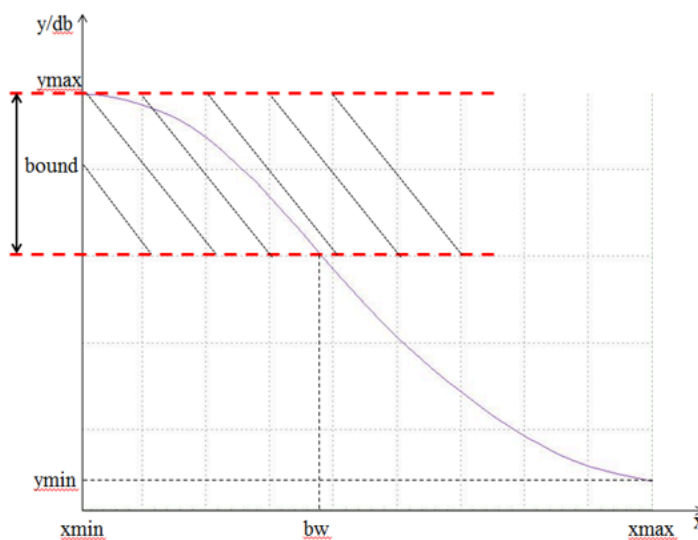
Errors

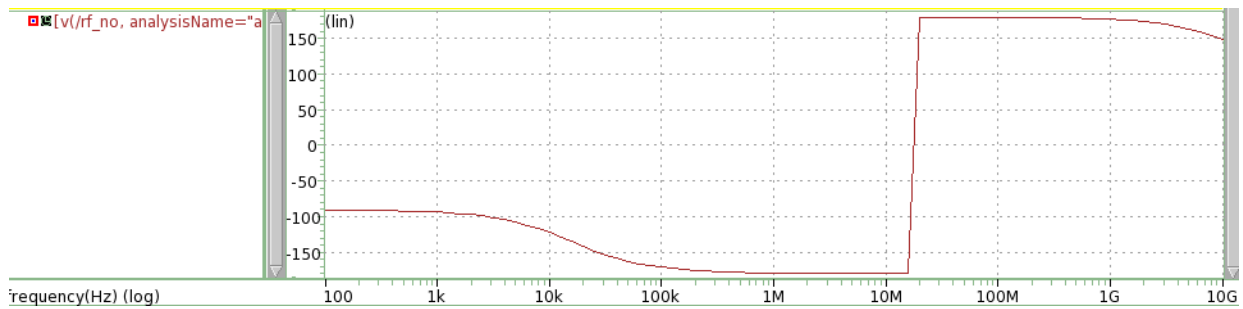
Possible error messages include:

- Signal not found

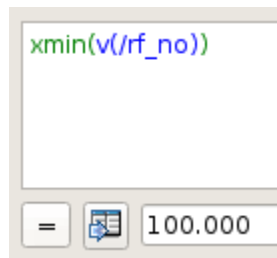
Examples

```
xmin(v(/rf_no))
```





`xmin(v(/rf_no))` will return the following:



The example inputs of the arguments shown are as follows:

- Signal: `v(/rf_no)`

yield

Measures the count of the value specified by `spec_min` and `spec_max` and returns the value for $(\text{valid_count}/\text{total_count}) \times 100$.

Where,

`total_count` = total number of input data point count

`valid_count` = `total_count` - `invalid_count`

If `spec_min` is defined and any one data point in the input (signal/measurement) < `spec_min`, or if `spec_max` is defined and the value > `spec_max`, then it is an `invalid_count`.

Also all NaN values and errors are viewed as `invalid_count`.

Syntax

```
yield(signal<string>,spec_min<string>,spec_max<string>)
```

Arguments

`signal`

The signal or measurement to be calculated.

`spec_min`

(Optional) The minimum spec value.

`spec_max`

(Optional) The maximum spec value.

ymax

Measures the maximum y value of a given signal.

Syntax

`ymax(<signal/scalar>, [<x_start>], [<x_stop>])`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

`[<x_start>]`

(Optional) Specifies the starting x value. The default is the signal start time.

`[<x_stop>]`

(Optional) Specifies the x value end time. The default is the signal end time.

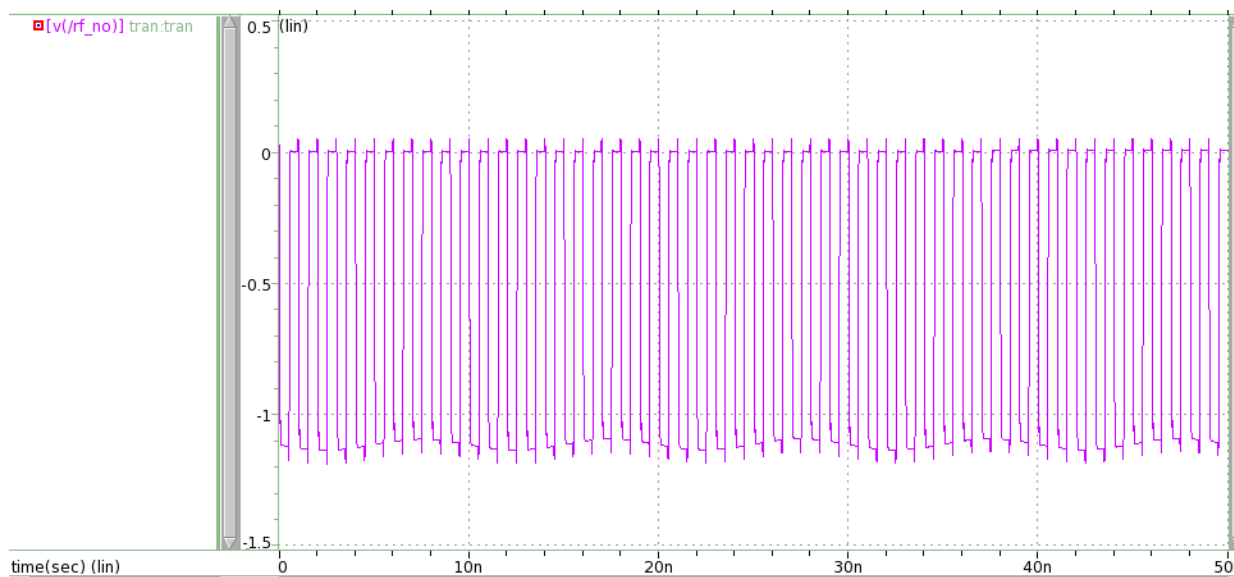
Errors

Possible error messages include:

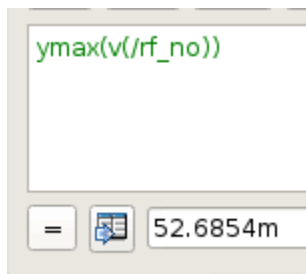
- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
ymax(v(/rf_no))  
ymax(v(/rf_no), 0, 2n)
```



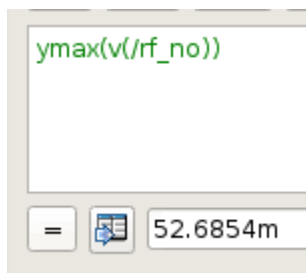
`ymin(v(/rf_no))` will return the following:



The example inputs of the arguments shown are as follows:

- Signal: `v(/rf_no)`

`ymin(v(/rf_no), 0, 2n)` will return the following:



The example inputs of the arguments shown are as follows:

- Signal: `v(/rf_no)`
- X_start: 0
- X_stop: 2n

ymin

Measures the minimum y value of a given signal.

Syntax

`ymin(<signal/scalar>, [<x_start>], [<x_stop>])`

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

[<x_start>]

(Optional) Specifies the starting x value. The default is the signal start time.

[<x_stop>]

(Optional) Specifies the x value end time. The default is the signal end time.

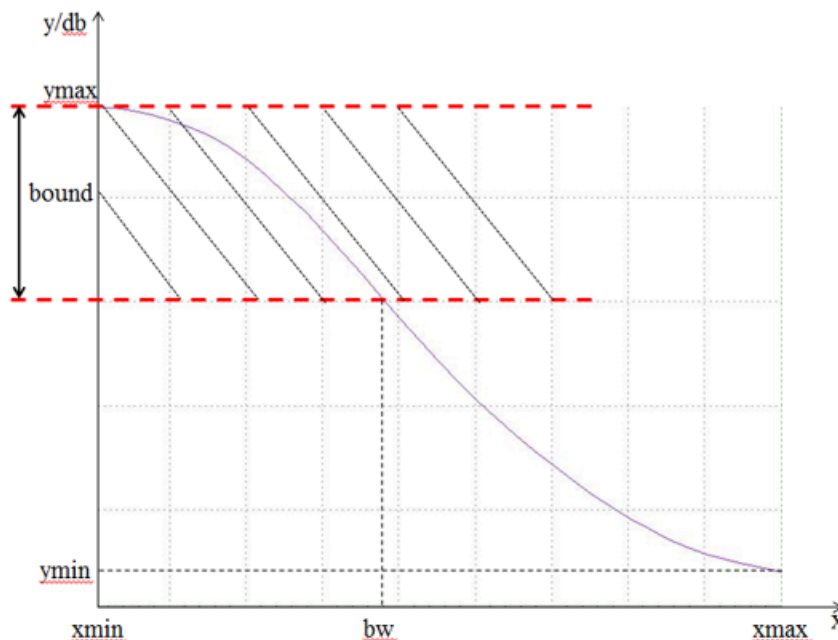
Errors

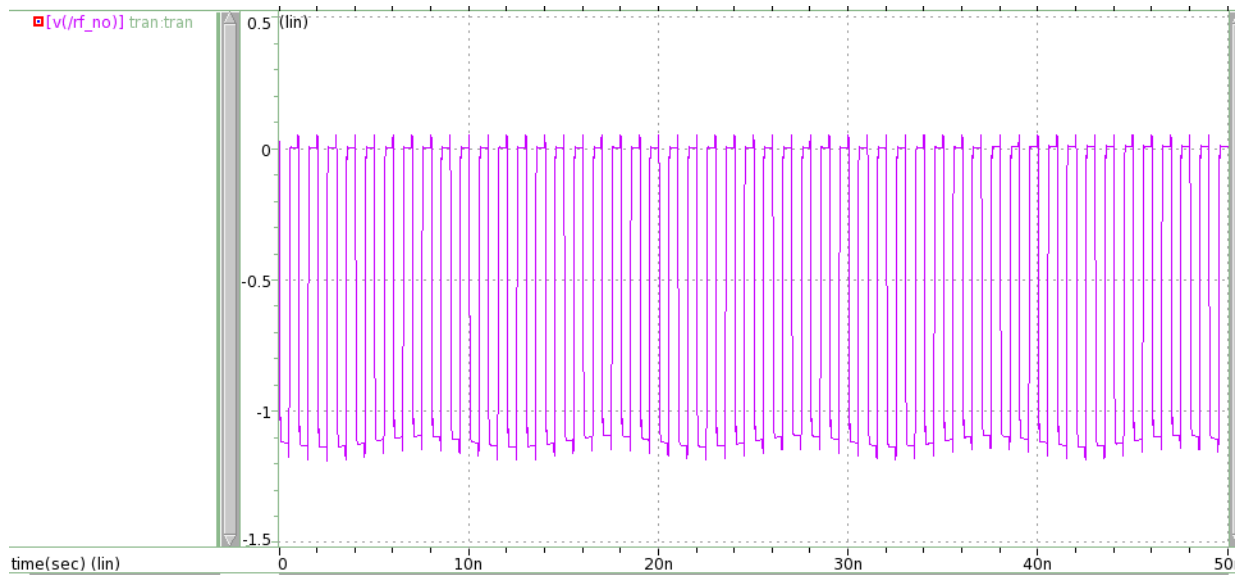
Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

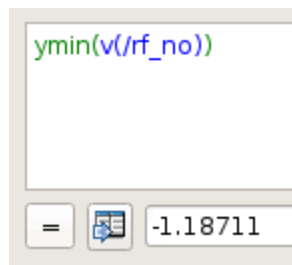
Examples

```
ymin(v(/rf_no))  
ymin(v(/rf_no),0,2n)
```





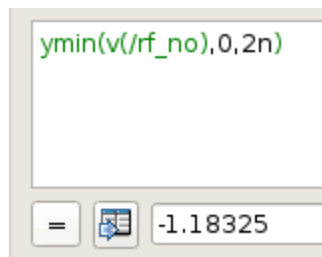
`ymin(v(/rf_no))` will return the following:



The example inputs of the arguments shown are as follows:

- Signal: `v(/rf_no)`

`ymin(v(/rf_no), 0, 2n)` will return the following:



The example inputs of the arguments shown are as follows:

- Signal: `v(/rf_no)`
- X_start: 0
- X_stop: 2n

yvalue

Measures the y value of a given signal at x.

Syntax

```
yvalue(<signal/scalar>,<x_value>,[<method>],  
[<occurrences>],[<period>],[<output_xAxisType>],[<extrapolate>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_value>`

The input x value.

`[<method>]`

(Optional) Specifies the interpolation method. The default is linear. If set to `round_down`, the point value is the lower bound of an interval that contained it.

`[<occurrences>]`

(Optional) Specifies the result to be only one occurrence of an interpolated value for the given waveform (`single`) or all interpolated values for the given waveform (`multiple`), which you can later plot or print. The default is `single`.

`[<period>]`

(Optional) Specifies the time interval to calculate the next Y value.

`[<output_xAxisType>]`

(Optional) Specifies the X axis type of the result waveform. The value can be `x` or `idx`.

`[<extrapolate>]`

(Optional) When enabled, ignores the boundary and always returns a value.

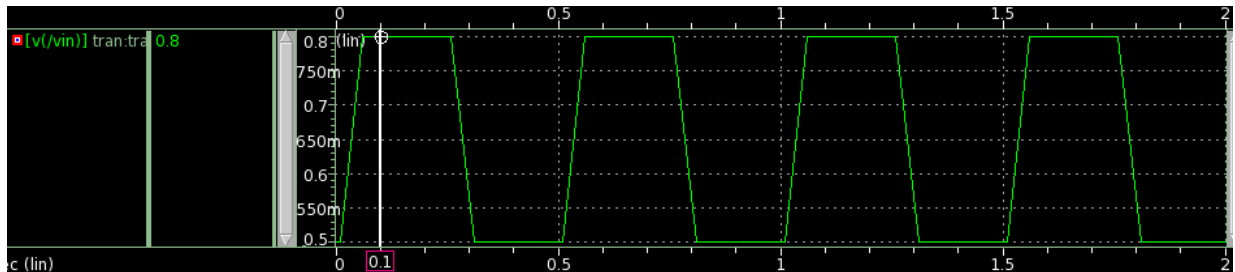
Errors

Possible error messages include:

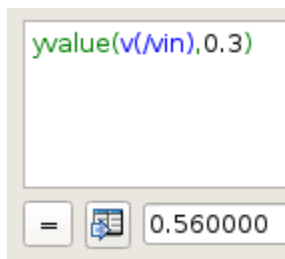
- Signal not found
- X value out of range

Examples

```
yvalue(v(/vin),0.3)  
yvalue(v(/vin),0.3,occu=multiple,period=0.1)
```



`yvalue(v(/vin),0.3)` will return the following:



The example inputs of the arguments shown are as follows:

- Signal: `v(/in)`
- X_value: 0.3

Waveform Functions

This topic contains information on how to use the supported waveform functions.

The following sections summarize the waveform functions available in the Calculator. Waveform functions must operate on waveforms (or derived waveforms). They always return new waveforms.

Function	Description
a2d	Converts the input signal from analog to digital.
average	Calculates the running average waveform of a given signal in the specified range.
clip	Generates a new waveform whose x range is limited to the given x_start and x_stop values. The y range is limited to the given y_min and y_max values.
cPwrContour	Generates contours with the same power for Z-Smith.
create_bus	Creates a bus signal with given name and up to 31 digital signals.
create_eye	Creates an eye object which can be plotted as an eye diagram.
crossings	Calculates the crossing points between signal1 and signal2 or a given ylevel value.
d2a	Converts the input signal from digital to analog.
dBc2V2	Converts phase noise signal from dBc/Hz to V ² /Hz. Only works for waveform files that contain carrier magnitude information.
delayvst	Measures the time difference versus the time between two signals at the specified switching edges from x_start to x_stop.
derivative	Calculates the derivative waveform of given signal.
dft	Measures the Fast-Fourier-Transform (FFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.
dftbb	Computes the fast Fourier transform (FFT) of a complex signal.
dnl	Computes the differential non-linearity of a transient simple or parametric waveform.
duty_vs_t	Measures the duty cycle versus time.
fallvst	Measures the time difference versus the time of the given signal starting from high y_level to low y_level within the range of x_start and x_stop.
fft	Measures the Fast-Fourier-Transform (FFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.
flip	Reverses the signal along the y-axis.
fourEval	Evaluates the Fourier series represented by the buffer expression.
func	Plots the waveform of signal1 versus signal2 for the two specified signals.
fvst	Calculates the frequency versus time for the given signal.

Function	Description
gdelay	Measures the group delay waveform of a given signal.
histo	Generates a histogram waveform for the input signal.
hold_times	This is used to plot a waveform that contains all of the hold time data points.
hold_times2	Plots a waveform containing all of the hold time data points.
ift	Calculates the Inverse Fourier transform (IFT) time domain waveform of the input frequency domain signal.
inl	Computes the integral non-linearity of a transient simple or parametric waveform.
integral	Measures the running integral waveform of the input signal.
ipnhVRICurves()	N dB intercept point VRI Curves.
join	Generates a joined waveform from signal1 and signal2, which depends on the starting x-axis value of the two signals.
jvst_freq	Calculates the jitter versus time given the frequency of an ideal clock.
jvst_ideal	Calculates the jitter versus time with an ideal clock.
jvst_llse	Calculates the jitter versus time with an estimated frequency of the ideal clock using LLSE (Linear Least Squares Estimator).
jvst_refsig	Calculates the jitter versus time using a reference signal as the clock.
loadpull	Generates load pull contour of waveform PSS analysis.
median	Measures the smoothed waveform of the given signal using median value method.
parametric	Gets the parametric plots of measure versus sweeping param_name.
pareto_r	Calculates the Pareto regression result for the s signal and the parameter file.
pareto_r2	Calculates the Pareto regression squared result for the s signal and the pfile parameter file.
pareto_s	Calculates the Pareto sensitivity result for the s signal and the pfile parameter file.
pavg	Calculates the average value of signal based on the given period.
peak	Calculates the peak of a given signal.

Function	Description
phasenoise	Calculates the phase noise of a given signal.
prms	Calculates the root-mean-square value of signal based on the given period.
psd	Power spectral density. Describes how the power (or variance) of a time series (signal) is distributed with frequency.
psdbb	Power spectral density baseband. Returns an estimate for the power spectral density of a waveform1+j * waveform2.
pstddev	Calculates the standard deviation value of signal based on the given period.
pwc	Measures the piece-wise constant value of a signal.
pzFreqRealFilter()	Returns a filtered pole or zero signal from the input poles/zeros signal.
reflectionContour	Constant reflection coefficient contour.
repeat	Creates a new waveform by repeating the given period signal for a specified number of times.
risevst	Calculates the time difference versus the time of the given signal between x_start and x_stop from the low threshold crossing to the high threshold crossing.
sample	Creates a new waveform which is a sample of a given signal starting from x with period for a given number of times.
setup_times	Generates a new waveform containing all of the setup time data points.
setup_times2	Generates a new waveform that contains all of the setup time data points.
signalAtFreq	Calculates the signal at frequency.
signalcompare	Compares the waveform outputs of the simulation.
slewratesvst	Calculates the slew rate versus time for the given signal within the y_initial and y_final.
smooth	Calculates the smoothed waveform from the given signal.
spectralPower	Calculates the spectral power for the specified current waveform and voltage waveform.
sPowerDiffVI	Calculates spectral power with two nodes voltage and current signals.
sPowerDiffVR	Calculates spectral power with two nodes voltage and resistance.
swapsweep	Swaps the x-axis value with the specified sweep variable.

Function	Description
sweepavg	Calculates the average waveform of a multi-trace sweep waveform.
sweepmax	Calculates the maximum bound waveform of a multi-trace sweep.
sweepmin	Calculates the minimum bound waveform of a multi-trace sweep.
tangent	Calculates a line that passes through x and y coordinates and the specified slope.
text2wave	Reads in a text file and plots the waveform.
trace	Returns a trace from a multi-trace of signal s based on the parameter condition parm_con.
v2todBc	Converts phase noise from V^2/Hz to dBc/Hz.
window_average	Calculates the window average.
x_at_y	Plots the crossing points of a given signal on a specified y_threshold.
xscale	Plots a new waveform identical to the given signal with the x component multiplied by a scaling factor.
xshift	Plots a new waveform identical to the given signal but shifted to the right by x delay.
xval	Measures the x value of the given signal and creates a new waveform equivalent to $y=x$.
yysi	Creates a new waveform with a y value versus a data point index signal.

a2d

Converts the input signal from analog to digital.

Syntax

```
a2d(<signal>,<low_bounds>,<high_bounds>,[<num_of_bits>],
    [<time_step>],[<x_start>],[<x_stop>],
    [<single_threshold>],[<delay_to_x>])
```

Returns

A logic signal.

Arguments

<signal>

The analog signal to input.

<low_bounds>

The low threshold of y.

<high_bounds>

The high threshold of y.

[<num_of_bits>]

(Optional) Number of bits to be set.

[<time_step>]

(Optional) The value of each time_step.

[<x_start>]

(Optional) The starting point of x.

[<x_stop>]

(Optional) The ending point of x.

[<single_threshold>]

(Optional) Specifies the center threshold of y. This argument conflicts with low_bounds and high_bounds.

[<delay_to_x>]

(Optional) Specifies time transferred to x.

Errors

Possible error messages include:

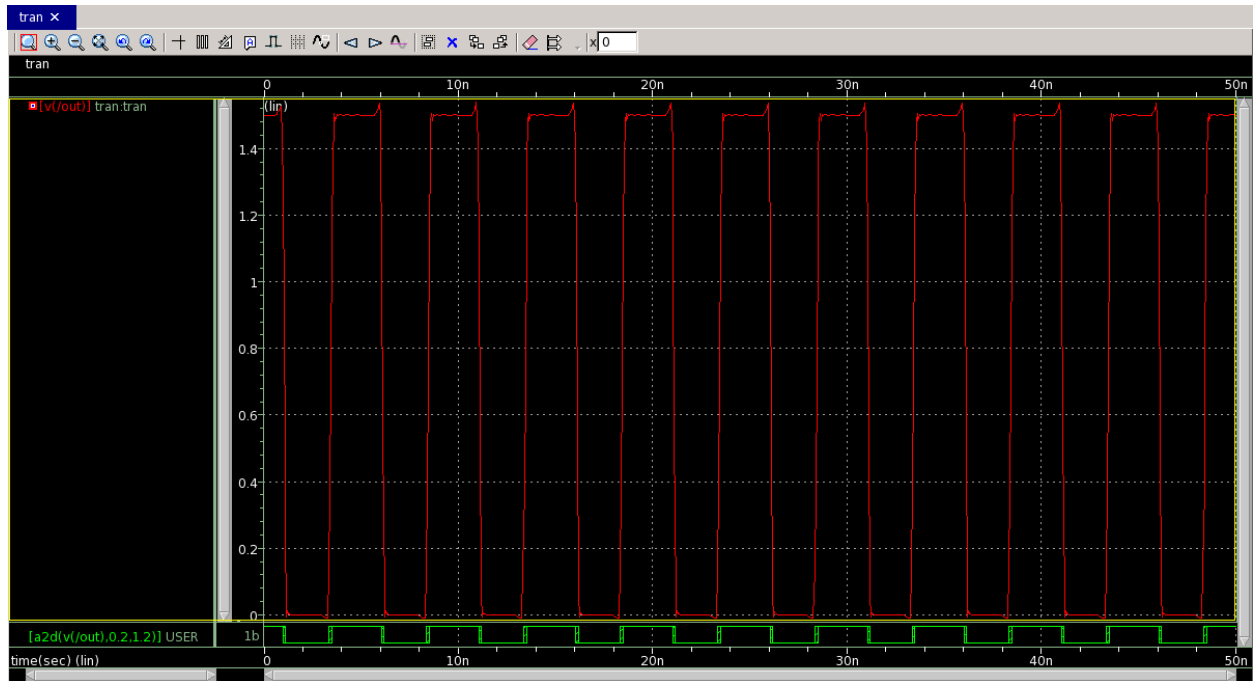
- Signal not found
- Y value out of range
- Bit number can't be greater than 32
- X value out of range

Examples

```
a2d('v(/out)',0.2,1.2);  
a2d('v(/out)',0.2,1.2,5,2n,0,10n)
```

The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 1:



The example inputs of the arguments are as follows:

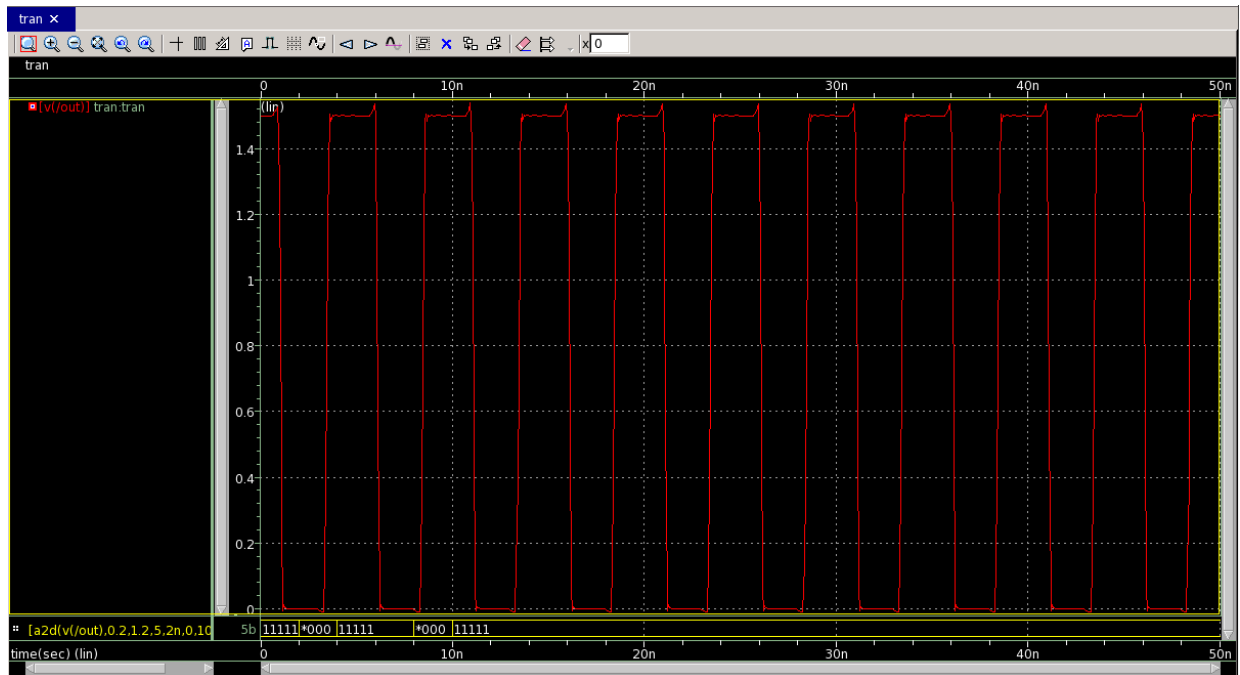
- Signal: v(out)
- Low bound: 0.2
- High bound: 1.2

The function expression is as follows:

```
a2d(v(out),0.2,1.2)
```

The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 2:



The example inputs of the arguments are as follows:

- Signal: v(/out)
- Low bound: 0.2
- High bound: 1.2
- Number of Bits: 5
- Time Step: 2n
- X start: 0
- X stop: 10n

The function expression shows as follows:

```
a2d(v(/out),0.2,1.2,5,2n,0,10n)
```

average

Calculates the running average waveform of a given signal in the specified range.

Syntax

```
average(<signal>, [<x_start>], [<x_stop>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

[<x_start>]

(Optional) The starting value of x.

[<x_stop>]

(Optional) The stopping value of x.

Errors

Possible error messages include:

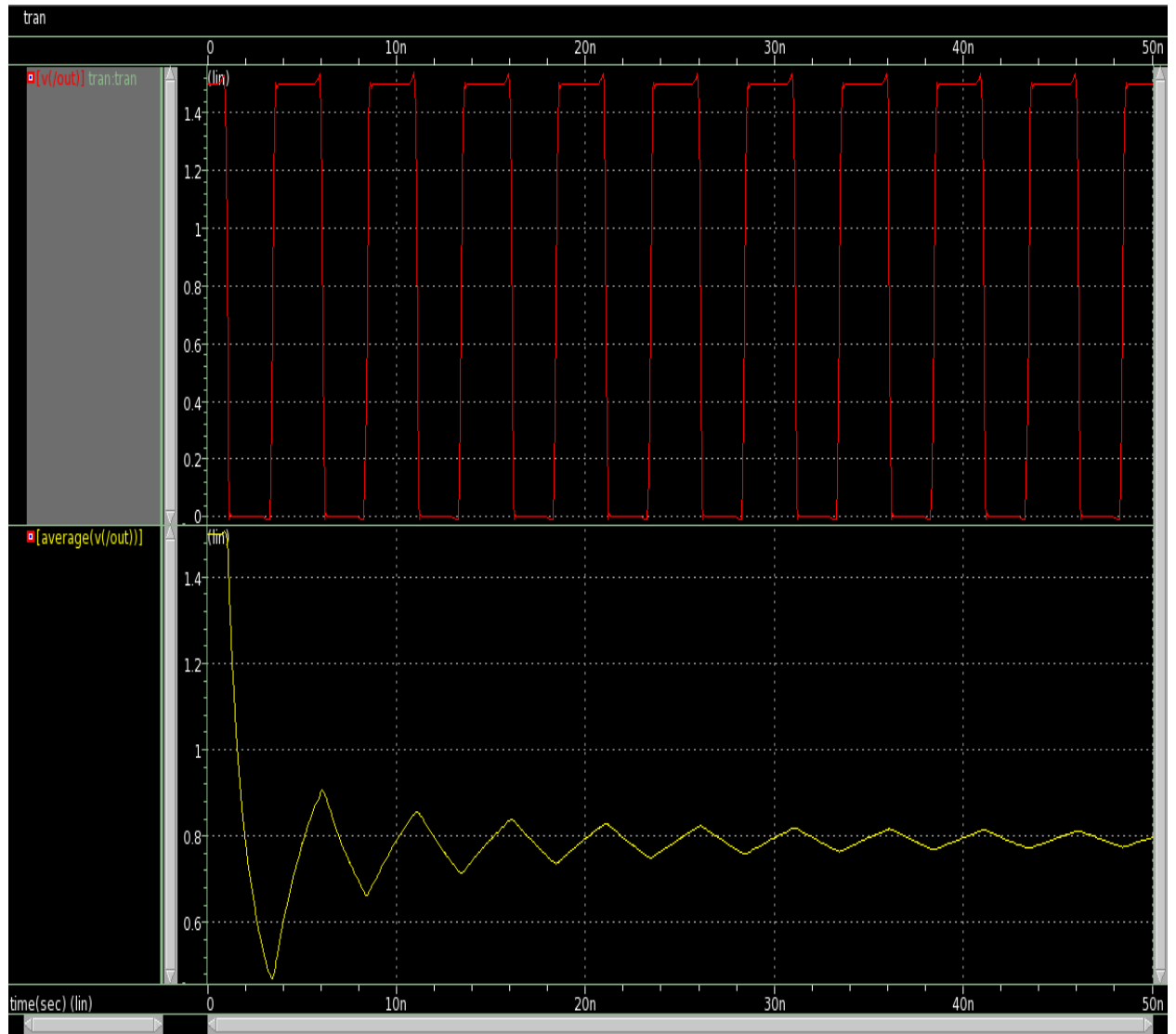
- Signal not found
- X value out of range

Examples

```
average(v(/out))  
average(v(/out), 20n, 30n)
```

The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 1:



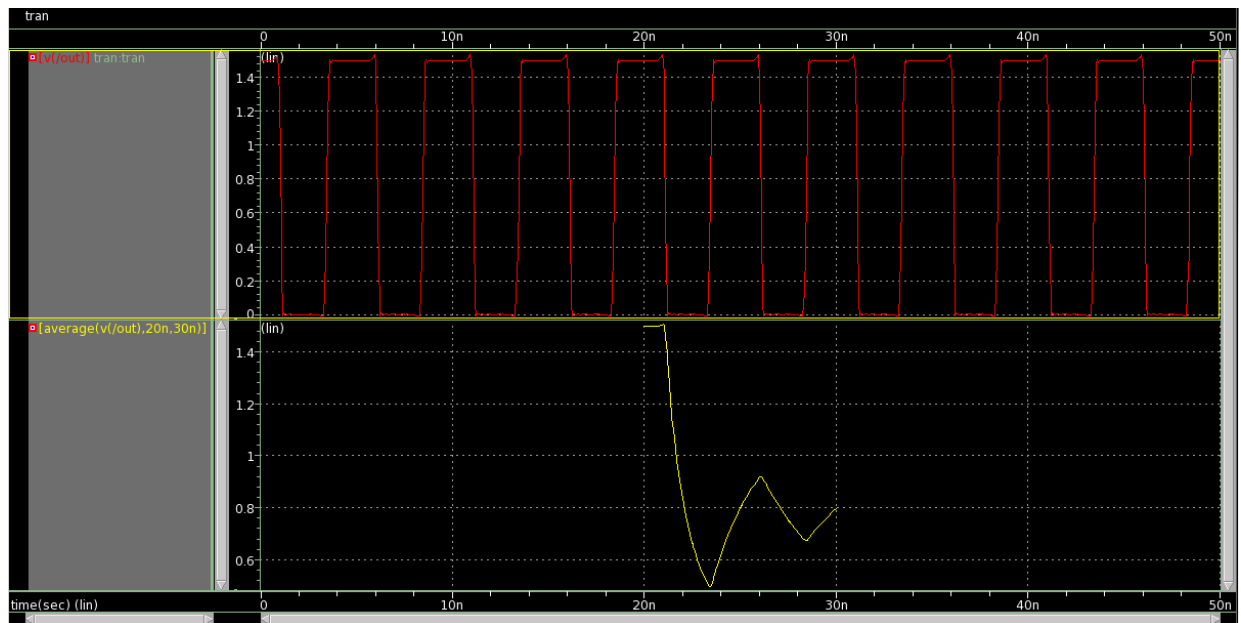
The example inputs of the arguments are as follows:

- Signal: `v (/out)`

The average expression is as follows:

`average (v (/out) )`

Example 2:



The example inputs of the arguments are as follows:

- Signal: `v(/out)`
- X start: `20n`
- X stop: `30n`

The function expression shows as follows:

```
average(v(/out),20n,30n)
```

clip

Generates a new waveform whose x range is limited to the given `x_start` and `x_stop` values. The y range is limited to the given `y_min` and `y_max` values.

Syntax

```
clip(<signal>,[<x_start>],[<x_stop>],[<y_min>],[<y_max>],[<percent>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be clipped.

[<x_start>]

(Optional) The point of x where the clip starts.

[<x_stop>]

(Optional) The value of x where the clip stops.

[<y_min>]

(Optional) The low threshold of y.

[<y_max>]

(Optional) The high threshold of y.

[<percent>]

(Optional) The value can be 0 or 1. If the value is specified as 1, the y_threshold is considered as a percentage value. If the value is specified as 0, the y_threshold is not considered a percentage value.

Errors

Possible error messages include:

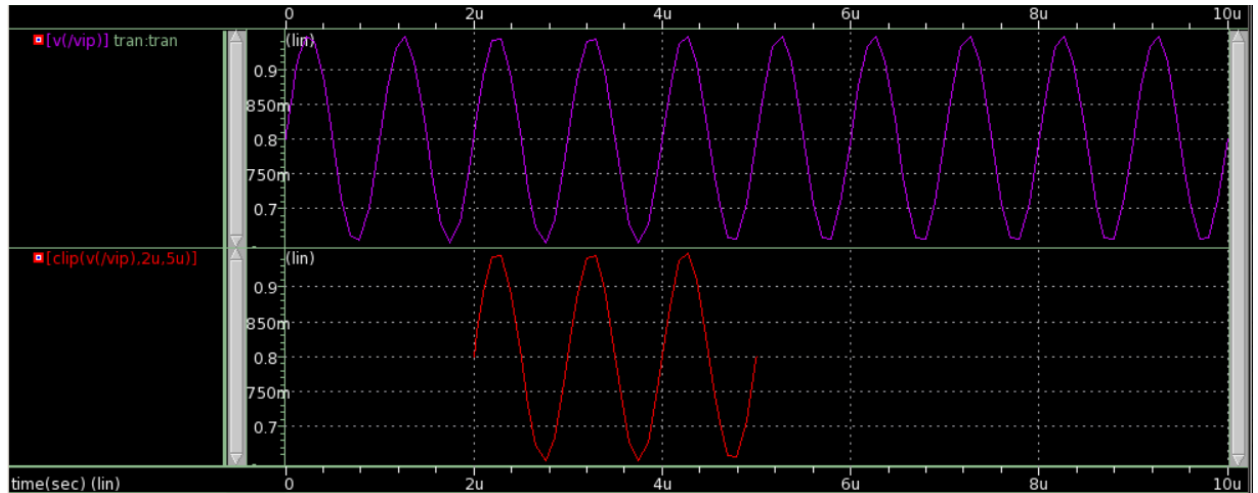
- Signal not found
- X value out of range
- Y value out of range

Examples

```
clip(v(/vip), 2u, 5u);  
clip(v(/vip), y1=0.7, y2=0.8);  
clip(v(/vip), y2=75, percent=1)
```


The following examples show how the function works on the input waveform $v(out)$ for different input parameters:

Example 1:



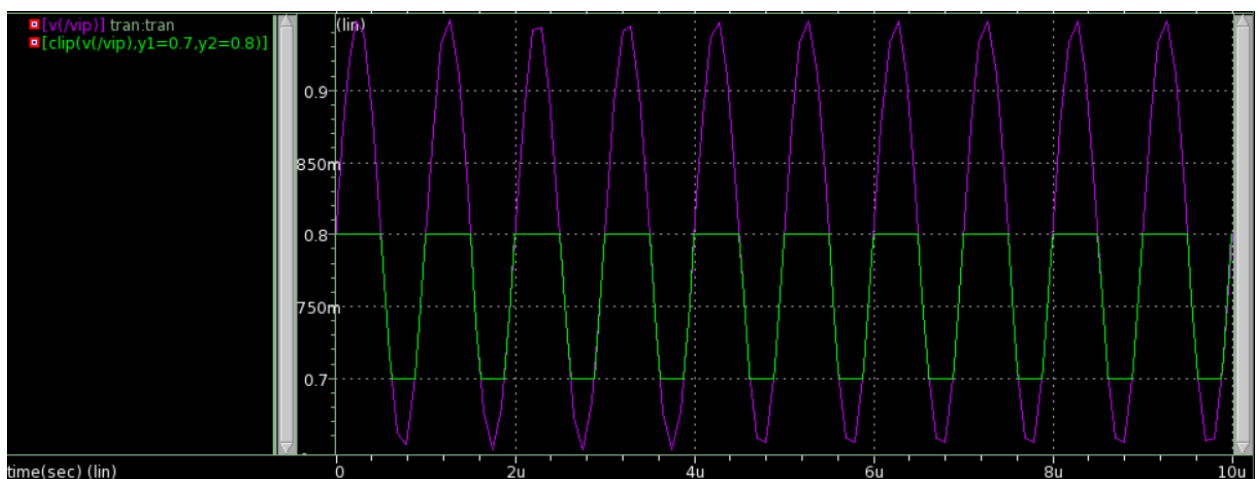
The inputs to the arguments in Example 1 are:

- Signal: $v(/vip)$
- x_start : 2u
- x_stop : 5u

The function expression shows as follows:

```
clip(v(/vip),2u,5u)
```

Example 2:



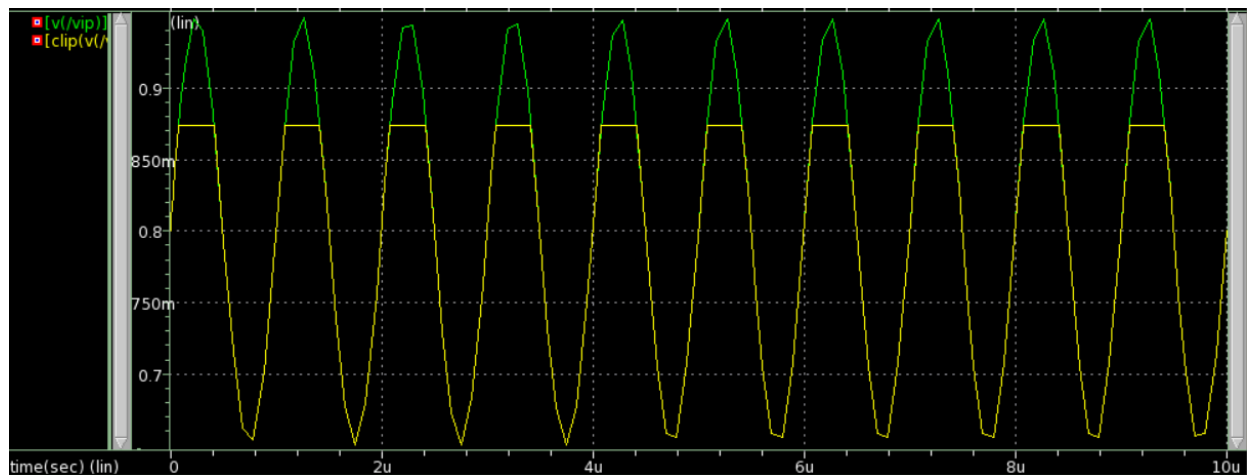
The inputs to the arguments in Example 2 are:

- Signal: $v(/vip)$
- y_min : 0.7
- y_max : 0.8

The function expression shows as follows:

```
clip(v(/vip),y1=0.7,y2=0.8)
```

Example 3:



The inputs to the arguments in Example 3 are:

- Signal: $v(/vip)$
- y_max : 75
- $percent=1$

The function expression shows as follows:

```
clip(v(/vip),y2=75,percent=1)
```

cPwrContour

Description

Generates contours with the same power for Z-Smith.

Syntax

```
cPwrContour(<current_signal>, <voltage_signal>, <harmonic_index>,  
[<current_load>], [<voltage_load>], [<max_power>], [<min_power>],  
[<num_contour>], [<resistance>], [<close>], [<modifier>])
```

Returns

A waveform.

Arguments

<current_signal>

Signal used to calculate power as current.

<voltage_signal>

Signal used to calculate power as voltage.

<harmonic_index>

Harmonic index of waves in current_signal and voltage_signal.

[<current_load>]

(Optional) Signal used to calculate reflection coefficient as current.

[<voltage_load>]

(Optional) Signal used to calculate reflection coefficient as voltage.

[<max_power>]

(Optional) The maximum power in contours.

[<min_power>]

(Optional) The minimum power in contours.

[<num_contour>]

(Optional) The number of contours.

[<resistance>]

(Optional) Resistance used to calculate reflection coefficients.

[<close>]

(Optional) The contours are closed contours (ON or OFF).

[<modifier>]

(Optional) Modifier function apply to power.

- mag
- dbm
- db10
- db20

create_bus

Description

Creates a bus signal with given name and up to 31 digital signals.

Syntax

```
create_bus(<name>, signal1, signal2, [signal3, ..., signal31])
```

Returns

A bus signal.

Arguments

<name>

A string to be used as a result signal's name.

signal1

First digital signal.

signal2

Second digital signal.

[signal3, ..., signal31]

Optional digital signals.

Examples

```
create_bus(tst, 'CLK', 'EZ', 'DED')
```

will create a 3 bits bus signal from inputs signals CLK', 'EZ', and, 'DED'. The result signal name is tst.

create_eye

Creates an eye object which can be plotted as an eye diagram.

Syntax

```
create_eye(<signal>, [<trigger_signal>], [eye_width],  
[<user_width/data_rate>], [<trigger_period>],  
[<start_i>], [<stop_i>], [<start_t>], [<stop_t>], [<phase>])
```

Returns

An eye object, which is a waveform object that contains eye parameter information.

Arguments

<signal>

A transient periodic signal.

[<trigger_signal>]

The external trigger signal.

[eye_width]

The eye width type: `auto`, `user`, or `data_rate`.

If `auto` is selected, the eye diagram width is calculated, which is the value to be used for folding the signal. The default is `auto`.

[<user_width/data_rate>]

The width of the eye diagram, which is a scalar value used to folding the signal. If `user` is selected as the `[eye_width]`, then the value input in this field is the eye width. If `data_rate` is selected as the `[eye_width]`, then the value input in this field is the data rate value.

[<trigger_period>]

The trigger period.

[<start_i>]

(Optional) The starting edge, which is an index-based filter.

[<stop_i>]

(Optional) The last edge, which is an index-based filter.

[<start_t>]

(Optional) The starting time, which is a time-based filter.

[<stop_t>]

(Optional) The ending time, which is a time-based filter.

[<phase>]

(Optional) The eye shift. Setting as `auto` centers the eye opening. See [Example 2](#).

Errors

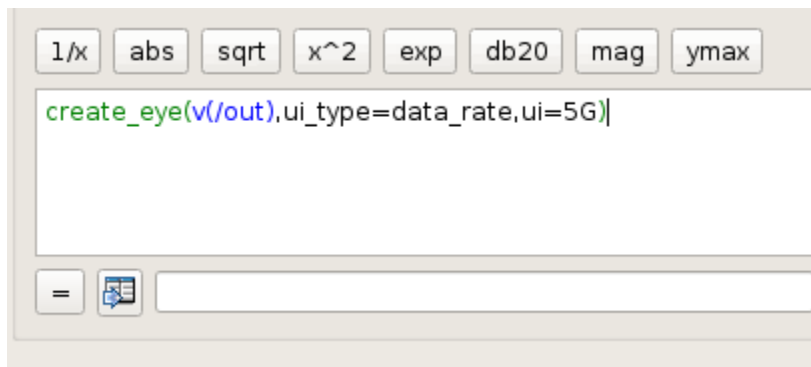
Possible error messages are:

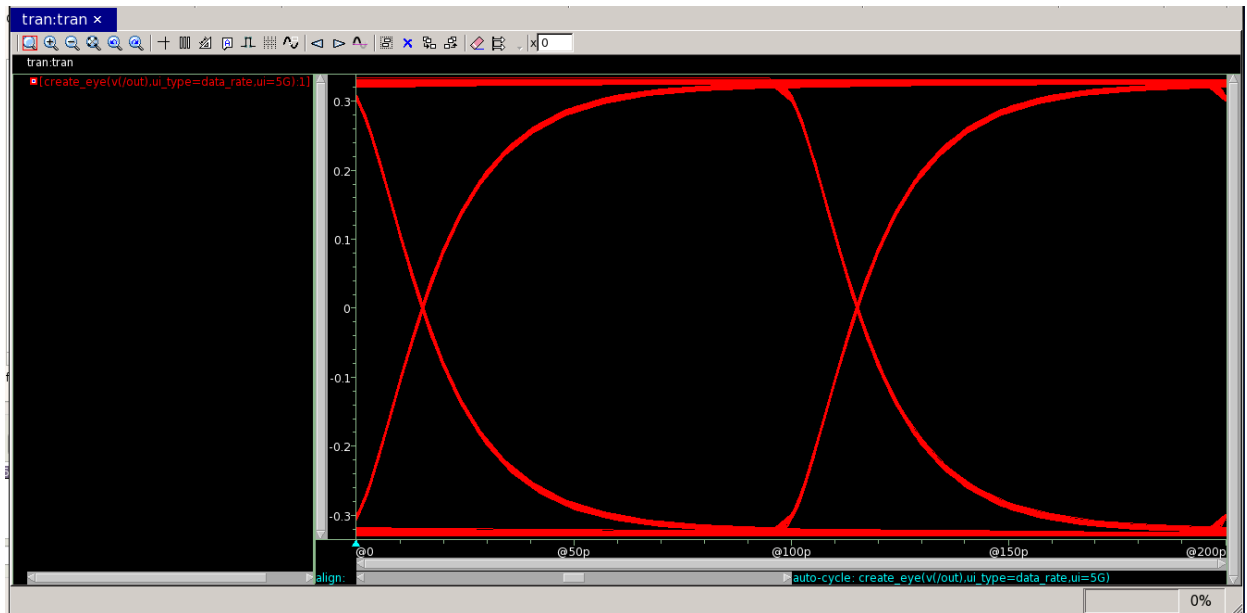
- Signal not found
- User width must be specified if not auto width
- Stop_i must be greater than start_i
- Stop_t must be greater than start_t
- Invalid user input shift value

Examples

Example 1

The following example shows how the function works on the input waveform "v(out)" for different input parameters:





The inputs to the arguments in the above example are:

- signal: v(/vout)
- ui_type (eye_width): data_rate
- ui (data rate): 5G

The function expression using these arguments is:

```
create_eye(v(/out),ui_type=data_rate,ui=5G)
```

Example 2

The following examples show the use of `auto` instead of a number in the `phase` field.

Plot Assist

Calculator

Measures Assist

create_eye : create an eye object

✓ Add

✗ Cancel

🔍 Help

signal:

v(/Vout)

trigger signal:

eye width:

user

user width/data rate:

900p

trigger period:

start_i:

stop_i:

start_t:

stop_t:

phase:

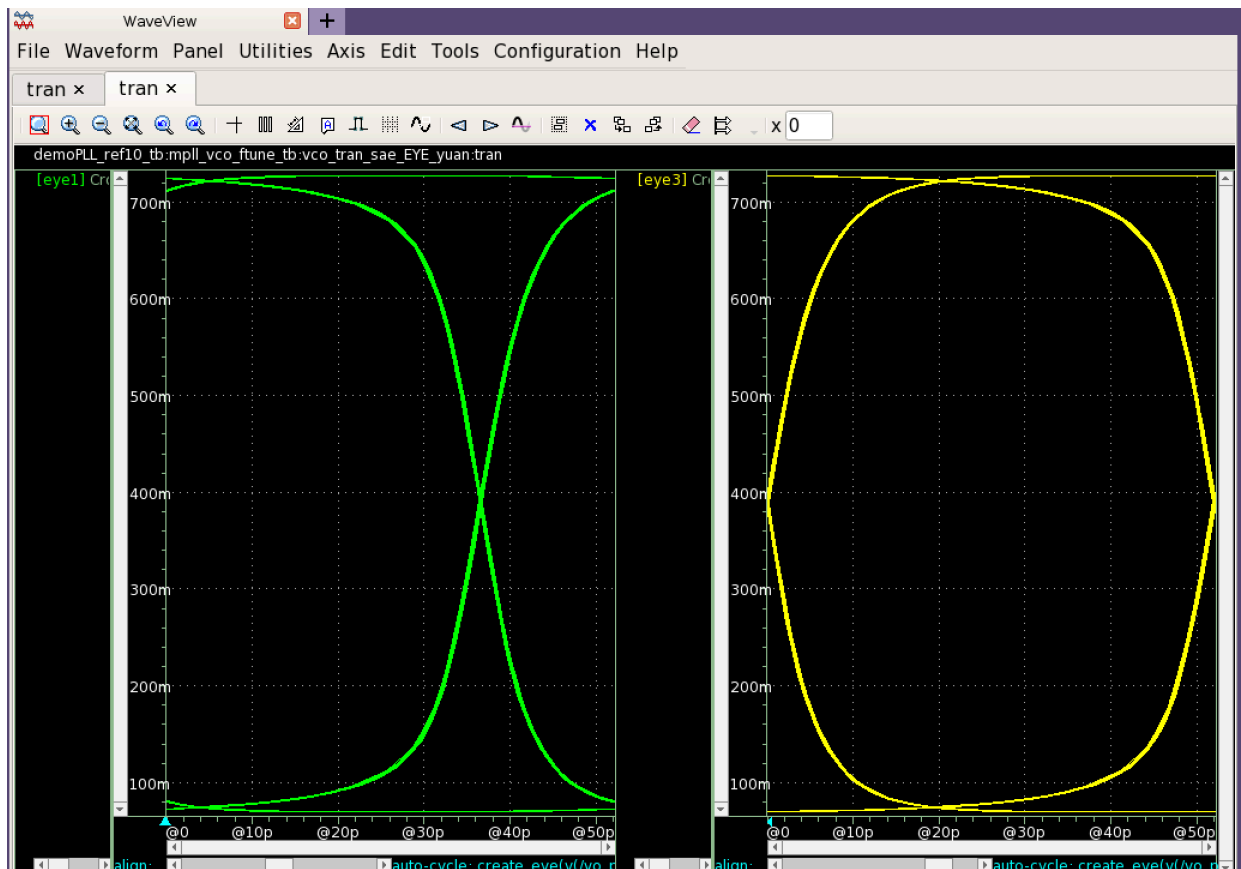
auto

This generates the following expression:

```
create_eye(v(/Vout),ui_type=user,ui=900p,phase=auto)
```

In the next example, eye1 and eye3 have the same configuration, except that eye3 is centered using the phase=auto option.

eye1	create_eye(v(/vo_p),ui_type=user,ui=5.207E-11,start_t=o(xstart),stop_t=o(xstop))		tran	☒	☒
eye3	create_eye(v(/vo_p),ui_type=user,ui=5.207E-11,start_t=o(xstart),stop_t=o(xstop),phase=auto)		tran	☒	☒



crossings

Calculates the crossing points between `signal1` and `signal2` or a given `ylevel` value.

Syntax

```
crossings(<signal1>,<signal2/ylevel>,[<edge_type>],[<output_yAxisType>],[<output_xAxisType>])
```

Returns

A waveform

Arguments

`<signal1>`

The first signal to be measured.

`<signal2/ylevel>`

The second signal to be measured. The value can also be a specified threshold.

`<edge_type>`

(Optional) The edge type. Valid values are `either`, `rise`, and `fall`. Default is `either`.

`<output_yAxisType>`

(Optional) The output signal `yAxisType`. Can be either signal1's Y axis or signal1's X axis. Default is the Y axis.

`<output_xAxisType>`

(Optional) The output signal `xAxisType`. Can be either signal1's X axis or index. Default is the X axis.

Errors

Possible error messages include:

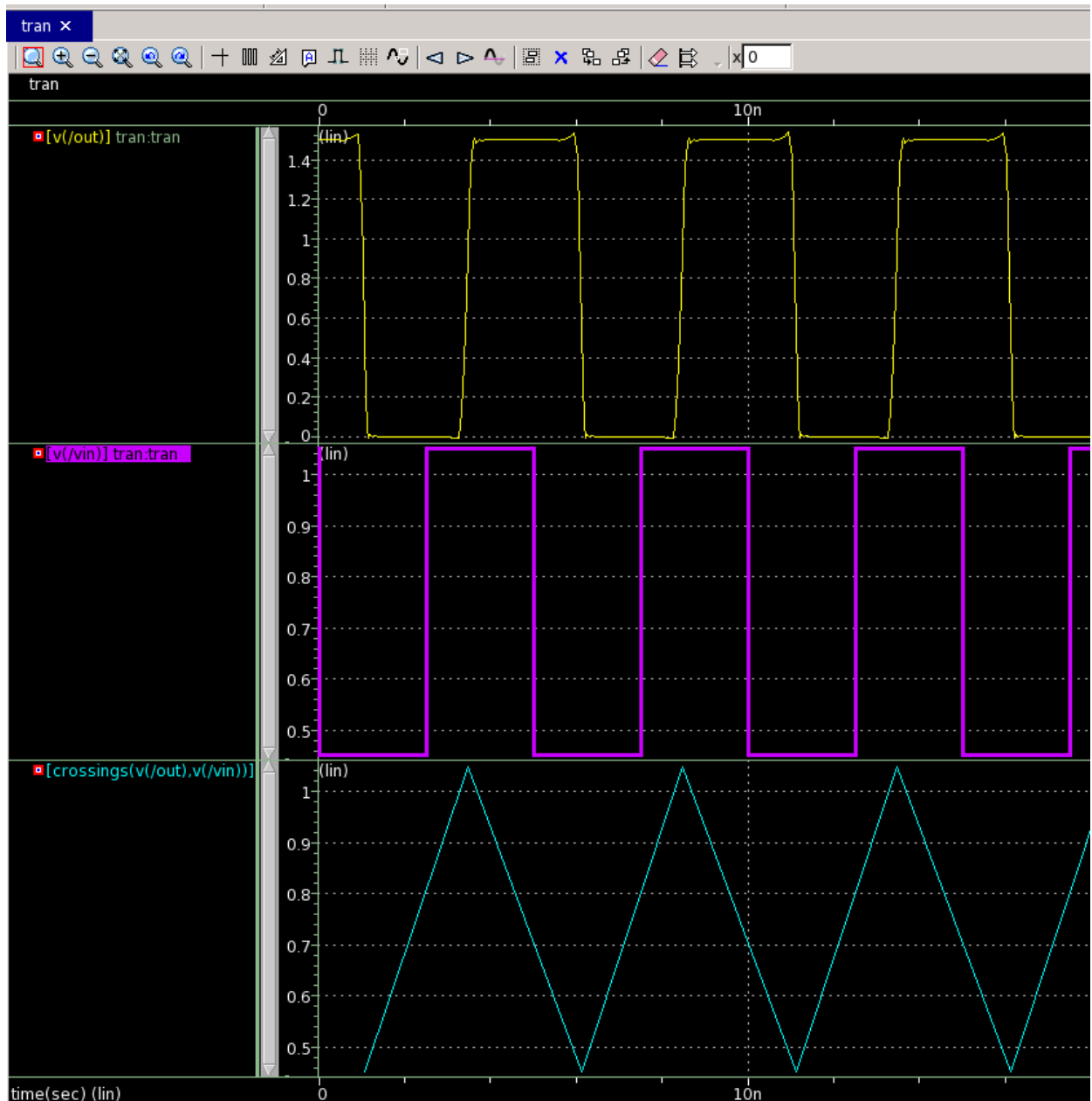
- Signal not found

Examples

```
crossings(v(/out), v(/vin))  
crossings(v(/out), 1, yAxisType=x)
```

The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 1:



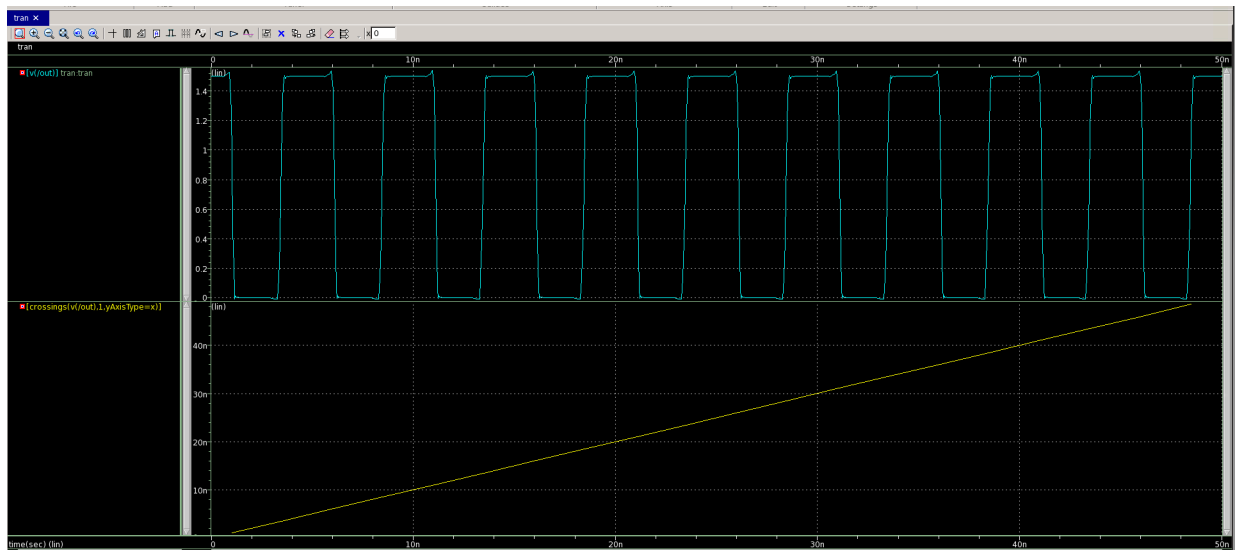
The example inputs of the arguments are as follows:

- Signal1: $v(/out)$
- Signal2: $v(/vin)$

The function expression is as follows:

```
crossings(v(/out), v(/vin))
```

Example 2:



The example inputs of the arguments are as follows:

- Signal1: $v(/out)$
- Y-level: 1
- X Stop: 40n
- Output $yAxisType$: x

The function expression is as follows:

```
crossings(v(/out), 1, yAxisType=x)
```

d2a

Converts the input signal from digital to analog.

Syntax

```
d2a(<signal>,<low_bounds>,<high_bounds>,  
    [<min_glitch_span>],[<time_step>],[<x_start>],  
    [<x_stop>],[<x_value>],[<bus_output_type>],[<display>])
```

Returns

A waveform.

Arguments

<signal>

An analog signal.

<low_bounds>

The low threshold of y.

<high_bounds>

The high threshold of y.

[<min_glitch_span>]

(Optional) The minimum glitch span to be set.

[<time_step>]

(Optional) The value of each `time_step`.

[<x_start>]

(Optional) The starting point of x.

[<x_stop>]

(Optional) The ending point of x.

[<x_value>]

(Optional) Determines state x of the digital waveform. This value can be a expression. For example, `vl` and `vh` can represent `low_bounds` and `high_bounds`.

[<bus_output_type>]

(Optional) Specifies output as an analog waveform or a list waveforms.

[<display>]

(Optional) Specifies whether display is `pwl` or `pwc`. Default is `pwc`.

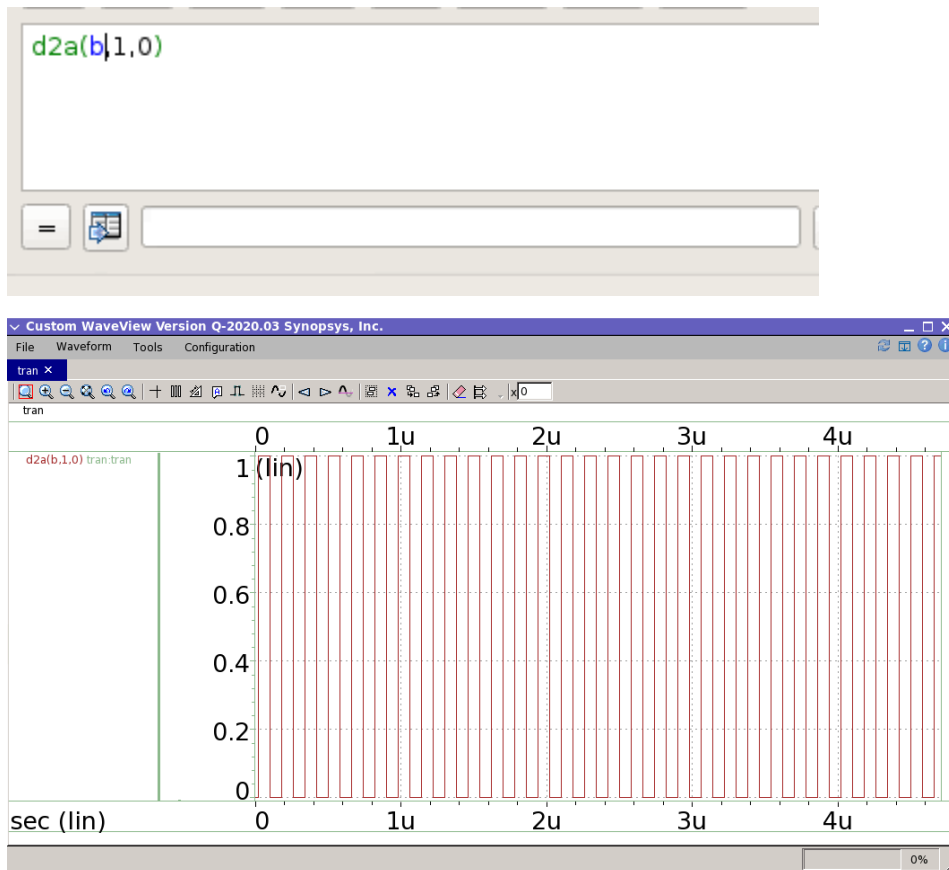
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

d2a(b, 1, 0)



The example inputs of the arguments shown are as follows:

- target signal: v(/out)
- high_bound: 1
- low_bound: 0

dBc2V2

Description

Converts phase noise signal from dBc/Hz to V²/Hz.

This only works for waveform files that contain carrier magnitude information. Otherwise, it will report: Carrier magnitude is not defined

Syntax

```
dBc2V2 (<signal>)
```

Returns

An analog signal.

Arguments

<signal>

The signal to be converted.

Examples

```
dBc2V2 ('PAC_L(f)')
```

delayvst

Measures the time difference versus the time between two signals at the specified switching edges from x_start to x_stop.

Syntax

```
delayvst (<signal>, <reference_signal>, <signal_y_level>,  
<reference_signal_y_level>, [<percent>], [<trigger_type>],  
[<reference_trigger_type>], [<x1_start>], [<x1_stop>],  
[<x2_start>], [<x2_stop>], [<premise>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

<reference_signal>

The reference signal.

`<signal_y_level>`

Threshold level for trigger signal.

`<reference_signal y_level>`

Threshold level for reference signal.

`[<percent>]`

(Optional) The value can be 1 or 0. When specified as 1, the `y_threshold` is a percentage value; when specified as 0, the threshold is not a percentage value.

`[<trigger_type>]`

(Optional) The `trigger_type` of measured signal, which can be `fall`, `rise`, or `either`. The default value is `either`.

`[<reference_trigger_type>]`

(Optional) The `trigger_type` of reference signal, which can be `fall`, `rise`, `same`, `opposite`, and `either`. Default is `either`.

`[<x1_start>]`

The starting value of `x` for trigger signal.

`[<x1_stop>]`

(Optional) The stopping value of `x` for trigger signal.

`[<x2_start>]`

(Optional) The starting value of `x` for reference signal.

`[<x2_stop>]`

(Optional) The stopping value of `x` for reference signal.

`[<premise>]`

Specifies the delay value as `positive`, `negative`, or `either`.

Errors

Possible error messages include:

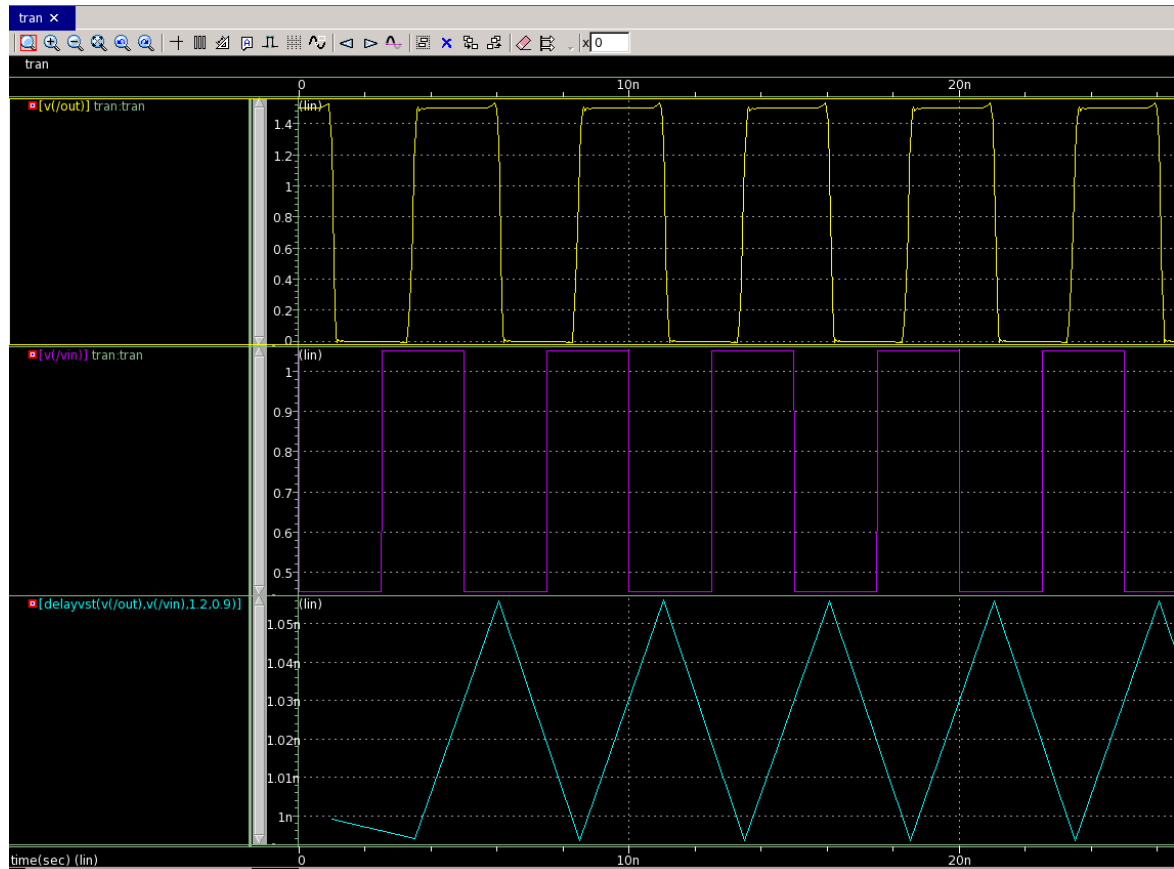
- Signal not found
- Y value out of range

Examples

```
delayvst(v(/out),v(/vin),1.2,0.9)
delayvst(v(/out),v(/vin),1.2,0.9, rise, same)
```


The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 1:



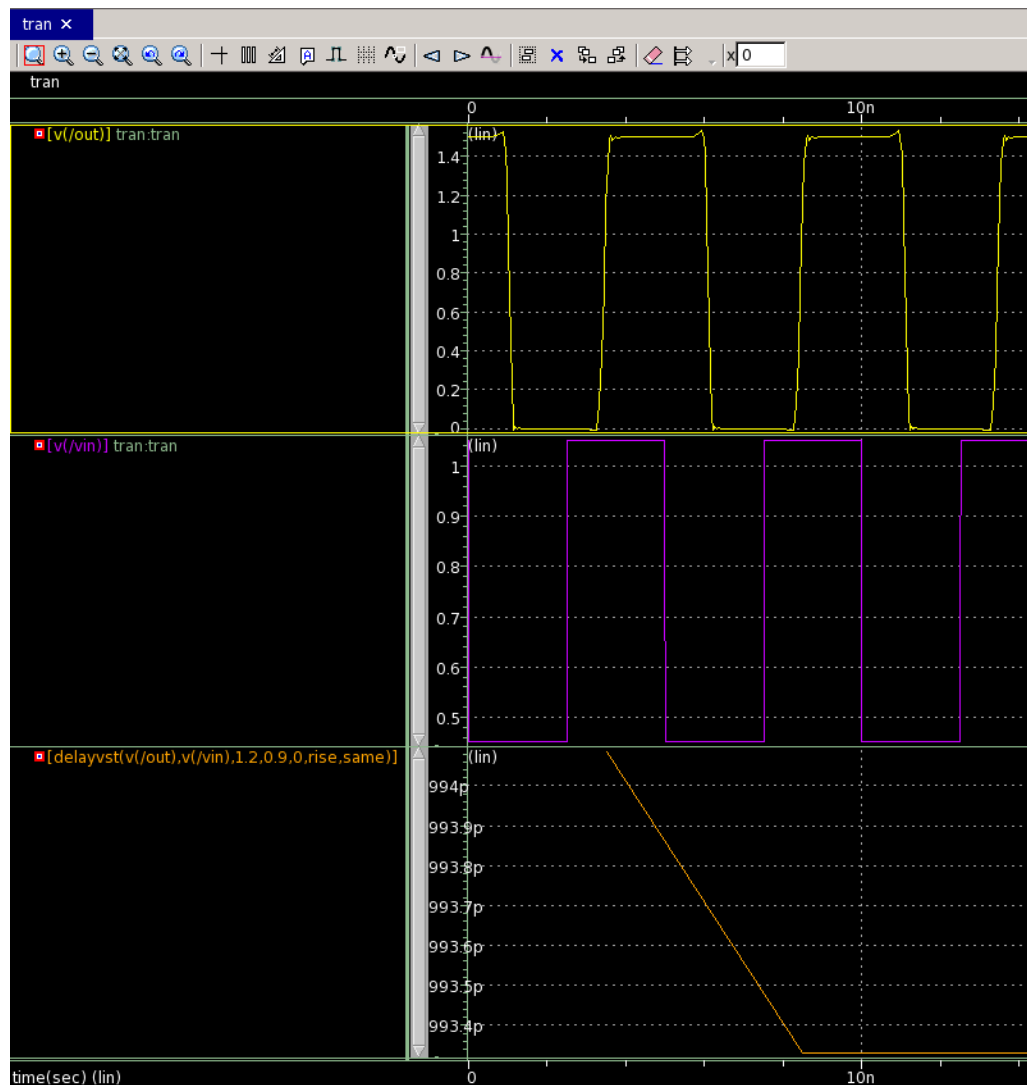
The example inputs of the arguments are as follows:

- Target Signal: v(/out)
- Reference Signal: v(/vin)
- Target Signal Y-level: 1.2
- Reference Signal Y-level: 0.9

The function expression is as follows:

```
delayvst(v(/out),v(/vin),1.2,0.9)
```

Example 2:



The example inputs of the arguments are as follows:

- Target Signal: `v(/out)`
- Reference Signal: `v(/vin)`
- Target Signal Y-level: 1.2
- Reference Signal Y-level: 0.9
- Target Signal Trigger Type: Rise
- Reference Signal Trigger Type: same

The function expression shows as follows:

```
delayvst(v(/out),v(/vin),1.2,0.9,rise,same)
```

derivative

Calculates the derivative waveform of given signal.

Syntax

```
derivative(<signal>, [<display>], [<type>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

[<display>]

(Optional) Displays the resulting waveform in either PWC or PWL.



[<type>]

(Optional) The values are tangent (the default) or secant. The tangent value has the same behavior as the current derivative function. When set to the secant value, the value for a point is the slope of line after that point. See the examples for more information.

Errors

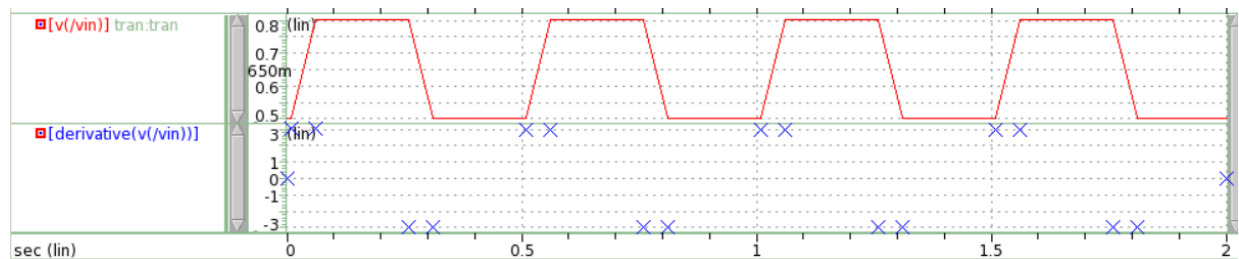
Possible error messages include:

- Signal not found

Examples

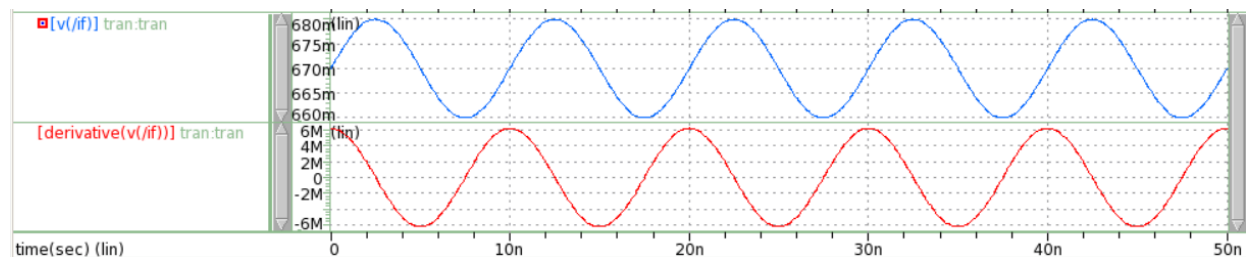
```
derivative(v(/vin))
```

The derivative is computed at every timepoint of the input waveform. At any given point, $\text{derivative}() = (yval(n) - yval(n-1)) / (xval(n) - xval(n-1))$



The example inputs of the arguments are as follows:

- Target Signal: v(/in)



The example inputs of the arguments are as follows:

- Target Signal: v(/if)

The following examples assume there are 3 continuous data points A, B, and C.

When `type` is set to `tangent` (the default):

For point B, the tool calculates the slopes of line AB and line BC, adds them together, then divides by 2.

Example 1:

$$\text{SlopeAB} = (f(t_B) - f(t_A)) / (t_B - t_A)$$

Example 2:

$$\text{SlopeBC} = (f(tC) - f(tB)) / (tC - tB)$$

$$\text{Derivative on B} = (\text{SlopeAB} + \text{SlopeBC}) / 2$$

For Example 1, the derivative is the slope of BC. For Example 2, the derivative is the slope of AB.

When `type` is set to `secant`:

The derivative on B is the slope of BC.

dft

Measures the Discrete-Fourier-Transform (DFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from `x_start` to `x_stop` for `n` points.

Syntax

```
dft(<signal>,<x_start>,<x_stop>,<num_of_points>,  
[<windowing_type>],[<window_alpha>],  
[<normalization>],[<user_defined_value>],  
[<suppress_DC>],[<dual_sideband>],  
[<coherent_gain>],[<coherent_gain_factor>],  
[<ADC_span>],[<window_sampling>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting point of x.

`<x_stop>`

The ending point of x.

`<num_of_points>`

The number of points needing to be measured.

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is a rectangle.

[<windowing_alpha>]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- none — FFT results is not normalized.
- normalized — FFT results is normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results is normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<coherent_gain>]

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1, if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor
- dB20: scaling with $\text{pow}(10, \text{coherent_gain_factor}/20)$
- dB10: scaling with $\text{pow}(10, \text{coherent_gain_factor}/10)$

[<coherent_gain_factor>]

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are $0 < 1$. Default is 0. You can use 1 if you do not want this scaling factor to be used.

[<ADC_span>]

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default value is 1.0.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. The default is symmetric.

Errors

Possible error messages include:

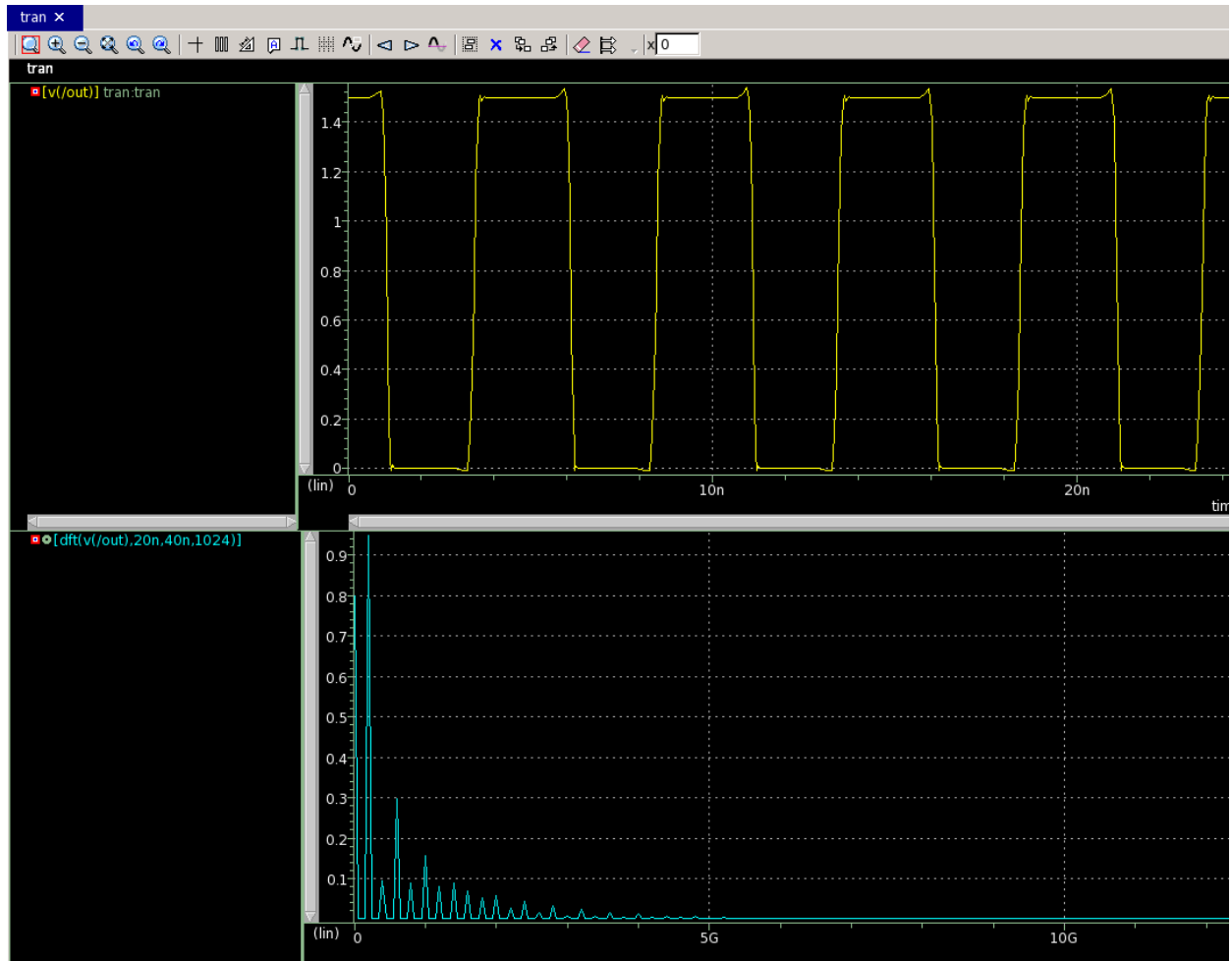
- Signal not found
- Y value out of range
- Number of sample points is non-positive
- X value out of range

Examples

```
dft(v(/out),20n,40n,1024)
dft(v(/out),20n,40n,1024,'hamming')
```

The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 1:



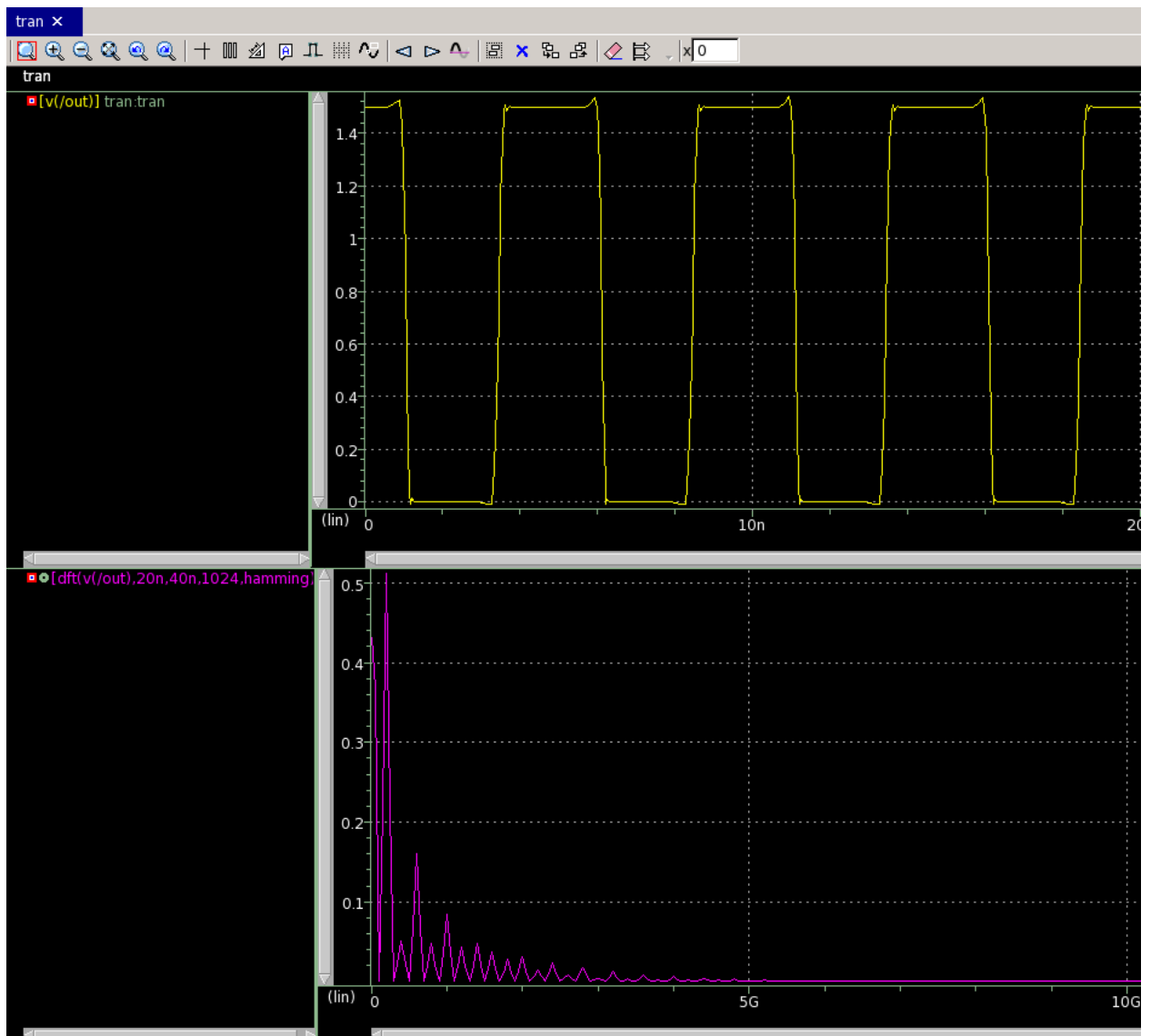
The example inputs of the arguments shown as follows:

- Target Signal: v(/out)
- X start: 20n
- X Stop: 40n
- Number of Sampling Points: 1024

The function expression shows as follows:

```
dft(v(/out),20n,40n,1024)
```


Example 2:



The example inputs of the arguments are as follows:

- Target Signal: $v(out)$
- X start: 20n
- X Stop: 40n
- Type of Window: Hamming
- Number of Sampling Points: 1024

The function expression is as follows:

```
dft(v(/out), 20n, 40n, 1024, hamming)
```

dftbb

Description

Computes the fast Fourier transform (FFT) of a complex signal $Z(n) = ReZ(n) + j \cdot ImZ(n)$ by reading in two signals, sampling evenly from x_{start} to x_{stop} for n points, and performs FFT separately.

Syntax

```
dftbb(<signal_1>, <signal_2>, <x_start>, <x_stop>,  
<number_of_points>, [<windowing_type>],  
[<window_alpha>], [<normalization>],  
[<user_defined_value>], [<suppress_DC>], [<dual_sideband>],  
[<coherent_gain>], [<coherent_gain_factor>], [<ADC_span>],  
[<window_sampling>])
```

Returns

A waveform which has N complex values.

Arguments

<signal_1>

The first signal.

<signal_2>

The second signal.

<x_start>

The starting point of x .

<x_stop>

The ending point of x .

<number_of_points>

The number of points needing to be measured.

[<windowing_type>]

(Optional) Specifies the windowing function (default is rectangle).

[<windowing_alpha>]

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

[<normalization>]

(Optional) There are three options for this argument:

- none — FFT results is not normalized.
- normalized — FFT results is normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results is normalized against a user-defined magnitude value.

[<user_defined_value>]

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<coherent_gain>]

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor
- dB20: scaling with $\text{pow}(10, \text{coherent_gain_factor}/20)$
- dB10: scaling with $\text{pow}(10, \text{coherent_gain_factor}/10)$

[<coherent_gain_factor>]

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by $1/(\text{Coherent Gain})$. Valid values are $0 < 1$. Default is 0. You can use 1 if you do not want this scaling factor to be used.

[<ADC_span>]

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default value is 1.0.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. The default is symmetric.

Examples

$$I(n) = ReI(n) + j \cdot ImI(n)$$

$$Q(n) = ReQ(n) + j \cdot ImQ(n)$$

and then computes:

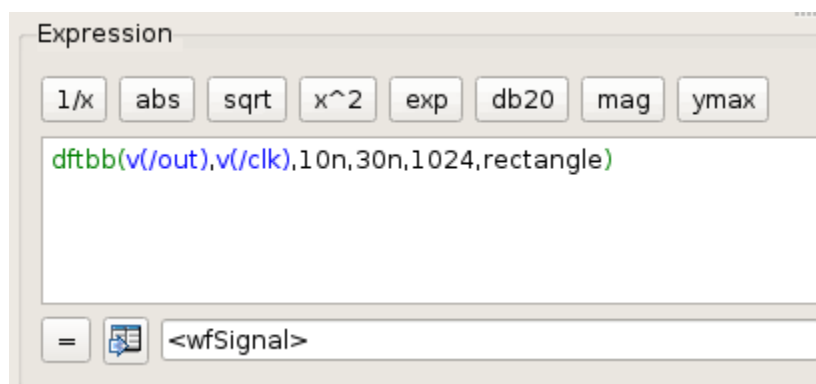
$$ReZ(n) = ReI(n) - ImQ(n)$$

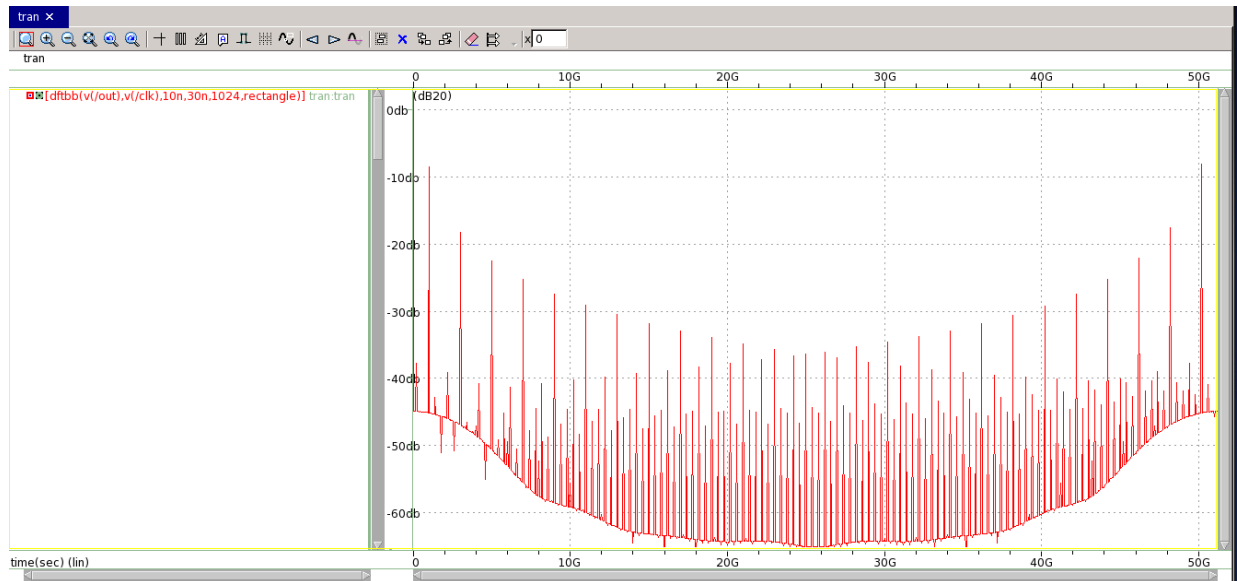
$$ImZ(n) = ImI(n) + ReQ(n)$$

For $n = 0, 1, N - 1$

The following example shows how the function works on the input waveform v(/out) and v(/clk):

Example 1:

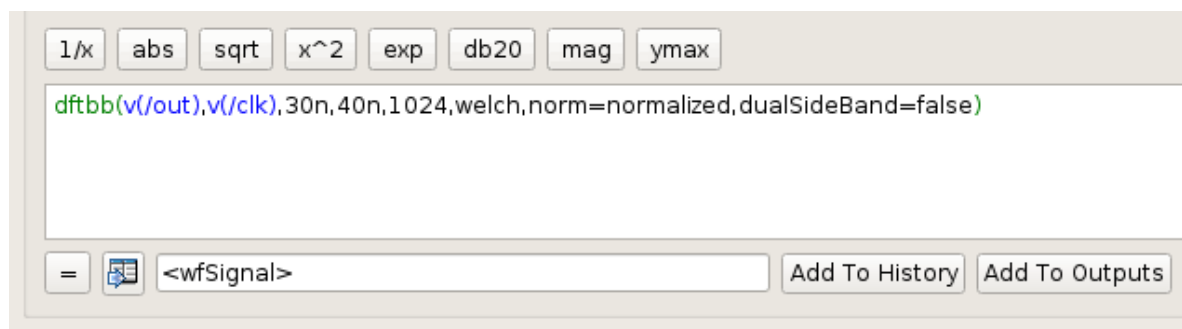


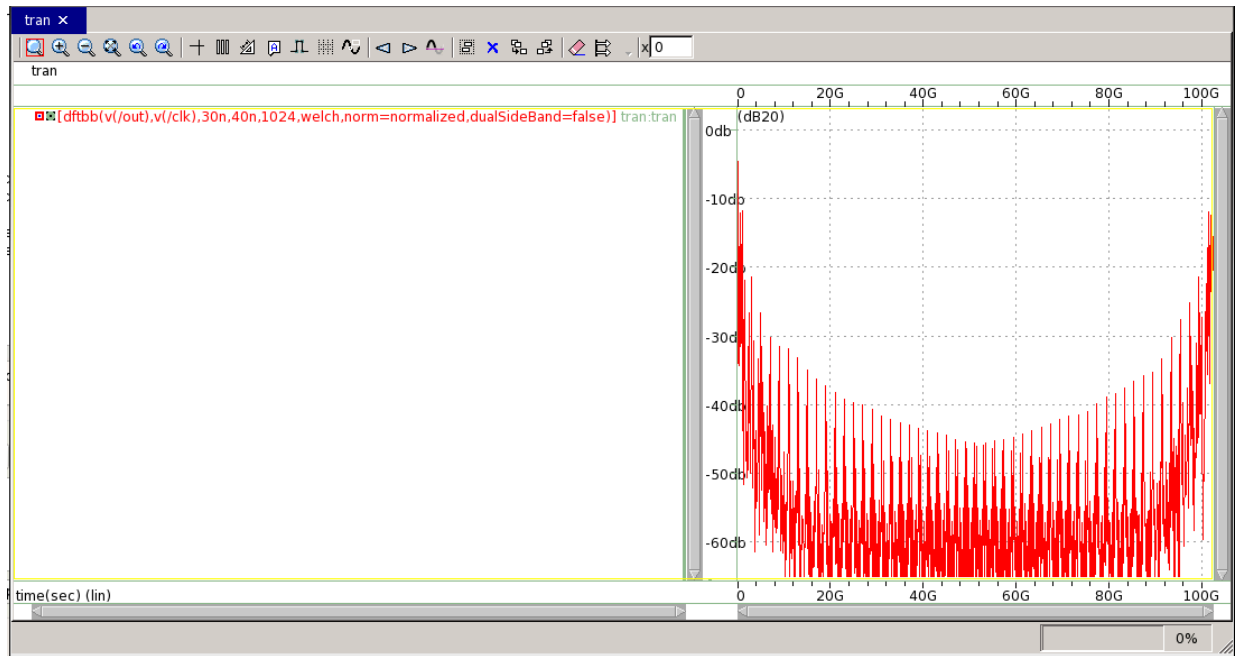


The inputs of the arguments in Example 1 are:

- Signal1: v(/out)
- Signal2: v(/clk)
- X start: 20n
- X stop: 30n
- Type of Window: Hamming
- Number of Sampling Points: 1024

Example 2:





The inputs of the arguments in Example 2 are:

- Signal1: v(/out)
- Signal2: v(/clk)
- X start: 30n
- X stop: 40n
- Type of Window: Welch
- Normalized: Yes
- dualSideBand: false

dnl

Description

Computes the differential non-linearity of a transient waveform. DNL is calculated as:

$$DNL(D+1)|_{LSB} = \frac{V(D+1) - V(D)}{V_{LSB}} - 1$$

where $0 < D < 2^N - 2$.

Syntax

```
dn1(<signal>,<sampling_signal|list|step>,<cross_type>],  
[<mode>], [<threshold>], [<delay>], [<method>], [<unit>],  
[<no_of_samples>])
```

Returns

An analog signal.

Arguments

<signal>

The signal to be measured.

<sampling_signal|list|step>

This argument specifies three ways to sampling the waveform:

- **sampling_signal**: the measured the signal is sampled according to the input sampling signal. The sample points is obtained at which the waveform crosses the threshold (specified with [<mode>] and [<threshold>] options) at either rising or falling edge (specified with [<cross_type>] option). The sampling can start with a delay (specified with [<delay>] option).
- **list**: list of x values at which the sample points are obtained for the measured signal.
- **step**: the interval value specified for sampling the measured signal.

[<cross_type>]

(Optional) Specifies the edge types at which the sampling signal intersect with the threshold. There are two options: rising or falling.

[<mode>]

(Optional) Specifies the mode used to calculate the threshold value. There are two options:

- **user**: Specify the threshold value with [<threshold>] option.
- **auto**: the threshold is calculated automatically by $(\text{integral_of_waveform} / \text{x_range})$.

[<threshold>]

(Optional) Specifies the threshold value used to sample the data. Only specified if mode is user.

[<delay>]

(Optional) Specifies time after which the sampling begins.

[<method>]

(Optional) Specifies whether the end-to-end (end) or straight line (fit) method is used.

[<unit>]

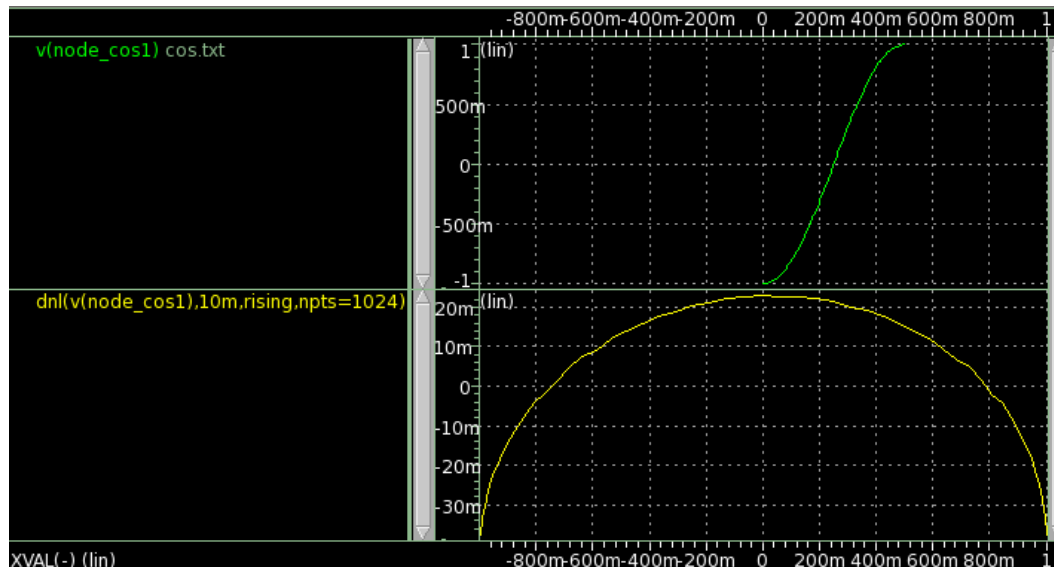
(Optional) Specifies whether the output waveform is to be output as an absolute value (abs) or multiples of least significant bit (lsb).

[<no_of_samples>]

(Optional) Specifies the samples used for calculating the non-linearity. If not specified, the samples are taken against the entire data window.

Examples

The following example shows how the function works on the input waveform “v(node_cos1)”:



The example inputs of the arguments show as follows:

- signal: v(node_cos1)
- step: 10m
- npts: 1024

The function expression shows as follows:

```
dnl('v(node_cos1)',10m,'rising',npts=1024)
```

duty_vs_t

Measures the duty cycle versus time. The duty cycle is the percentage of one period in which a signal is active. A period is the total time for a signal to complete an on-and-off cycle. Duty cycle can be expressed as where T is the active time signal and P is the total period of the signal: $D = \frac{T}{P} \times 100$

Syntax

```
duty_vs_t(<signal>,<y_threshold>,<rise_edge>,[<x_start>],  
[<x_stop>],[<percent>],[<output_xAxisType>],[<time_value>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<y_threshold>`

The threshold value of y.

`<rise_edge>`

This has two values 0 and 1 (0='fall', 1='rise').

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The ending point of x.

`[<percent>]`

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. When the value is 0, the `y_threshold` is not a percentage value.

`[<output_xAxisType>]`

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx.

`[<time_value>]`

(Optional) Specifies the time value, which is the cycle begin value or cycle end value. Default is `cycle_end`.

Errors

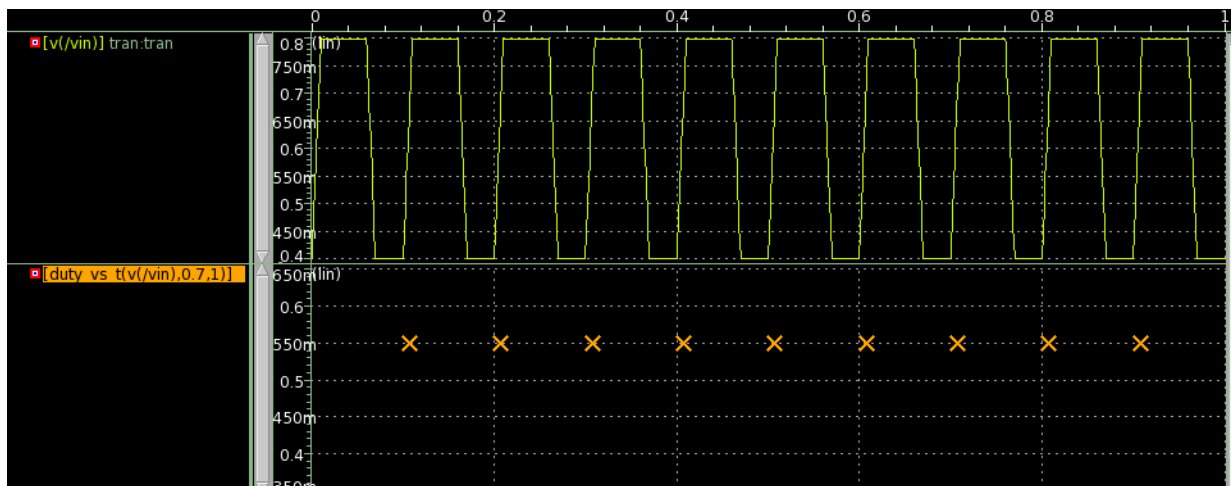
Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
duty_vs_t(v(/vin),0.7,1) ;  
duty_vs_t(v(/vin),0.7,csample=cycle_end
```

Example 1:



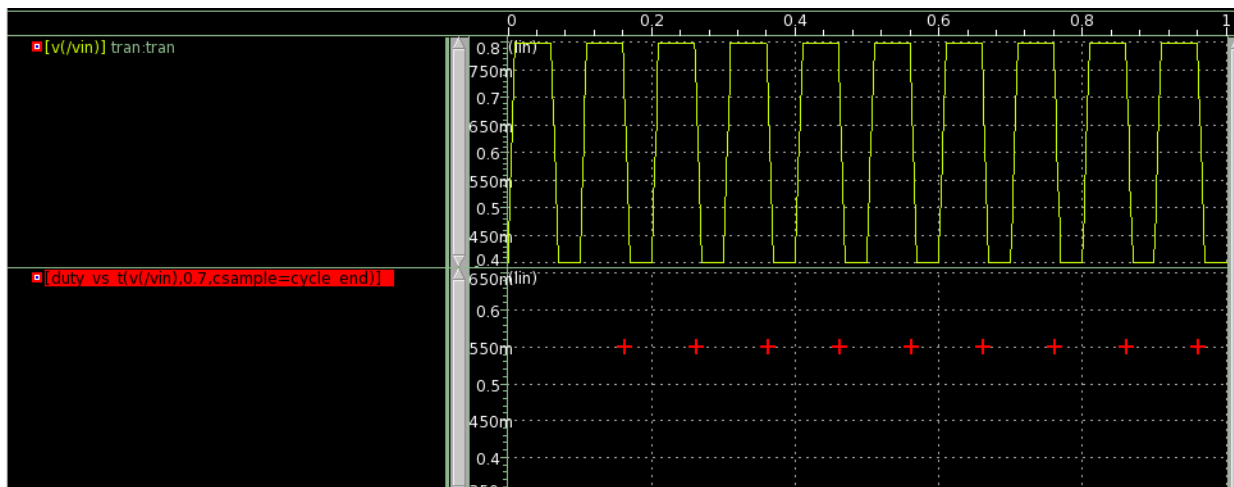
The input arguments in Example 1 are:

- Signal: vin
- y threshold: 0.7
- Rise edge: 1

The function expression is:

```
duty_vs_t(v(/vin),0.7,1)
```

Example 2:



The input arguments in Example 1 are:

- Signal: vin
- y threshold: 0.7
- Time Value: cycle end

The function expression is:

```
duty_vs_t(v(/vin),0.7,csample=cycle_end)
```

fallvst

Measures the time difference versus the time of the given signal starting from high y_level to low y_level within the range of x_start and x_stop.

Syntax

```
fallvst(<signal>,<high_y_level>,<low_y_level>,[<percent>],  
[<x_start>],[<x_stop>])
```

Arguments

<signal>

The signal to be measured.

<high_y_level>

The high threshold value of y

<low_y_level>

The low threshold value of y.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the y_threshold is a percentage value. If the value is 0, the y_threshold is not a percentage value.

[<x_start>]

(Optional) The starting point of x.

[<x_stop>]

(Optional) The ending point of x.

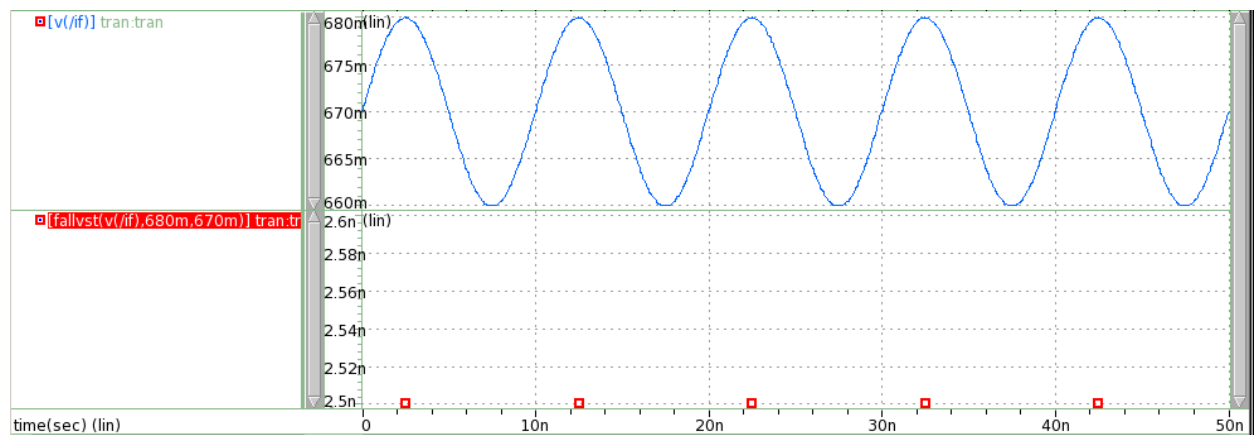
Errors

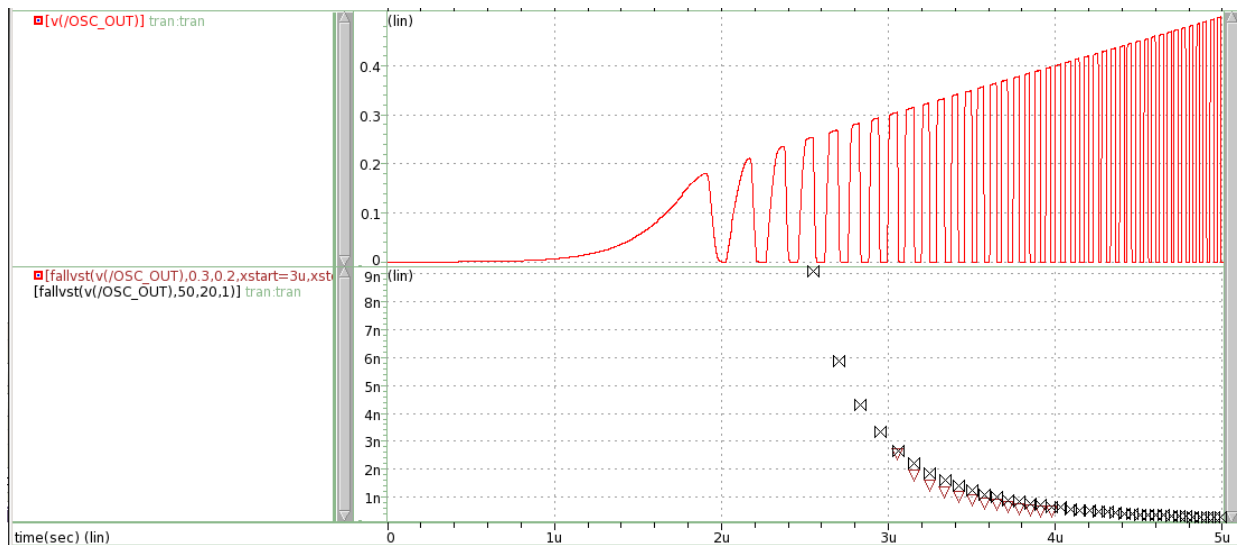
Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
fallvst(v(/if),680m,670m)
fallvst(v(/OSC_OUT),50,20,1)
fallvst(v(/OSC_OUT),0.3,0.2,xstart=3u,xstop=4u)
```





fft

Measures the Fast-Fourier-Transform (FFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.

Syntax

```
fft(<signal>,<x_start>,<x_stop>,<num_of_points>,<windowing_type>,<window_alpha>,<normalization>,<user_defined_value>,<suppress_DC>,<dual_sideband>,<coherent_gain>,<coherent_gain_factor>,<ADC_span>,<window_sampling>)
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

<x_start>

The starting point of x .

<x_stop>

The ending point of x .

`<num_of_points>`

The number of points needing to be measured. Must be a power of 2.

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is a rectangle.

`[<window_alpha>]`

(Optional) Window alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- none — FFT results are not normalized.
- normalized — FFT results are normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<coherent_gain>]`

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with `coherent_gain_factor`

- dB20: scaling with $\text{pow}(10, \text{coherent_gain_factor}/20)$
- dB10: scaling with $\text{pow}(10, \text{coherent_gain_factor}/10)$

[<coherent_gain_factor>]

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by $1/(\text{Coherent Gain})$. Valid values are $0 < 1$. Default is 0. You can use 1 if you do not want this scaling factor to be used.

[<ADC_span>]

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default value is 1.0.

[<window_sampling>]

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. Default is symmetric.

Errors

Possible error messages include:

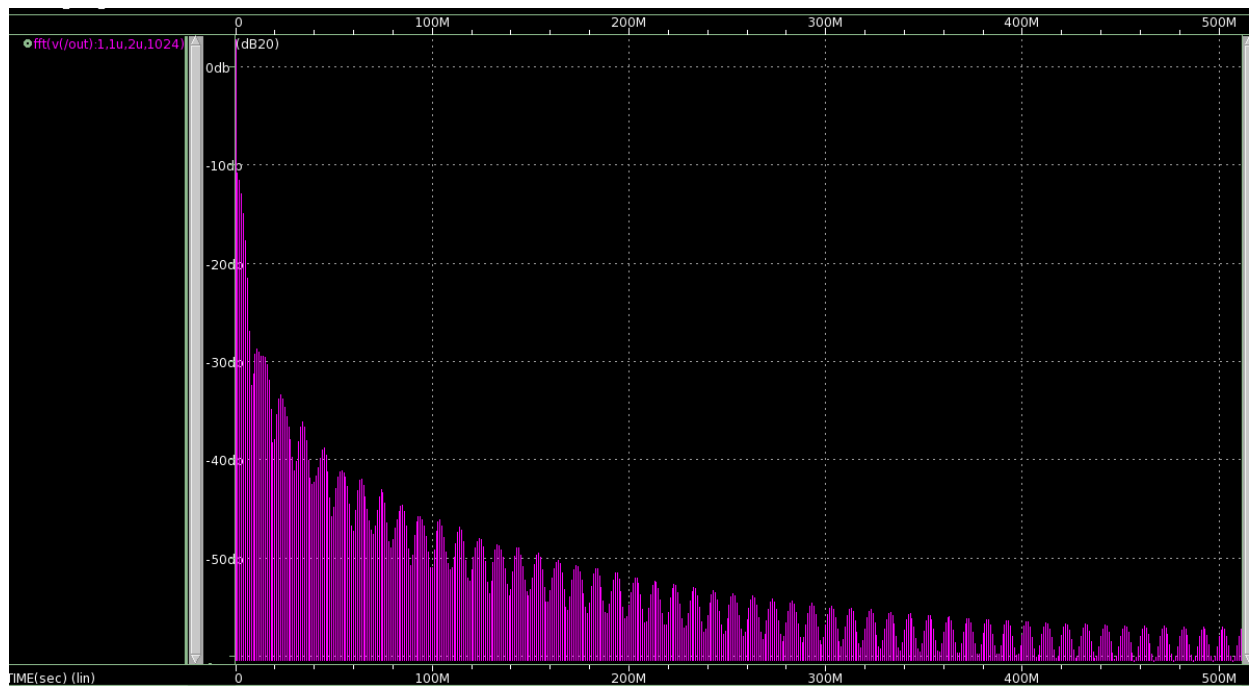
- Signal not found
- Y value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2
- X value out of range

Examples

Example 1:

```
fft('v(/out)', 1u, 2u, 1024)
```

will appear as follows:



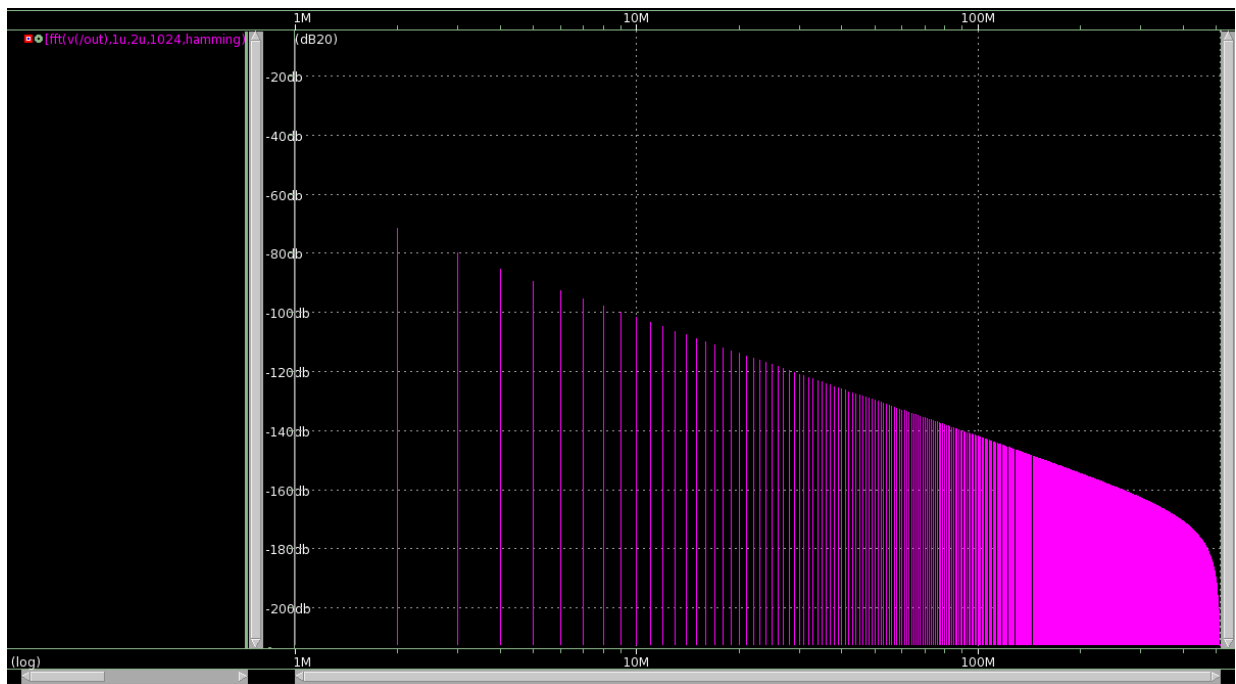
The example inputs of the arguments are as follows:

- Signal: `v(/out)`
- X start: `1u`
- X stop: `2u`
- Number of Points: `1024`

Example 2:

```
fft(v(/out),1u,2u,1024,hamming)
```


will appear as follows:



The example inputs of the arguments shown are as follows:

- Signal: v(/out)
- X start: 1u
- X stop: 2u
- Number of Points: 1024
- Window Type: Hamming

flip

Description

Reverses the signal along the Y-axis.

Syntax

```
flip(<signal>)
```

Returns

An analog signal.

Arguments

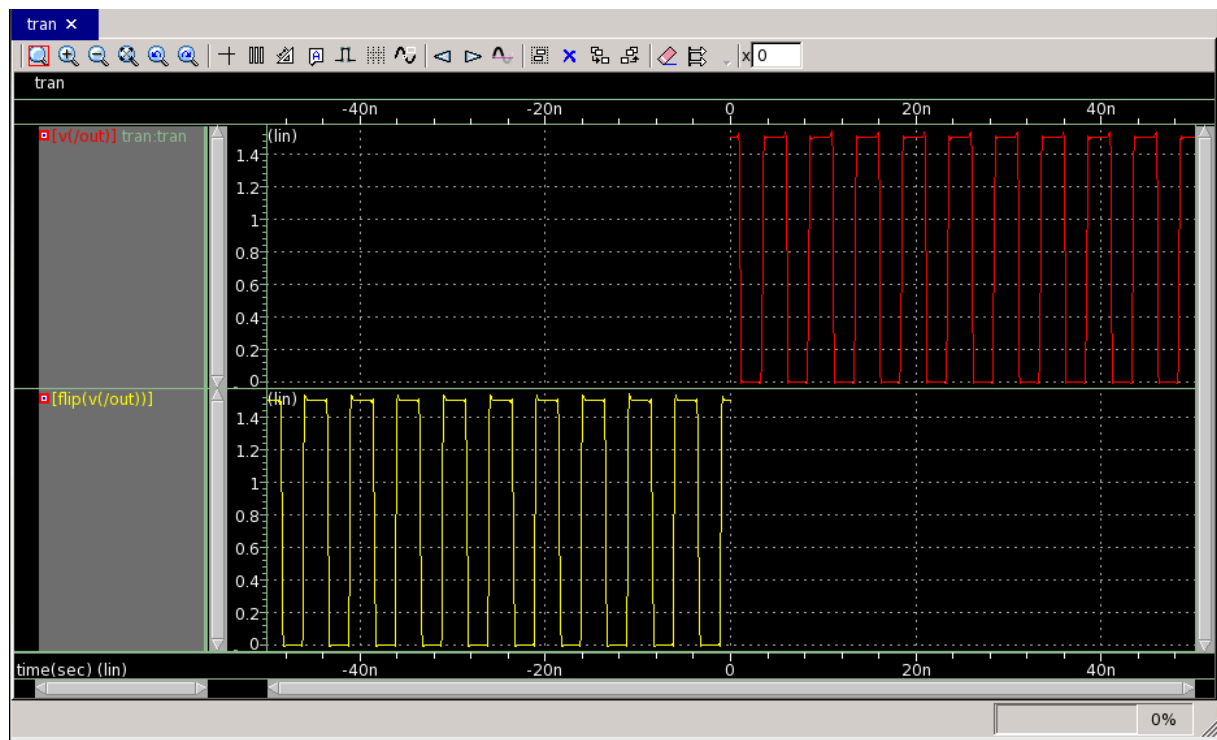
<signal>

The signal to be reversed.

Examples

```
flip(v(/out))
```

The following example shows how the function works on the input waveform "v(out)":



The example inputs of the arguments are as follows:

- Signal: v(/out)

The function expression is as follows:

```
flip(v(/out))
```

fourEval

Description

This function is an inverse Fourier transformation. It transforms the specified waveform from the frequency domain to the time domain.

Syntax

```
fourEval(<signal>,<x_start>,<x_stop>,<time_step>,  
[<baseband>])
```

Returns

An analog signal.

Arguments

<signal>

The signal to be calculated.

<x_start>

Specifies time at which to begin evaluating the series.

<x_stop>

Specifies time at which to stop evaluating the series.

<time_step>

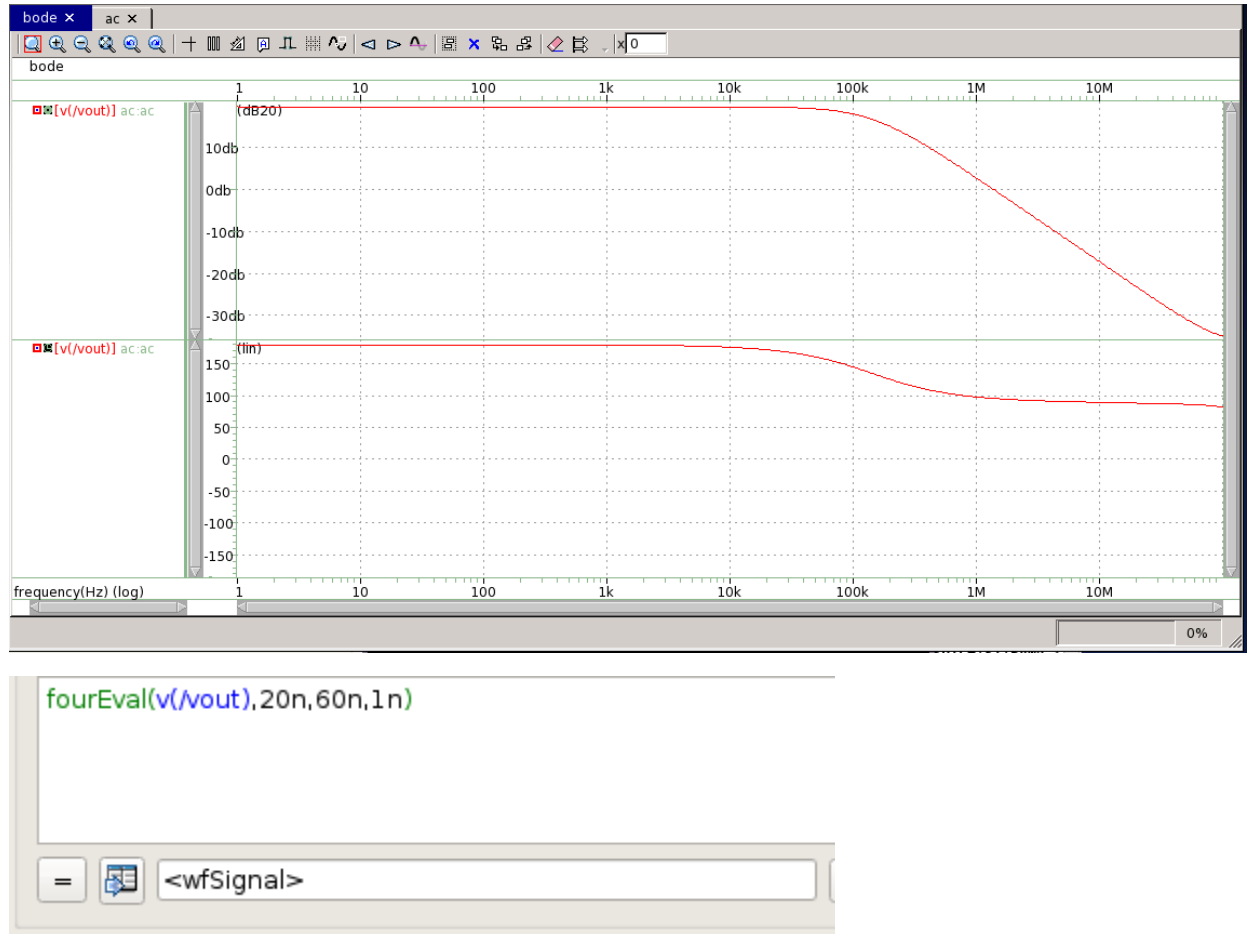
Specifies time increment.

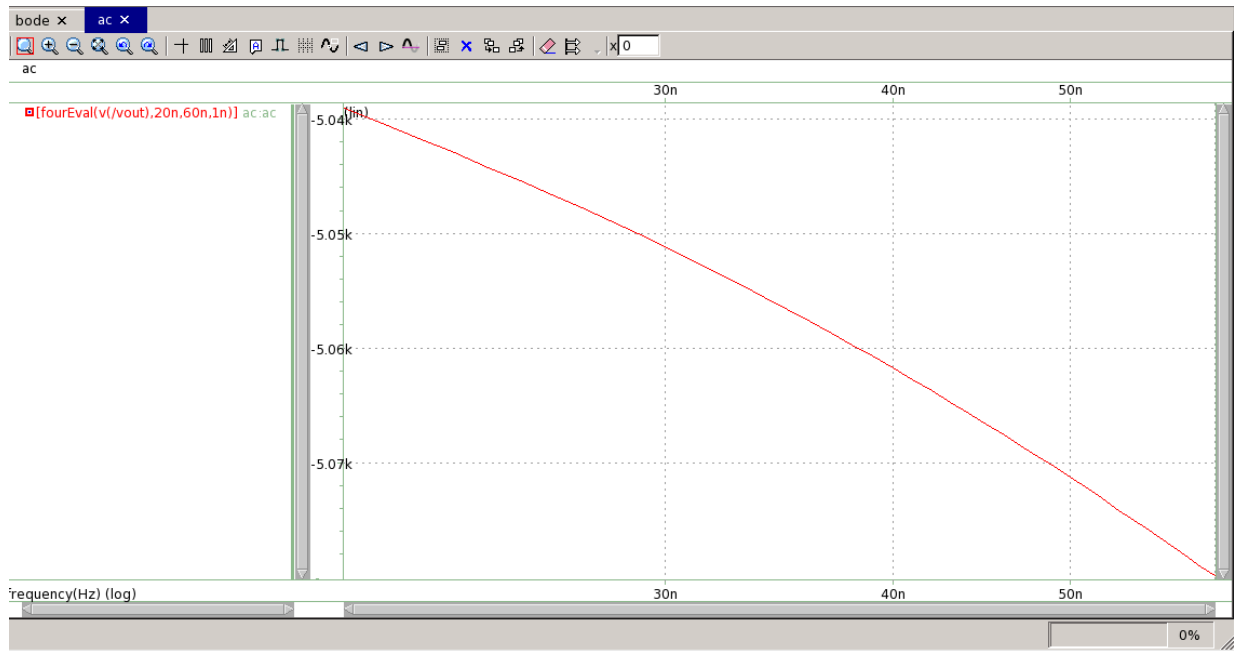
[<baseband>]

(Optional) Evaluates the baseband version of the inverse of the DFT function.

Examples

The following example shows how the function works on the input waveform "v(/vout)":





func

Plots the waveform of `signal1` versus `signal2` for the two specified signals.

Syntax

```
func(<signal1>,<signal2>)
```

Returns

A waveform.

Arguments

`<signal1>`

The first signal to be measured.

`<signal2>`

The second signal to be measured.

Errors

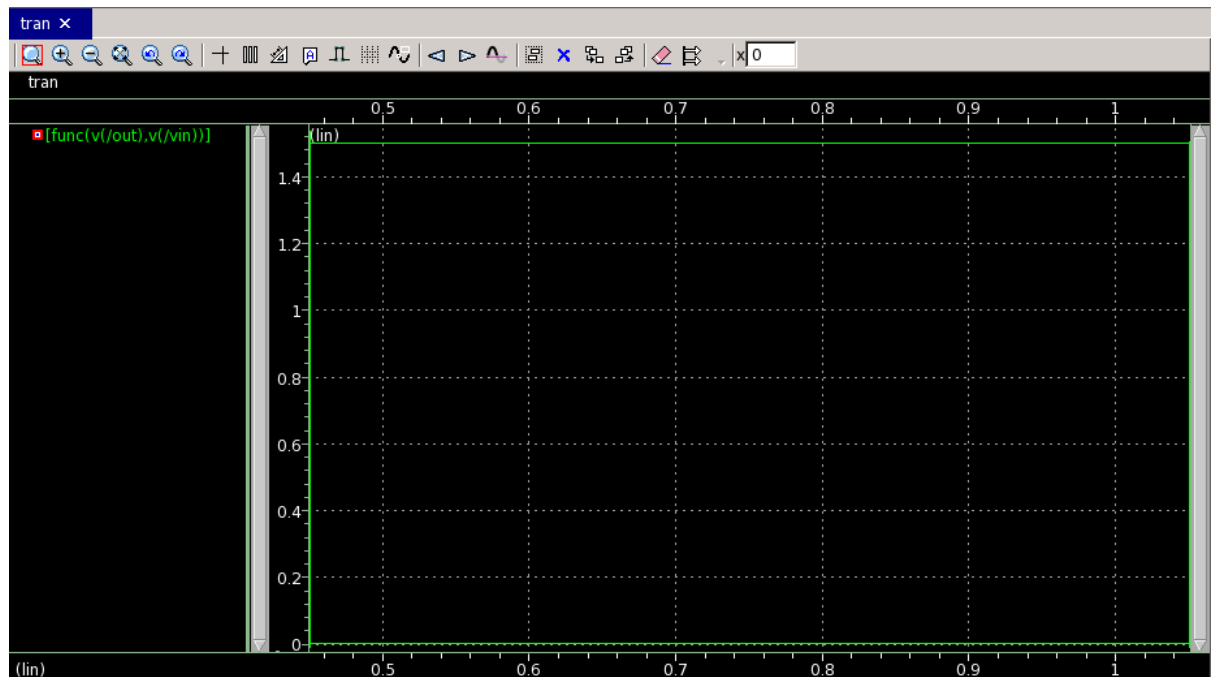
Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
func(v(/out),v(/vin))
```

The following example shows how the function works on the input waveforms "v(/out)" and "v(/vin)":



The example inputs of the arguments are as follows:

- Signal1: v(/out)
- Signal2: v(/vin)

The function expression is as follows:

```
func(v(/out),(/vin))
```

fvst

Calculates the frequency versus time for the given signal.

Syntax

```
fvst(<signal>,[<y_threshold>],<x_start>,<x_stop>,  
[<percent>],[<edge_type>],  
[output_xaxistype],[<time_value>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<y_threshold>]`

(Optional) The threshold value of y. If not specified, the default value will be the integral of the waveform divided by the X range.

`<x_start>`

The starting value of x.

`<x_stop>`

The ending value of x.

`[<percent>]`

(Optional). The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. If the value is 0, the `y_threshold` is not a percentage value.

`[<edge_type>]`

(Optional) Specifies the trigger type (fall, rise, or either). The fall argument specifies the wave crossing from above the `y_level` to below the `y_threshold`.

`[<output_xaxistype>]`

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx. Default is time.

`[<time_value>]`

Specifies whether X values in result waveform are equal to the start time or end time of each period.

Errors

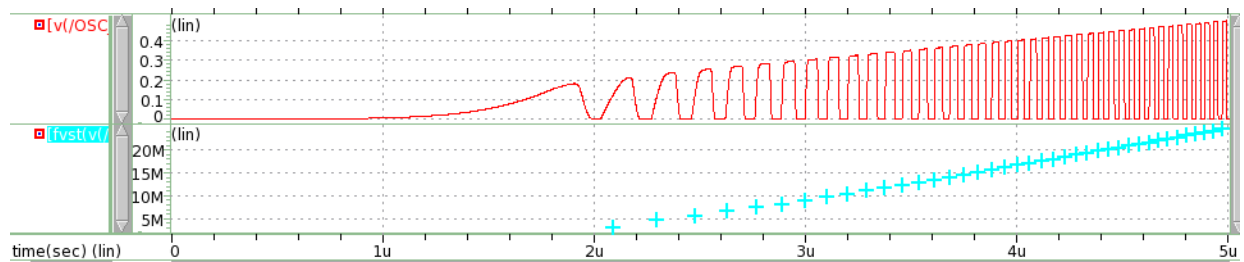
Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

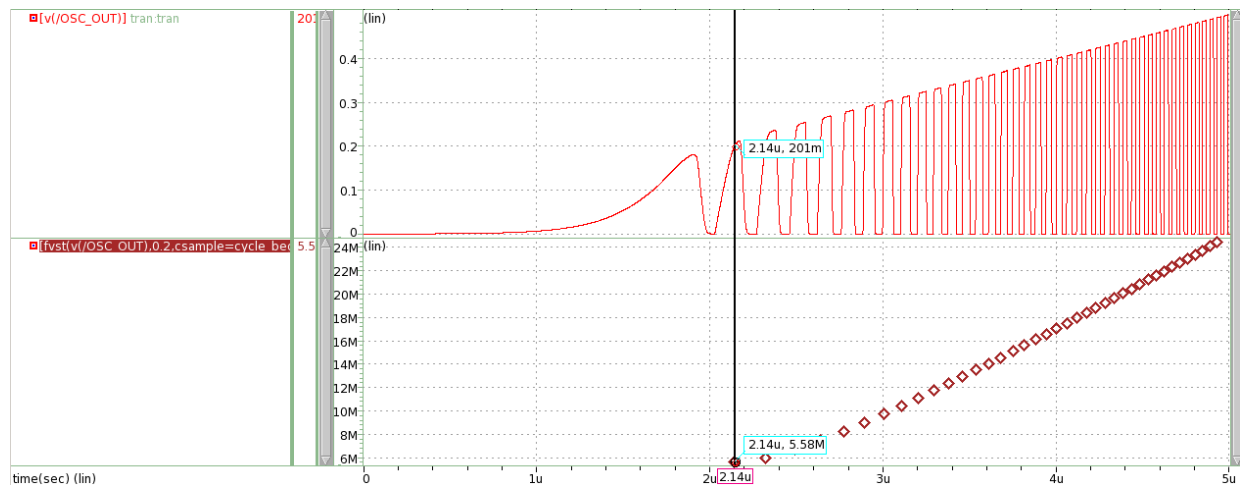
Examples

```
fvst(v(/OSC_OUT))
fvst(v(/OSC_OUT),0.2,csample=cycle_begin)
```

```
fvst(v(/OSC_OUT))
```



```
fvst(v(/OSC_OUT),0.2,csample=cycle_begin)
```



gdelay

This is an AC domain function which measures the group delay waveform of a given signal. The group delay can be defined as where is the phase and is the angular

frequency. $\frac{d\phi}{d\omega} \frac{d\phi}{d\omega} \omega (2\pi f)$

Syntax

```
gdelay(<signal>)
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

Errors

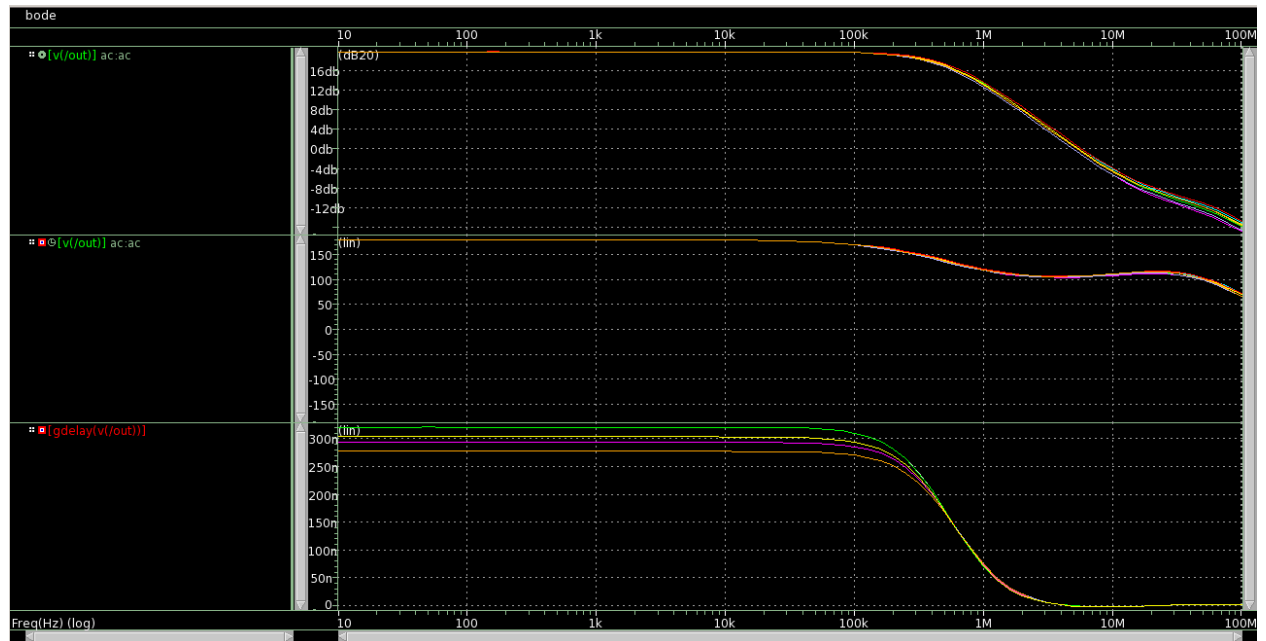
Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
gdelay('0|v(out)')
```

The following example shows how the function works on the input waveforms "v(/out)":



The example inputs the arguments as follows:

- Signal: v(/out)

The function expression is as follows:

```
gdelay(v(/out))
```

histo

Generates a histogram waveform for the input signal.

Syntax

```
histo(<signal>, [<nbins>], [<min>], [<max>], [<type>],  
      [<annotations>], [<density_estimator>], [<bandwidth>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<nbins>]`

(Optional) The number of bins in the histogram.

`[<min>]`

(Optional) The minimum threshold value.

`[<max>]`

(Optional) The maximum threshold value.

`[<type>]`

(Optional) Specifies the type of histogram:

- standard
- cumulative line — connects center of cumulative boxes.
- cumulative box

`[<annotations>]`

(Optional) Specifies cursor annotation type:

- none
- std dev lines — show where are average + (-3, -2, -1, 0, 1, 2, 3) times of standard deviation.
- % markers — show where are (0.3%, 4.5%, 32%, 50%, 68%, 95.5%, 99.7%) of the data.
- std dev lines and % markers

[<density_estimator>]

(Optional) Shows density estimation curve (on or off).

[<bandwidth>]

(Optional) Controls the bandwidth used in density estimation.

Errors

Possible error messages include:

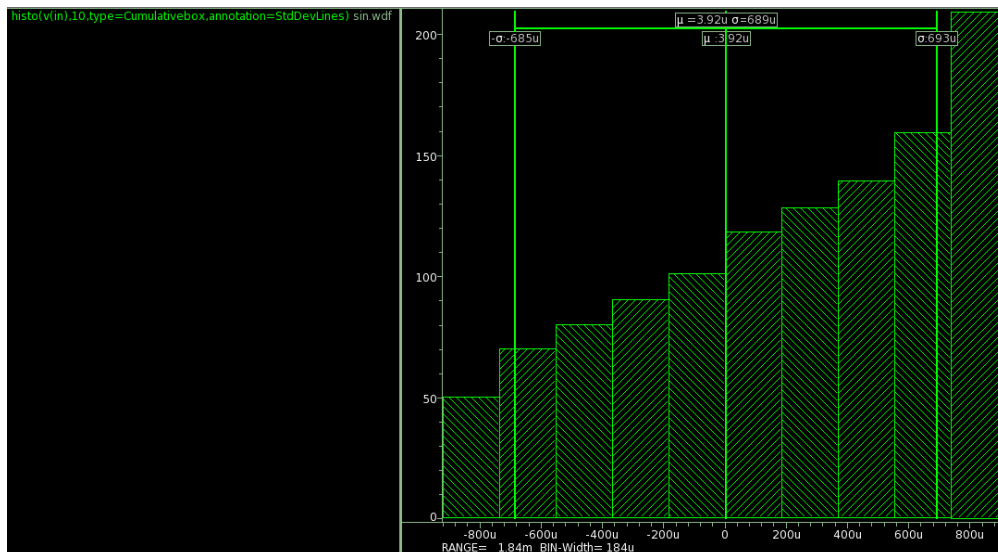
- Signal not found
- Nbins must be greater than 0
- High threshold must be greater than low threshold
- It should be in range(0, Inf)

Examples

Example 1:

```
histo(v(in),10,type=Cumulativebox,annotation=StdDevLines)
```

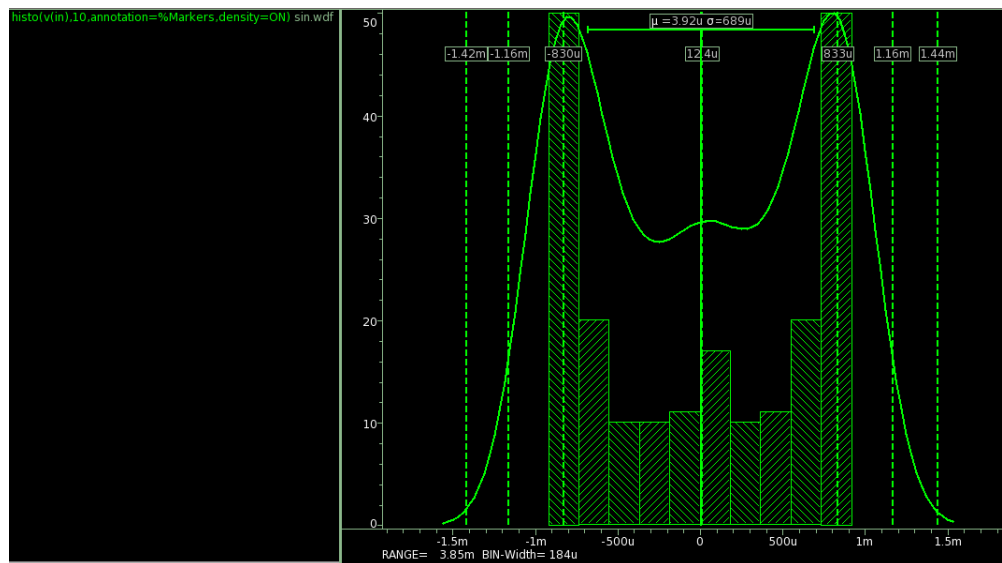
will appear as follows:



Example 2:

```
histo(v(in),10,annotation=%Markers,density=ON)
```

will appear as follows:



hold_times

Plots a waveform that contains all of the hold time data points. The x values in the waveform indicate the time at which the hold times occur.

Syntax

```
hold_times(<signal>,<clock_signal>,<clock_period>,<y_threshold>,<signal_edge>,<clock_edge>,[<x_start>],<percent>)]
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured (logic/digital signals or complex data is not supported).

<clock_signal>

The reference clock signal.

<clock_period>

The period of the reference clock_signal.

If the input value for the `clock_signal` period is 0 or smaller than 0, the `clock_period` is calculated using the `clock_signal`.

<`y_threshold`>

The threshold value of y.

<`signal_edge`>

The type of measured `signal_edge`. The valid `signal_edge` can be rise, fall, or both. The default `signal_edge` is rise.

<`clock_edge`>

The type of reference `clock_signal` edge. The valid `signal_edge` can be rise, fall, or both. The default `signal_edge` is rise.

[<`x_start`>]

(Optional) The starting value of x.

[<`x_stop`>]

(Optional) The ending value of x.

[<`percent`>]

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. When the value is 0, the `y_threshold` is not a percentage value.

Errors

Possible error messages include:

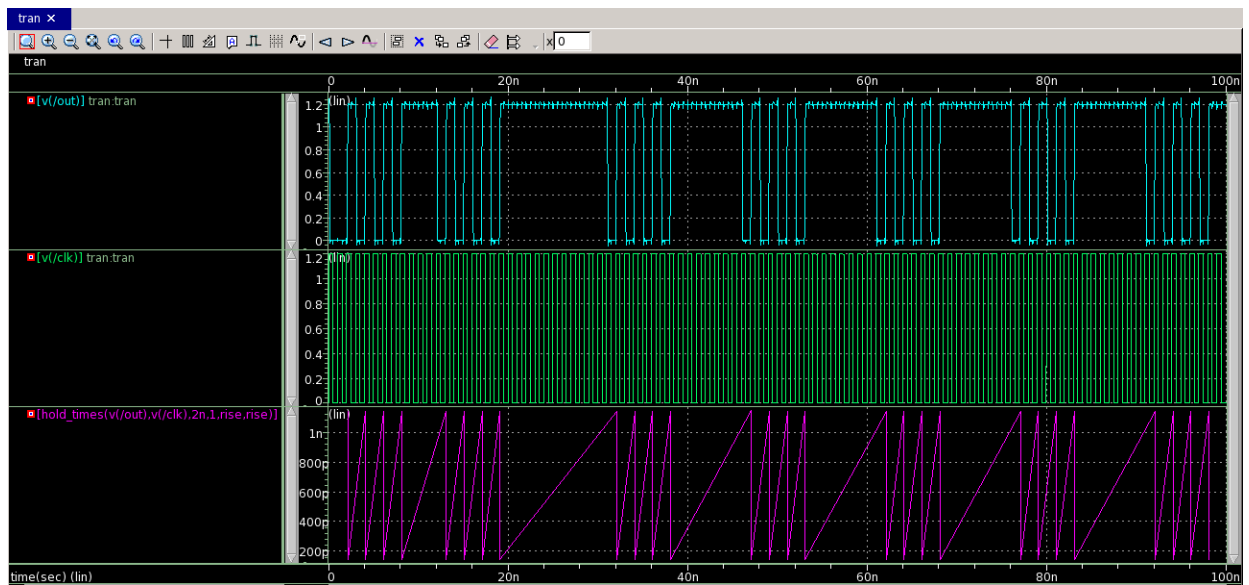
- Signal not found
- Incorrect signal type
- Y value out of range

Examples

Example 1:

```
hold_times(v(/out),v(/clk),2n,1,rise,rise)
```

will appear as follows:



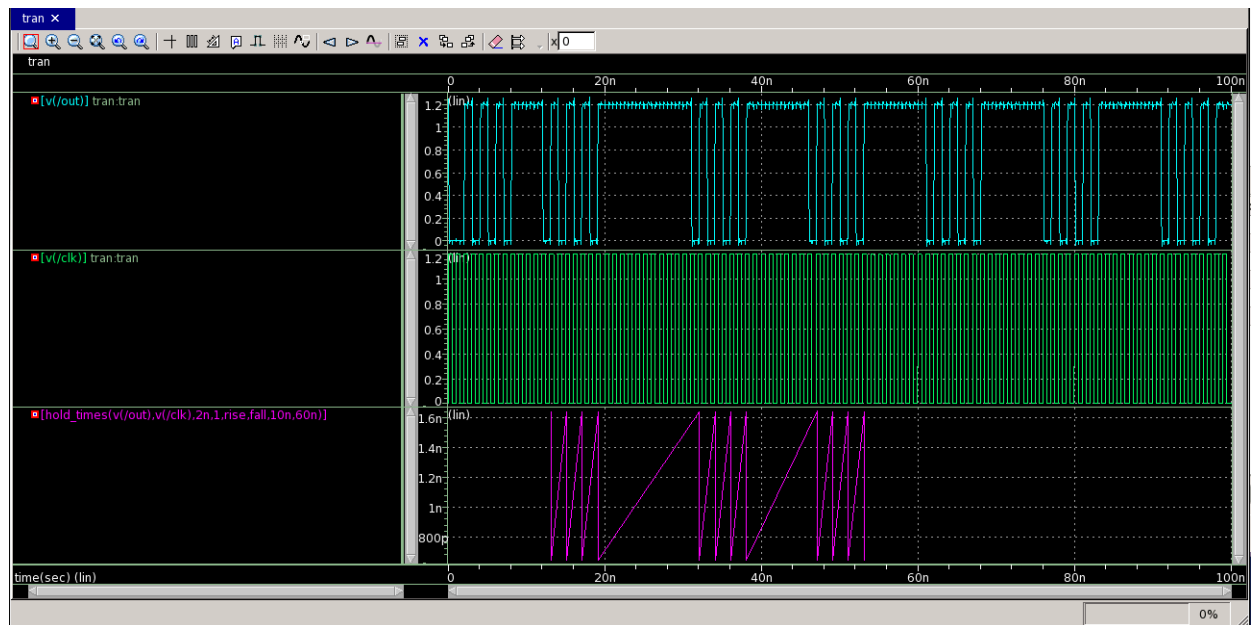
The example inputs of the arguments are as follows:

- Signal: `v(/out)`
- Clock: `v(/clk)`
- Clock period: `2n`
- Y-Threshold: `1`
- Signal Edge: `Rise`
- Clock Edge: `Rise`

Example 2:

```
hold_times(v(/out),v(/clk),2n,1,rise,fall,10n,60n)
```

will appear as follows:



The example inputs of the arguments are as follows:

- Signal: v(/out)
- Clock: v(/clk)
- Clock period: 2n
- Y-Threshold: 1
- Signal Edge: Rise
- Clock Edge: Fall
- X-start: 10n
- X-Fall: 60n

hold_times2

Like `hold_times`, plots a waveform containing all of the hold time data points. The x values in the waveform indicate the time at which hold times occur. `hold_times2` differs from `hold_times` in that it gives you the ability to specify a different threshold for the clock signal as well as the measured signal, whereas `hold_times` allows selecting the threshold of only the measured signal.

Syntax

```
hold_times2(<signal>,<clock_signal>,<clock_period>,  
            <data_threshold>,<clock_threshold>,<signal_edge>,<clock_edge>,[<x_start>],[<x_stop>],[<percent>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

<clock_signal>

The reference clock signal.

<clock_period>

The period of the reference `clock_signal`.

If the input value for the `clock_signal` period is 0 or smaller than 0, the `clock_period` is calculated using the `clock_signal`.

<data_threshold>

The threshold value of y for the measured signal.

<clock_threshold>

The threshold value of y for the `clock_signal`.

<signal_edge>

The type of measured `signal_edge`. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

<clock_edge>

The type of reference `clock_signal` edge. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

[<x_start>]

(Optional) The starting time of measurement.

[<x_stop>]

(Optional) The ending time of measurement.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. When the value is 0, the `y_threshold` is not a percentage value.

Description

Possible error messages include:

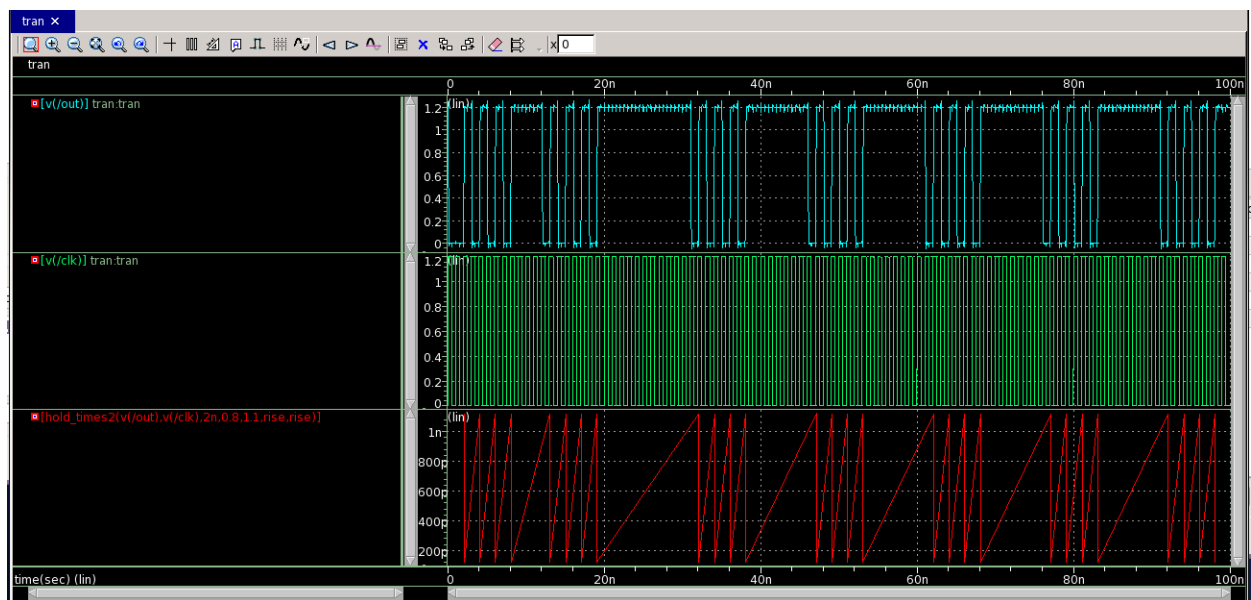
- Signal not found
- Incorrect signal type
- Y value out of range

Examples

Example 1:

```
hold_times2(v(/out),v(/clk),2n,0.6,1.1,rise,rise)
```

will appear as follows:



The example inputs of the arguments are as follows:

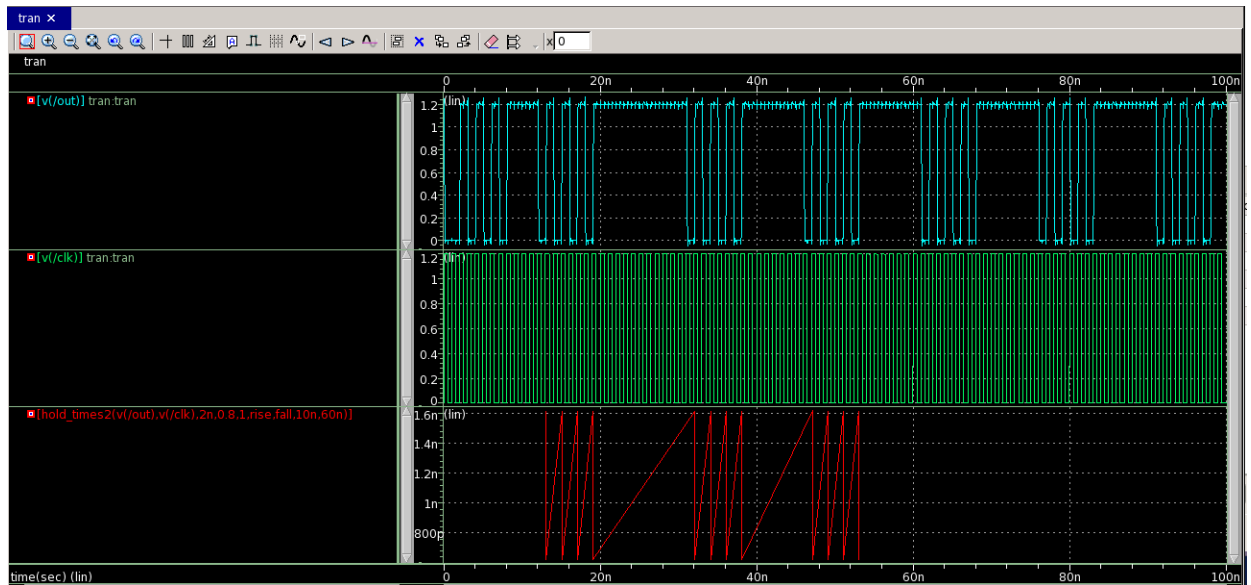
- Signal: `v(/out)`
- Clock: `v(/clk)`
- Clock period: 2n
- Y-Threshold of data: 0.6

- Y-Threshold of clock: 1.1
- Signal Edge: Rise
- Clock Edge: Rise

Example 2:

```
hold_times2(v(/out),v(/clk),2n,0.8,1,rise,fall,10n,60n)
```

will appear as follows:



The example inputs of the arguments are as follows:

- Signal: v(/out)
- Clock: v(/clk)
- Clock period: 2n
- Y-Threshold of data: 0.8
- Y-Threshold of clock: 1
- Signal Edge: Rise
- Clock Edge: Fall
- X-start: 10n
- X-Fall: 60n

ift

Calculates the Inverse Fourier transform (IFT) time domain waveform of the input frequency domain signal.

Syntax

```
ift(<signal>,<fundamental_freq>,<max_harmonic>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal.

`<fundamental_freq>`

The fundamental frequency in the frequency domain spectrum.

`<max_harmonic>`

The maximum harmonic index to be included in the IFT. For DC, the value is 0.

Errors

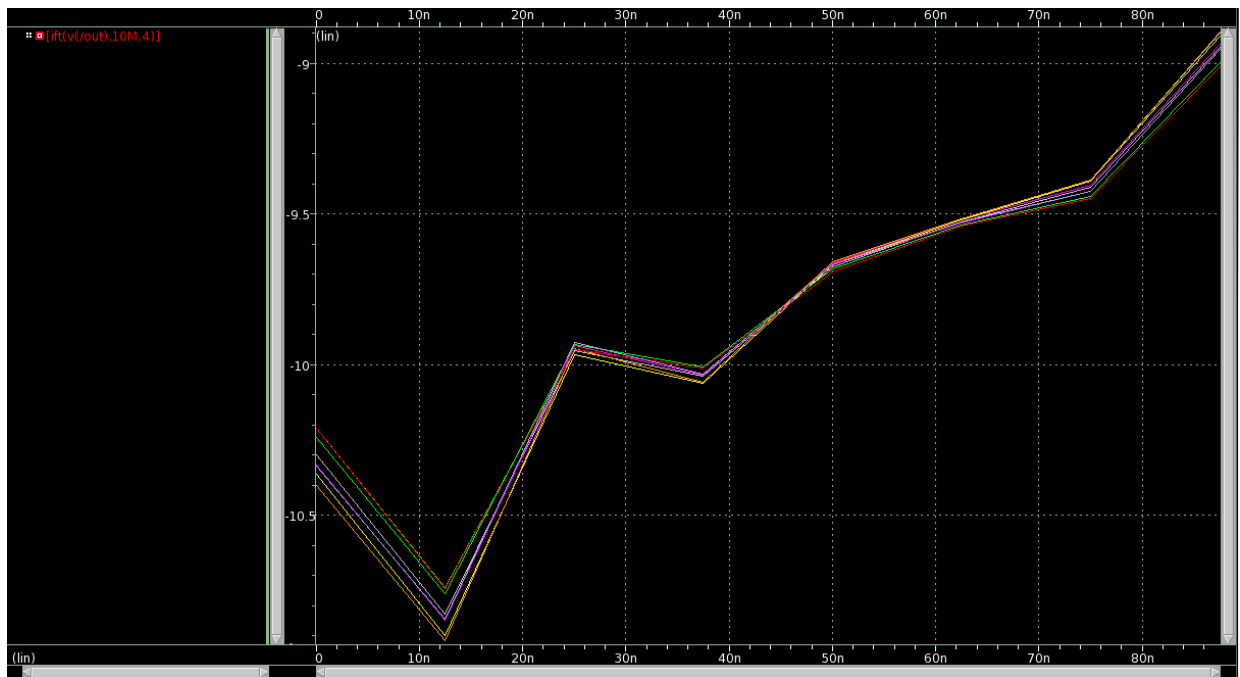
Possible error messages include:

- Signal not found
- Incorrect signal type
- Number of harmonics is negative

Examples

```
ift('0|v(out)',10M,4)
```

will return the following:



The example inputs of the arguments are as follows:

- Signal: v(/out)
- Fundamental frequency: 10M
- Maximum harmonic: 4

inl

Description

Computes the integral non-linearity of a transient simple or parametric waveform. INL is calculated as:

$$INL(D)|_{LSB} = \frac{V(D) - V(0)}{V_{LSB}} - D$$

where $0 < D < 2^N - 1$.

Syntax

```
inl(<signal>,<sampling_signal|list|step>,<cross_type>],
[<mode>],[<threshold>],[<delay>],[<method>],[<unit>],
[<no_of_samples>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be measured.

`<sampling_signal/list/step>`

This argument specifies three ways to sampling the waveform:

- `sampling_signal`: the measured the signal is sampled according to the input sampling signal. The sample points is obtained at which the waveform crosses the threshold (specified with `<mode>` and `<threshold>` options) at either rising or falling edge (specified with `<cross_type>` option). The sampling can start with a delay (specified with `<delay>` option).
- `list`: list of x values at which the sample points are obtained for the measured signal.
- `step`: the interval value specified for sampling the measured signal.

`[<cross_type>]`

(Optional) Specifies the edge types at which the sampling signal intersect with the threshold. There are two options: rising or falling.

`[<mode>]`

(Optional) Specifies the mode used to calculate the threshold value. There are two options:

- `user`: Specify the threshold value with `<threshold>` option.
- `auto`: the threshold is calculated automatically by $(\text{integral_of_waveform} / \text{x_range})$.

`[<threshold>]`

(Optional) Specifies the threshold value used to sample the data. Only specified if mode is `user`.

`[<delay>]`

(Optional) Specifies time after which the sampling begins.

`[<method>]`

(Optional) Specifies whether the end-to-end (end) or straight line (fit) method is used.

[<unit>]

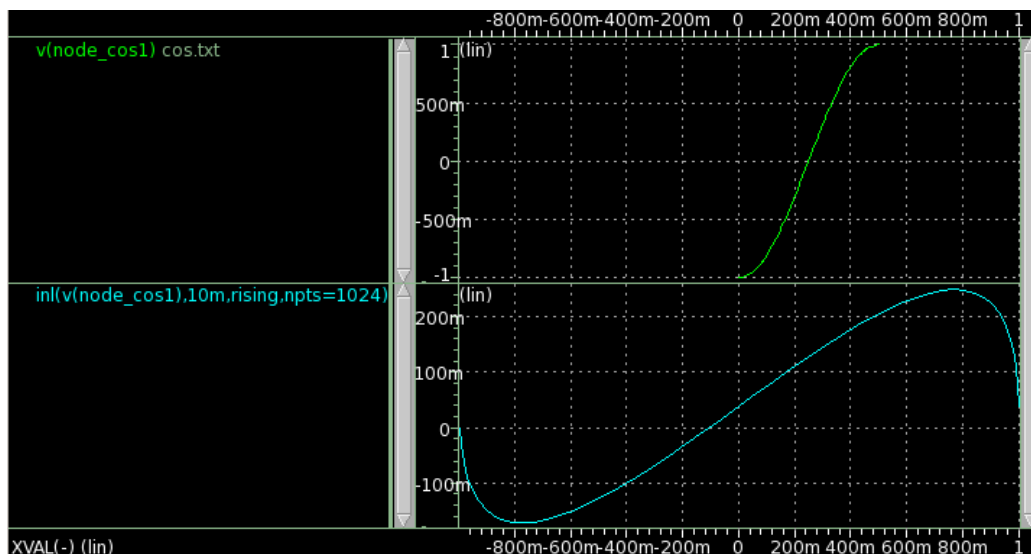
(Optional) Specifies whether the output waveform is to be output as an absolute value (abs) or multiples of least significant bit (lsb).

[<no_of_samples>]

(Optional) Specifies the samples used for calculating the non-linearity. If not specified, the samples are taken against the entire data window.

Examples

The following example shows how the function works on the input waveform “v(node_cos1)”:



The example inputs of the arguments show as follows:

- signal: v(node_cos1)
- step: 10m
- npts: 1024

The function expression shows as follows:

```
inl('v(node_cos1)',10m,'rising',npts=1024)
```

integral

Measures the running integral waveform of the input signal.

Syntax

```
integral(<signal>)
```

Returns

A waveform

Arguments

```
<signal>
```

The input signal.

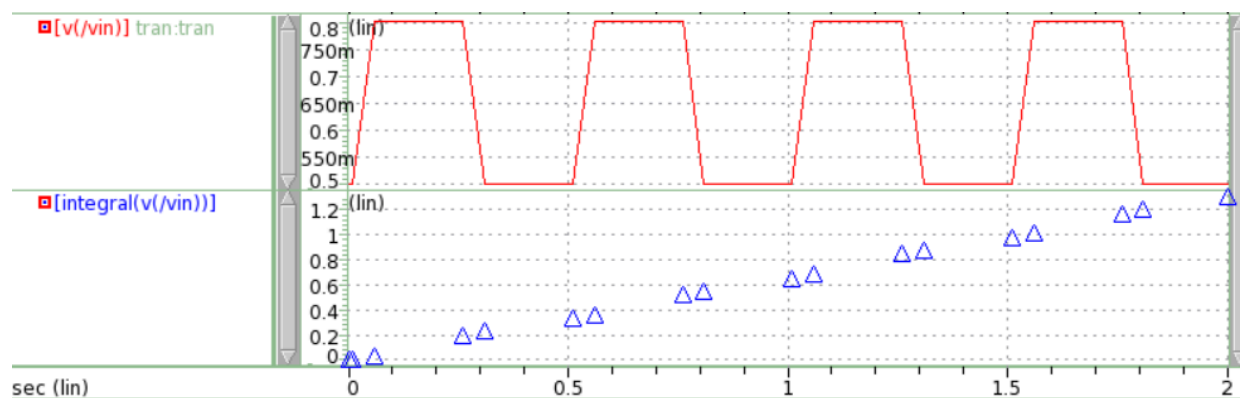
Errors

Possible error messages include:

- Signal not found

Examples

```
integral(v(/vin))
```



ipnhVRICurves()

Description

N dB intercept point VRI Curves.

Syntax

```
ipnhVRICurves (voltage signal <string>,
    harmonic indices 1 <string>,
    harmonic indices 2 <string>
    [, current signal]
    [, resistance]
    [, order 1 <integer>])
```

```
[, order 2 <integer>]  
[, extrapolationPoint 1 <string>]  
[, extrapolationPoint 2 <string>]  
[, sweepUnits <enum>])
```

Arguments

voltage signal

Input voltage signal.

harmonic indices 1

The first indices, space delimited.

harmonic indices 2

The second indices, space delimited.

current signal

(Optional) The input current signal.

resistance

(Optional) The resistance.

order 1

(Optional) Harmonic indices 1 signal's order. (Default: 1)

order 2

(Optional) Harmonic indices 2 signal's order. (Default: 3)

extrapolationPoint 1

(Optional) Harmonic indices 1 signal extrapolation point. If unspecified, defaults to lowest value.

extrapolationPoint 2

(Optional) Harmonic indices 2 extrapolation point. If unspecified, defaults to lowest value.

sweepUnits

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none (Default: dBm)

join

Generates a joined waveform from `signal1` and `signal2`, which depends on the starting x-axis value of the two signals. The waveform that has the greater starting x value is

always appended to the one with the smaller x starting value. If `signal1` and `signal2` overlap, the overlapped portion of the latter waveform is removed before joining. For the reversed sweep data whose x starting value > x stopping value from the two signals, `signal1` and `signal2` is reordered into a forward sweep in the joined waveform.

Syntax

```
join(<signal1>,<signal2>)
```

Returns

A waveform.

Arguments

<signal1>

The first input signal.

<signal2>

The second input signal.

Errors

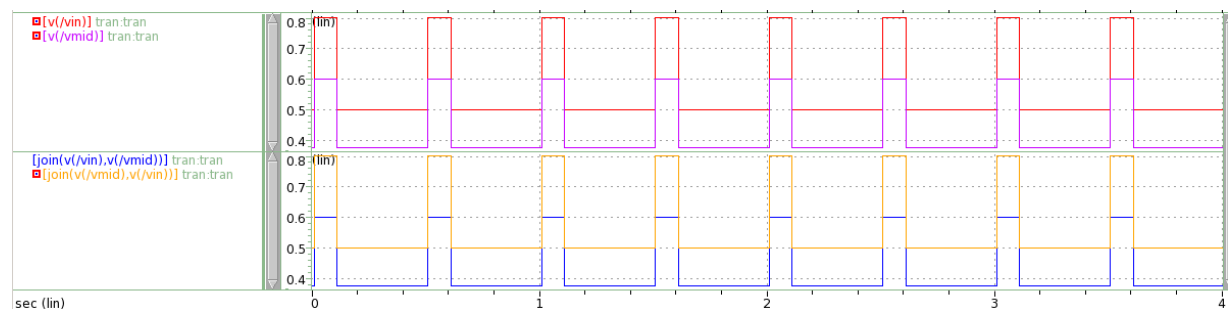
Possible error messages include:

- Signal not found

Examples

```
join(v(/vin),v(/vmid))
```

```
join(v(/vmid),v(/vin))
```



The example inputs of the arguments are as follows:

- Signal 1: `v(/vin)`
- Signal 2: `v(/vmid)`

Signal 1 and Signal 2 overlap. So, the later waveform is removed before appending. Only Signal 1 is visible. Vice-versa is true for `join(v(/vmid),v(/vin))`, where the arguments are reversed. Here, the arguments are reversed.

jvst_freq

Description

Calculates the jitter versus time given the frequency of an ideal clock for the following jitter types:

- N-Cycle and N-Period Jitter

N-Cycle jitter extends cycle-to-cycle jitter. Instead of measuring the difference between adjacent periods, a multiple grouping of N periods is measured. Where N = 3, the width of three periods is measured. N can be any value from 2 to n where only the available sample population limits n. Once again, after taking a significant sample of periods and measuring the differences in the multiple groupings, the standard deviation and peak values can be resolved. This statistical information is the N-Cycle "jitter." There are two permutations of the N-Cycle jitter. The first reports statistics from the difference between adjacent groupings (best called N-Cycle jitter). The second reports statistics from the variation of the multiple periods (best called N-Period jitter).

- Pulse Width Jitter

Measures the pulse differences between the reference signal and the target signal at the reference edge.

- Period Jitter

Measures the period differences between the reference signal and the target signal at the reference edge.

- Cycle-to-cycle Jitter

The application of simple arithmetic to the period measurements just taken. If the timing information for the two adjacent periods is known, the difference between them is the cycle-to-cycle change: period one minus period two. Again, after taking a significant sample of periods, and measuring the difference between them, the standard deviation and peak values can be resolved. This statistical information is the cycle-to-cycle "jitter".

Syntax

```
jvst_freq(<signal>,<jitter_type>,<clock_freq>,<y_at_rise>,  
<y_at_fall>,<clock_edge>,[<signal_edge>],  
[<number_of_periods>],[<search_forward>],  
[<search_range>],[<value_sign>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<jitter_type>`

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

`<clock_freq>`

Frequency for an ideal reference clock.

`<y_at_rise>`

The input signal threshold level at the rising edge.

`<y_at_fall>`

The input signal threshold level at the falling edge.

`<clock_edge>`

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<signal_edge>]`

The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<number_of_periods>]`

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is 1 for `n period`.

`[<search_forward>]`

(Optional) Always search forward for the target edge. Values can be 0 or 1. The default value is 0.

`[<search_range>]`

(Optional) The maximum target edge search range.

`[<value_sign>]`

(Optional) The forward or backward positive value sign. Values can be 0 or 1. The default value is 0, which represents the forward positive `value_sign`.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. If the value is 0, the `y_threshold` is not percentage value.

Errors

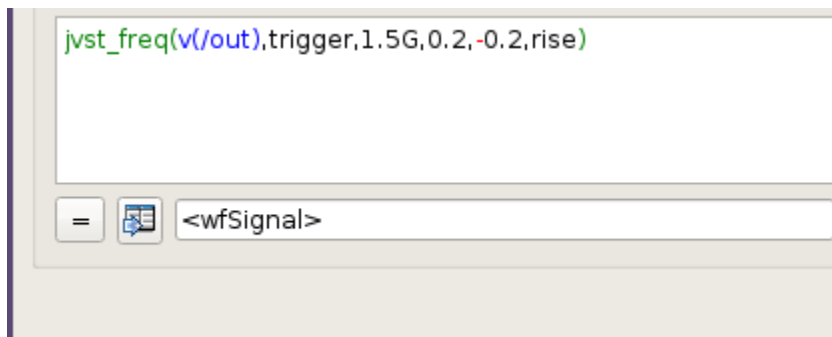
Possible error messages include:

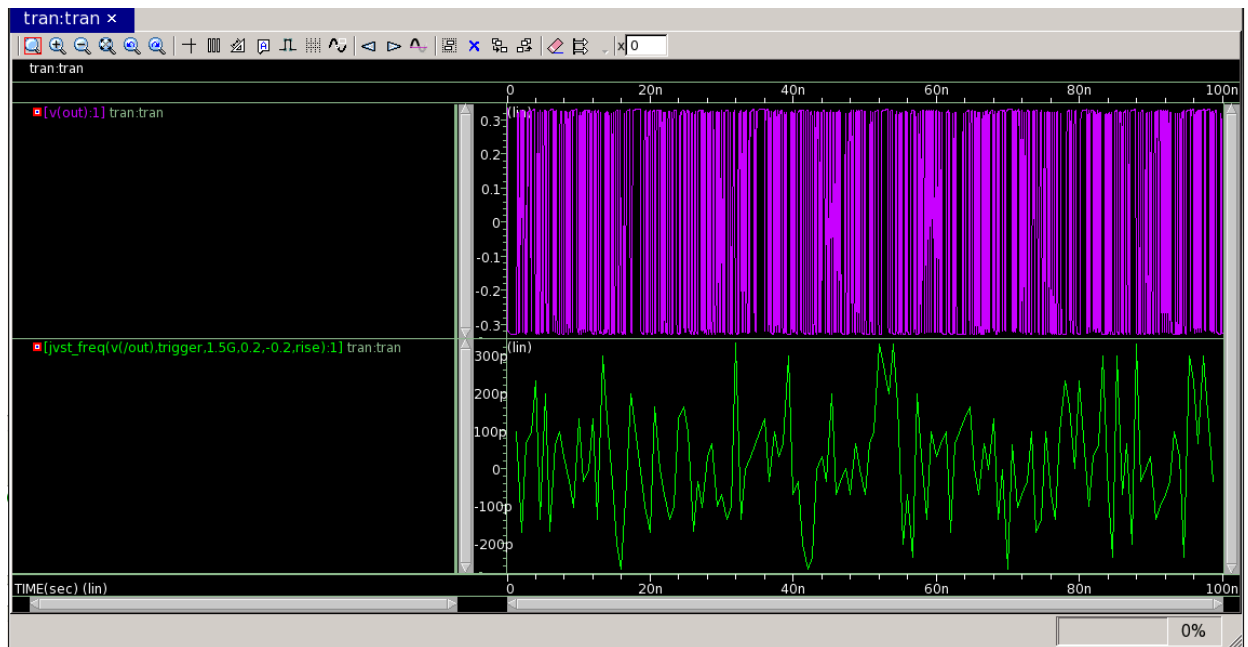
- Signal not found
- Incorrect signal type
- Invalid input value
- Frequency value can't be 0
- Unknown target edge type, should be same, rise, fall, or cross

Examples

```
jvst_freq('v(/out)', 'trigger', 1.5G, 0.2, -0.2, 'rise')  
jvst_freq(v(/out), trigger, 1.5G, 0.2, -0.2, rise, same, 1)
```

The following example shows how the function works on the input waveform "v(/out)":





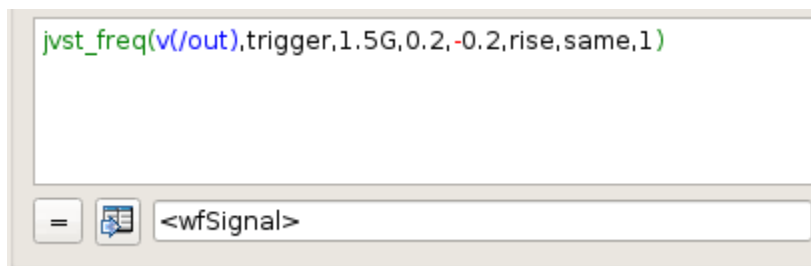
The example inputs of the arguments are:

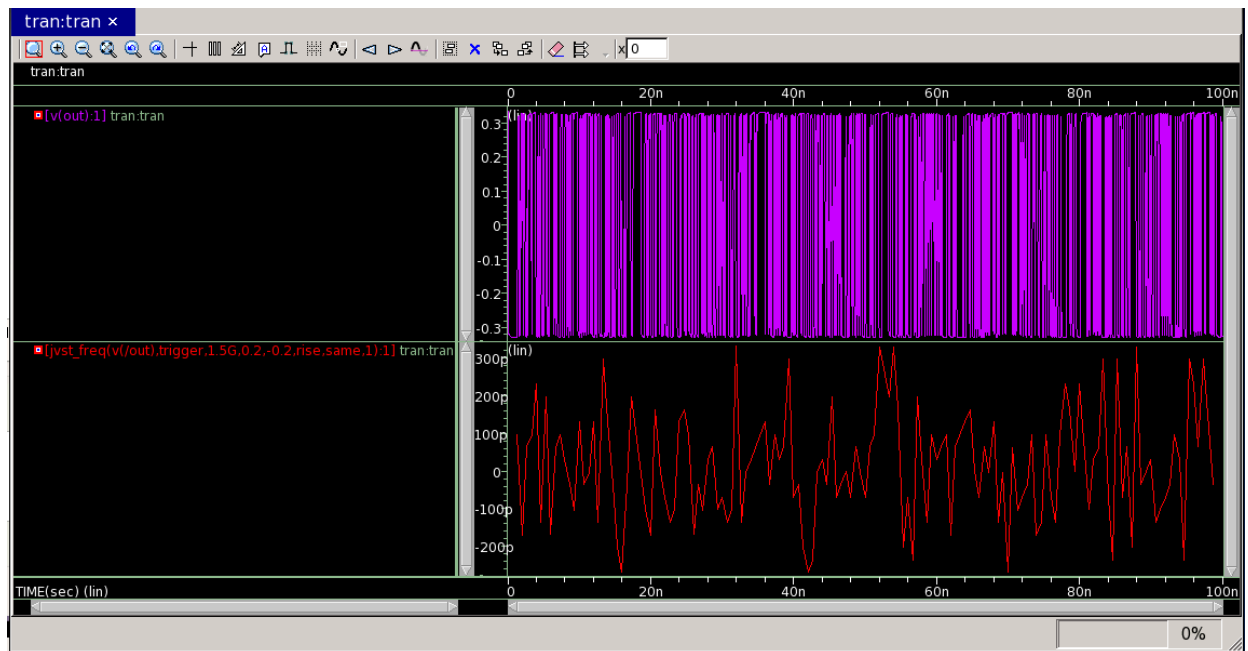
- signal: `v(/out)`
- Jitter type: `trigger`
- Ideal Clock freq: `1.5G`
- Y at rise: `0.2`
- Y at fall: `-0.2`
- Clock Edge: `Rise`

The function expression shows as follows:

```
jvst_freq(v(/out), trigger, 1.5G, 0.2, -0.2, rise)
```

Another example shows how the function works with different inputs:





The example inputs of the arguments are:

- signal: v(/out)
- Jitter type: trigger
- Ideal Clock freq: 1.5G
- Y at rise: 0.2
- Y at fall: -0.2
- Clock Edge: Rise
- Signal Edge: Same as clock
- Number of periods: 1

The function expression shows as follows:

```
jvst_freq(v(/out),trigger,1.5G,0.2,-0.2,raise,same,1)
```

jvst_ideal

Calculates the jitter versus time with an ideal clock.

Syntax

```
jvst_ideal(<signal>,<jitter_type>,<clock_delay>,  
<clock_period>,<clock_width>,<y_at_rise>,<y_at_fall>,<
```

```
<clock_edge>[,<signal_edge>][,<num_of_periods>][  
<search_forward>][,<search_range>][,<value_sign>][,<percent>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

<jitter_type>

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

<clock_delay>

Delays the time value for an ideal reference clock with a period value.

<clock_period>

Period value for reference ideal clock.

<clock_width>

Width value for an ideal reference clock. Note that the value of `clock_width` should always be smaller than the value of `clock_period`.

<y_at_rise>

The input signal threshold level at the rising edge.

<y_at_fall>

The input signal threshold level at the falling edge.

<clock_edge>

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

[<signal_edge>]

The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

[<number_of_periods>]

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is 1 for `n period`.

[<search_forward>]

(Optional) Always search forward for the target edge. Values can be 0 or 1. The default value is 0.

[<search_range>]

(Optional) Maximum target edge search range.

[<value_sign>]

(Optional) Forward or backward positive value sign. The value can be 0 or 1. The default value is 0, which represents the forward positive `value_sign`.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. If the value is 0, the `y_threshold` is not percentage value.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Invalid input value
- Frequency value can't be 0
- Unknown jitter type, should be trigger, width, period, cycle, nperiod or ncycle
- Unknown ref edge type, should be rise, fall, or both

Examples

The following example calculates jitter versus time using the `trigger` jitter type.

```
jvst_ideal(v(/out),trigger,2n,2.4n,1.2n,0.8,0.6,rise)
```

The example inputs of the arguments are:

- signal: `v(/out)`
- Jitter type: `trigger`
- Clock delay: `2n`
- Clock period: `2.4n`
- Clock width: `1.2n`
- Y at rise: `0.8`

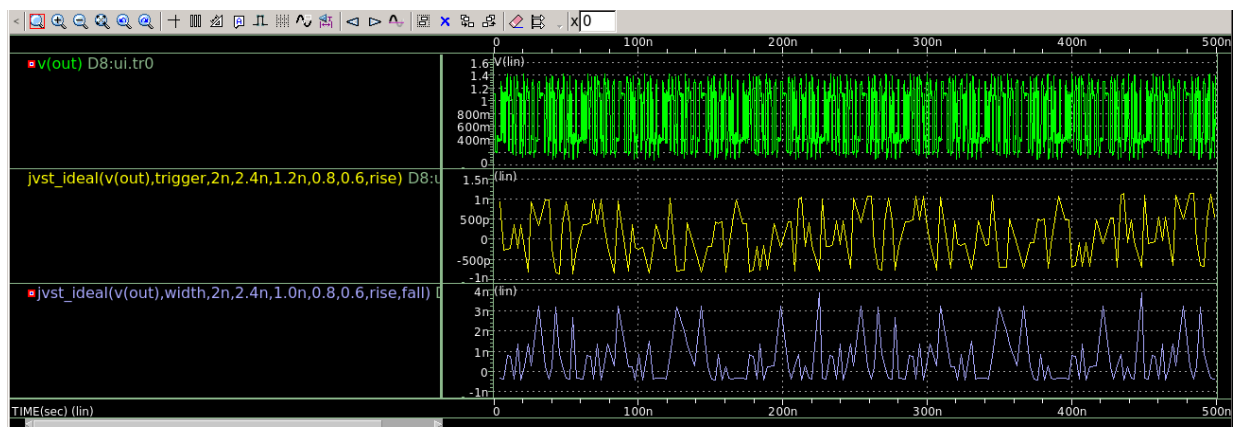
- Y at fall: 0.6
- Clock Edge: rise

The following example calculates jitter versus time using the `width` jitter type.

```
jvst_ideal(v(/out),width,2n,2.4n,1.0n,0.8,0.6,rise,fall)
```

- signal: v(/out)
- Jitter type: width
- Clock delay: 2n
- Clock period: 2.4n
- Clock width: 1.0n
- Y at rise: 0.8
- Y at fall: 0.6
- Clock Edge: rise
- Signal Edge: fall

The waveforms below illustrate how the example functions work on the input waveform "v(/out)". The yellow waveform shows the `trigger` jitter type and the blue waveform shows the `width` jitter type.



jvst_llse

Description

Calculates the jitter versus time with an estimated frequency of the ideal clock using LLSE (Linear Least Squares Estimator).

LLSE creates an X-Y plot, where X is edge number and Y is time of edge occurrence. For the ideal clock, the result is a linear line whose slope equals the period time. For the real clock, the result is a point scattered plot.

To find the closest frequency around which these points are scattered, use LLSE estimator to get the $y=ax+b$ formula of this line. This is useful for the PLL jitter calculation, because sometimes the target frequency of the PLL is unknown, and the jitter can be calculated around the optimal frequency.

Syntax

```
jvst_llse(<signal>,<jitter_type>,<y_at_rise>, <y_at_fall>,  
<clock_edge>,[<signal_edge>],  
[<number_of_periods>],[<search_forward>],  
[<search_range>],[<value_sign>],[<percent>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

<jitter_type>

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

<y_at_rise>

The input signal threshold level at the rising edge.

<y_at_fall>

The input signal threshold level at the falling edge.

<clock_edge>

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

[<signal_edge>]

(Optional) The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

[<number_of_periods>]

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is 1 for `nperiod`.

[<search_forward>]

(Optional) Always search forward for the target edge. Values can be 0 or 1. The default value is 0.

[<search_range>]

(Optional) Maximum target edge search range.

[<value_sign>]

(Optional) Forward or backward positive value sign. The value can be 0 or 1. The default value is 0, which represents the forward positive `value_sign`.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. If the value is 0, the `y_threshold` is not percentage value.

Errors

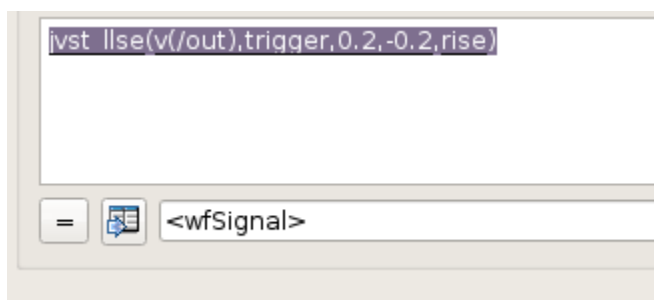
Possible error messages include:

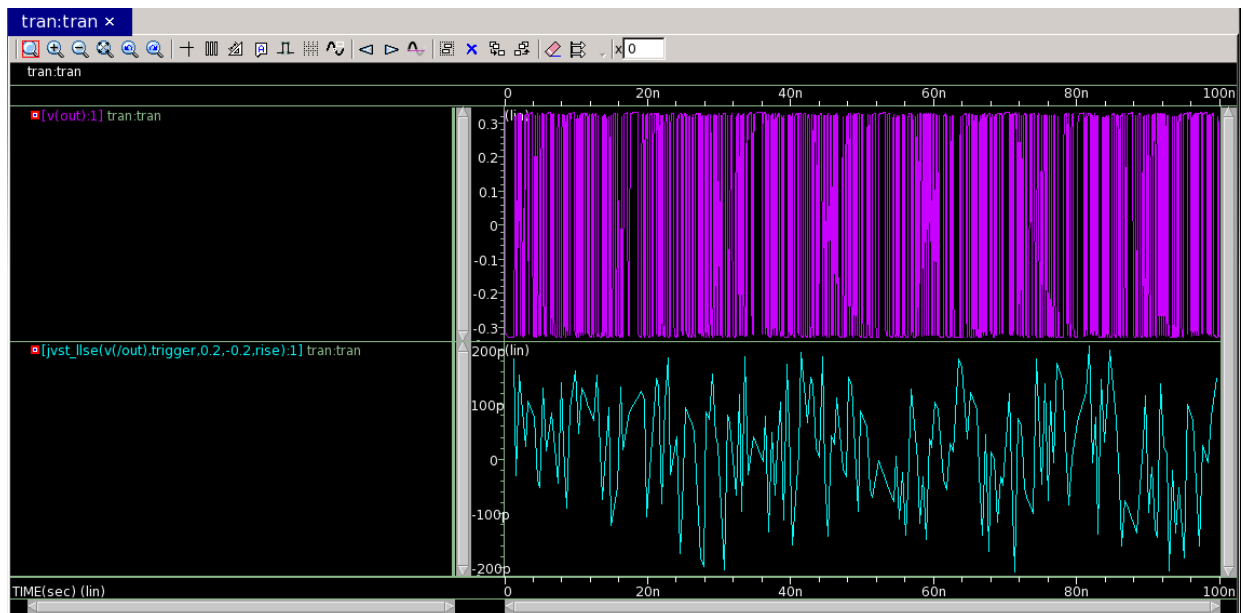
- Signal not found
- Incorrect signal type
- Invalid input value
- No valid frequency found by Linear Square Estimator
- Unknown jitter type, should be trigger, width, period, cycle, nperiod or ncycle
- Unknown ref edge type, should be rise, fall, or both

Examples

```
jvst_llse(v(/out),trigger,0.2,-0.2,rise)
```

The following example shows how the function works on the input waveform "v(/out)":





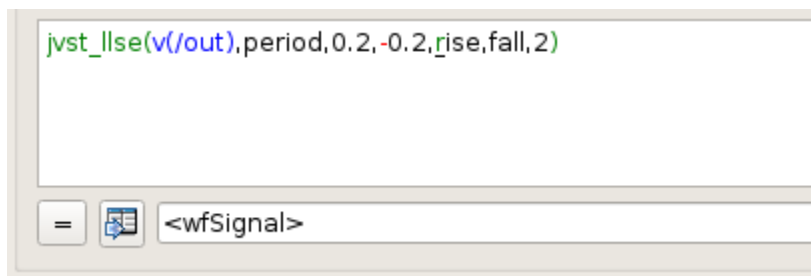
The example inputs of the arguments are:

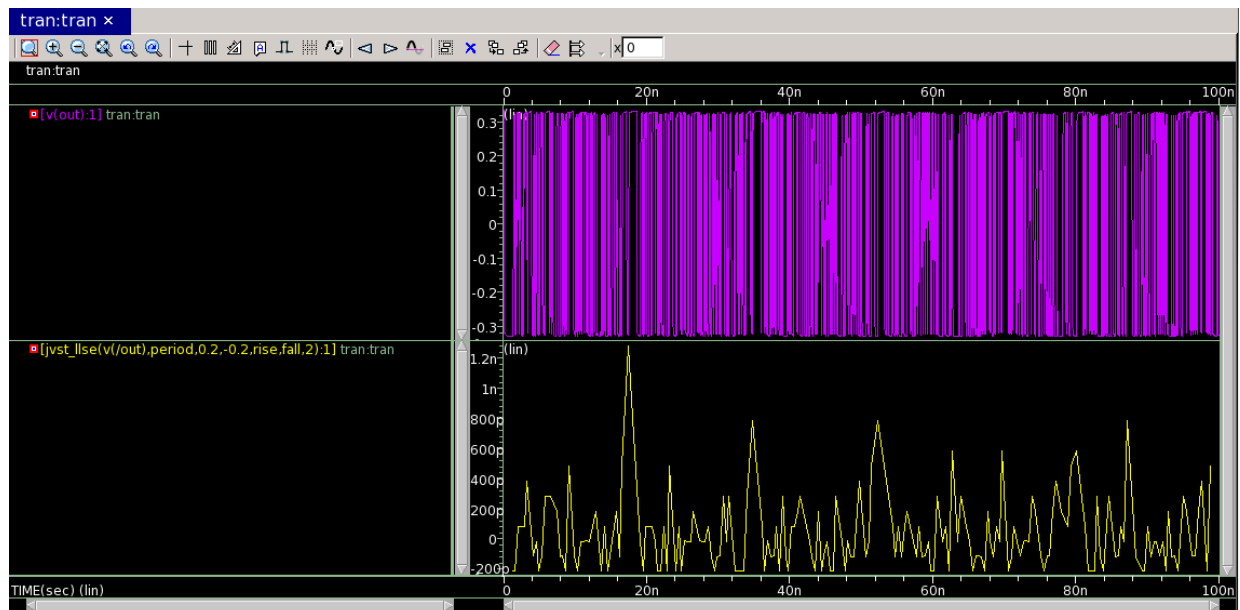
- signal: `v(/out)`
- Jitter type: `trigger`
- Y at rise: `0.2`
- Y at fall: `-0.2`
- Clock Edge: `Rise`

The function expression shows as follows:

```
jvst_llse(v(/out),trigger,0.2,-0.2,rise)
```

Another example shows how the function works with different inputs:





The example inputs of the arguments are:

- signal: v(/out)
- Jitter type: period
- Y at rise: 0.2
- Y at fall: -0.2
- Clock Edge: rise
- Signal Edge: fall
- Number of periods: 2

The function expression shows as follows:

```
jvst_llse(v(/out),period,0.2,-0.2,rise,fall,2)
```

jvst_refsig

Calculates the jitter versus time using a reference signal as the clock.

Syntax

```
jvst_refsig(<signal>,<jitter_type>,<clock_signal>,<y_at_rise>,<y_at_fall>,<clock_edge>,<signal_edge>,<clock_y_at_rise>,<clock_y_at_fall>,<number_of_periods>,<search_forward>,<search_range>,<value_sign>,<percent>)
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<jitter_type>`

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

`<clock_signal>`

The reference clock signal.

`<y_at_rise>`

The input signal threshold level at the rising edge.

`<y_at_fall>`

The input signal threshold level at the falling edge.

`<clock_edge>`

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<signal_edge>]`

The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<clock_at_y_rise>]`

(Optional) The reference signal threshold level at the rising edge.

`[<clock_at_y_fall>]`

(Optional) The reference signal threshold level at the falling age.

`[<number_of_periods>]`

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is 1 for `n period`.

`[<search_forward>]`

(Optional) Always search forward for the target edge. Values can be 0 or 1. The default value is 0.

`[<search_range>]`

(Optional) Maximum target edge search range.

[<value_sign>]

(Optional) Forward or backward positive value sign. The value can be 0 or 1.
The default value is 0, which represents the forward positive value_sign.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the y_threshold is a percentage value. If the value is 0, the y_threshold is not percentage value.

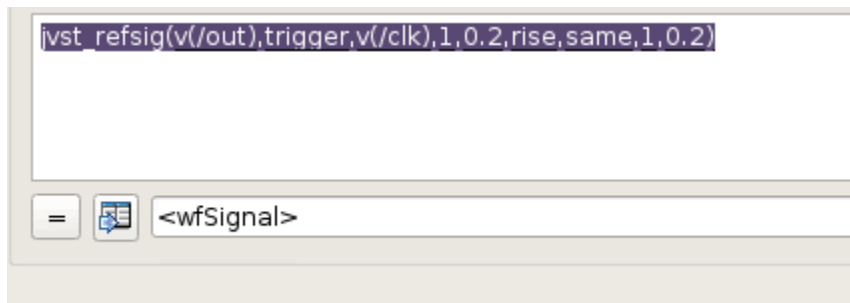
Errors

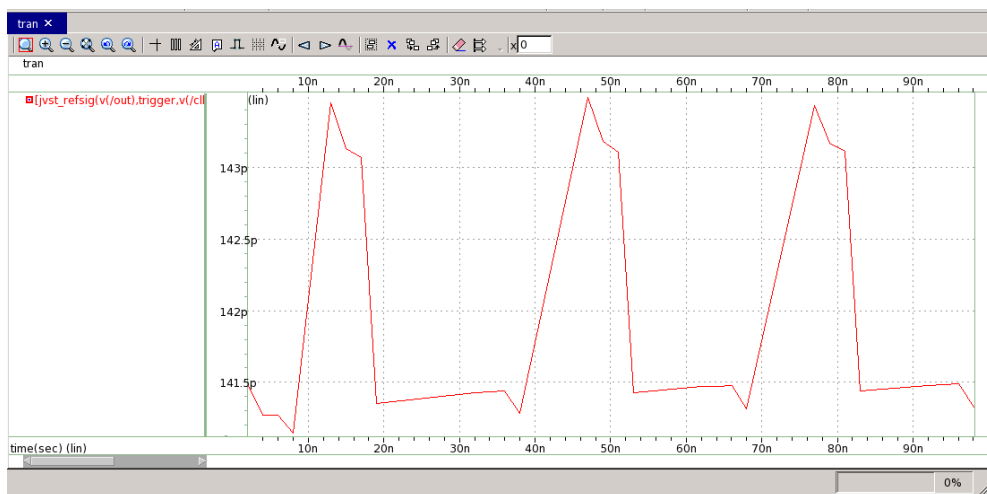
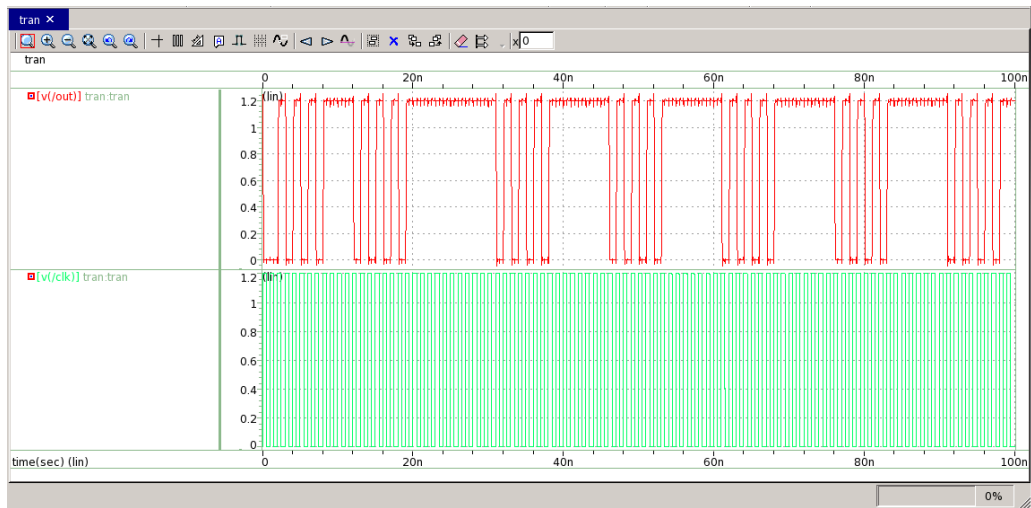
Possible error messages include:

- Signal not found
- Incorrect signal type
- Invalid input signal value
- No valid frequency found by Linear Square Estimator
- Unknown jitter type, should be trigger, width, period, cycle, nperiod or ncycle
- Unknown ref edge type, should be rise, fall, or both
- Unknown target edge type, should be same, rise, fall, or cross

Examples

```
jvst_refsig(v(/out),trigger,v(/clk),1,0.2,raise,same,1,0.2)
```





loadpull

Generates load pull contour of waveform PSS analysis.

Syntax

```
loadpull(<signal>, [<max_value>], [<min_value>], [<num_contour>],
        [<close>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<max_value>]`

(Optional) The maximum threshold value.

`[<min_value>]`

(Optional) The minimum threshold value.

`[<num_contour>]`

(Optional) The number of contours.

`[<close>]`

(Optional) The contours are closed contours (ON or OFF).

max_window_average

Maximum Window Average: calculates the maximum window average using step size between a start and a stop time.

Syntax

```
max_window_average(<signal>  
[<x_start>][<x_stop>][<time_step>][<window_size>]
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`x_start`

(Optional) The start time (`x1`). Default: `xmin`.

`x_stop`

(Optional) The stop time (`x2`). Default: `xmax`.

`time_step`

(Optional) The step size (`step`). Default: `(x_stop-x_start)`.

`window_size`

The window size (`size`). Default: `time_step`

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Input signal has no data point

Examples

```
window_average(v(/in),time_step=1n)
```

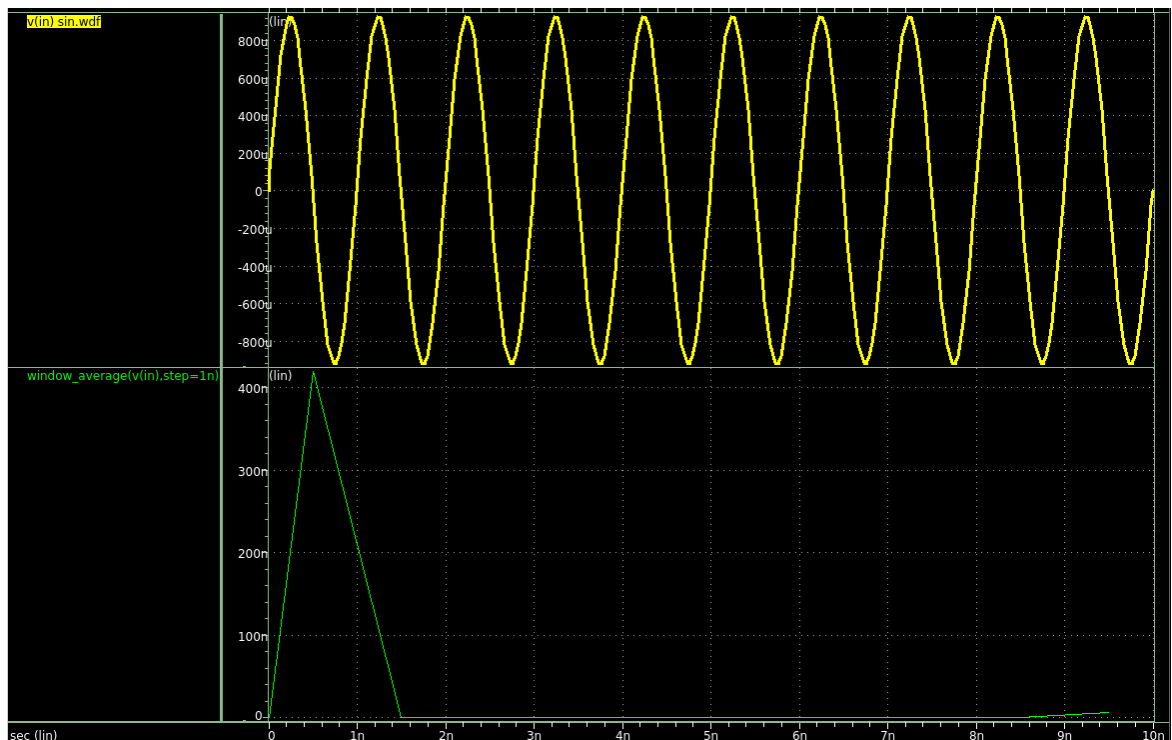
This example calculates the waveform within a range.

The first point ($x=0$) is (0, 0).

The value of `time_step` is 1n, so the second point is the average of the waveform between 0~1n, and the value is 419n, therefore the second point is (0.5n, 419n).

The last point is $((9n+10n)/2, \text{average}(9n\sim 10n)) = (9.5n, 6.07n)$.

The following example shows how the function works on the input waveform "v(/in)":



median

Measures the smoothed waveform of the given signal using median value method. For every data point in the original signal, the median value method sorts the values of the original data point and *w* points window before and after the original point. The median value of the sorted result is then used to replace the original data value.

Syntax

```
median(<signal>,<points_window>)
```

Returns

A waveform.

Arguments

<signal>

The input signal.

<points_window>

The number of points in the window.

Errors

Possible error messages include:

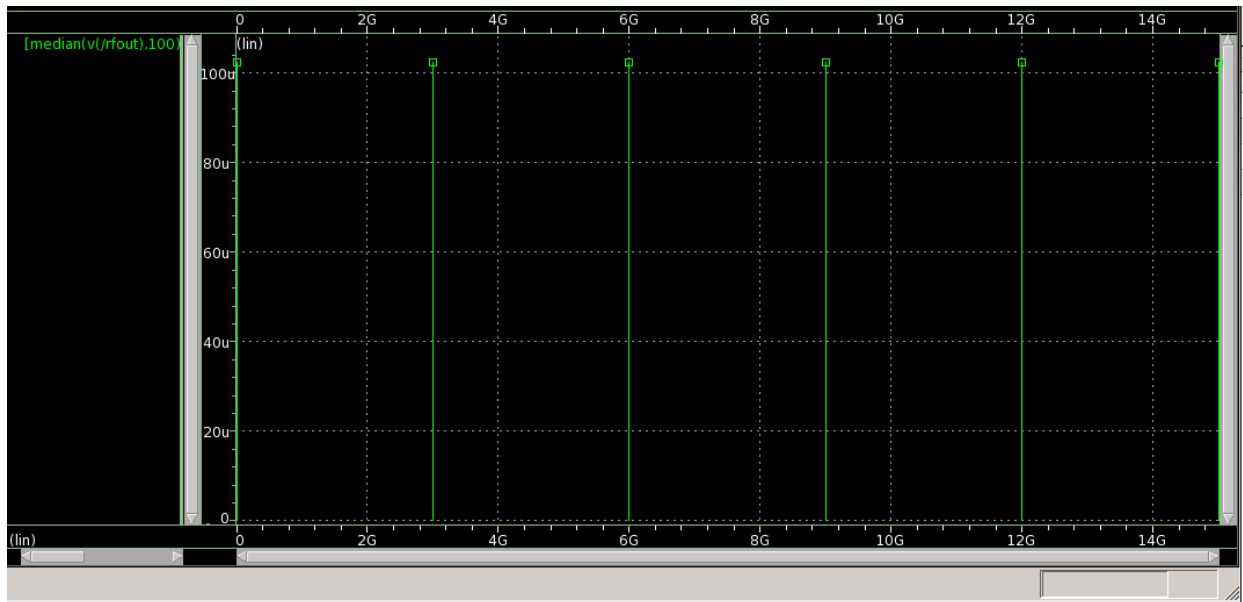
- Signal not found
- Invalid input value

Examples

```
median(v(/rfout),100)
```

The following examples show how the function works on the input waveform "v(/rfout)" for input parameters:





parametric

Gets the parametric plots of `measure` versus sweeping `param_name`.

Syntax

```
parametric(<measure>,<param_name>)
```

Returns

A waveform.

Arguments

`<measure>`

The sweeping signal to be measured.

`<param_name>`

The parameter name in the sweeping signal set to be the x component.

Errors

Possible error messages include:

- Signal not foundIncorrect signal type

Examples

```
parametric('0|v(out)','T')
```

pareto_r

Description

Calculates the Pareto regression result for the s signal and the parameter file. Pareto Analysis is a simple method for separating the major causes (the “vital few”) of a problem, from the minor ones (“trivial many”). Regression analysis is used to examine the relation of the problem (output/dependent variable) to the causes (parameters/independent variables). For the pareto calculation, the 1st order fit calculates the sensitivity values (Main Effect), R^2 , and correlation coefficient R by fitting a straight line to the measured output versus each of the parameters. This method is a 1st order polynomial least square fitting. The straight line is: $f(x) = \beta_0 + \beta_1 x$

Where β_1 is the non-normalized sensitivity.
$$\frac{\sum (xy) - (\sum (x) \sum (y)) / n}{\sum x^2 - \frac{(\sum x)^2}{n}}$$

$$\text{Adjusted: } R = \frac{\sum (xy) - \frac{\sum (y)}{n}}{\sqrt{(\sum (x^2) - \frac{(\sum x)^2}{n}) (\sum (y^2) - \frac{(\sum y)^2}{n})}} \quad R^2 = 1 - (1 - R \times R) \times \frac{n-1}{n-2}$$

Where:

- x is the vector of parameter values.
- y is the vector of waveform values,
- n is the number of points of the vectors.

Syntax

```
pareto_r(<measure_signal>,<parameter_file>,  
[<normalized_by>],[<sorted by>],[<filter_for_s>],  
[<filter_for_r2>],[<filter_for_r>],[<histbar_plot>],  
[<scatter_plot>],[<textWindow>],[<text_file>])
```

Returns

A waveform value which can have the behavior of a histbar, plot (default), scatter plot, text window, or text file.

Arguments

<measure_signal>

The input signal to be measured (sweep signals are not supported).

<parameter_file>

The name of the file containing all of the parameters.

[<normalized_by>]

(Optional) For non-interactive regressions, the sensitivity values (effects) that the Pareto calculation results in can be normalized by multiplying a normalization factor. When the factor has a negative value, the sensitivity value signs are changed by the normalization. The normalization factor can be Param/Meas , $\text{abs}(\text{Param}/\text{Meas})$, Param , $1/\text{Meas}$, or un-normalized.

The default normalization factor is Param/Meas , where Param/Meas is: $\frac{\sum x}{\sum y}$

Where $\text{abs}(\text{Param}/\text{Meas})$ is: $\frac{\sum x}{\sum y}$

Where Param is: $\sum x$

Where $1/\text{Meas}$ is: $\frac{1}{\sum y}$

For example, when calculating an absolute general normalization, the 1st order regression main effect normalized value is:

$$\text{sensitivity} = \frac{\sum (xy) - (\sum x \times \sum \frac{y}{n})}{\sum x^2 - (\frac{(\sum x)^2}{n})} \times \left| \frac{\sum x}{\sum y} \right|$$

[<sorted_by>]

(Optional) The sorted values, which can be S, R^2 , or R. For this function, you can only choose R.

[<filter_for_s>]

(Optional) The minimum value of the sensitivity filter.

[<filter_for_r2>]

(Optional) The minimum value of the regression square filter.

[<filter_for_r>]

(Optional) The minimum value of the regression filter.

[<histbar_plot>]

(Optional) Organizes the result output by histogram plot, which can be 0 or 1. The default value is 1.

[<scatter_plot>]

(Optional) Organizes the result output by scatter plot, which can be 0 and 1. The default value is 0.

[<text_window>]

(Optional) Organizes the result output by text window, which can be 0 and 1. The default value is 0.

[<text_file>]

(Optional) Organizes the result output by text file, which can be the text file name or can be missing.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot open waveform file
- Param signal not specified
- Data count are not the same
- Sweep signal is not supported

pareto_r2

Description

Calculates the Pareto regression square result for the s signal and the parameter file. The following equations for Pareto analysis calculations assume:

- x is the vector of parameter values
- y is the number of waveform values
- n is the number of points of the vectors

$$R = \frac{\sum (xy) - \left(\sum (x) \sum \frac{(y)}{n}\right)}{\sqrt{\left(\sum (x^2) - \frac{(\sum (x))^2}{n}\right) \left(\sum (y^2) - \frac{(\sum (y))^2}{n}\right)}} \quad R^2 = 1 - (1 - R \times R) \times \frac{n-1}{n-2}$$

Syntax

```
pareto_r2(<measure_signal>,<parameter_file>,  
[<normalized_by>],[<sorted_by>],[<filter_for_s>],  
[<filter_for_r2>],[<filter_for_r>],[<histbar_plot>],  
[<scatter_plot>],[<text_window>],[<text_file>])
```

Returns

A waveform which can be a histbar, plot (default), scatter plot, text window, or text file.

Arguments

<measure_signal>

The input signal to be measured (sweep signals are not supported).

<parameter_file>

The name of the file containing all of the parameters.

[<normalized_by>]

(Optional) For non-interactive regressions, the sensitivity values (effects) that the Pareto calculation results in can be normalized by multiplying a normalization factor. When the factor has a negative value, the sensitivity value signs are changed by the normalization. The normalization factor can be Param/Meas , $\text{abs}(\text{Param}/\text{Meas})$, Param , $1/\text{Meas}$, or un-normalized.

The default normalization factor is Param/Meas , where $\text{Param}/\text{Meas} = \frac{\sum x}{\sum y}$

Where $\text{abs}(\text{Param}/\text{Meas})$ is: $\frac{\sum x}{\sum y}$

Where Param is: $\sum x$

Where $1/\text{Meas}$ is: $\frac{1}{\sum y}$

For example, when calculating an absolute general normalization, the 1st order regression main effect normalized value is:

$$\text{sensitivity} = \frac{\sum (xy) - (\sum x \times \sum \frac{y}{n})}{\sum x^2 - (\frac{(\sum x)^2}{n})} \times \left| \frac{\sum x}{\sum y} \right|$$

[<sorted_by>]

(Optional) The sorted values, which can be S, R^2 , or R. You can only choose R^2 for this function.

[<filter_for_s>]

(Optional) The minimum value of the sensitivity filter.

[<filter_for_r2>]

(Optional) The minimum value of the regression square filter.

[<filter_for_r>]

(Optional) The minimum value of the regression filter.

[<histbar_plot>]

(Optional) Organizes the result output by histogram plot, which can be 0 or 1.
The default value is 1.

[<scatter_plot>]

(Optional) Organizes the result output by scatter plot, which can be 0 and 1. The default value is 0.

[<text_window>]

(Optional) Organizes the result output by text window, which can be 0 and 1.
The default value is 0.

[<text_file>]

(Optional) Organizes the result output by text file, which can be the text file name or can be missing.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot open waveform file
- Param signal not specified
- Data count are not the same
- Sweep signal is not supported

pareto_s

Description

Calculates the Pareto sensitivity result for the s signal and the parameter file. The sensitivity values can be normalized by the multiplying factor. When the factor has a

negative value, the signs of sensitivity values are changed by the normalization. The options for sensitivity normalizations are as follows:

- Multiply $\frac{\sum (x)}{\sum (y)}$
- ABS $\left(\frac{\sum (x)}{\sum (y)} \right)$
- When pseminormalized, multiply $\sum (x)$:
- When fseminormalized, multiply $\frac{1}{\sum (y)}$:
- un-normalized

For example, for absolute general normalization, the following equation applies:

$$sensitivity = \frac{\sum (x \times y) - \left(\sum x \times \sum \frac{y}{n} \right)}{\sum (x^2) - \frac{(\sum x)^2}{n}} \times \left| \frac{\sum x}{\sum y} \right|$$

Syntax

```
pareto_s(<measure_signal>,<parameter_file>,[<normalized_by>],[<sorted_by>],
[<filter_for_s>],[<filter_for_r2>],[<filter_for_r>],[<histbar_plot>],
[scatter_plot],[<text_window>],[<text_file>])
```

Returns

A waveform which can be a histbar, plot (default), scatter plot, text window, or a text file.

Arguments

<measure_signal>

The input signal to be measured (sweep signals are not supported).

<parameter_file>

The name of the file containing all of the parameters.

[<normalized_by>]

(Optional) For non-interactive regressions, the sensitivity values (effects) that the Pareto calculation results in can be normalized by multiplying a normalization factor. When the factor has a negative value, the sensitivity value signs are changed by the normalization. The normalization factor can be Param/Meas , $\text{abs}(\text{Param}/\text{Meas})$, Param , $1/\text{Meas}$, or un-normalized.

The default normalization factor is Param/Meas , where $\text{Param/Meas} = \frac{\sum x}{\sum y}$

Where $\text{abs}(\text{Param/Meas})$ is: $\frac{\sum x}{\sum y}$

Where Param is: $\sum x$

Where $1/\text{Meas}$ is: $\frac{1}{\sum y}$

For example, when calculating an absolute general normalization, the 1st order regression main effect normalized value is:

$$\text{sensitivity} = \frac{\sum (xy) - (\sum x \times \sum \frac{y}{n})}{\sum x^2 - (\frac{(\sum x)^2}{n})} \times \left| \frac{\sum x}{\sum y} \right|$$

[<sorted_by>]

(Optional) The sorted values, which can be S, R², or R. You can only choose R² for this function.

[<filter_for_s>]

(Optional). The minimum value of the sensitivity filter.

[<filter_for_r2>]

(Optional) The minimum value of the regression square filter.

[<filter_for_r>]

(Optional) The minimum value of the regression filter.

[<histbar_plot>]

(Optional) Organizes the result output by histogram plot, which can be 0 or 1. The default value is 1.

[<scatter_plot>]

(Optional) Organizes the result output by scatter plot, which can be 0 and 1. The default value is 0.

[<text_window>]

(Optional) Organizes the result output by text window, which can be 0 and 1. The default value is 0.

[<text_file>]

(Optional) Organizes the result output by text file, which can be the text file name or can be missing.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot open waveform file
- Param signal not specified
- Data count are not the same
- Sweep signal is not supported

pavg

Description

Calculates the average value of signal based on the given period. Applies the sample first, then performs the calculation.

Syntax

```
pavg(<signal>,<x_start>,<x_end>,[<period>],  
[<sampling_factor>])
```

Returns

An analog signal.

Arguments

<signal>

The signal to be measured.

<x_start>

The start of x value to be measured.

<x_end>

The end of x value to be measured.

[<period>]

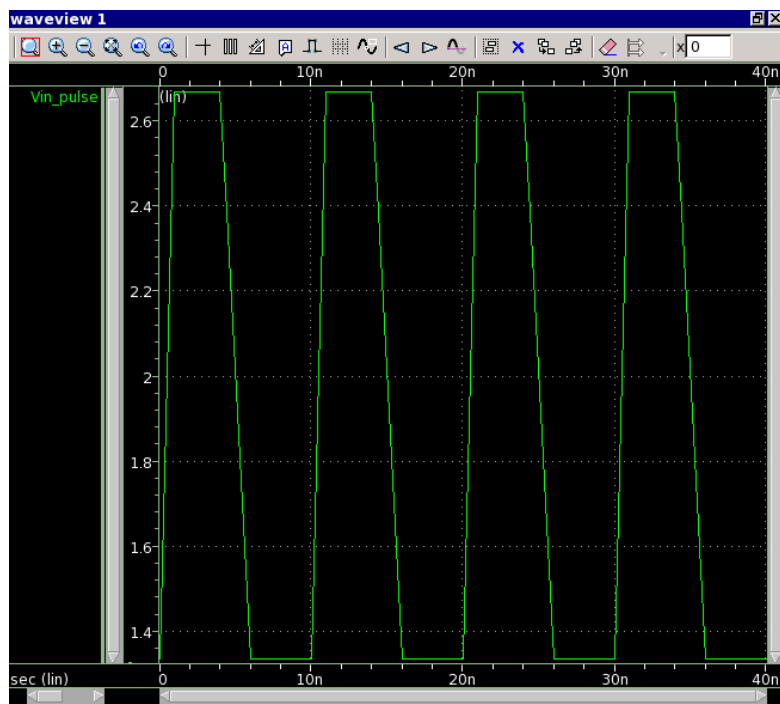
(Optional) Specifies the period of the waveform. The default value is the range of x.

[<sampling_factor>]

(Optional) Specifies the value of the sampling factor. The larger the factor, the more points to be included. The value should be larger than 0. The default value is 1.

Examples

The following example shows how the `pavg` function works on the input waveform.



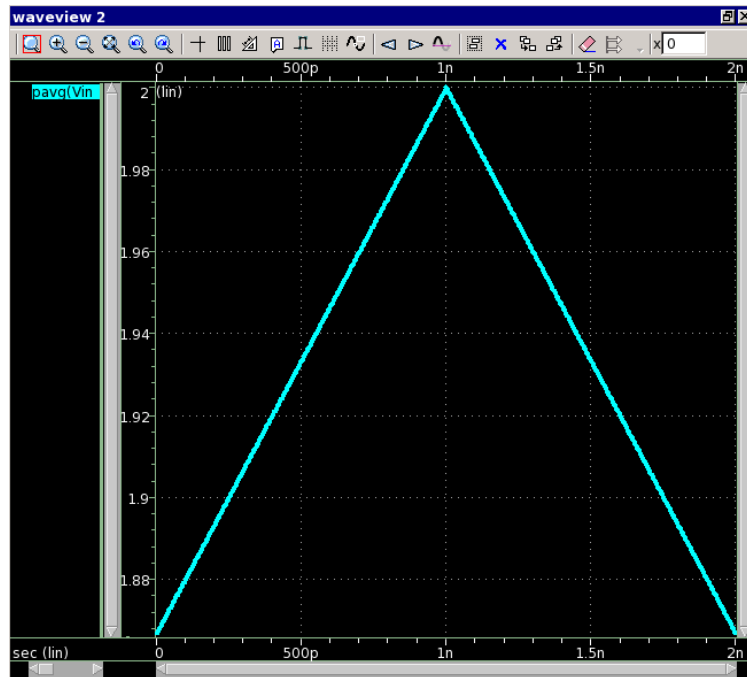
The example inputs of the arguments are as follows:

- signal: `Vin_pulse`
- `x_start`: 0
- `x_end`: 40n
- period: 2n
- sampling factor: 1

The function expression shows as follows:

```
pavg('Vin_pulse',0,40n,2n,1)
```

with the output waveform being plotted as the following:



peak

Description

Calculates the peaks of a given signal.

Syntax

```
peak(<signal>,[<x_start>],[<x_end>],[<x_tolerance>],[<y_tolerance>],[<last>])
```

Returns

An analog signal.

Arguments

<signal>

The signal to be measured.

[<x_start>]

(Optional) The starting value of x. Default is first x point.

[<x_end>]

(Optional) The ending value of x. Default is last x point.

[<x_tolerance>]

(Optional) Filters all peaks within the distance on the x-axis.

[<y_tolerance>]

(Optional) Filters all peaks within the distance on the y-axis.

[<last>]

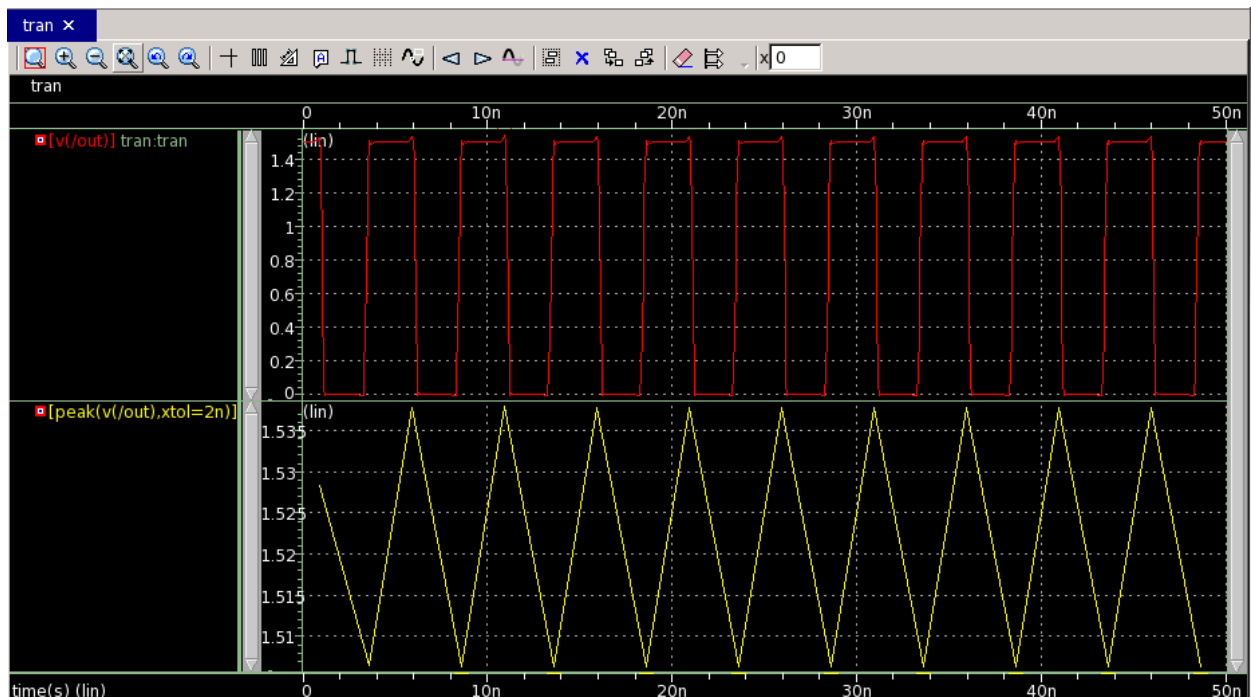
(Optional) Determines whether to count the last point as a peak.

Examples

```
peak(v(/out),xtol=2n)  
peak(v(/out),10n,30n,2n)
```

The following examples show how the function works on the input waveform "v(out)" for different input parameters:

Example 1:



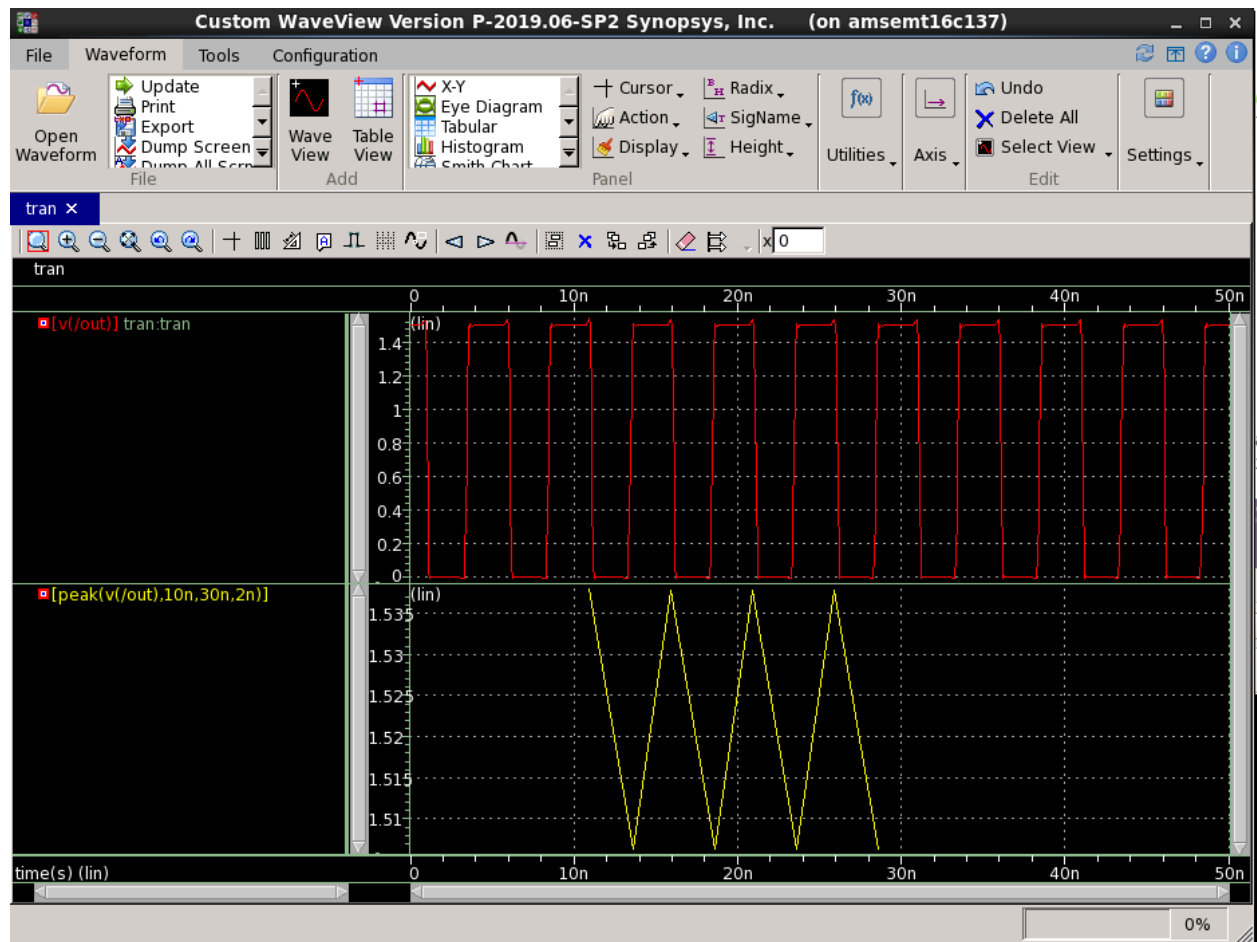
The example inputs of the arguments shown are as follows:

- Signal1: v(/out)
- X tolerance=2n

The function expression shows as follows:

```
peak(v(/out),xtol=2n)
```

Example 2:



The example inputs of the arguments shown are as follows:

- Signal1: v(/out)
- X Start=10n

- X Stop=30n
- X tolerance=2n

The function expression shows as follows:

```
peak(v(/out),10n,30n,2n)
```

phasenoise

Calculates the single-sideband noise of a given signal.

Syntax

```
phasenoise(<signal>,<fundamental_freq>,[<x_start>],  
[<x_stop>],[<sampling_freq>],[<min_offset_freq>],  
[num_of_cycles],[<windowing_type>])
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured (complex signals are not supported).

<fundamental_freq>

The fundamental frequency.

[<x_start>]

(Optional) The starting time for the measured signal.

[<x_stop>]

(Optional) The ending time for the measured signal.

[<sampling_freq>]

(Optional) The sampling frequency. The default is 128*fundamental frequency.

[<min_offset_freq>]

(Optional) Minimum offset frequency. The default is frequency is 10.

[<num_of_cycles>]

(Optional) Number of delay clock cycles. The default number is 0.

[<windowing_type>]

(Optional) The type of window in measuring the signal. The default is "rectangle."

Errors

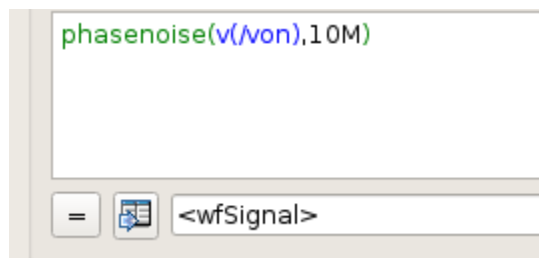
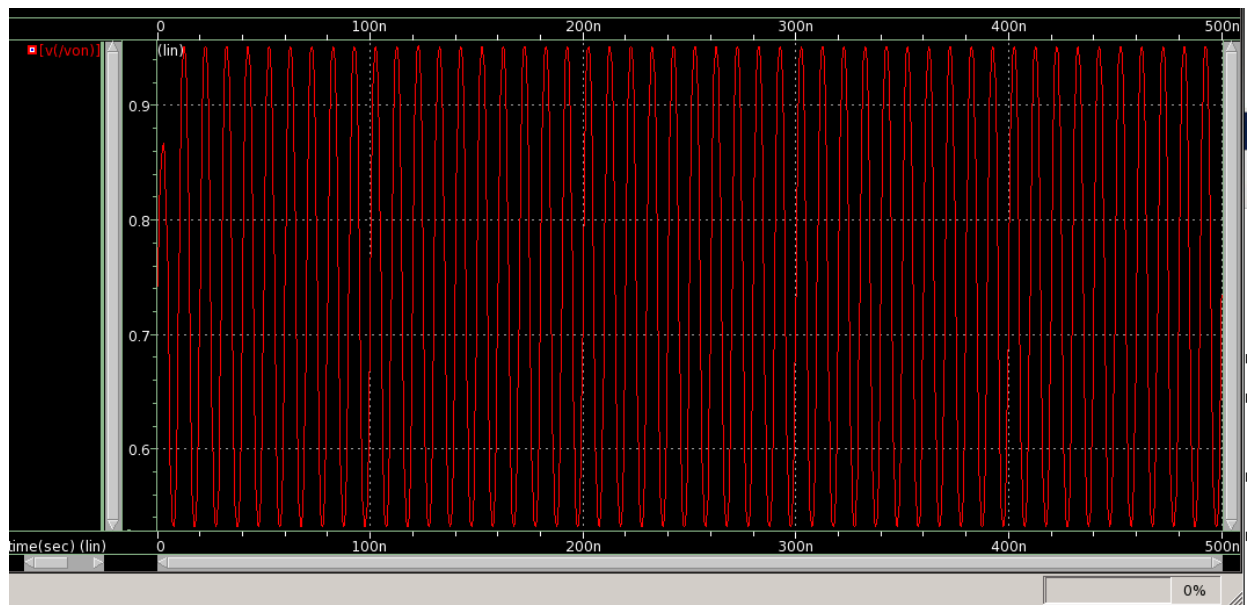
Possible error messages include:

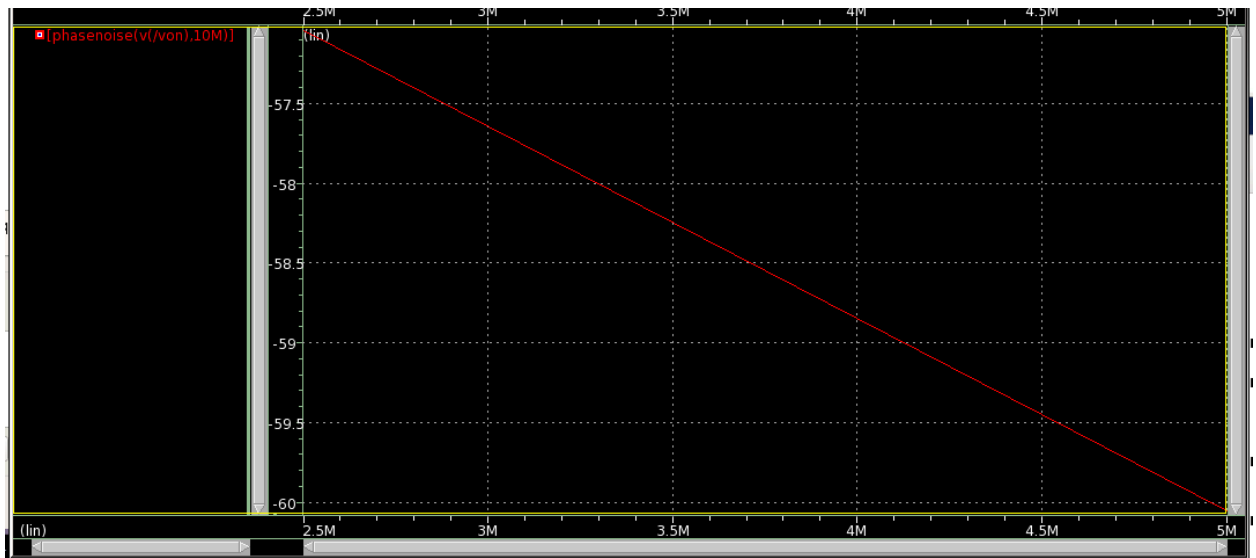
- Signal not found
- Incorrect signal type
- Fundamental frequency must be > 0X value out of range

Examples

```
phasenoise(v(/von),10M)
```

The following example shows how the function works on the input waveform "v(/von)" on input parameters:





prms

Description

Calculates the root-mean-square value of signal based on the given period. Applies the sample first then performs the calculation.

Syntax

```
prms(<signal>,<x_start>,<x_end>,[<period>],  
[<sampling_factor>])
```

Returns

An analog signal.

Arguments

<signal>

The signal to be measured.

<x_start>

The start of x value to be measured.

<x_end>

The end of x value to be measured.

[<period>]

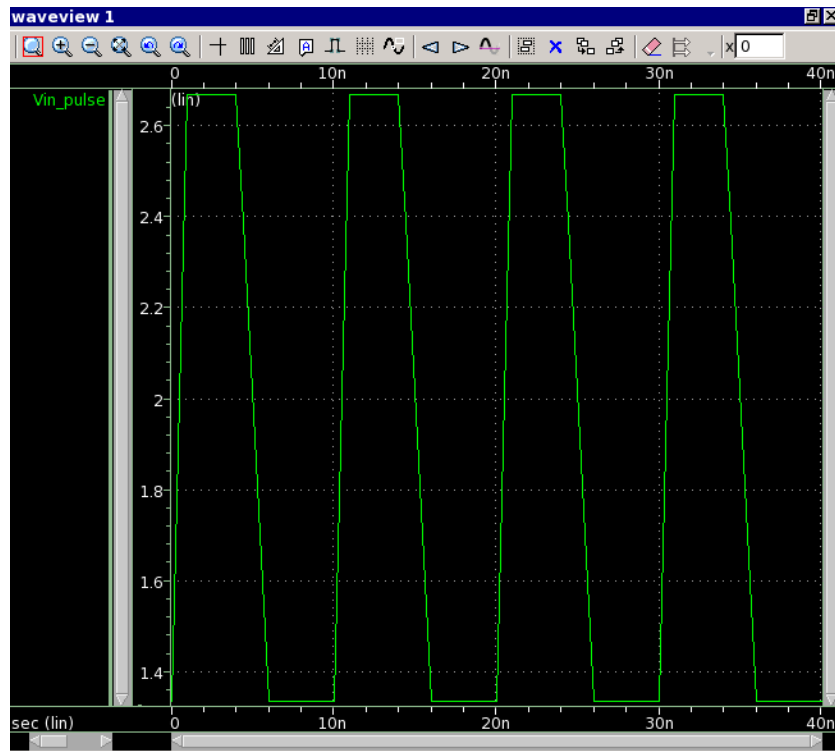
(Optional) Specifies the period of the waveform. The default value is the range of x.

[<sampling_factor>]

(Optional) Specifies the value of the sampling factor. The larger the factor, the more points to be included. The value should be larger than 0. The default value is 1.

Examples

The following example shows how the function works on the input waveform.



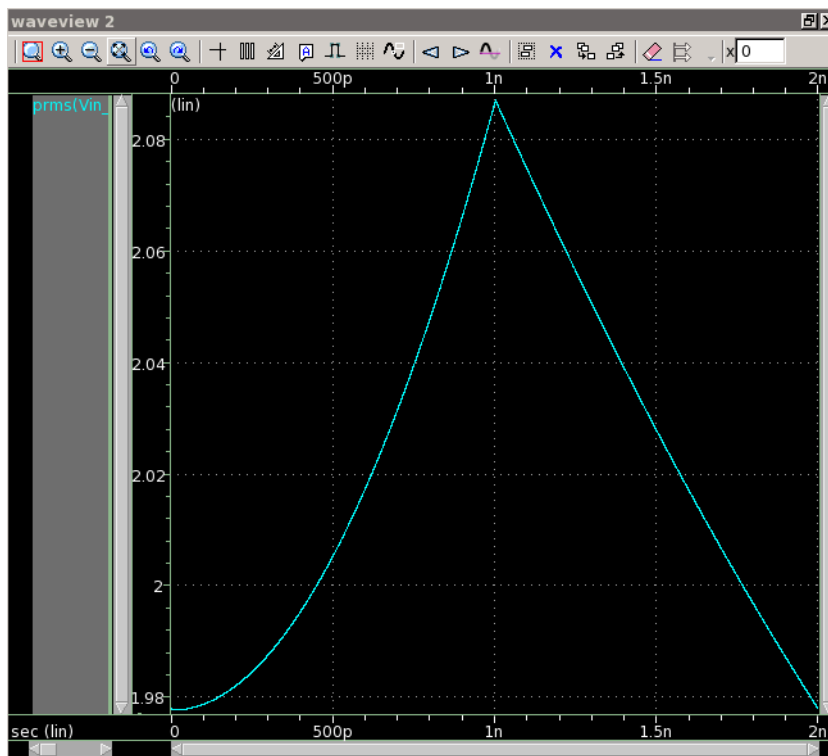
The example inputs of the arguments are as follows:

- signal: Vin_pulse
- x_start: 0
- x_end: 40n
- period: 2n
- sampling factor: 1

The function expression shows as follows:

```
prms('Vin_pulse',0,40n,2n,1)
```

with the output waveform being plotted as the following:



psd

Description

Power spectral density. Describes how the power of a signal or time series is distributed with frequency. This function estimates Power Spectral Density using the overlapped segmented averaging of periodograms method. A periodogram is the DFT of one segment of the signal or time series.

Syntax

```
psd(<signal>,<x_start>,<x_stop>,<num_of_points>,  
[<windowing_type>],[<window_alpha>],  
[<window_size>],[<detrend_mode>],  
[<coherent_gain>],[<coherent_gain_factor>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be calculated.

`<x_start>`

Starting time for the spectral analysis interval.

`<x_stop>`

Ending time for the spectral analysis interval.

`<num_of_points>`

Number of time domain points to be used. Default value is 512.

[`<windowing_type>`]

(Optional) Specifies window to be used.

[`<windowing_alpha>`]

This argument is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 0 to 15. The value 0 implies no smoothing and is equivalent to a rectangular window. The default value is 1.

[`<window_size>`]

(Optional) Number of frequency domain points to use in the Fourier analysis.

[`<detrend_mode>`]

(Optional) Specifies the trend mode (linear, mean, none).

- linear: compute a least squares regression line for each segment and subtract that line from the data points of the segment
- mean: subtract the average value from the data points of the segment
- none (default): no trend removal

[`<coherent_gain>`]

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with `coherent_gain_factor`

- dB20: scaling with $\text{pow}(10, \text{coherent_gain_factor}/20)$
- dB10: scaling with $\text{pow}(10, \text{coherent_gain_factor}/10)$

[<coherent_gain_factor>]

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by $1/(\text{Coherent Gain})$. Valid values are $0 < 1$. Default is 0. You can use 1 if you do not want this scaling factor to be used.

Examples

```
psd(v(/von), 200n, 300n, 30)
```

The following example shows how the function works on the input waveform:



The example inputs of the arguments shown as follows:

- Signal1: `v(/von)`
- X Start: `200n`

- X Stop:400n
- Number of points: 30

psdbb

Description

Power spectral density baseband. Returns an estimate for the power spectral density of a waveform1+j * waveform2.

Syntax

```
psdbb(<signal1>,<signal2>,<x_start>,<x_stop>,  
<num_of_points>,[<windowing_type>],[<window_alpha>],  
[<window_size>],[<detrending_mode>],  
[<coherent_gain>],[<coherent_gain_factor>])
```

Returns

An analog signal.

Arguments

<signal1>

First signal to be calculated.

<signal2>

Second signal to be calculated.

<x_start>

Starting time for the spectral analysis interval.

<x_stop>

Ending time for the spectral analysis interval.

<num_of_points>

Number of time domain points to be used. Default value is 512.

[<windowing_type>]

(Optional) Specifies window to be used.

[<windowing_alpha>]

This argument is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 0 to 15. The value 0 implies no smoothing and is equivalent to a rectangular window. The default value is 1.

[<window_size>]

(Optional) Specifies number of frequency domain points to use in the Fourier analysis. A larger window size results in an expectation operation over fewer samples, which leads to larger variations in the power spectral density. A small window size can smear out sharp steps in the power spectral density that might be present. Default value is 256.

[<detrending_mode>]

(Optional) Specifies the trend mode (linear, mean, none). If there is a deterministic trend to the underlying data, you might want to remove the trend before performing the spectral analysis. For example, consider analyzing phase noise in a VCO model. Without the noise, the phase increases more or less linearly with time, so it is appropriate to set the mode to linear.

To subtract an average value, set the to mean. If the spectrum of raw data is desired, set the mode to none. Default value is none.

[<coherent_gain>]

(Optional) Specifies the zero frequency gain (or the DC gain) of the window. When performing dft, the applied window type (aside from the rectangular type) changes the signal amplitude. Applying a coherent gain factor is a way to get consistent results regardless of the window type. Valid Values are none, default, magnitude, dB20, or dB10.

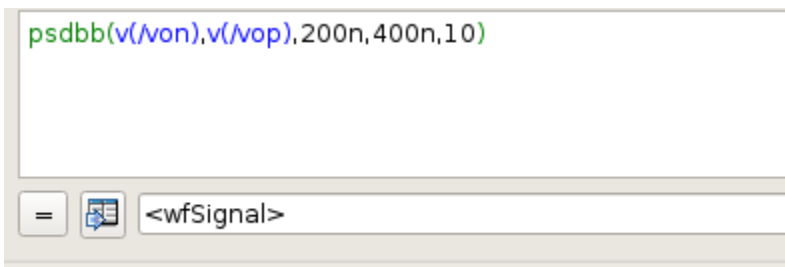
[<coherent_gain_factor>]

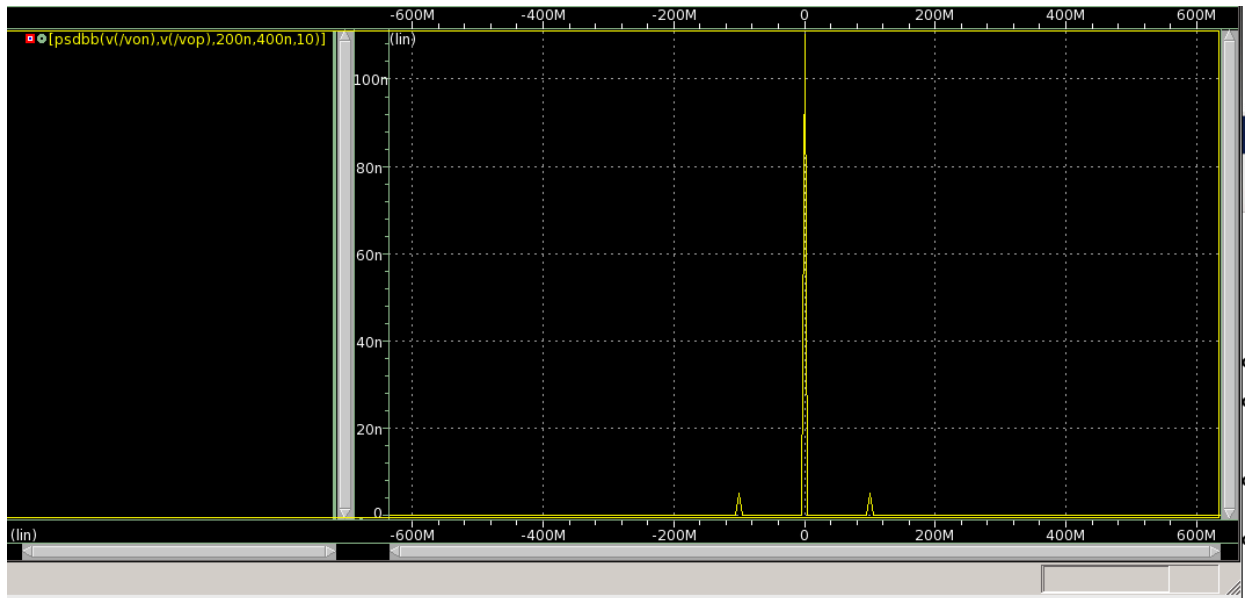
(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are $0 < 1$. Default is 0. You can use 1 if you do not want this scaling factor to be used.

Examples

```
psdbb(v(/von),v(/vop),200n,400n,10)
```

The following example shows how the function works on the input waveform "v(/von)" on input parameters:





The example inputs of the arguments shown as follows:

- Signal1: v(/von)
- Signal2: v(/vop)
- X Start: 200n
- X Stop:400n
- Number of points: 10

pstddev

Description

Calculates the standard deviation value of signal based on the given period. Applies the sample first then performs the calculation.

Syntax

```
pstddev(<signal>,<x_start>,<x_end>,[<period>],  
[<sampling_factor>])
```

Returns

An analog signal.

Arguments

<signal>

The signal to be measured.

<x_start>

The start of x value to be measured.

<x_end>

The end of x value to be measured.

[<period>]

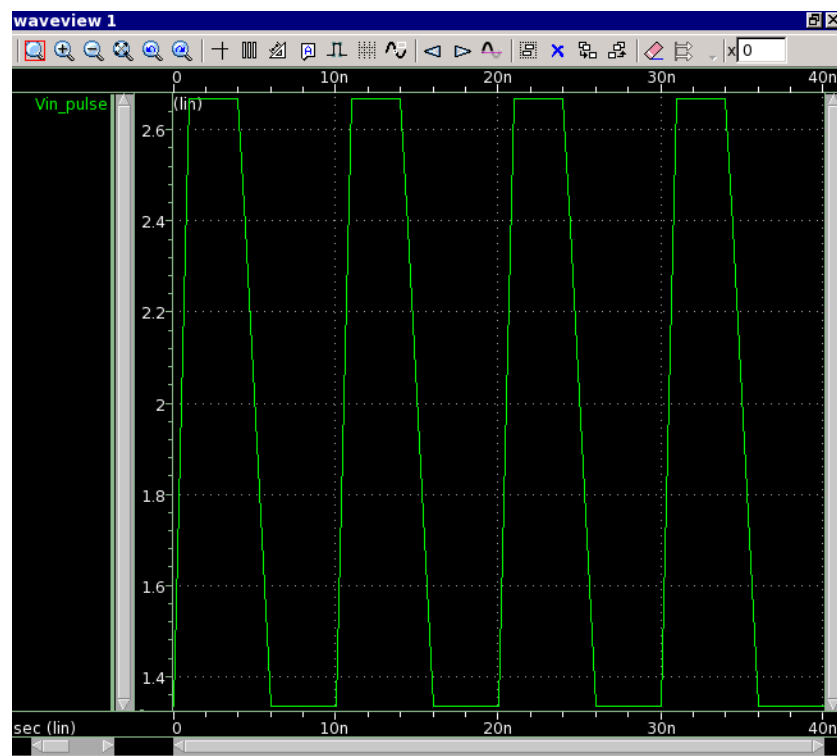
(Optional) Specifies the period of the waveform. The default value is the range of x.

[<sampling_factor>]

(Optional) Specifies the value of the sampling factor. The larger the factor, the more points to be included. The value should be larger than 0. The default value is 1.

Examples

The following example shows how the function works on the input waveform.



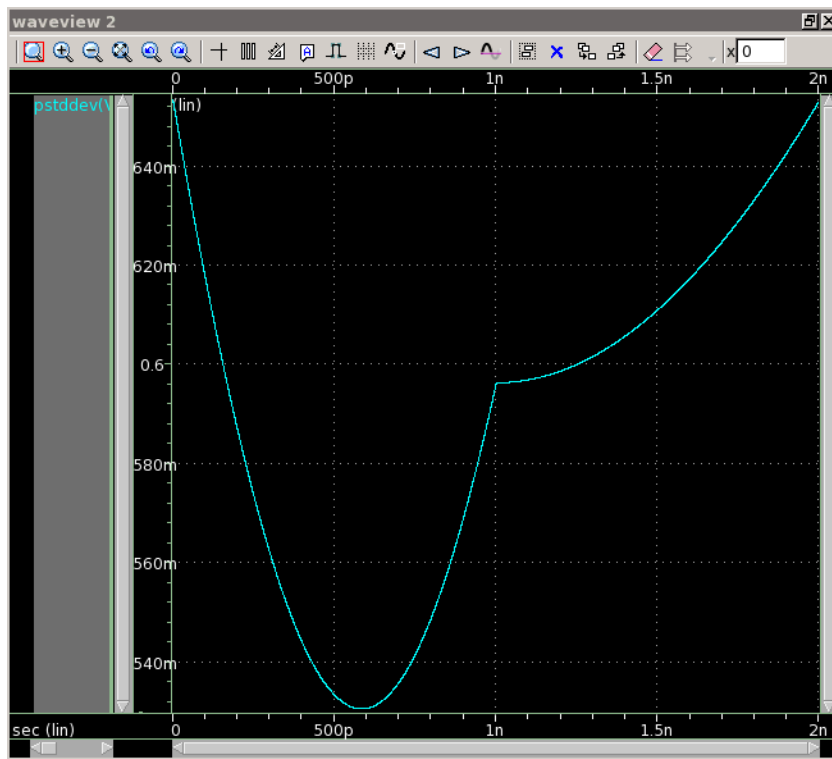
The example inputs of the arguments are as follows:

- signal: Vin_pulse
- x_start: 0
- x_end: 40n
- period: 2n
- sampling factor: 1

The function expression shows as follows:

```
pstddev('Vin_pulse',0,40n,2n,1)
```

with the output waveform being plotted as the following:



pwc

Measures the piece-wise constant value of a signal.

Syntax

```
pwc(<signal>)
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

Description

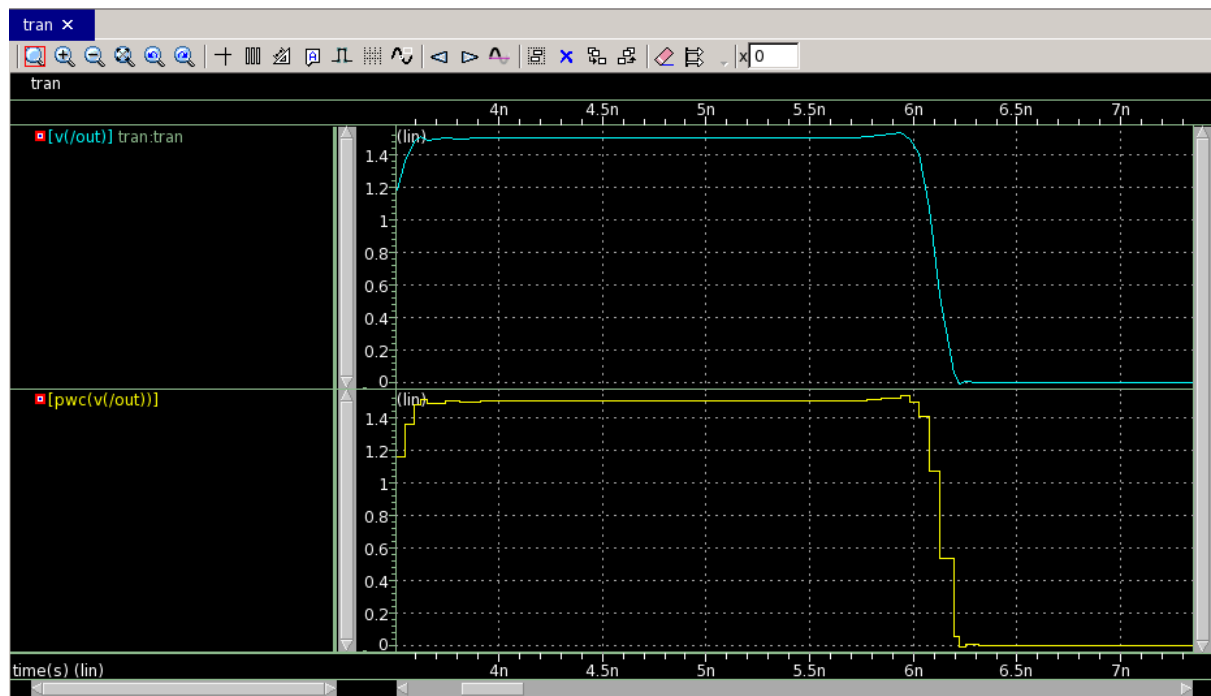
Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
pwc('v(/out)')
```

The following example shows how the function works on the input waveform "v(/out)":



The example inputs of the arguments shown as follows:

- Signal: v(/out)

The function expression shows as follows:

```
pwc(v(/out))
```

pzFreqRealFilter()

Description

Returns a filtered pole or zero signal from the input poles/zeros signal.

Syntax

```
pzFreqRealFilter(<signal>, [<Min Freq>], [<Max Freq>], [<Min abs(Real)>],  
[<Max abs(Real)>])
```

Arguments

<signal>

The input signal poles/zeros.

<Min Freq>

(Optional) The minimum frequency value.

<Max Freq>

(Optional) The maximum frequency value.

<Min abs(Real)>

(Optional) The minimum real value, abs(real).

<Max abs(Real)>

(Optional) The maximum real value, abs(real).

Errors

Possible error messages include:

- Signal not found or variable/type undefined.
- Incorrect signal type.
- Cannot add waveform data.

Examples

```
pzFreqRealFilter(v(/out))
```

```
pzFreqRealFilter(v(/out), 5M, 10M)
```

The example inputs of the arguments shown are as follows:

- Signal: `v (/out)`
- Min Freq: 5M
- Max Freq: 10M

reflectionContour

Description

Constant reflection coefficient contour.

Syntax

```
reflectionContour(<current_signal>, <voltage_signal>, <harmonic_index>,  
[<current_load>], [<voltage_load>], [<max_power>], [<min_power>],  
[<num_contour>], [<resistance>], [<close>], [<modifier>])
```

Returns

A waveform.

Arguments

`<current_signal>`

The current used to get the reflection coefficient.

`<voltage_signal>`

The voltage used to get the reflection coefficient.

`<harmonic_index>`

Harmonic index of waves.

`[<current_load>]`

(Optional) The current used to get the reference reflection coefficient.

`[<voltage_load>]`

(Optional) The voltage used to get the reference reflection coefficient.

`[<max_ref>]`

(Optional) The maximum reflection coefficient.

`[<min_ref>]`

(Optional) The minimum reflection coefficient.

[<num_contour>]

(Optional) The number of contours. Default is 8.

[<resistance>]

(Optional) Resistance used to get the reflection coefficient. Default is 50.

[<close>]

(Optional) The contours are closed contours (ON or OFF). Default is OFF.

repeat

Creates a new waveform by repeating the given period signal for a specified number of times.

Syntax

```
repeat (<signal>, <period>, <repeat_count>)
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

<period>

The period of the given signal.

<repeat_count>

The number of cycle times to be repeated.

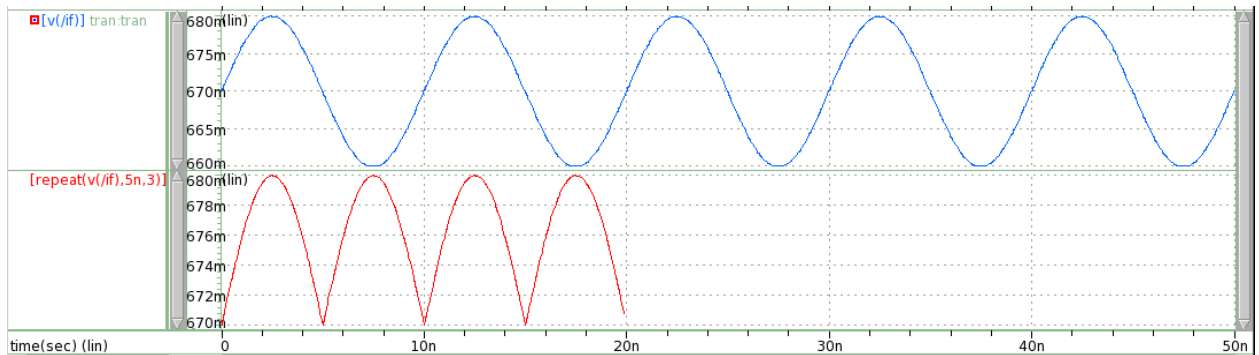
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
repeat (v (/if), 5n, 3)
```

The example inputs of the arguments shown as follows:

- Signal: $v(/f)$
- Period: $5n$
- Repeat_count: 3

The function expression shows as follows:

```
repeat(v(/f),5n,3)
```

risevst

Calculates the time difference versus the time of the given signal between x_start and x_stop from the low threshold crossing to the high threshold crossing.

Syntax

```
risevst(<signal>,<low_y_level>,<high_y_level>,[percent],  
[x_start],[x_stop])
```

Returns

A waveform.

Arguments

<signal>

The input signal to be calculated.

<low_y_level>

The low threshold level of y.

<high_y_level>

The high threshold level of y.

[<percent>]

(Optional) The value can be 0 and 1. When specified as 1, the y_threshold is considered a percentage value.

[<x_start>]

(Optional) The starting value of x.

[<x_stop>]

(Optional) The ending value of x.

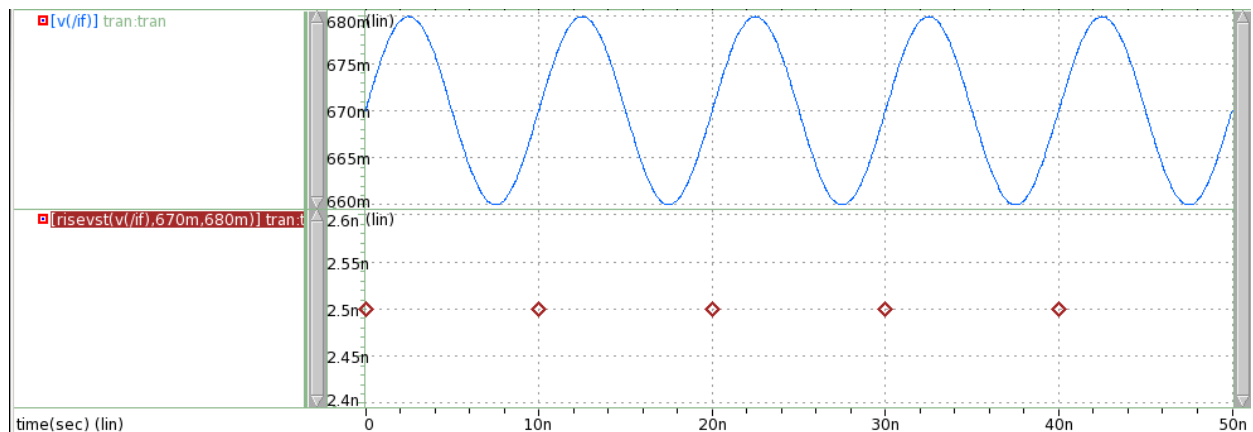
Errors

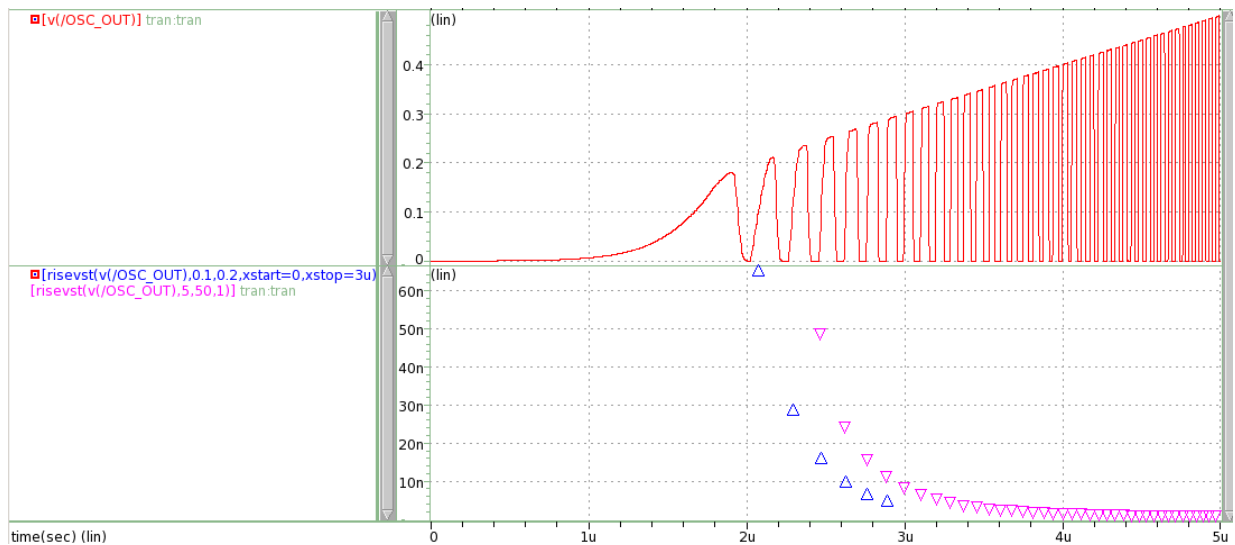
Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

```
risevst(v(/if), 670m, 680m)
risevst(v(/OSC_OUT), 5, 50, 1)
risevst(v(/OSC_OUT), 0.1, 0.2, xstart=0, xstop=3u)
```





sample

Creates a new waveform which is a sample of a given signal starting from x with period for a given number of times.

Syntax

```
sample(<signal>,<x_start>,<period>,[<num_of_points>],  
[<x_end>],[<type>])
```

Returns

A waveform.

Arguments

<signal>

The input signal to be calculated.

<x_start>

The starting value of x.

<period>

The period of a given signal.

[<num_of_points>]

(Optional) Specifies the number of sample times.

[<x_end>]

(Optional) Specifies the ending value of x. Disabled if `num_of_points` is set.

[<type>]

(Optional) Specifies scale of x-axis: linear or logarithmic. Default is linear.

Errors

Possible error messages include:

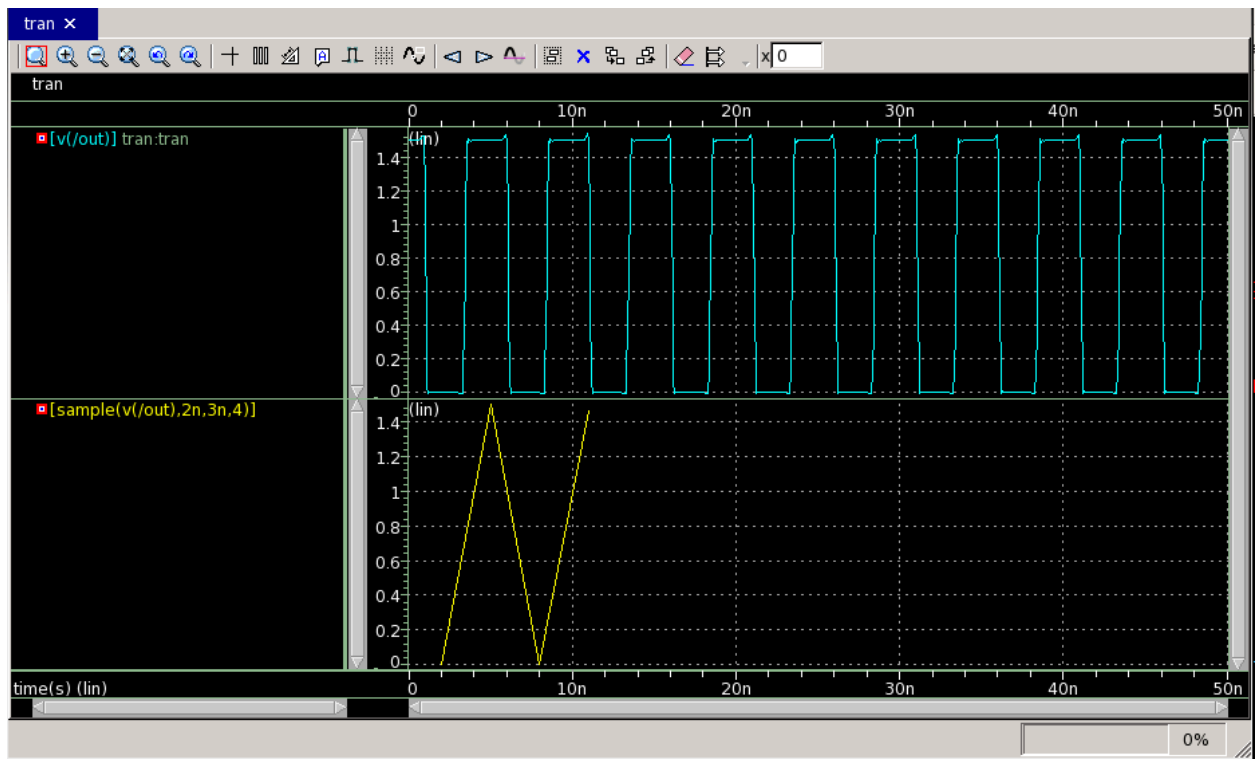
- Signal not found
- X value out of range
- Sampling step is non-positive

Examples

```
sample('v(/out)', 2n, 4n, 4)  
sample('v(/out)', 5n, 4n, 10)
```

The following examples show how the function works on the input waveforms "v(/out)":

Example 1:



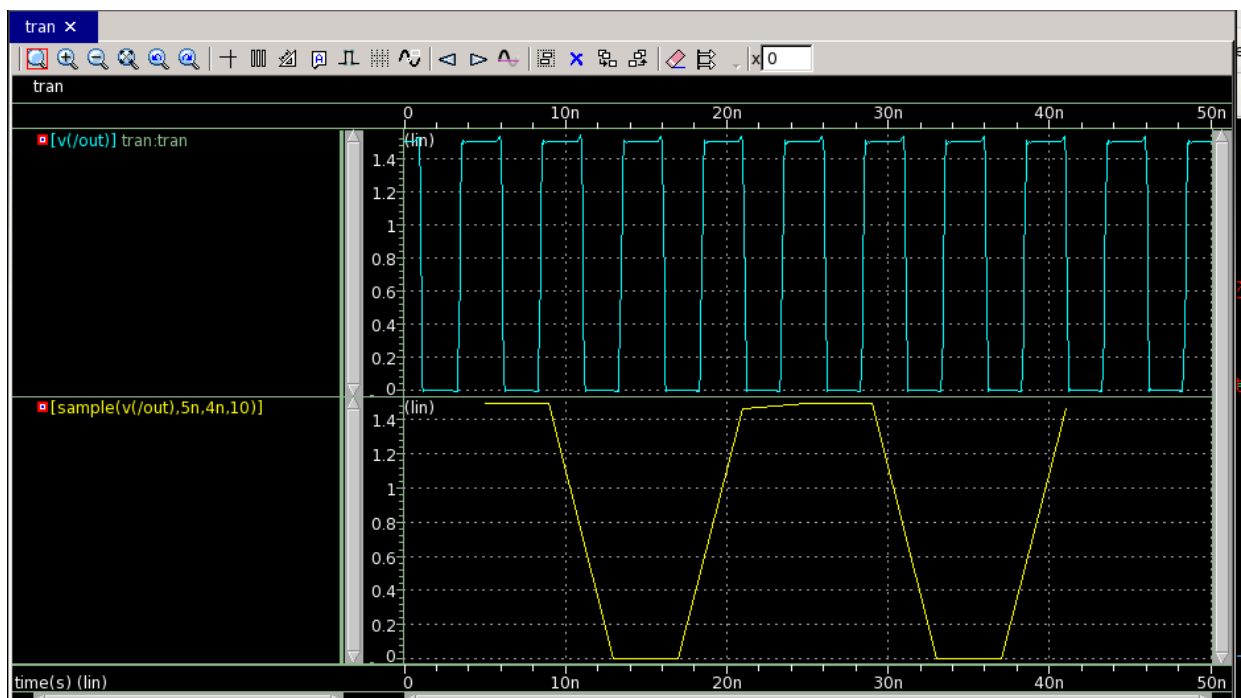
The example inputs of the arguments shown as follows:

- Signal: $v(/out)$
- X start: $2n$
- Period: $4n$
- Number of points: 4

The function expression shows as follows:

```
sample(v(/out), 2n, 4n, 4)
```

Example 2:



The example inputs of the arguments shown as follows:

- Signal: $v(/out)$
- X start: $5n$
- Period: $4n$
- Number of points: 10

The function expression shows as follows:

```
sample(v(/out), 5n, 4n, 10)
```

setup_times

Generates a new waveform containing all of the setup time data points.

Syntax

```
setup_times(<signal>,<clock_signal>,<clock_period>,  
<y_threshold>,<signal_edge>,<clock_edge>,[<x_start>],  
[<x_stop>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<clock_signal>`

The reference clock signal.

`<clock_period>`

The period of the reference `clock_signal`. If the input value for the `clock_signal` period is 0 or less than 0, the `clock_period` is calculated using the `clock_signal`.

`<y_threshold>`

The threshold value of y for the measured signal.

`<signal_edge>`

The type of measured `signal_edge`. The valid `signal_edge` can be rise, fall, or both. The default `signal_edge` is rise.

`<clock_edge>`

The type of reference `clock_signal` edge. The valid `signal_edge` can be rise, fall, or both. The default `signal_edge` is rise.

`[<x_start>]`

(Optional) The starting time of measurement.

`[<x_stop>]`

(Optional) The ending time of measurement.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the `y_threshold` is a percentage value. If the value is 0, the `y_threshold` is not a percentage value.

Errors

Possible error messages include:

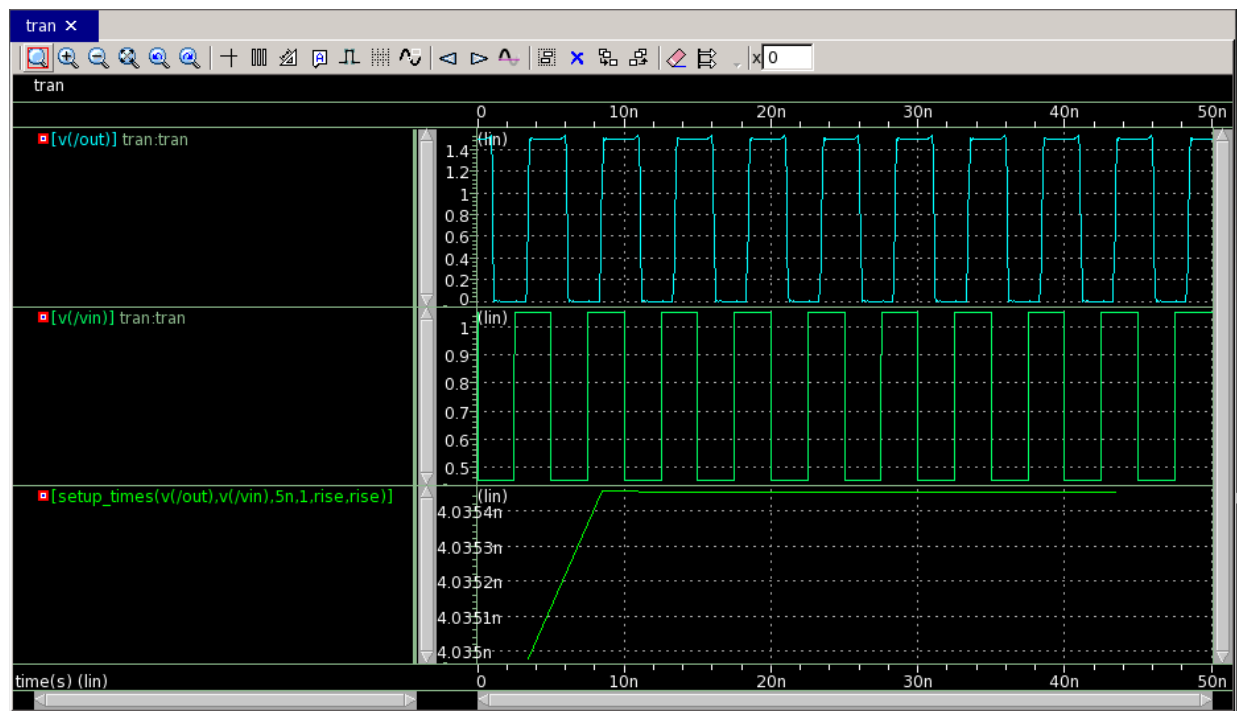
- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

```
setup_times(v(/out),v(/vin),5n,1,raise,raise)
setup_times(v(/out), v(/vin),5n,1,raise,raise,2n,20n)
```

The following examples show how the function works on the input waveforms "v(/out)":

Example 1:



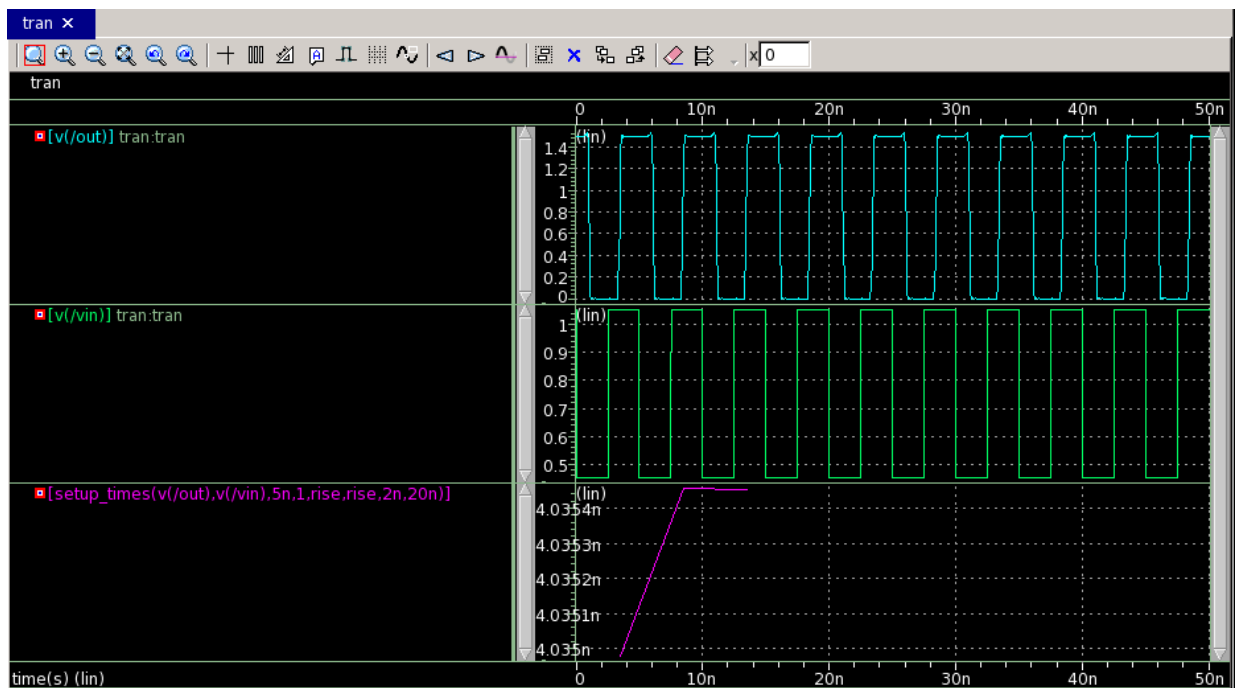
The example inputs of the arguments shown as follows:

- Target Signal: v(/out)
- Clock Signal: v(/vin)
- Period: 5n
- Y threshold: 1
- Target Signal edge: rise
- Clock signal edge: rise

The function expression shows as follows:

```
setup_times(v(/out), v(/vin), 5n, 1, rise, rise)
```

Example 2:



The example inputs of the arguments shown as follows:

- Target Signal: v(/out)
- Clock Signal: v(/vin)
- Period: 5n
- Y threshold: 1

- Target Signal edge: rise
- Clock signal edge: rise
- X start: 2n
- X stop: 20n

The function expression shows as follows:

```
setup_times(v(/out), v(/vin), 5n, 1, rise, rise, 2n, 20n)
```

setup_times2

Generates a new waveform that contains all of the setup time data points. `setup_times2` differs from `setup_times` in that it gives you the ability to specify a different threshold for the clock signal as well as the measured signal, whereas `setup_times` allows selecting the threshold of only the measured signal.

Syntax

```
setup_times2(<signal>, <clock_signal>, <clock_period>,  
             <data_threshold>, <clock_threshold>, <signal_edge>,  
             <clock_edge>, [<x_start>], [<x_stop>], [<percent>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be measured.

<clock_signal>

The reference clock signal.

<clock_period>

The period of the reference `clock_signal`. If the input value for the `clock_signal` period is 0 or less than 0, the `clock_period` is calculated using the `clock_signal`.

<data_threshold>

The threshold value of y for the measured signal.

<clock_threshold>

The threshold value of y for the `clock_signal`.

<signal_edge>

The type of measured signal_edge. The valid signal_edge can be rise, fall, or both. The default signal_edge is rise.

<clock_edge>

The type of reference clock_signal edge. The valid signal_edge can be rise, fall, or both. The default signal_edge is rise.

[<x_start>]

(Optional) The starting time of measurement.

[<x_stop>]

(Optional) The ending time of measurement.

[<percent>]

(Optional) The value can be 1 or 0. When the value is 1, the y_threshold is a percentage value. If the value is 0, the y_threshold is not a percentage value.

Errors

Possible error messages include:

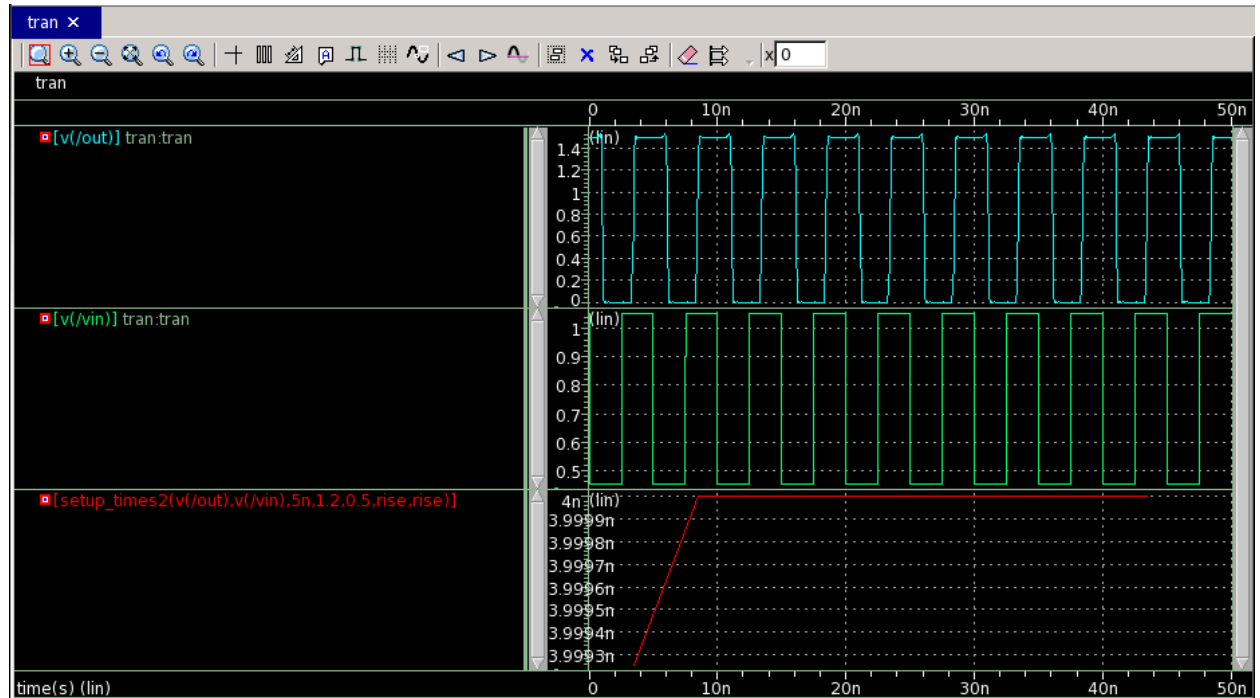
- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

```
setup_times2('0|v(out)', '0|v(out)', 4n, 2, 3, 'rise', 'rise')
setup_times2('0|v(out)', '0|v(out)', 4n, 2, 3, 'rise', 'rise', 2n
6n, 1)
```

The following examples show how the function works on the input waveforms "v(/out)":

Example 1:



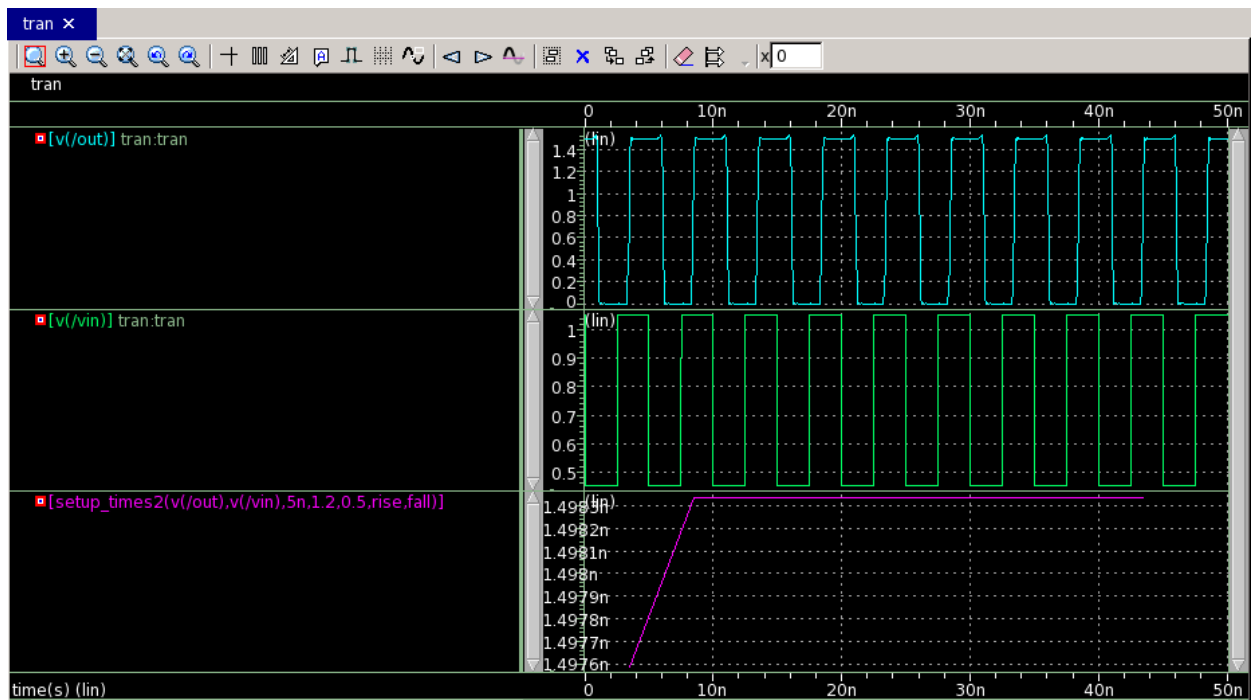
The example inputs of the arguments shown as follows:

- Target Signal: v(/out)
- Clock Signal: v(/vin)
- Period: 5n
- Target Signal Y-threshold: 1.2
- Clock Signal Y-threshold: 0.5
- Target Signal edge: rise
- Clock signal edge: rise

The function expression shows as follows:

```
setup_times2(v(/out),v(/vin),5n,1.2,0.5,rise,rise)
```

Example 2:



The example inputs of the arguments shown as follows:

- Target Signal: `v(/out)`
- Clock Signal: `v(/vin)`
- Period: `5n`
- Target Signal Y-threshold: `1.2`
- Clock Signal Y-threshold: `0.5`
- Target Signal edge: `rise`
- Clock signal edge: `fall`

The function expression shows as follows:

```
setup_times2(v(/out),v(/vin),5n,1.2,0.5,rise,fall)
```

signalAtFreq

Calculates the signal at frequency.

Syntax

```
signalAtFreq(<signal>, [<frequency>])
```

Arguments

<signal>

The input signal to be measured.

<frequency>

(Optional) The frequency.

signalcompare

Compares the waveform outputs of the simulation.

Syntax

```
signalcompare(<signal1>, <signal2>, [<absolute_tolerance>], [<relative_tolerance>])
```

Arguments

<signal1>

The first signal to be measured.

<signal2>

The second signal to be measured.

<absolute_tolerance>

(Optional) The absolute tolerance.

<relative_tolerance>

(Optional) The relative tolerance.

slewratesvst

Calculates the slew rate versus time for the given signal within the `y_initial` and `y_final`.

Syntax

```
slewratesvst(<signal>, <y_initial>, <y_final>, [<percent>],  
[<x_start>], [<x_stop>], [<trigger_type>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<y_initial>`

The initial threshold value of y.

`<y_final>`

The final threshold value of y.

`[<percent>]`

(Optional) The value can be 1 or 0. If the value is 1, the `y_threshold` is a percentage value. If the value is 0, the `y_threshold` is not a percentage value.

`[<x_start>]`

(Optional) The starting time of measurement.

`[<x_stop>]`

(Optional) The ending time of measurement.

`[<trigger_type>]`

(Optional) The `trigger_type` of the signal within `y_initial` and `y_final`, which can be `rise`, `fall`, or `either`. The default `trigger_type` is `either`.

Errors

Possible error messages include:

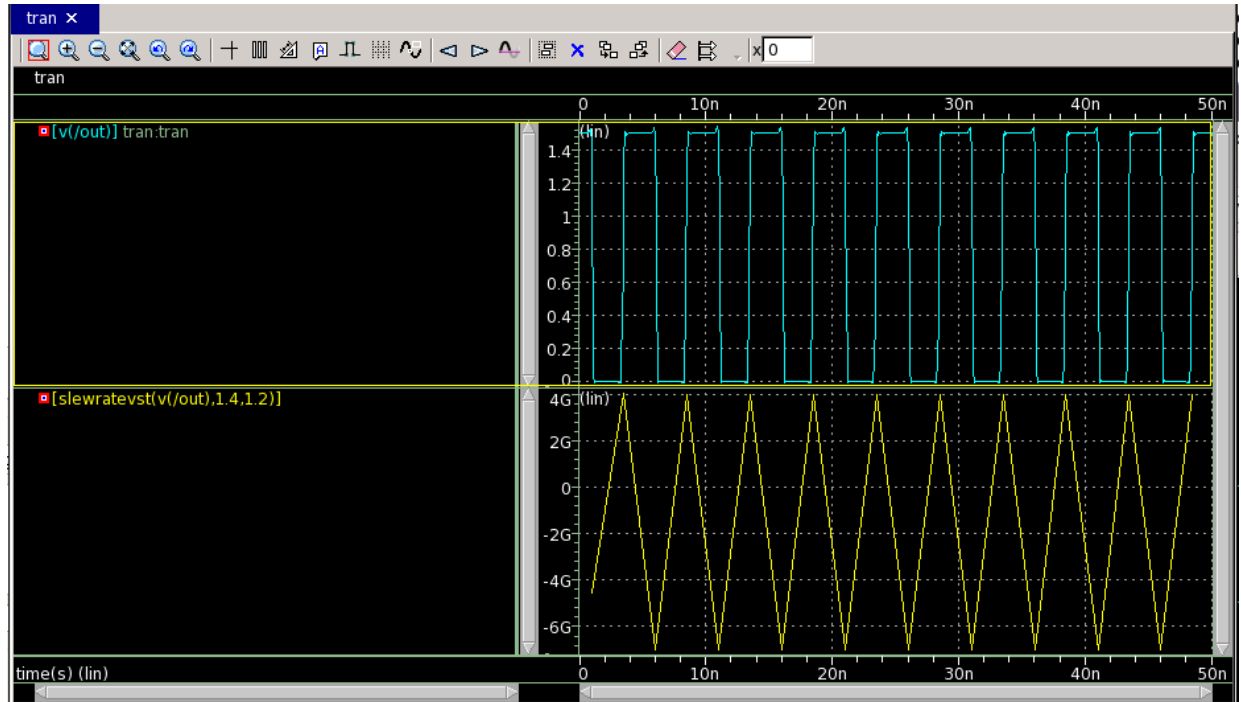
- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

```
slewrtevst(v(/out),1.4,1.2)  
slewrtevst(v(/out),75,25,1)
```

The following examples show how the function works on the input waveforms "v(/out)":

Example 1:



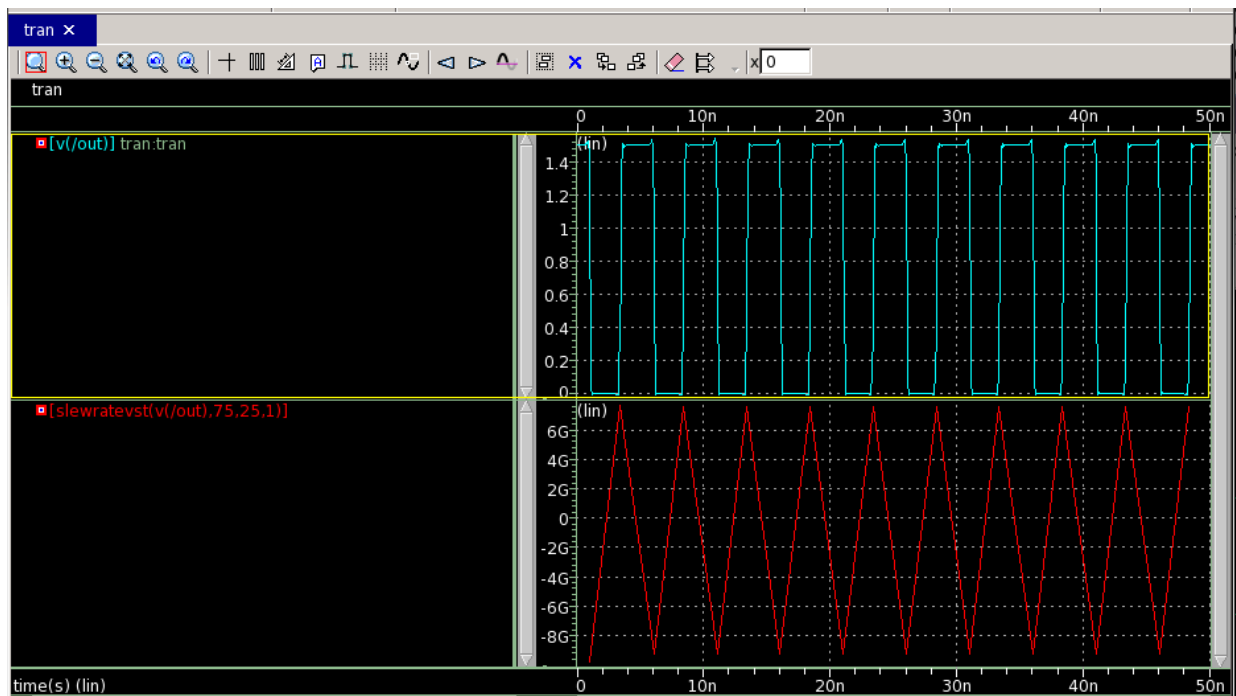
The example inputs of the arguments shown as follows:

- Signal: v(/out)
- Initial Y value: 1.4
- Final Y value: 1.2

The function expression shows as follows:

```
slewrtevst(v(/out),1.4,1.2)
```

Example 2:



The example inputs of the arguments shown as follows:

- Signal: v(/out)
- Initial Y value as percentage: 75
- Final Y value as percentage: 25
- Percentage=true/1

The function expression shows as follows:

```
slewrtevst(v(/out),75,25,1)
```

smooth

Calculates the smoothed waveform from the given signal. For each point of the input data of the signal, the average value from (x-x_window) to (x + window) is calculated to generate the smoothed output.

Syntax

```
smooth(<signal>,<x_window>)
```


Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

<x_window>

The smoothing window in x value.

Description

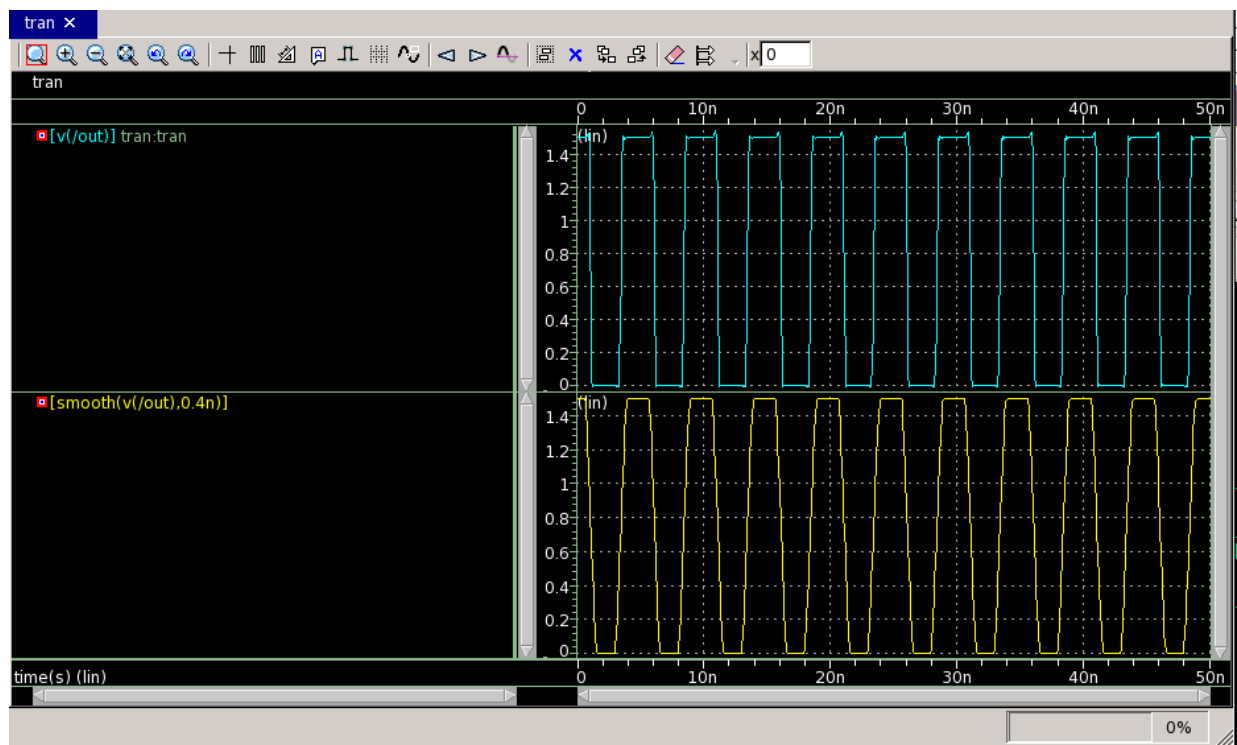
Possible error messages include:

- Signal not found
- X value out of range

Examples

```
smooth('v(/out) ',0.4n)
```

The following example shows how the function works on the input waveforms "v(/out)":



The example inputs of the arguments shown as follows:

- Signal: v(/out)
- X window: 0.4n

The function expression shows as follows:

```
smooth(v(/out),0.4n)
```

spectralPower

Description

Calculates the spectral power for the specified current waveform and voltage waveform.

Syntax

```
spectralPower(<current_signal>,<voltage_signal>)
```

Returns

An analog signal.

Arguments

<current_signal>

The current source.

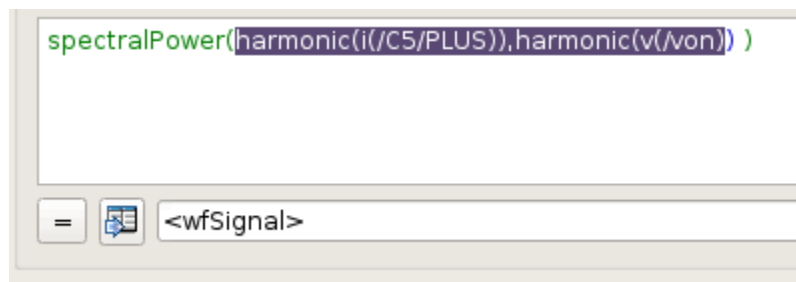
<voltage_signal>

The voltage source.

Examples

```
spectralPower(harmonic(i(/C5/PLUS)),harmonic(v(/von)))
```

The following example shows how the function works on the input waveforms harmonic(i(/C5/PLUS)) and harmonic(v(/von)):





sPowerDiffVI

Description

Calculates spectral power with two nodes voltage and current signals.

Syntax

```
sPowerDiffVI(<voltage_signal1>, <voltage_signal2>, <current_signal1>,  
<current_signal2>)
```

Returns

An analog signal.

Arguments

<voltage_signal1>

The input positive voltage signal.

<voltage_signal2>

The input negative voltage signal.

<current_signal1>

Current signal 1.

<current_signal2>

Current signal 2.

sPowerDiffVR

Description

Calculates spectral power with two nodes voltage and resistance.

Syntax

```
sPowerDiffVR(<voltage_signal1>, <voltage_signal2>, <resistance>)
```

Returns

An analog signal.

Arguments

<voltage_signal1>

The input positive voltage signal.

<voltage_signal2>

The input negative voltage signal.

<resistance>

Resistance.

swapsweep

Description

Swaps the X-axis value with the specified sweep variable.

Syntax

```
swapsweep(<signal>, <parameter_name>, [<x_value>])
```

Returns

An analog signal.

Arguments

<signal>

The output waveform with the x-axis values to be swapped.

<parameter_name>

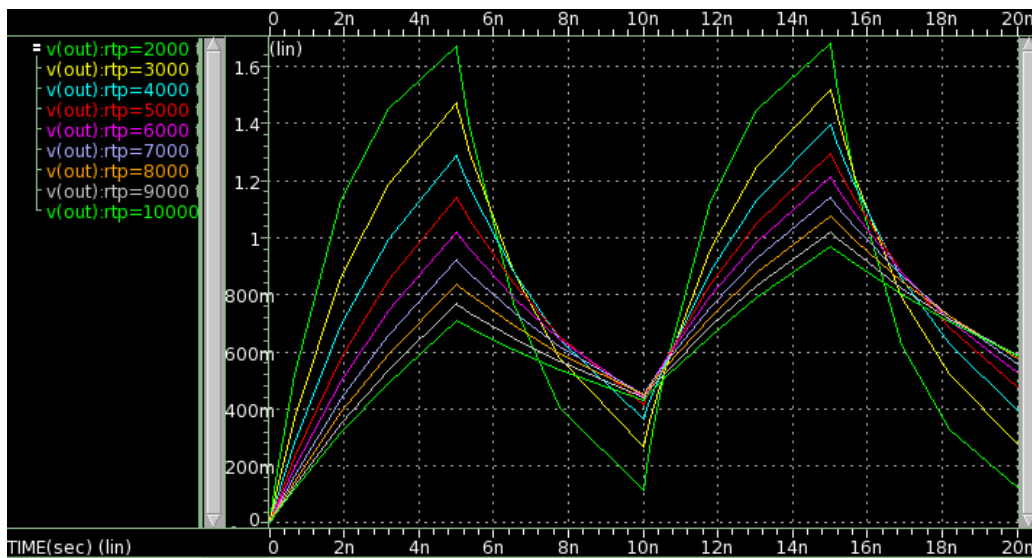
The sweep variable for which the x-axis of the given waveform is to be swapped.
For example, CAP or temp.

[<x_value>]

(Optional) x-axis value at which the waveform is swapped with the specified sweep variable.

Examples

The following example shows how the function works on the input waveform “v(out)”:



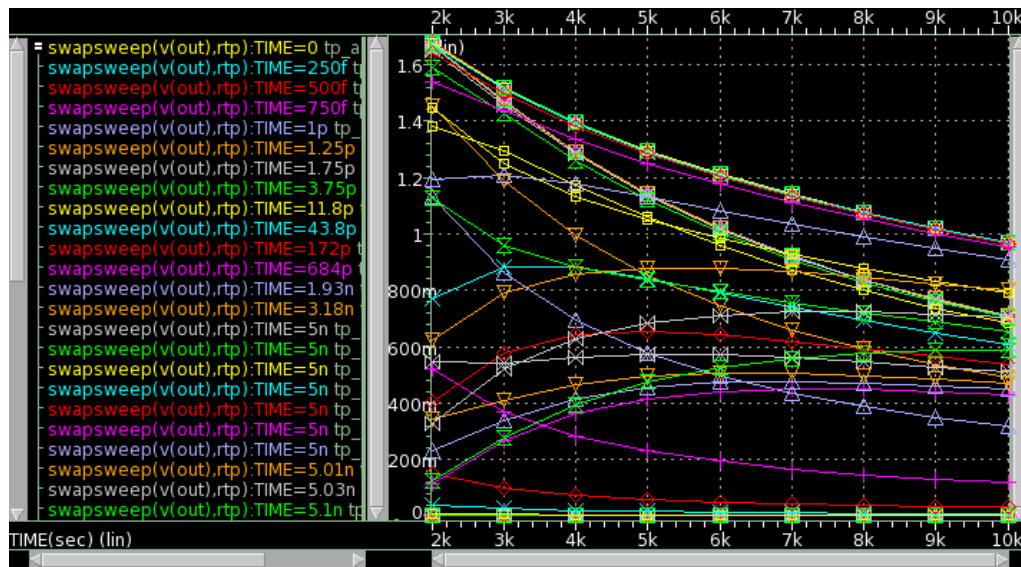
The example inputs of the arguments show as follows:

- signal: v(out)
- param name: rtp

The function expression shows as follows:

```
swapsweep('v(out)', 'rtp')
```

The output waveform will be plotted in the following:



sweepavg

Calculates the average waveform of a multi-trace sweep waveform.

Syntax

`sweepavg (<signal>)`

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

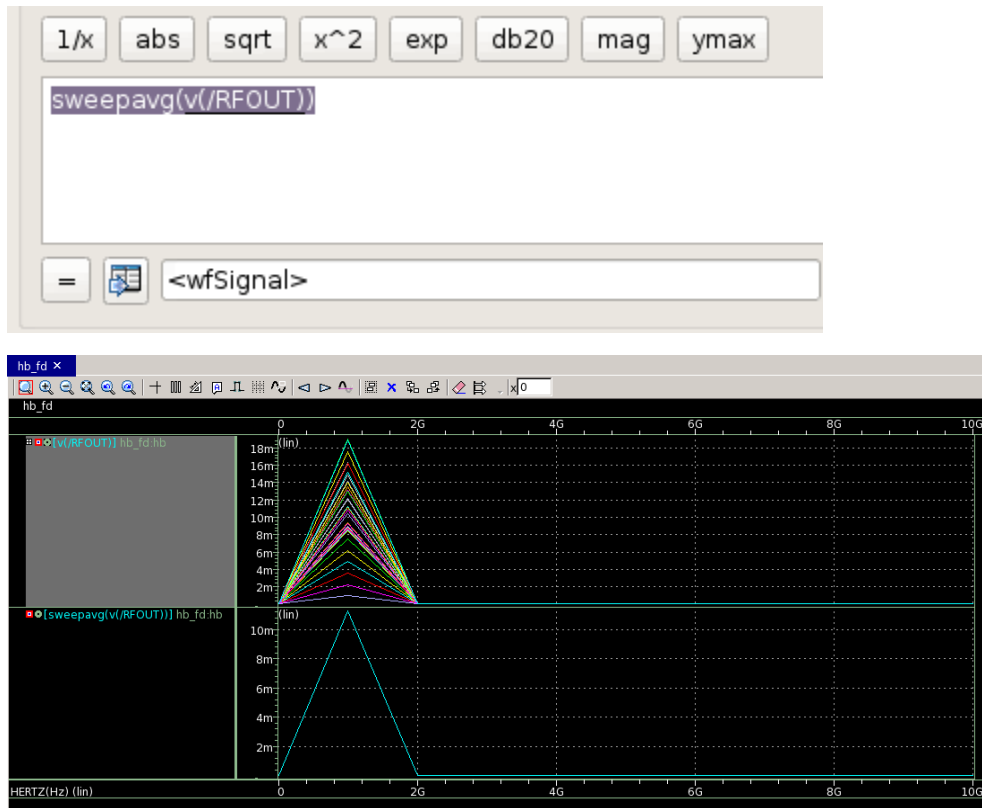
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

`sweepavg (v (/RFOUT) )`



sweepmax

Calculates the maximum bound waveform of a multi-trace sweep.

Syntax

```
sweepmax(<signal>)
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

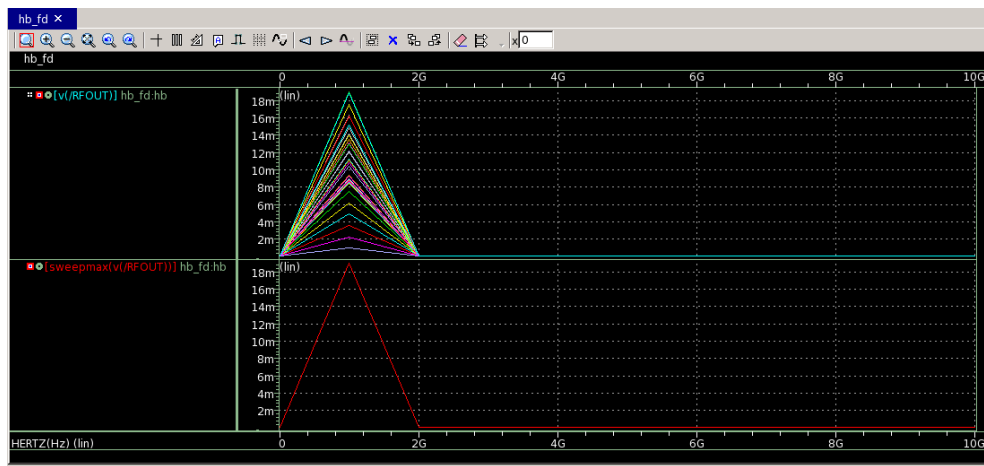
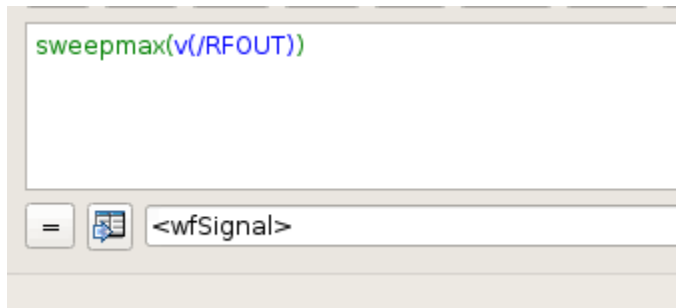
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

```
sweepmax(v(/RFOUT))
```



sweepmin

Calculates the minimum bound waveform of a multi-trace sweep.

Syntax

```
sweepmin(<signal>)
```


Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

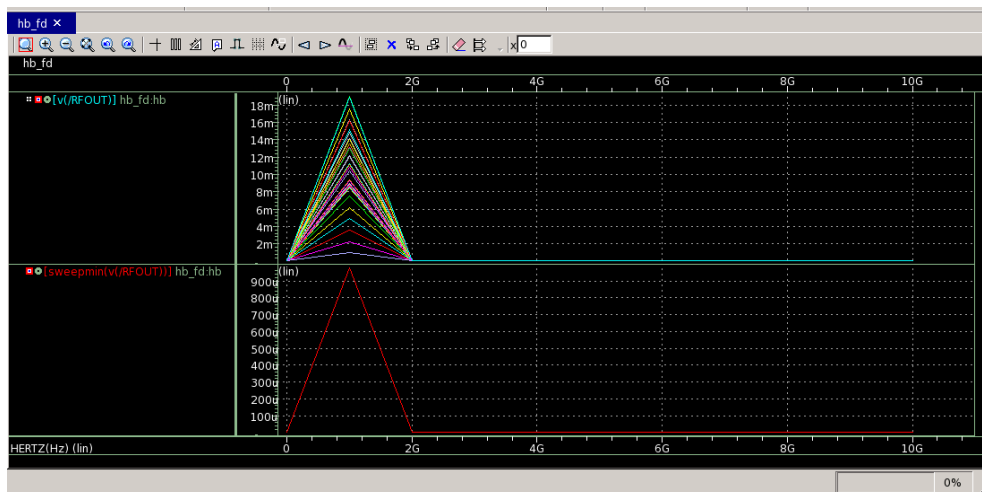
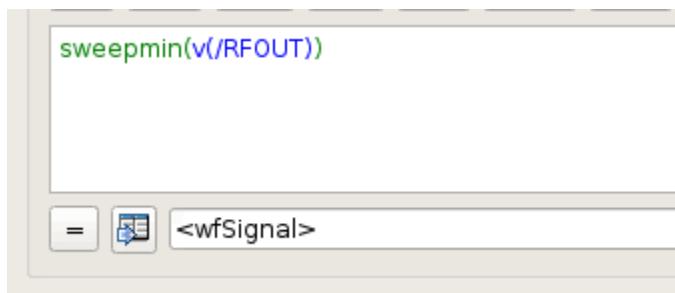
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

```
sweepmin(v(/RFOUT))
```



tangent

Description

Calculates a line that passes through x and y coordinates and the specified slope.

Syntax

```
tangent(<signal>, [<x_value>], [<y_value>], [<slope>])
```

Returns

An analog signal.

Arguments

<signal>

The target signal as the background.

<x_value>

Specifies the x-axis value. Default is x value of first point on the waveform.

<y_value>

Specifies y-axis value. Default is y of the specified x value or default x value.

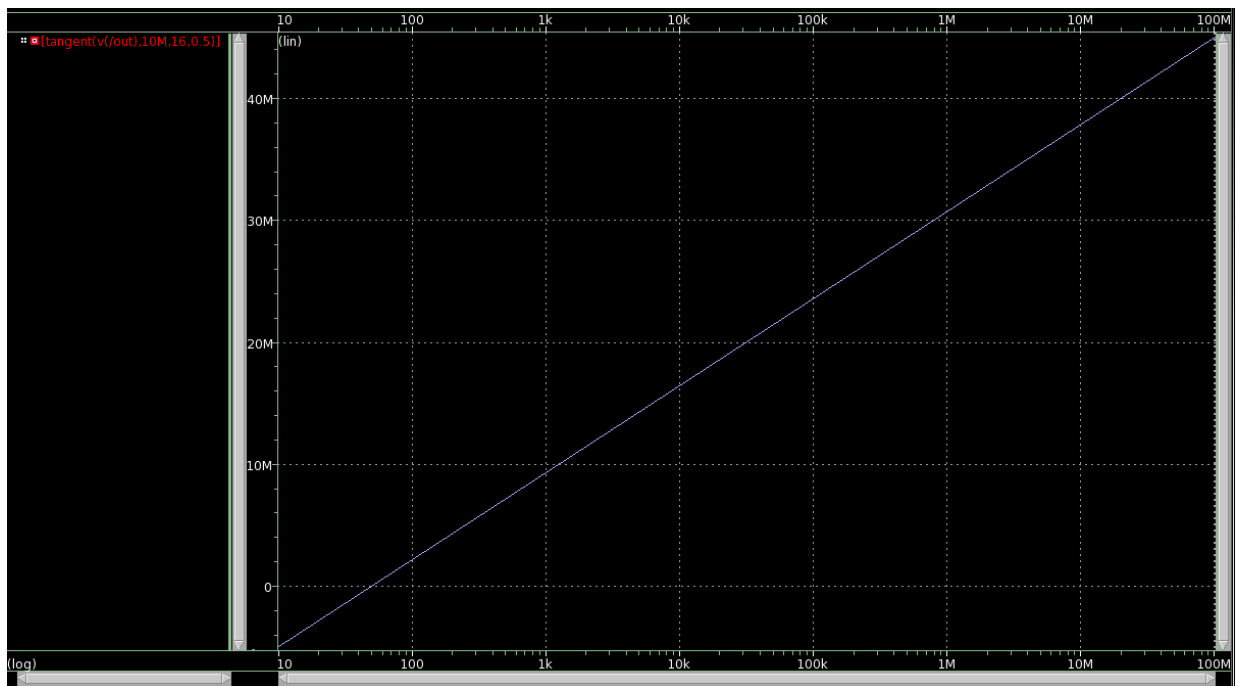
<slope>

Specifies slope. Default is 1.0.

Examples

```
tangent(v(/out), 10M, 16, 0.5)
```

will return the following:



The example inputs of the arguments shown as follows:

- Signal: v(/out)
- X axis value: 10M
- Y axis value: 16
- Slope: 0.5

text2wave

Description

Reads specified columns from a text file and creates a signal.

Syntax

```
text2wave(<path>, [<X_column>], [<Y_column>], [<X_skip>], [<Y_skip>], [<X_label>],  
[<Y_label>], [<X_unit>], [<Y_unit>])
```

Returns

A signal.

Arguments

`<path>`

Specifies the file path.

`<X_column>`

(Optional) Specifies the column of X values. Default is 1.

`<Y_column>`

(Optional) Specifies the column of Y values. Default is 2.

`<X_skip>`

(Optional) The number of lines skipped for reading X values.

`<Y_skip>`

(Optional) The number of lines skipped for reading Y values.

`<X_label>`

(Optional) Specifies the signal's X-axis label.

`<Y_label>`

(Optional) Specifies the signal's Y-axis label.

`<X_unit>`

(Optional) Specifies the signal's X-axis unit.

`<Y_unit>`

(Optional) Specifies the signal's Y-axis unit.

trace

Returns a trace from a multi-trace of signal `s` based on the parameter condition `parm_con`.

Syntax

`trace(<signal>,<parameters>)`

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured. The signal must be a sweep signal with different parameters.

<parameters>

The parameter condition of the input signal.

Description

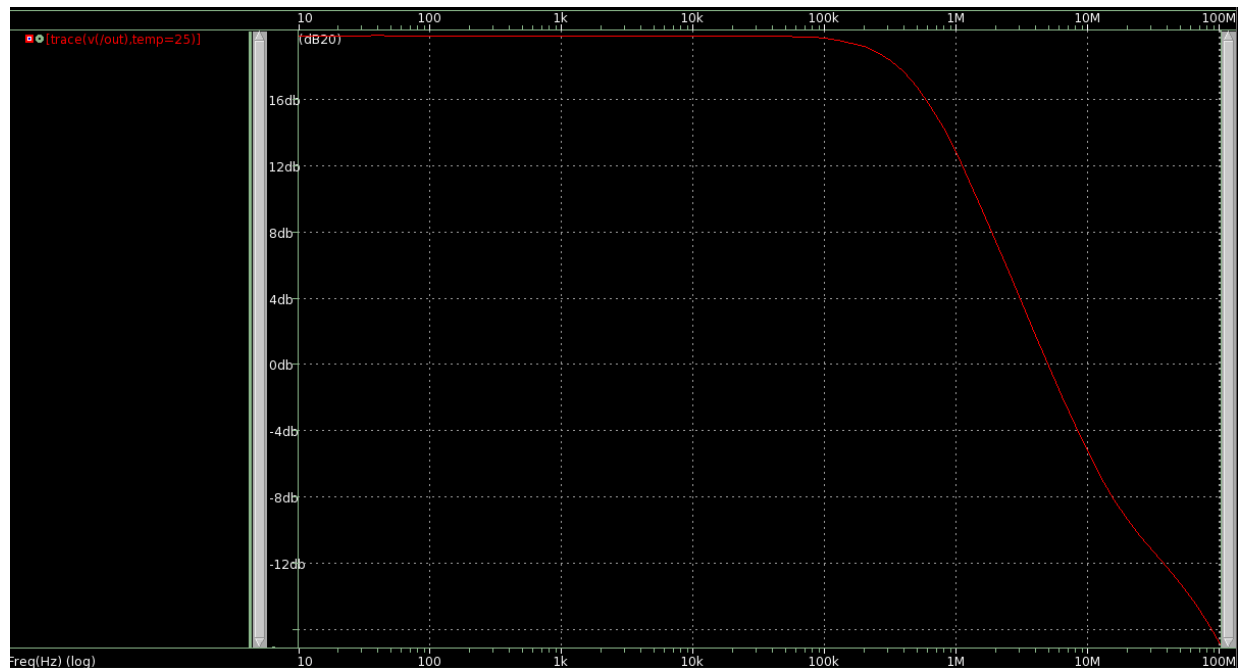
Possible error messages include:

- Signal not found
- Incorrect signal type
- Can't find trace with matching conditions

Examples

```
trace(v(/out),temp=25)
```

will return the following:



The example inputs of the arguments shown as follows:

- Signal: v(/out)
- Parameters: temp=25

v2todBc

Converts phase noise from V^2/Hz to dBc/Hz . This function only works for waveform files which contain carrier magnitude information; otherwise, the waveform viewer will report that carrier magnitude is not defined.

Syntax

```
v2todBc(<signal>)
```

Returns

N/A

Arguments

<signal>

The signal to be converted.

Examples

```
v2todBc(dBc2V2('PAC_L(f)'))
```

window_average

Window average: calculates the average using step size between a start and a stop time.

Syntax

```
window_average(<signal> [<x_start>] [<x_stop>] [<time_step>]  
[<window_size>])
```

Returns

A waveform.

Arguments

<signal>

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

x_start

(Optional) The start time (x_1). Default: $x_{\min}$.

x_stop

(Optional) The stop time (x_2). Default: $x_{\max}$.

`time_step`

(Optional) The step size (`step`). Default: (`x_stop-x_start`).

`window_size`

The window size (`size`). Default: `time_step`

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Input signal has no data point

x_at_y

Plots the crossing points of a given signal on a specified `y_threshold`.

Syntax

`x_at_y(<signal>,<y_level>)`

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

`<y_level>`

The threshold value of y.

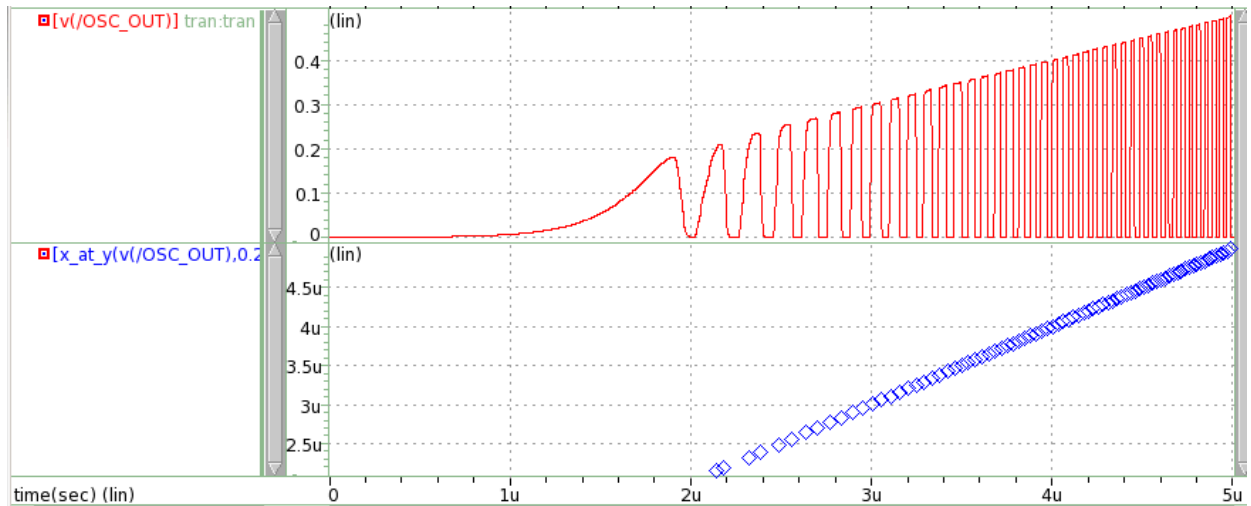
Errors

Possible error messages include:

- Signal not found
- Y value out of range

Examples

`x_at_y(v(/OSC_OUT),0.2)`



xscale

Plots a new waveform identical to the given signal with the x component multiplied by a scaling factor.

Syntax

```
xscale(<signal>,<scaling_factor>)
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

<scaling_factor>

The multiplier to cross the x value of a given signal.

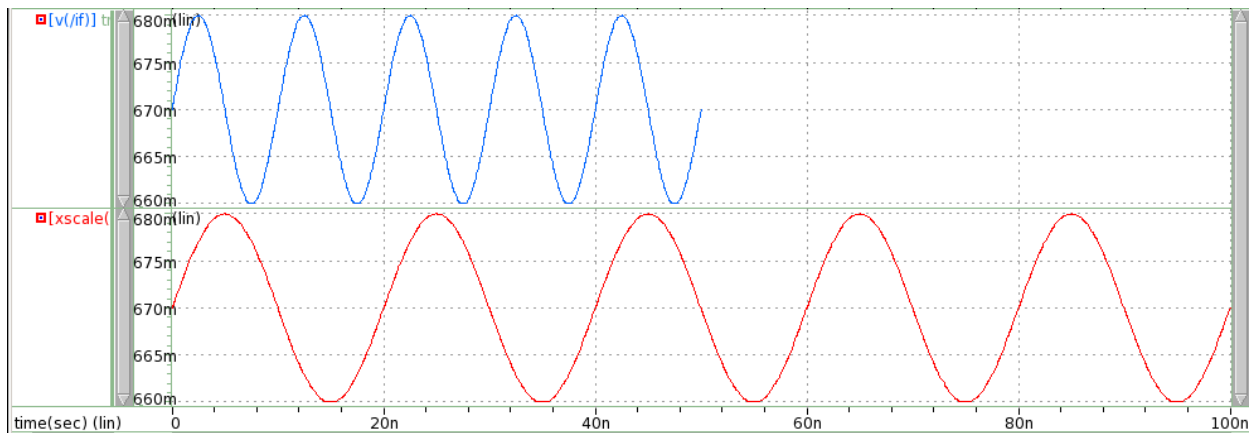
Errors

Possible error messages include:

- Signal not found

Examples

```
xscale(v(/if),2)
```

xshift

Plots a new waveform identical to the given signal but shifted to the right by x delay.

Syntax

```
xshift(<signal>,<x_delay>)
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<x_delay>`

The x value by which the signal shifts to right.

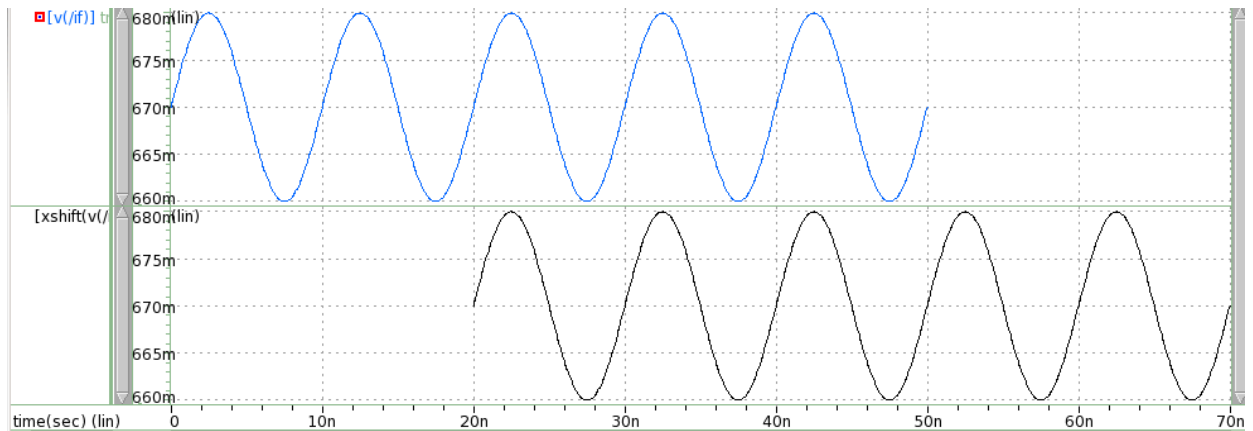
Errors

Possible error messages include:

- Signal not found

Examples

```
xshift(v(/if),20n)
```



xval

Measures the x value of the given signal and creates a new waveform equivalent to $y=x$.

Syntax

```
xval(<signal>)
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

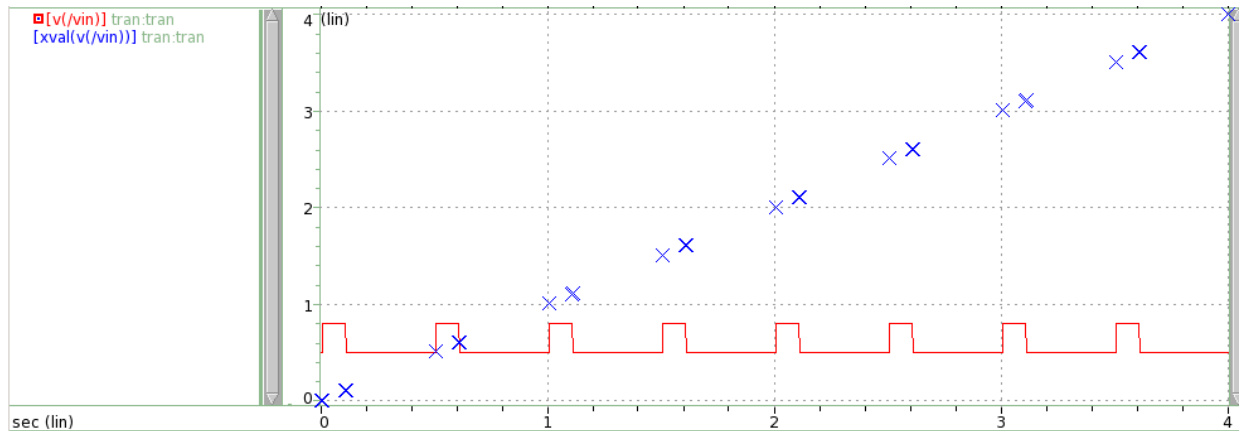
Errors

Possible error messages include:

- Signal not found

Examples

```
xval('0|v(out)')
```



yvsi

Creates a new waveform with a y value versus a data point index signal.

Syntax

```
yvsi(<signal>)
```

Returns

A waveform.

Arguments

<signal>

The input signal to be measured.

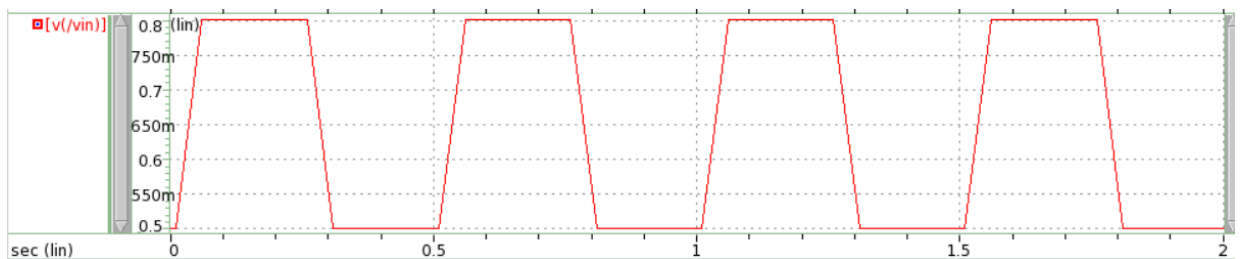
Errors

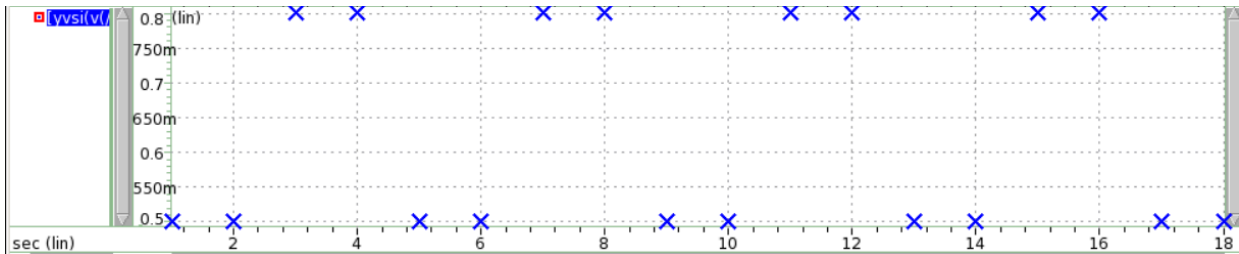
Possible error messages include:

- Signal not found

Examples

```
yvsi(v(/vin))
```





RF Functions

This topic contains information on how to use the supported RF functions.

The following table summarizes the supported RF functions. Complex mathematical functions can also operate on both scalars and waveforms.

Note:

The RF functions support S-parameter results from Spectre, ELDO-RF, and ADS.

Function	Description
apgc	Calculates the available power gain circles (APGC) over S-parameter values and gain values at specified frequency values.
b1f	A 2-port stability factor B1.
dBCPhVRI	dB compression point VRI.
dBCPVRI Curves	Creates a signal associated with an nth compression point measurement.
ga	A 2-port available power gain.
gac_freq	The available power gain circles where the gain is fixed and frequency is swept.
gac_gain	The available power gain circles where the frequency is fixed and gain is swept.
gmax	A 2-port maximum available power gain.
gmin	The optimum noise reflection coefficient for nfmin.
gmsg	A 2-port max. stable power gain.
gp	A 2-port power gain.

Function	Description
gpc_freq	The operating power gain circles where the gain is fixed and frequency is swept.
gpc_gain	The operating power gain circles where the frequency is fixed and gain is swept.
gt	A 2-port transducer power gain.
gumx	A 2-port unilateral max. available power gain.
hp	2-port H-parameter, p1/p2 1-based port index. z0: port characteristic impedance.
kf	A 2-port stability factor.
lsb	Returns the load stability circles.
lq	Calculates the Inductance value L and Quality factor Q.
mu1	U-test for stability circle derivation.
mu2	U-test for stability circle derivation.
nc_freq	Noise circles with fixed frequency.
nc_gain	Noise circles with fixed frequency.
nf	The noise figure of the waveform file.
nfmin	The minimum noise figure.
noisec	Calculates the noise circle over input Noise Figure values at specified frequency values.
opgc	Calculates the Operating Power Gain Circles over S-parameter values and gain values at specified frequency values.
rn	Normalized equivalent noise resistance vs. frequency waveform.
Rn	Equivalent noise resistance vs. frequency waveform.
sp	S-parameter, p1 and p2 are 1-based port index.
spm	The differential S-parameter function (scattering parameters).
ssb	Returns the source stability circles.
stabc_in	Calculates the Input stability circles over S-parameters, (S11, S12, S21, and S22), at the specified frequency values.

Function	Description
stabc_out	Calculates the output stability circles over S-parameters, (S11, S12, S21 and S22), at the specified frequency values.
vswr	Measures the voltage standing wave ratio.
vswrc_in	Calculates the input voltage standing wave ratio cycle circles calculated over S-parameter values, as well as the input value of voltage standing wave ratios at specified frequency values.
vswrc_out	Calculates the output voltage standing wave ratio cycle circles over S-parameter values, as well as the output value of a voltage standing wave ratio at specified frequency values.
ym	Y parameter at port1 with all other ports matched.
yp	Y-parameter, p1 and p2 are 1-based port index. z0 is the port characteristic impedance.
zm	Z-parameter at port p1 with all other ports matched.
zp	Z-parameter, p1 and p2 are 1-based port index. z0 is the port characteristic impedance.

apgc

Description

Calculates the available power gain circles (APGC) over S-parameter values and gain values at specified frequency values. The circles are implemented as sweep signals with `gain` as a parameter if gain sweep is the input, or with frequency as a parameter if multiple frequency values are input.

The result waveform can be one circle if the waveform file is not a sweep waveform file. One gain value is input, and only one frequency value is selected; otherwise, the resulting waveform can be multiple circles.

Syntax

```
apgc (<s11>,<s12>,<s21>,<s22>,<gain>,<freq_list>)
```

Returns

A set of circles.

Arguments

<s11>

Input S-parameter s11.

<s12>

Input signal S-parameter s12.

<s21>

Input signal S-parameter s21.

<s22>

Input signal S-parameter s22.

<gain>

The gain value in dB units, which can be one single value, two values separated by space in the format of “step number”, which means to sweep gain values from maximum gain value at a loss of “Step” (Gmax – Step) with “Number” times, or three values separated by a space in the format of “start stop step”. The maximum gain value is calculated with associated S-Parameter values at specified frequency values. For example, Gmax=0.6, Step=0.1, Number=2, gain values is {0.5 0.4}.

<freq_list>

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

- Ga value is greater than Gamax at trace#=' ', freq=' ', Ga=' ', Gamax=' '
- Ga value is greater than Gamax at freq=' ', Ga=' ', Gamax=' '
- Gamax can't be defined, use Gmsg at trace#=' ', freq=' ', K=' ', |delta|=' '
- Zero again, |S21| is too small at trace#=' ', freq=' ', S21=(' ', ' ')"
- Zero again, |S21| is too small at freq=' ', S21=(' ', ' ')"
- Zero again, |S12| is too small at trace#=' ', freq=' ', S12=(' ', ' ')"
- Zero again, |S12| is too small at freq=' ', S12=(' ', ' ')"
- All frequency values are not in the files
- Signal is not valid or cannot find the loaded signal

b1f

A 2-port stability factor for B1.

Syntax

```
blf(<file>)
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

dBCPhVRI

Description

dB compression point VRI.

Syntax

```
dBCPhVRI(<voltage_signal>,<harmonic_index>,[<current_signal>],  
[<resistance>],[<gainDecrease>],[<extrapolationPoint>],[<sweepUnits>],[<  
format>],[<referred>])
```

Returns

A multiple traces signal.

The first trace is the power or gain curve. The second trace is the extrapolation line.

Arguments

<voltage_signal>

The input voltage signal.

<harmonic_index>

Harmonic index.

<current_signal>

(Optional) The input current signal.

<resistance>

(Optional) Resistance.

`<gainDecrease>`

(Optional) The gain decrease in db. The default is 1.

`<extrapolationPoint>`

(Optional) The input signal extrapolation point.

`<sweepUnits>`

(Optional) Can be `dBm`, `W`, or `none`. Default is `dBm`.

`<format>`

(Optional) Can be `power` or `gain`. Default is `power`.

`<referred>`

(Optional) Input/Output referred measurement. Default is `input`.

dBCPVRICurves

Description

Creates a signal associated with an nth compression point measurement.

This function extracts the specified harmonic from the input signals, calculates a power waveform through the `spectralPower` function for given voltage signal and current signal or resistance.

This function then uses the power waveform to extrapolate a constant slope line.

The result signal is a multiple traces signal containing power waveform and the extrapolation line.

Syntax

```
dBCPVRICurves(<voltage_signal>,<harmonic_index>,[<current_signal>],  
[<resistance>],[<gainDecrease>],[<extrapolationPoint>],[<sweepUnits>],  
[<format>])
```

Returns

A multiple traces signal.

The first trace is the power or gain curve. The second trace is the extrapolation line.

Arguments

`<voltage_signal>`

The input voltage signal, which must be a multiple traces spectrum waveform.

<harmonic_index>

Harmonic index.

<current_signal>

(Optional) The input current signal, which must be a multiple traces spectrum waveform.

<resistance>

(Optional) Resistance into the output port. Default value is 50 Ohm.

<gainDecrease>

(Optional) The delta between power waveform and ideal gain line.

<extrapolationPoint>

(Optional) The X value (in dBm) indicating the power waveform point that constant slope extrapolation line crosses. Default is the first point of the power waveform.

<sweepUnits>

(Optional) Can be dBm, W, or none. Default is dBm.

<format>

(Optional) Can be power or gain. Default is power.

ga

A 2-port available power gain. The algorithm for power gain is where P_{load} is the maximum time average for power to be delivered to the load and the maximization of power is over the load impedance, and P_{input} is the time averaged power entering the network:

$$G_p = \frac{P_{load}}{P_{input}}$$

Syntax

ga(<file>, <load_impedance>)

Returns

A waveform or a scalar value.

Arguments

<load_impedance>

The characteristic load impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

gac_freq

Description

The available power gain circles where the gain is fixed and frequency is swept.

Syntax

```
gac_freq(<file>, <level_db>, <start_freq>, <stop_freq>, <step_freq>)
```

Returns

A set of circles.

gac_gain

Description

The available power gain circles where the frequency is fixed and gain is swept.

Syntax

```
gac_gain(<file>, <freq_Hz>, <start_gain>, <stop_gain>, <step_gain>)
```

Returns

A set of circles.

gmax

A 2-port maximum available power gain.

Syntax

```
gmax(<file>)
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

gmin

Description

The optimum noise reflection coefficient for `nfmin`.

Syntax

```
gmin(<file>)
```

Returns

Returns a waveform or a scalar value.

gmsg

A 2-port maximum stable power gain.

Syntax

```
gmsg(<file>)
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

gp

A 2-port power gain. The algorithm for power gain is where P_{load} is the average power delivered to the load and P_{source} is the average available power at the source:

$$G_p = \frac{P_{source}}{P_{load}}$$

Syntax

```
gp(<file>,<source_impedance>,<load_impedance>)
```

Returns

A waveform or a scalar value.

Arguments

<source_impedance>

The characteristic source impedance.

<load_impedance>

The characteristic load impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

gpc_freq

Description

The operating power gain circles where the gain is fixed and frequency is swept.

Syntax

```
gpc_freq(<file>, <level_db>, <start_freq>, <stop_freq>, <step_freq>)
```

Returns

A set of circles.

gpc_gain

Description

The operating power gain circles where the frequency is fixed and gain is swept.

Syntax

```
gpc_gain(<file>, <freq>, <start_gain>, <stop_gain>, <step_gain>)
```

Returns

A set of circles.

gt

A 2-port transducer power gain. The algorithm for power gain is where P_{load} is the average power delivered to the load and $P_{source\ max}$ is the maximum available power at the source:

$$G_p = \frac{P_{load}}{P_{source\ max}}$$

Syntax

```
gt(<file>, <source_impedance>)
```

Returns

A waveform or a scalar value.

Arguments

<source_impedance>

The characteristic source impedance.

Errors

Possible error messages include:

- Data file has no data point
 - The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
-

gumx

A 2-port unilateral maximum power gain.

Syntax

`gumx (<file>)`

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

hp

Description

2 port H parameter function (hybrid parameters)
$$\begin{bmatrix} V_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} I_1 \\ V_2 \end{bmatrix}$$

where: $h_{11} = \frac{V_1}{I_1} \Big|_{V_2=0}$, $h_{12} = \frac{V_1}{V_2} \Big|_{I_1=0}$, $h_{21} = \frac{I_2}{I_1} \Big|_{V_2=0}$, $Z_{22} = \frac{I_2}{V_2} \Big|_{I_1=0}$.

Syntax

`hp (<file>, <port1>, <port2>, <impedance>)`

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

`<impedance>`

The port characteristic impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of H parameter size (no H parameter found)

kf

A 2-port stability factor.

Syntax

```
kf(<file>)
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

lsb

Description

Returns the load stability circles.

Syntax

```
lsb(<file>, <start_freq>, <stop_freq>, <step_freq>)
```

Returns

A set of circles.

lq

Description

Calculates the Inductance value L and Quality factor Q.

Syntax

```
lq(<file>,<type> [,<inductor_mode>] [,<port>] [,<port2_index>])
```

Returns

When type is L, returns the Inductance value L. When type is Q, returns the Quality factor value.

Arguments

`<file>`

The waveform file.

`<type>`

Valid types are L and Q. The default is L.

`<inductor mode>`

(Optional) The inductor mode.

`<port>`

(Optional) Port 1 index.

`<port2_index>`

(Optional) Port index.

Errors

Possible error messages include:

- `lq(): unknown type '%s', expect SingleEnded/Differential/CommonFloating/CommonGND: Indicates that the type is unexpected.`
- `Input value of port is out the range of LQ size (no LQ found): Indicates that the port is unexpected.`
- `Evaluation failed: Indicates that the SP signal was not found.`

Examples

```
lq(file(~\test.sp),L,SingleEnded)
```

Calculates the Inductance value in single-ended.

```
lq(file(~/test.s3p),Q, Differential)
```

Calculates the Quality factor value of three port inductors in differential mode.

For an example of how to plot the inductance value L and Quality factor Q by S-parameter, see SolvNetPlus article, [L&Q Plot by S-Parameter in the PrimeWave Design Environment](#)".

mu1

U-test for stability circle derivation.

Syntax

```
mu1(<file>)
```

Returns

A waveform or a scalar value.

mu2

U-test for stability circle derivation.

Syntax

```
mu2(<file>)
```

Returns

A waveform or a scalar value.

nc_freq

Description

Noise circles with fixed gain.

Syntax

```
nc_freq(<file>,<level>,<start_freq>,<stop_freq>,<step_freq>)
```

Returns

A complex analog signal that can be plotted on Smith Chart.

Arguments

`<level>`

Noise level in dB.

`<start_freq>`

Starting frequency.

`<stop_freq>`

Ending frequency.

`<step_freq>`

Frequency step value.

nc_gain

Description

Noise circles with fixed frequency.

Syntax

```
nc_gain(<file>,<frequency>,<start_gain>,<stop_gain>,<step_gain>)
```

Returns

A complex analog signal that can be plotted on Smith Chart.

Arguments

`<frequency>`

Noise level in Hz.

`<start_gain>`

Starting noise level.

`<stop_gain>`

Ending noise level.

`<step_gain>`

Noise level step value.

nf

Description

The noise figure of the waveform file.

Syntax

```
nf(<file>)
```

Returns

An analog signal.

nfmin

Description

The minimum noise figure.

Syntax

```
nfmin(<file>)
```

Returns

An analog signal.

noisec

Description

Calculates the noise circle over input Noise Figure values at specified frequency values. The circles are implemented as a sweep signal with `noise_figure` as a parameter if multiple noise figures are specified. Frequency is another parameter if multiple frequency values are specified.

Other needed values include NFmin, Rn and Zopt. These values come from signals in the active waveform file (for example, `S-Par.NFmin`, `S-Par.Rn`, and `S-Par.Zopt`). If these signals are not located in a file, the function fails.

The resulting waveform can be one circle if the waveform file is not a sweep waveform file, one noise figure value is input and only one frequency value is input; otherwise, the resulting waveform can be multiple circles.

Syntax

```
noisec(<file>,<noise_figures>,<freq_list>)
```

Returns

A set of circles.

Arguments

`<noise_figures>`

A value in dB scale which can either be a single value or three values separated by space in the format of `start stop step`.

`<freq_list>`

A frequency list which can be one single value or a list of frequency values separated by space.

Errors

Possible error messages include:

- Signal not found
- Some frequency values are not in the file
- No noise parameter information in the file
- Imaginary radius values at `trace#=' ', freq=' ', nf=' '`
- No system impedance `Zo` in waveform file
- Invalid noise figure inputs

opgc

Description

Calculates the Operating Power Gain Circles over S-parameter values and gain values at specified frequency values. The circles are implemented as sweep signals with `gain` as a parameter if gain sweep is input or frequency as a parameter if multiple frequency values are input.

The result waveform can be one circle if the waveform file is not a sweep waveform file. One gain value is input if only one frequency value is selected; otherwise, the result waveform can be multiple circles.

Syntax

```
opgc (<s11>, <s12>, <s21>, <s22>, <gain>, <freq_list>)
```

Returns

A set of circles.

Arguments

<s11>

Input S-parameter s11.

<s12>

Input signal S-parameter s12.

<s21>

Input signal S-parameter s21.

<s22>

Input signal S-parameter s22.

<gain>

The gain value in dB scale, which can be a single value, two values separated by spaces in the “step number” format (sweep gain values from maximum gain value at a loss of “Step” (Gmax – Step) with “Number” times), or three values separated by a space in the format of “start stop step”. The maximum gain value is calculated with associated S-Parameter values at specified frequency values. For example, Gmax=0.6, Step=0.1, Number=2, gain values are {0.5 0.4}.

<freq_list>

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

- Gp value is greater than Gpmax at trace#=' ', freq=' ', Gp=' ', Gpmax=' '
- Gpmax can't be defined, use Gmsg at trace#=' ', freq=' ', K=' ', |delta|=' '
- S11/S12/S21/S22 is not S-Parameter
- Zero again, |S21| is too small at trace#=' ', freq=' ', S21=(' ', ' ')
- Zero again, |S21| is too small at freq=' ', S21=(' ', ' ')
- Zero again, |S12| is too small at trace#=' ', freq=' ', S12=(' ', ' ')

- Zero again, |S12| is too small at freq=' ', S12=(' ', ' ')"
- All frequency values are not in the files
- Signal is not valid or cannot find the loaded signal

rn

Description

Normalized equivalent noise resistance vs. frequency waveform.

It is available on 2-port S-Parameter analysis.

Syntax

```
rn(<file>)
```

Returns

An analog signal.

Rn

Description

Equivalent noise resistance vs. frequency waveform. It is available on 2-port S-Parameter analysis.

Syntax

```
Rn(<file>)
```

Returns

An analog signal.

sp

Description

The S-parameter function (scattering parameters) describes the electrical behavior of linear electrical networks when undergoing various steady state stimuli by electrical signals. For a generic multi-port network, the ports are numbered from 1 to N, where N is the total number of ports. For port n, the associated S-parameter definition is in terms of incident and reflected 'power waves', a_n and b_n respectively. The incident power wave for each port is as follows: $a = \frac{1}{2}k(V + Z_p I)$

The reflected wave for each port is defined as: $b = \frac{1}{2}k(V - Z_p I)$

Z_p is the diagonal matrix of the complex reference impedance for each port. Z_p^* is a conjugate of Z_p . V and I are the column vectors of the voltage and currents at each port.

An S-parameter matrix can be defined as where S is an $N \times N$ matrix containing elements that can be indexed using conventional matrix notation. $k = \left(\sqrt{|\Re\{Z_p\}|} \right)^{-1}$ $b = Sa$

For two S-Parameters, the following S-parameter matrix $\begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$ and $b_1 = S_{11}a_1 + S_{12}a_2$; $b_2 = S_{21}a_1 + S_{22}a_2$ equation applies.

Syntax

`sp(<file>, <port1>, <port2>)`

Returns

A waveform or a scalar value.

Arguments

`<file>`

Waveform file.

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of S parameter size (no S parameter found)

spm

Description

The differential S-parameter function (scattering parameters).

Syntax

```
spm(<file>,<port1>,<port2>,[<port1_instance>],[<port2_instance>])
```

Returns

A waveform or a scalar value.

Arguments

<file>

Waveform file.

<port1>

The first 1-base port index.

<port2>

The second 1-base port index.

[<port1_instance>]

(Optional) The first port instance.

[<port2_instance>]

(Optional) The second port instance.

ssb

Description

Returns the source stability circles.

Syntax

```
ssb(<file>, <start_freq>, <stop_freq>, <step_freq>)
```

Returns

A set of circles.

stabc_in

Description

Calculates the Input stability circles over S-parameters, (S11, S12, S21, and S22), at the specified frequency values. The circles are implemented as a sweep signal with frequency as a parameter if multiple frequency values are specified.

The resulting waveform can be one circle if the waveform file is not a sweep waveform file and only one frequency value is input; otherwise, the resulting waveform can be multiple circles.

Syntax

```
stabc_in(<s11>,<s12>,<s21>,<s22>,<freq_list>)
```

Returns

A set of circles.

Arguments

<s11>

Input S-parameter s11.

<s12>

Input signal S-parameter s12.

<s21>

Input signal S-parameter s21.

<s22>

Input signal S-parameter s22.

<freq_list>

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

- All frequency values are not in the file
- The circuit is unconditionally stable at trace#=", freq=' ', K=' ', |delta="|
- Signal is not valid or cannot find the loaded signal
- Invalid frequency inputs

stabc_out

Description

Calculates the output stability circles over S-parameters, (S11, S12, S21 and S22), at the specified frequency values. The circles are implemented as a sweep signal with frequency as a parameter if multiple frequency values are specified.

The resulting waveform can be one circle if the waveform file is not a sweep waveform file and only one of the frequency values is input; otherwise, the resulting waveform can be multiple circles.

Syntax

```
stabc_out(<s11>,<s12>,<s21>,<s22>,<freq_list>)
```

Returns

A set of circles.

Arguments

<s11>

Input S-parameter s11.

<s12>

Input signal S-parameter s12.

<s21>

Input signal S-parameter s21.

<s22>

Input signal S-parameter s22.

<freq_list>

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

- All frequency values are not in the file
- The circuit is unconditionally stable at trace#=", freq=' ', K=' ', |delta="|
- Signal is not valid or cannot find the loaded signal

VSWR

Description

Measures the voltage standing wave ratio. The algorithm is: $VSWR = \frac{1+|\Gamma|}{1-|\Gamma|}$

where Γ is a reflection coefficient $\Gamma = \frac{V_r}{V_f}$.

V_r is a complex amplitude of the reflected wave and V_f is a complex amplitude of a forward wave.

Syntax

```
vswr(<file>,<port1>)
```

Returns

A waveform or a scalar value.

Arguments

<port1>

The first 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of VSWR size (no VSWR found)

vswrc_in

Calculates the Input voltage standing wave ratio cycle circles calculated over S-parameter values, as well as the input value of voltage standing wave ratios at specified frequency values. The circles are implemented as sweep signals with frequency as a parameter if multiple frequency values are input.

Syntax

```
vswrc_in(<s11>,<s12>,<s21>,<s22>,<input_vswr>,<freq_list>)
```

Returns

A set of circles.

Arguments

<s11>

Input S-parameter s11.

<s12>

Input signal S-parameter s12.

<s21>

Input signal S-parameter s21.

<s22>

Input signal S-parameter s22.

<input_vswr>

The input value of voltage standing wave ratio.

<freq_list>

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

- All frequency values are not in the file

vswrc_out

Calculates the output voltage standing wave ratio cycle circles over S-parameter values, as well as the output value of a voltage standing wave ratio at specified frequency values. The circles are implemented as sweep signals with frequency as parameter if multiple frequency values are input.

Syntax

```
vswrc_out(<s11>,<s12>,<s21>,<s22>,<output_vswr>, <freq_list>)
```

Returns

A set of circles.

Arguments

<s11>

Input S-parameter s11.

<s12>

Input signal S-parameter s12.

<s21>

Input signal S-parameter s21.

<s22>

Input signal S-parameter s22.

`<output_vswr>`

The output value of voltage standing wave ratio.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

- All frequency values are not in the file

ym

Y parameter at port1 with all other ports matched. The formula is as follows: $y_m = (1 - s) / (1 + s) / Z_0$

Syntax

`ym(<file>, <port1>)`

Returns

A waveform or a scalar value.

Arguments

`<file>`

The waveform file.

`<port1>`

The 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of YM size (no YM found)

yp

Description

The Y parameter function (admittance parameters) are used in electrical engineering, electronic engineering, and communication systems engineering. This function describes the electrical behavior of linear electrical networks and also describes the small-signal response of non-linear networks.

Two-port networks, the relationship between the port currents, port voltages, and the Z-parameter matrix is given by:

$$\begin{bmatrix} I_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix}$$

where $Y_{11} = \frac{I_1}{V_1} \Big|_{V_2=0}$, $Y_{12} = \frac{I_1}{V_2} \Big|_{V_1=0}$, $Y_{21} = \frac{I_2}{V_1} \Big|_{V_2=0}$, $Y_{22} = \frac{I_2}{V_2} \Big|_{V_1=0}$.

Relation to S-parameters (two

port): $Y_{11} = \frac{((1-S_{11})(1+S_{22})+S_{12}S_{21})}{\Delta s}$, $Y_{01}Y_{12} = \frac{-2S_{12}}{\Delta s}$, $Y_{01}Y_{21} = \frac{-2S_{12}}{\Delta s}$, $Y_{01}Y_{22} = \frac{((1+S_{11})(1-S_{22})+S_{12}S_{21})}{\Delta s}$

where $\Delta s = (1-S_{11})(1-S_{22})-S_{12}S_{21}$

Syntax

`yp(<file>, <port1>, <port2>, [<impedance>])`

Returns

A waveform or scalar value.

Arguments

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

`<impedance>`

The port characteristic impedance.

Errors

Possible error messages include:

- Data file has no data point
 - The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
 - Input value of port is out the range of Y parameter size (no Y parameter found)
-

zm

Z parameter at port1 with all other ports matched.

Syntax

```
zm(<file>,<port1>)
```

Returns

A waveform or a scalar value.

Arguments

<file>

The waveform file.

<port1>

The 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
 - The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
 - Input value of port is out the range of ZM size (no ZM found)
-

zp

Description

The Z parameter function (impedance parameters) are used in electrical engineering, electronic engineering, and communication systems engineering. This function describes

the electrical behavior of linear electrical networks and also describes the small-signal response of non-linear networks.

Two-port networks, the relationship between the port currents, port voltages, and the Z-parameter matrix is given by:

$$\begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = \begin{bmatrix} Z_{11} & Z_{12} \\ Z_{21} & Z_{22} \end{bmatrix} \begin{bmatrix} I_1 \\ I_2 \end{bmatrix}$$

where $Z_{11} = \frac{V_1}{I_1} \Big|_{I_2=0}$, $Z_{12} = \frac{V_1}{I_2} \Big|_{I_1=0}$, $Z_{21} = \frac{V_2}{I_1} \Big|_{I_2=0}$, $Z_{22} = \frac{V_2}{I_2} \Big|_{I_1=0}$.

Relation to S-parameters (two

port): $Z_{11} = \frac{((1+S_{11})(1-S_{22})+S_{12}S_{21})}{\Delta S} Z_0$, $Z_{12} = \frac{2S_{12}}{\Delta S} Z_0$, $Z_{21} = \frac{2S_{12}}{\Delta S} Z_0$, $Z_{22} = \frac{((1+S_{11})(1-S_{22})+S_{12}S_{21})}{\Delta S} Z_0$

where $\Delta S = (1-S_{11})(1-S_{22})-S_{12}S_{21}$

Syntax

`zp(<file>,<port1>,<port2>,<impedance>)`

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

`<impedance>`

The port characteristic impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of Z parameter size (no Z parameter found)